

台灣自編大學教科書

影 像 處 理

Digital Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程

大學部 智慧機器人學程 | 高中 AI 機器人課程

編著 李世淵（機器人叫獸）

助教 李天宇、李宇晴

課程資訊與補充教材 [@prof](#)

序言 從像素到世界

這本書是為國立雲林科技大學智慧科技學院智慧機器人學程的《影像處理》課程而寫的,同時也希望能夠為高中的 AI 機器人課程提供一份可用的教材。它從一張影像最基本的像素講起,一路走到讓機器自己學習、自己辨識,最後把這一切裝進機器人裡,讓它能在真實世界中看見與行動。

寫這本書的起心動念很單純。這些年在課堂上,我常看到兩種讓人惋惜的情況。一種是學生很快學會了呼叫函式庫,能做出漂亮的成果,卻在遇到問題時完全不知道該從哪裡查起——因為他不知道那行程式底下發生了什麼事。另一種是學生把公式背得滾瓜爛熟,卻答不出「這個方法什麼時候該用、什麼時候不該用」。這本書想做的,是把「理解原理」與「動手做出來」重新縫合在一起。

因此本書有幾個刻意的安排,想在這裡先說明。

其一,深度學習被放在第 15 章,而不是開頭。這在今天或許顯得保守,但我相信:唯有先理解卷積、頻率、多尺度、特徵這些基本觀念,你才能真正看懂卷積神經網路在做什麼。事實上,當你讀到第 15 章,會發現網路自己學出來的第一層濾波器,與前面各章人工設計的邊緣偵測器驚人地相似——深度學習沒有推翻傳統影像處理的智慧,而是把它自動化了。

其二,每一章開頭都有一則「叫獸開講」。它們不是花絮。傅立葉的論文被退了稿,他繼續改;哈伯太空望遠鏡近乎失明,工程師用數學把影像救回來;張正友用一張列印的棋盤格,取代了昂貴的精密標定物;一群創客翻譯著沒人看得懂的技術文件,意外打開了物聯網的大門;YOLO 的創造者在成就的頂點,因為對技術用途的疑慮而選擇放下。這些故事想告訴你:技術從來不是冷冰冰的公式,而是活生生的人,帶著各自的處境、熱情與判斷,一步步做出來的。而每一則故事的史料,都經過查證。

其三,本書採用 ESP32-CAM 與 NVIDIA Jetson 雙平台實作。前者只要一頓便當的價錢,後者則能讓深度學習模型真正「上車」。這樣的安排是希望你建立一種判斷力:選對工具,遠比用最強的工具重要。這也是全書反覆出現的主題——適合勝於最強。

其四,課程的教學設計遵循 MIT 媒體實驗室 Mitchel Resnick 提出的 4P 創意學習理念:專題 (Projects)、熱情 (Passion)、同儕 (Peers)、遊玩 (Play)。每章都有動手做的 Lab、每題習題都附上提示與參考解答、而全書以一個由你自己選題的期末專題收尾。我希望你做出來的東西是你的,不是老師的、不是課本的。

書中也有一條不那麼顯眼、卻同樣重要的暗線:技術背後的責任。從第 1 章那張退役的測試影像、第 7 章「還原或腦補」的區分、第 14 章模型學到的無意義捷徑,到第 16 章關於軍事應用的抉擇——技術能力愈強大,對後果的想像力就愈是專業素養不可分割的一部分。這門課教你把東西做出來,但沒有任何一行程式碼能替你決定「該不該做」。

最後要說明的是,本書的每一章都經過「加厚」的改寫:除了正文之外,每章都附有與全書其他章節的觀念連結總表、多個應用案例、四個程式實作 Lab,以及附提示與參考解答的習題。這是因為我相信,影像處理真正困難的地方不在單一演算法,而在於看見那些反覆出現的共通結構——換一個域看問題、多解析度、投票對抗雜訊、正則化對抗病態、精確率與召回率的取捨。技術會更迭,這些思考方式不會。

感謝助教李天宇與李宇晴在教材整理與實驗驗證上的協助,也感謝歷屆修課同學的提問——很多章節的補充,都是因為你們問了好問題。書中若有錯漏,責任在我,也歡迎透過 @prof 與我聯繫指正。

祝你讀得愉快,做得盡興。我們在你的作品裡見。

李世淵 (機器人叫獸)

國立雲林科技大學 智慧機器人學程

目錄

(頁碼為正文頁碼)

第一篇 基礎

第 1 章 緒論	1
1.1 什麼是影像處理	2
1.2 影像處理發展史：四個階段	3
1.3 應用領域巡禮	5
1.4 標準測試影像與技術社群文化	5
1.5 本書架構與學習地圖	6
1.6 應用案例	8
1.7 本章與全書的觀念連結總表	9
第 2 章 開發環境與程式基礎	14
2.1 開發環境的建立	15
2.2 影像就是陣列：NumPy 基礎	16
2.3 OpenCV 基本操作	18
2.4 三種 AI 辨識架構與雙平台	20
2.5 工程習慣的養成	22
2.6 應用案例	23
2.7 本章與全書的觀念連結總表	24
第 3 章 數位影像基礎	29
3.1 從連續到離散：取樣與量化	30
3.2 取樣定理與混疊	31
3.3 量化與位元深度	33
3.4 影像的產生：感測器與色彩	34
3.5 影像品質的客觀評估	35
3.6 像素的鄰域、連通性與距離	36
3.7 應用案例	37
3.8 本章與全書的觀念連結總表	38

第二篇 核心

第 4 章 幾何轉換	43
4.1 基本幾何轉換與齊次座標	44
4.2 轉換家族：從剛體到透視	45
4.3 影像內插：轉換後的像素從哪裡來	47
4.4 相機模型、鏡頭畸變與張氏標定	48
4.5 幾何轉換在全書中的角色	50
4.6 應用案例	51
4.7 本章與全書的觀念連結總表	52
第 5 章 空間域影像強化	57
5.1 空間域處理的框架與灰階轉換	58
5.2 直方圖處理	60
5.3 空間濾波：卷積與相關	61
5.4 平滑濾波：對付雜訊	62
5.5 銳化濾波：強調細節	63
5.6 多影格合成：以軟體突破硬體限制	65

5.7 應用案例.....	66
5.8 本章與全書的觀念連結總表.....	67
第 6 章 頻率域影像處理.....	73
6.1 從弦波到影像：傅立葉的基本觀念.....	74
6.2 二維離散傅立葉轉換(DFT)與 FFT.....	76
6.3 頻率域濾波：低通、高通、帶通.....	78
6.4 帶阻與陷波濾波：狙擊週期性雜訊.....	81
6.5 卷積定理：兩個域的橋梁.....	82
6.6 頻率域觀點與全書的觀念連結.....	83
6.7 應用案例.....	85
6.8 本章與全書的觀念連結總表.....	88
第 7 章 影像還原.....	94
7.1 影像還原的定位、模型與病態性.....	95
7.2 節)並不成立，因為它的強度隨訊號變化。第三，雜訊是零均值的。若感測器有直.....	96
7.3 空間域去雜訊：從排序統計到非局部平均.....	98
7.4 頻率域觀點：還原與濾波的血緣關係.....	100
7.5 反卷積：逆濾波、維納濾波與約束最小平方.....	102
7.6 應用案例.....	104
7.7 影像還原對後續處理的影響.....	106
7.8 AI 影像修復：從還原到生成.....	107
7.9 本章與全書的觀念連結總表.....	108
第 8 章 色彩影像處理.....	113
8.1 色彩的基礎：從人眼到 RGB.....	114
8.2 HSL 與 HSV：符合人類直覺的色彩模型.....	115
8.3 YCbCr 與 CIE Lab.....	117
8.4 虛擬色彩：讓看不見的資訊現形.....	117
8.5 白平衡與色彩校正.....	118
8.6 顏色分割與物件追蹤.....	120
8.7 應用案例.....	121
8.8 本章與全書的觀念連結總表.....	122
第 9 章 影像分割.....	127
9.1 什麼是影像分割.....	128
9.2 門檻法：最簡單的分割.....	129
9.3 邊緣偵測.....	130
9.4 霍夫轉換：參數空間的投票.....	132
9.5 基於區域的分割.....	133
9.6 從傳統分割到深度學習語意分割.....	135
9.7 應用案例.....	136
9.8 本章與全書的觀念連結總表.....	137
第 10 章 二值影像處理.....	142
10.1 二值影像與型態學的基礎.....	143
10.2 兩個基本運算：侵蝕與膨脹.....	144
10.3 兩個組合運算：斷開與閉合.....	145
10.4 進階型態學運算.....	145
10.5 連通元件標記與距離轉換.....	146
10.6 Haar 級聯分類器與人臉偵測.....	147
10.7 應用案例.....	149
10.8 本章與全書的觀念連結總表.....	150
第 11 章 小波與正交轉換.....	154
11.1 正交轉換的共同思想.....	155
11.2 傅立葉的弱點與小波的動機.....	155

11.3 Haar 小波與多解析度分析.....	156
11.4 離散餘弦轉換與 JPEG 壓縮.....	158
11.5 小波壓縮與 JPEG 2000.....	159
11.6 小波的其他應用.....	160
11.7 小波在深度學習時代的定位.....	160
11.8 應用案例.....	161
11.9 本章與全書的觀念連結總表.....	162
第 12 章 視訊與動態影像處理.....	167
12.1 視訊的本質：多了一個時間維度.....	168
12.2 視訊的讀取、處理與寫出.....	169
12.3 移動偵測：影格差分與背景相減.....	170
12.4 光流：估計像素的運動.....	171
12.5 物件追蹤.....	173
12.6 進階主題：穩定、動作辨識與時序深度學習.....	174
12.7 應用案例.....	175
12.8 本章與全書的觀念連結總表.....	176
 第三篇 學習	
第 13 章 特徵擷取.....	181
13.1 什麼是好的特徵.....	182
13.2 角點偵測：Harris 與 Shi-Tomasi.....	183
13.3 尺度不變特徵：SIFT 與 ORB.....	184
13.4 特徵匹配與外點去除.....	185
13.5 應用：影像拼接、模板比對與物件定位.....	187
13.6 案例深探：車牌偵測.....	187
13.7 從手工特徵到學習特徵.....	189
13.8 本章與全書的觀念連結總表.....	189
第 14 章 機器學習基礎.....	194
14.1 什麼是機器學習.....	195
14.2 影像分類的完整流程.....	196
14.3 影像資料集：機器學習的燃料.....	196
14.4 兩個經典分類器：kNN 與 SVM.....	197
14.5 資料切分與模型評估.....	199
14.6 過度擬合、擬合不足與偏差-變異權衡.....	200
14.7 應用案例.....	201
14.8 本章與全書的觀念連結總表.....	201
第 15 章 深度學習.....	206
15.1 從手工特徵到學習特徵：典範轉移.....	207
15.2 神經網路的基本構件.....	207
15.3 卷積神經網路(CNN).....	208
15.4 經典 CNN 架構譜系.....	209
15.5 三大框架與 MNIST 實作.....	210
15.6 遷移學習與資料增強.....	212
15.7 深度學習的侷限與展望.....	213
15.8 本章與全書的觀念連結總表.....	213
第 16 章 物件偵測 YOLO 實戰.....	218
16.1 四種視覺任務.....	219
16.2 物件偵測的演進與 YOLO 的核心思想.....	219
16.3 物件偵測的關鍵概念.....	220
16.4 YOLO 實戰：推論、訓練與驗證.....	221
16.5 延伸功能：分割、姿態估計與追蹤.....	223

16.6 應用案例.....	224
16.7 技術倫理：工程師的責任.....	225
16.8 本章與全書的觀念連結總表.....	226

第四篇 整合

第 17 章 邊緣 AI 與機器人視覺部署.....	231
17.1 為什麼需要邊緣運算.....	232
17.2 兩大實作平台：ESP32-CAM 與 Jetson.....	233
17.3 模型最佳化：讓大模型跑在小裝置上.....	234
17.4 效能量測與系統設計.....	235
17.5 整合進機器人系統：ROS 2.....	236
17.6 應用案例.....	237
17.7 部署的實務課題.....	238
17.8 本章與全書的觀念連結總表.....	238
第 18 章 期末專題：4P 實踐.....	244
18.1 4P 創意學習.....	245
18.2 創意學習螺旋.....	246
18.3 專題的規劃與執行.....	246
18.4 專題方向與技術地圖.....	248
18.5 成果展示與評估.....	248
18.6 專題案例參考.....	249
18.7 全書技術整合地圖.....	250

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 1 章 緒論

本章學習目標

- 說明影像處理的定義與範疇，區分影像處理、電腦視覺與電腦圖學三者的關係。
- 闡述低階、中階、高階視覺的三層架構，並理解它與本書章節安排的對應。
- 概述影像處理的發展史，從報紙傳真、太空探索到深度學習的四個階段。
- 舉例說明影像處理在醫療、製造、農業、交通、機器人等領域的應用。
- 認識標準測試影像的角色，並反思技術社群的文化與包容性議題。
- 掌握本書的學習地圖、雙平台實作架構與 4P 學習理念。

叫獸開講|萊娜圖退役：一張測試影像與一個領域的自省

如果你翻開 1980 至 2000 年代的任何一本影像處理教科書或論文集，幾乎必然會遇到同一張照片：一位側身回眸、戴著羽飾寬邊帽的年輕女子。她的名字叫萊娜(Lenna)，而她的臉，曾被戲稱為「比蒙娜麗莎還要被研究得更透徹的一張臉」。半個世紀以來，無數的濾波器、壓縮演算法、邊緣偵測器，都是先在她身上試過才發表的。

這張影像的來歷，是一個關於「偶然」的故事。1973 年六、七月間，南加州大學訊號與影像處理研究所的助理教授 Alexander Sawchuk 與同事，正急著替一位同僚的研討會論文找一張測試影像。實驗室裡現成的測試圖都是 1960 年代電視標準時代留下的老素材，單調乏味；他們想要一張印刷精美、動態範圍好、而且有人臉的照片。就在此時，有人拿著一本當期的《花花公子》雜誌走進實驗室。他們撕下了中頁摺頁的上方三分之一（僅保留頭肩部分），把它捲上一台 Muirhead 滾筒式傳真掃描器——那台機器被他們自行改裝過，接上了紅綠藍三組類比數位轉換器與一台 HP 2100 迷你電腦——掃出了一張 512×512 的數位影像。

接下來的事出乎所有人意料。這張影像因為兼具平滑的皮膚、銳利的羽毛紋理、豐富的色彩層次與明暗對比，成了絕佳的測試素材；南加大甚至會把拷貝發送給來訪的研究者。它就這樣一傳十、十傳百，成了整個領域不成文的標準測試影像，甚至參與了 JPEG 壓縮標準的發展歷程。照片中的瑞典模特兒 Lena Forsén 對此渾然不知，直到多年後才知道自己在一個素未謀面的技術圈裡如此有名——1997 年，她還受邀出席了影像科學與技術學會的年會。

但這個故事有它的另一面。從 2010 年代中期起，愈來愈多研究者提出質疑：一個以《花花公子》中頁作為半世紀標準素材的領域，對想要進入這個領域的女性傳遞了什麼樣的訊息？2019 年的紀錄短片《失去萊娜》(Losing Lena)直指這張影像象徵了科技產業長期的偏狹文化。多家期刊陸續勸退或禁用它。而 Lena Forsén 本人也表達了希望「退休」的意願。

2024 年 3 月，全球最大的計算機專業學會之一——IEEE 計算機協會——正式發函通知會員：自 4 月 1 日起，不再接受包含萊娜圖的投稿論文。函中援引了學會的多元共融聲明與倫理守則，並明確表示這是「尊重影像當事人 Lena Forsén 的意願」。一張見證了整個數位影像時代的照片，就這樣正式退役。

叫獸把這個故事放在全書的第一頁，有三個用意。第一，它讓你看見這個領域的真實歷史——不是無菌的教科書敘事，而是由一群人在特定時空下的偶然選擇所堆疊起來的。第二，它預告了本書的一條暗線：技術從來不只是技術，它承載著使用者的價值觀，也形塑著誰會覺得自己屬於這裡。從第 7 章的「還原或腦補」、第 14 章的資料偏誤、到第 16 章 YOLO 之父的選擇，我們會不斷回到這個主題。第三，它示範了一個健康的專業社群該有的樣子：能夠回頭檢視自己的習慣，即使那個習慣已經存在了五十年。

思考題：萊娜圖之所以流行，一部分原因是它「技術上很好用」（紋理豐富、對比鮮明）。當「技術上好用」與「對某些人不友善」發生衝突時，你認為該怎麼權衡？有沒有可能找到兩全的方案？（提示：今天已有 Kodak 影像庫、DIV2K、BSDS 等大量高品質的替代測試集）

1.1 什麼是影像處理

1.1.1 定義與範疇

數位影像處理(Digital Image Processing)是以電腦對數位影像進行分析與操作的技術。最基本的定義是：輸入一張影像，輸出另一張影像或從中萃取出的資訊。前者例如去除雜訊、增強對比、校正變形；後者例如判斷影像中有幾個物件、它們在哪裡、是什麼。

為什麼需要它？因為人眼雖然強大，卻有三個限制。其一，人眼只能看見電磁波譜中極窄的一段(可見光)，而 X 光、紅外線、超音波、雷達所攜帶的資訊，必須經過處理才能轉換成人眼可讀的形式。其二，人會疲勞、判斷會不一致——一位品檢員連續工作八小時後的判斷，與剛上班時未必相同，而機器不會。其三，人的處理速度有限——一條產線每秒通過數十個工件、一台自駕車每秒接收數十影格影像，這是人力無法應付的規模與速度。

1.1.2 三個相鄰領域的關係

影像處理常與另外兩個領域混淆，釐清它們有助於建立整體樣貌：

1. 影像處理(Image Processing)：影像進、影像出。重點在改善影像品質或轉換其表示方式，例如去雜訊、銳化、壓縮、幾何校正。本書第 4 至 12 章大多屬於此範疇。
2. 電腦視覺(Computer Vision)：影像進、資訊或理解出。重點在讓機器「看懂」影像的內容，例如辨識物件、估計距離、理解場景。本書第 13 至 17 章走向此範疇。
3. 電腦圖學(Computer Graphics)：資訊進、影像出。重點在由模型與參數合成影像，例如 3D 渲染、動畫、遊戲畫面。它與影像處理恰好方向相反。

三者的界線在實務上早已模糊。第 15 章的生成式模型「由文字生成影像」，某種意義上正是圖學與視覺的融合；而第 16 章的物件偵測，既需要影像處理的前處理，也需要電腦視覺的理解。本書採取的立場是：以影像處理為地基，自然地延伸到電腦視覺——這也是為什麼書名是《影像處理》，但後半部大量談論辨識與學習。

1.1.3 低階、中階與高階視覺

一個更有用的分類方式，是依「抽象程度」把整個處理流程分成三層。這個三層架構是理解本書章節安排的鑰匙，請務必牢記：

層級	輸入與輸出	典型工作	本書對應章節
低階視覺	影像進、影像出	去雜訊、增強、還原、幾何轉換、壓縮	第 3-8、11 章
中階視覺	影像進、屬性出	分割、型態學、特徵擷取、追蹤	第 9、10、12、13 章
高階視覺	屬性進、理解出	辨識、偵測、場景理解、決策	第 14-17 章

表 1-1 低階、中階、高階視覺與本書章節的對應

這三層不是彼此獨立的階段，而是層層堆疊的支撐關係：低階處理的品質決定了中階分割的成敗(第 7.7 節會量化這一點)，而中階萃取的特徵又決定了高階辨識的上限。這也解釋了為什麼本書要花費大量篇幅在看似「基礎」的前半部——沒有穩固的地基，再先進的深度學習模型也蓋不高。

1.2 影像處理發展史：四個階段

理解一項技術的歷史，能幫助你看清它「為什麼長成現在這樣」。影像處理的發展大致可分為四個階段，而每個階段的驅動力都不相同。

1.2.1 萌芽期：報紙傳真(1920 年代)

數位影像的最早應用之一，出現在 1920 年代橫越大西洋的海底電纜。當時的巴特蘭(Bartlane)傳輸系統把照片編碼成訊號，經由電纜傳送，再在另一端重建成影像——原本需要一週船運的新聞照片，縮短到數小時。早期系統只能表現五個灰階，後來提升到十五階。這個階段還談不上「處理」，但它建立了一個關鍵觀念：影像可以被離散化成數字來傳輸與儲存。這正是第 3 章取樣與量化的先聲。

1.2.2 奠基期：太空探索與醫學影像(1960-1970 年代)

真正推動影像處理成為一門學科的，是兩股力量。第一股來自太空：1964 年，美國噴射推進實驗室(JPL)處理由徘徊者七號探測器傳回的月球照片，以電腦校正相機造成的幾何失真與雜訊——這被公認為現代數位影像處理的起點。往後的水手號、阿波羅任務持續推動技術演進，而第 7 章叫獸開講中哈伯望遠鏡的反卷積救援，正是這條血脈的延續。

第二股力量來自醫學。1970 年代電腦斷層掃描(CT)問世，它從不同角度拍攝的 X 光投影中「重建」出人體的橫切面影像——這項成就讓 Hounsfield 與 Cormack 獲得 1979 年諾貝爾生理醫學獎，也讓影像處理第一次直接關乎人命。此後的磁振造影(MRI，第 7 章的維納濾波案例)、超音波、正子斷層造影，共同構成了現代醫學不可或缺「看見身體內部」的能力。

值得一提的是，這個階段的驅動力都不是「為了做影像處理而做影像處理」，而是有一個非做不可的真實需求——看清月球、看見身體內部。這個模式在往後的每個階段都不斷重演，也是叫獸希望你在期末專題(第 18 章)中記住的：好的技術，總是從真實的問題出發。

1.2.3 普及期：個人電腦與數位相機(1980-2000 年代)

隨著電腦運算能力提升與成本下降，影像處理從國家級實驗室走進了企業與家庭。這個階段的關鍵發展包括：JPEG 壓縮標準的制定(1992 年，第 11 章)讓數位影像得以在有限的儲存與頻寬中流通；數位相機取代底片，使影像的產生本身就是數位的；工業自動光學檢測(AOI)開始取代人工目檢；而 1999 年 Intel 發起的 OpenCV 專案(第 2 章叫獸開講)則把影像處理的工具開放給了全世界。這也是萊娜圖最風光的年代——叫獸開講中那些一代代被測試的演算法，大多誕生於此時。

1.2.4 智慧期：深度學習與邊緣運算(2012 年至今)

2012 年是分水嶺。AlexNet 在 ImageNet 競賽上的壓倒性勝利(第 15 章叫獸開講)，讓「讓機器自己學特徵」取代了「人工設計特徵」成為主流。此後十餘年，辨識準確率快速逼近甚至超越人類，應用從實驗室湧向產業：自駕車、人臉支付、醫療輔助診斷、智慧製造。

近年的發展則呈現兩個方向：一是往「更大」走——基礎模型、多模態大模型、生成式 AI 模糊了辨識與創造的界線；二是往「更小」走——邊緣運算讓 AI 走出資料中心，進入手機、感測器與機器人(第 17 章)。而對本學程的學生而言，後者尤其重要：機器人要在真實世界中行動，運算就必須發生在現場。

階段	年代	驅動力	代表性成果
萌芽期	1920 年代	新聞照片的跨洋傳輸	巴特蘭電纜傳真系統
奠基期	1960-70 年代	太空探索、醫學診斷	月球照片校正、CT 斷層重建
普及期	1980-2000 年代	個人電腦、數位相機、工業自動化	JPEG 標準、AOI 檢測、OpenCV
智慧期	2012 年至今	大數據、GPU 算力、深度學習	AlexNet、YOLO、生成式 AI、邊緣 AI

表 1-2 影像處理發展的四個階段

1.3 應用領域巡禮

影像處理已滲透到現代生活的每個角落。以下巡禮不求詳盡，而是希望讓你看見「這門課能做什麼」，並在其中找到你想投入的方向(第 18 章的專題選題會用到)。

- 醫療與生醫：CT/MRI 影像的重建與去噪(第 7 章)、病灶分割與量測(第 9 章)、細胞計數(第 10 章)、輔助診斷模型(第 15 章)。這是影像處理最早也最深刻改變人類生活的領域。
- 智慧製造：自動光學檢測(AOI)找出工件瑕疵(第 6、9、16 章)、零件計數與尺寸量測(第 10 章)、機械手臂的視覺定位(第 4、13 章)、產線人員的安全監控(第 16 章)。這是本學程畢業生最主要的就業領域之一。
- 農業與環境：果實計數估產、病蟲害辨識、雜草偵測、無人機田間巡檢(第 16、17 章)、衛星遙測的植生分析。對雲林這個農業大縣而言，這是最具在地價值的應用方向。
- 交通與自駕：車道線偵測(第 9 章)、車輛與行人偵測追蹤(第 12、16 章)、車牌辨識(第 13 章)、車流分析與測速。
- 機器人：視覺導航與避障、視覺里程計與 SLAM(第 12、13、17 章)、物件抓取的姿態估計、人機協作的安全防護。這是本書的最終目的地。
- 消費性應用：手機的夜景模式與計算攝影(第 5 章)、人臉解鎖(第 10、15 章)、影像美化與濾鏡、擴增實境。這些是你每天都在使用、卻可能沒意識到的影像處理。
- 科學研究：天文影像的還原(第 7 章)、顯微影像分析、材料表面檢測、生態調查的自動物種辨識。

觀察這份清單，你會發現一個共通點：影像處理很少是獨立的產品，它總是「某個更大系統中的一環」。醫師需要的不是分割演算法，而是更準確的診斷；工廠需要的不是 YOLO 模型，而是更低的不良率。這個認知很重要——它提醒我們，技術的價值取決於它解決了什麼問題，而不是它有多先進(這個判準會在第 9、11、16、17 章反覆出現)。

1.4 標準測試影像與技術社群文化

承叫獸開講，本節深入談一個容易被忽略、卻對整個領域影響深遠的主題：標準測試影像。

1.4.1 為什麼需要標準測試影像

當研究者甲宣稱他的去雜訊演算法比研究者乙的好，如何驗證？若兩人各自用自己的照片測試，結果無從比較——也許甲的照片本來就比較容易處理。因此，領域內需要共同的基準影像與資料集，讓不同方法能在相同的立足點上比較。這與第 14 章的 MNIST、ImageNet、COCO 扮演的角色完全一致，只是規模不同：標準測試影像是「單張」的基準，而資料集是「大規模」的基準。

好的測試影像需要具備多樣的視覺特性：平滑區域(檢驗是否過度平滑)、銳利邊緣(檢驗是否模糊)、細緻紋理(檢驗細節保留)、豐富色彩與明暗層次(檢驗動態範圍)。萊娜圖恰好同時具備這些特性——這正是它當年被廣泛採用的技術理由。

1.4.2 現代的替代選項

今天已有大量高品質且無爭議的替代選擇，建議同學在作業與專題中優先採用：柯達(Kodak)影像庫的二十四張自然影像、DIV2K 高解析度資料集、BSDS 邊緣與分割基準集、Set5 與 Set14 超解析度基準，以及各領域的專用資料集。OpenCV 與 scikit-image 等函式庫也內建了可自由使用的範例影像。技術上完全不遜色，而且沒有任何倫理包袱——這正是叫獸開講思考題所說的「兩全方案」。

1.4.3 從測試影像看社群文化

萊娜圖事件真正的意義，不在於一張照片的存廢，而在於它揭示了一個道理：一個專業社群的日常習慣，會無聲地傳遞「誰屬於這裡」的訊息。當一個領域的入門教材、研討會投影片、示範程式碼中反覆出現某一類影像時，它就在定義這個領域的文化樣貌。

這個道理在人工智慧時代更加重要，因為現在我們不只是「選擇測試影像」，而是「選擇訓練資料」——而模型會忠實地學習資料中的一切，包括其中的偏誤。第 14 章的肺炎 X 光案例、第 15 章談到的人臉超解析度偏誤，都是這個問題的延伸。身為未來的工程師，你的每一個資料選擇，都在形塑技術的樣貌與它服務的對象。

1.5 本書架構與學習地圖

1.5.1 四篇十八章

本書共十八章，分為四篇，對應 1.1.3 節的三層視覺架構，並以整合實踐收尾：

篇	章節	內容與定位
第一篇 基礎	第 1-3 章	緒論、開發環境與程式基礎、數位影像基礎(取樣量化、色彩、品質評估)
第二篇 核心	第 4-12 章	幾何轉換、空間域強化、頻率域、影像還原、色彩處理、分割、二值型態學、小波、視訊
第三篇 學習	第 13-16 章	特徵擷取、機器學習基礎、深度學習、物件偵測 YOLO 實戰
第四篇 整合	第 17-18 章	邊緣 AI 與機器人視覺部署、期末專題 4P 實踐

表 1-3 本書四篇架構

這個安排有其刻意：深度學習被放在第 15 章而非開頭，是因為叫獸相信——唯有先理解卷積、頻率、多尺度、特徵這些基本觀念，你才能真正看懂 CNN 在做什麼(第 15.1 節會回頭印證這一點)。反之，若一開始就直接學 YOLO，你會很快做出成果，卻只是在呼叫別人寫好的函式，遇到問題時無從診斷。

1.5.2 貫穿全書的核心思想

比起個別的演算法，更重要的是那些會在不同章節以不同面貌反覆出現的思想。請在閱讀時留意它們，遇到時在心中打個記號：

- 換一個域看問題：空間域解不開的問題，到頻率域可能一目了然(第 6 章);傅立葉看不見位置，小波補上了(第 11 章)。
- 多解析度與由粗到細：小波金字塔(第 11 章)、SIFT 尺度空間(第 13 章)、CNN 的層級特徵(第 15 章)，骨子裡是同一種思維。
- 投票對抗雜訊：霍夫轉換讓邊緣點投票給直線(第 9 章)、RANSAC 讓匹配投票給幾何模型(第 13 章)——以多數證據壓制少數錯誤。
- 正則化對抗病態：維納濾波在分母加項以避免雜訊爆炸(第 7 章)、資料增強與 Dropout 避免模型死背(第 14、15 章)，都是「以約束換穩定」。
- 精確率與召回率的取舍：Canny 的雙門檻(第 9 章)、偵測器的信心門檻(第 10、16 章)、評估指標的選擇(第 14 章)——沒有最好的設定，只有最適合任務的設定。
- 適合勝於最強：ESP32 上跑得動的 Haar 級聯勝過跑不動的 YOLO(第 10、17 章)、夠好又普及的 JPEG 勝過技術更優的 JPEG 2000(第 11 章)。
- 技術背後的責任：從本章的萊娜圖、第 7 章的還原與腦補、第 14 章的資料偏誤，到第 16 章 YOLO 之父的選擇——技術能力愈強，選擇就愈重要。

1.5.3 雙平台實作架構

本書的所有實作採用兩個平台，對應第 2 章與第 17 章詳述的三種 AI 辨識架構：

4. ESP32-CAM(入門平台)：極低成本、極低功耗的視覺感測節點。用於理解「端上運算」的能力與限制，以及大規模佈署的可能性。第 17 章叫獸開講會告訴你這個五美元的小東西如何改變了創客世界。
5. NVIDIA Jetson(進階平台)：內建 GPU 的邊緣 AI 電腦，能即時執行深度學習模型與 ROS 2，是機器人的視覺大腦。

多數章節的 Lab 可以先在個人電腦上完成(使用 Python 與 OpenCV，第 2 章會建立環境)，再移植到這兩個平台。這個「先在電腦上驗證邏輯、再移植到目標硬體」的流程，本身就是嵌入式開發的標準工作方式。

1.5.4 4P 學習理念

本課程的教學設計遵循 MIT 媒體實驗室 Mitchel Resnick 提出的 4P 創意學習理念——專題(Projects)、熱情(Passion)、同儕(Peers)、遊玩(Play)。具體體現在：每章都有動手做的 Lab(Projects)、鼓勵你選擇自己在乎的專題題目(Passion)、分組協作與互相回饋(Peers)、以及允許失敗、鼓勵嘗試的評量方式(Play)。

這也是為什麼每章開頭都有「叫獸開講」——它們不是花絮，而是想讓你看見：每一項技術背後都有活生生的人，帶著各自的困境、熱情與選擇。第 18 章會完整展開這套理念，並帶你把它應用到期末專題上。

1.6 應用案例

1.6.1 案例一：從月球照片到哈伯望遠鏡

1964 年，徘徊者七號傳回的月球照片因相機特性與傳輸雜訊而失真，JPL 的工程師以電腦逐一校正——這是現代數位影像處理公認的起點。二十六年後的 1990 年，哈伯太空望遠鏡因主鏡研磨誤差而近乎失明，天文學家再次以數學方法(反卷積)從模糊資料中救回科學成果(第 7 章叫獸開講)。

這兩件相隔四分之一世紀的事，體現了同一個精神：當硬體有其物理極限時，數學與運算可以把資訊救回來。這正是第 7 章影像還原的核心命題，也是叫獸希望你在遇到「感測器不夠好」的困境時，能想起的另一條路。

1.6.2 案例二：AOI 如何改變製造業

在自動光學檢測普及之前，工廠的品質檢驗仰賴人工目視。問題不只是速度慢，更在於一致性——不同檢驗員的標準不同、同一個人在不同時段的判斷也不同，且長時間注視細小物件極易疲勞。AOI 系統以固定的打光、相機與演算法，提供了穩定且可追溯的檢測。

這個案例的教學價值在於：它示範了影像處理的典型價值主張——不是「比人更聰明」，而是「比人更穩定、更快、不會累」。這個定位很重要，它也提醒我們設計系統時該把人放在哪裡：讓機器做重複而穩定的判斷，讓人做需要脈絡與經驗的決策(第 9 章醫學影像、第 16 章人機協作都會回到這個主題)。

1.6.3 案例三：手機裡的計算攝影

你口袋裡的手機，是當代最普及的影像處理裝置。按下快門的那一刻，它其實做了遠不只「拍一張照」的事：連拍多張影像對齊後合併以降低雜訊、依場景自動調整白平衡與色調、以深度學習分離主體與背景製造淺景深、甚至用超解析度重建細節。第 5 章叫獸開講會完整拆解夜景模式的原理。

這個案例說明了一件事：當硬體受限(手機鏡頭與感光元件都很小)時，軟體可以補上差距。它也是本書多章技術的綜合展示——多影格平均(第 5 章)、影像對齊(第 6、13 章)、色彩處理(第 8 章)、分割(第 9 章)、深度學習(第 15 章)全都在你按下快門的那一秒內發生。

1.6.4 案例四：雲林在地的農業視覺

雲林是台灣重要的農業縣，而農業正面臨人口老化與缺工的挑戰。影像處理在此有明確的著力點：以無人機或定點相機自動巡檢作物、辨識病蟲害的早期徵兆、估計產量以協助銷售規劃、辨識雜草以精準施藥減少用量。

這類應用的技術挑戰很典型：戶外光線變化劇烈(第 5、8 章的前處理成為必要)、標註資料稀少(第 15 章的遷移學習與資料增強)、田間沒有穩定網路(第 17 章的邊緣運算)。它們不是最尖端的研究題目，卻是最能產生在地價值的方向——也是叫獸最鼓勵同學在期末專題(第 18 章)嘗試的領域。

1.7 本章與全書的觀念連結總表

作為全書的開篇，本章的每一節都是後續章節的入口。這張表既是本章的整理，也是你日後回頭查找的索引。

本章觀念	後續展開的章節	如何深化
影像的數位化	第 3 章	取樣定理、量化、位元深度、品質評估指標
低階視覺	第 4-8、11 章	幾何轉換、強化、頻率域、還原、色彩、壓縮
中階視覺	第 9、10、12、13 章	分割、型態學、追蹤、特徵擷取
高階視覺	第 14-16 章	機器學習、深度學習、物件偵測
太空與醫學的推力	第 7、9 章	哈伯反卷積、MRI 去噪、病灶分割
標準測試與基準	第 3、14 章	PSNR/SSIM 指標、MNIST/ImageNet/COCO 資料集
資料與文化偏誤	第 14、15、16 章	捷徑學習、模型偏誤、技術倫理與工程師責任
三種辨識架構	第 2、10、17 章	端上、近端邊緣、雲端的取捨與混合架構
4P 學習理念	第 18 章與各章 Lab	專題、熱情、同儕、遊玩;創意學習螺旋

表 1-4 第 1 章與全書其他章節的觀念連結

程式實作 Lab 1

Lab 1-1 影像處理應用觀察

目標：建立對這門課的整體感受。(a)在你的日常生活中，找出五個「背後一定用到影像處理」的產品或服務(例如手機相機、超商的商品辨識、停車場的車牌辨識、社群軟體的濾鏡、醫院的檢查儀器);(b)對每一個，推測它可能用到本書哪幾章的技術，並說明理由;(c)挑其中一個做深入調查：它解決什麼問題?若沒有它，人們原本怎麼做?它的限制是什麼?(d)小組分享並互相補充。

Lab 1-2 我的第一張影像

目標：在正式學習前先建立直覺(第 2 章會建立完整環境，本 Lab 可用線上環境或手機 App 完成)。(a)用手機拍攝三張照片：一張明亮清晰、一張昏暗、一張模糊;(b)使用任何影像編輯工具(手機內建、小畫家、GIMP 或線上工具)，嘗試調整亮度、對比、銳利度，觀察哪些問題「救得回來」、哪些「救不回來」;(c)特別觀察：昏暗的照片調亮後，是否出現原本看不見的雜訊?模糊的照片銳化後，細節是真的回來了嗎?

(d)把你的觀察寫下來——第 5 章(強化)與第 7 章(還原)會回頭解釋這些現象背後的原理，屆時請翻回你的筆記對照。

Lab 1-3 標準測試影像調查

目標：實踐叫獸開講的反思。(a)上網查詢 Kodak 影像庫、DIV2K、BSDS500 等公開測試資料集，各下載幾張影像;(b)觀察它們具備哪些視覺特性(平滑區、銳利邊緣、細緻紋理、色彩層次)，說明為什麼這些特性對測試演算法很重要;(c)選出三張你認為最適合作為本課程測試影像的圖，說明理由，並在往後各章的 Lab 中固定使用它們——這樣你就能跨章比較不同演算法在同一張影像上的效果;(d)討論題：承叫獸開講，當「技術上好用」與「對某些人不友善」衝突時該如何權衡？請提出你的看法與理由。

Lab 1-4 學習地圖與目標設定

目標：為這門課設定屬於你的方向。(a)瀏覽本書目錄與表 1-3 的四篇架構，標出你最感興趣的三章與最擔心的三章;(b)參考 1.3 節的應用領域，選出一個你最想投入的方向，寫下你想解決的一個具體問題;(c)為自己設定一個學期目標(例如「做出能辨識我家果園病害的系統」、「弄懂深度學習到底在學什麼」);(d)把這份紀錄收好，第 18 章期末專題與反思時會請你回頭檢視——那時你會很驚訝自己走了多遠。

本章重點整理

- 數位影像處理以電腦分析與操作影像，輸出可以是影像(強化、還原)或資訊(辨識、量測);它彌補了人眼在光譜範圍、一致性與速度上的三個限制。
- 影像處理(影像進影像出)、電腦視覺(影像進資訊出)、電腦圖學(資訊進影像出)三者方向不同但界線日益模糊;本書以影像處理為地基延伸到電腦視覺。
- 三層視覺架構：低階(去雜訊、強化、還原)、中階(分割、特徵)、高階(辨識、理解)，層層堆疊——低階品質決定中階成敗，中階特徵決定高階上限。
- 發展史四階段：萌芽期(1920 年代電纜傳真)、奠基期(1960-70 年代太空與 CT)、普及期(1980-2000 年代 PC、JPEG、AOI、OpenCV)、智慧期(2012 年 AlexNet 至今的深度學習與邊緣運算);每個階段都由真實需求驅動。
- 標準測試影像與資料集讓不同方法能公平比較;萊娜圖因其技術特性流行半世紀，亦因文化包容性爭議於 2024 年被 IEEE 計算機協會停用，現有 Kodak、DIV2K、BSDS 等替代選項。
- 本書四篇十八章對應三層視覺並以專題整合;貫穿全書的核心思想包括換域看問題、多解析度、投票對抗雜訊、正則化對抗病態、精確率召回率取捨、適合勝於最強，以及技術背後的責任。

習題

- 請以自己的話定義數位影像處理，並說明為什麼需要它(人眼有哪三個限制)。

提示與參考解答：定義：以電腦對數位影像進行分析與操作，輸入影像、輸出另一張影像或從中萃取的資訊。人眼的三個限制：一、只能看見可見光，X 光/紅外線/超

音波等須經處理才能判讀；二、會疲勞且判斷不一致(檢驗員八小時後的標準與剛上班不同)；三、速度有限，無法應付產線每秒數十件或自駕每秒數十影格的規模。

2. 請區分影像處理、電腦視覺與電腦圖學，各以「輸入與輸出」說明，並各舉一例。

提示與參考解答：影像處理：影像進、影像出，重點在改善品質或轉換表示(如去雜訊、壓縮)。電腦視覺：影像進、資訊或理解出，重點在看懂內容(如辨識物件、估計距離)。電腦圖學：資訊進、影像出，由模型與參數合成影像(如3D渲染、遊戲畫面)，方向與影像處理相反。三者界線在生成式AI與現代系統中日益模糊。

3. 說明低階、中階、高階視覺的差異與對應的典型工作。為什麼說三者是「層層堆疊」而非彼此獨立？

提示與參考解答：低階：影像進影像出，如去雜訊、強化、還原、幾何轉換。中階：影像進屬性出，如分割、型態學、特徵擷取、追蹤。高階：屬性進理解出，如辨識、偵測、場景理解。層層堆疊是因為低階處理的品質決定中階分割的成敗(雜訊產生假邊緣、模糊使邊界偏移)，而中階萃取的特徵又決定高階辨識的上限——地基不穩，上層再先進也蓋不高。

4. 影像處理發展史的四個階段分別由什麼力量驅動？這對「如何找到好的研究或專題題目」有什麼啟示？

提示與參考解答：萌芽期(1920s)：新聞照片跨洋傳輸的需求。奠基期(1960-70s)：太空探索(看清月球)與醫學診斷(看見身體內部)。普及期(1980-2000s)：個人電腦、數位相機、工業自動化。智慧期(2012-)：大數據、GPU 算力與深度學習。啟示：每個階段都不是「為做技術而做技術」，而是有一個非解決不可的真實需求——好的題目應從真實問題出發(第18章專題選題的核心原則)。

5. 1964 年的月球照片校正為什麼被視為現代數位影像處理的起點？它與第 7 章的哈伯望遠鏡案例有什麼共通的精神？

提示與參考解答：因為JPL 首次以電腦系統性地校正探測器傳回影像的幾何失真與雜訊，是數位影像處理作為學科的開端。與哈伯案例的共通精神：當硬體有其物理極限(相機特性失真、主鏡研磨誤差)時，數學與運算可以把資訊救回來——這正是第7章影像還原的核心命題，提醒工程師在「感測器不夠好」時還有另一條路。

6. 為什麼一個領域需要「標準測試影像」？一張好的測試影像應具備哪些視覺特性？

提示與參考解答：因為若各研究者用自己的影像測試，結果無從比較(也許某人的影像本來就容易處理)；需要共同基準讓不同方法在相同立足點上競爭。這與第14章MNIST、ImageNet、COCO 的角色一致，只是規模不同。好的測試影像應具備：平滑區域(檢驗是否過度平滑)、銳利邊緣(是否模糊)、細緻紋理(細節保留)、豐富色彩與明暗層次(動態範圍)。

7. 承叫獸開講：萊娜圖為什麼會流行半世紀？IEEE 計算機協會又為什麼在 2024 年停用它？

提示與參考解答：流行原因：1973 年南加大Sawchuk 團隊掃描後，發現它兼具平滑皮膚、銳利羽毛紋理、豐富色彩與明暗對比，是絕佳測試素材；拷貝在社群中廣為流傳，甚至參與JPEG 標準的發展歷程。停用原因：自2010 年代中期起，愈來愈多人質疑以《花花公子》中頁作為領域標準素材對女性傳遞的訊息；2024 年3 月IEEE 計算機協會發函，自4 月1 日起不再接受含該影像的投稿，援引學會多元共融聲明與倫理守則，並尊重當事人Lena Forsén 的意願。

8. 請提出至少三個可取代萊娜圖的現代測試影像或資料集，並說明為什麼「技術上不遜色」很重要。

提示與參考解答：替代選項：Kodak 影像庫(二十四張自然影像)、DIV2K(高解析度)、BSD500(邊緣與分割基準)、Set5/Set14(超解析度基準)，以及OpenCV 與scikit-image 內建範例影像。「技術上不遜色」很重要，是因為它證明了叫獸開講思考題所問的「兩全方案」確實存在——不必在技術品質與文化包容之間二選一，這使得改變沒有正當的技術阻力。

9. 本書為什麼把深度學習安排在第 15 章而非開頭?請說明這個安排的理由與可能的代價。

提示與參考解答：理由：唯有先理解卷積、頻率、多解析度、特徵等基本觀念，才能真正看懂CNN 在做什麼(第15.1 節會印證卷積層即可學習的濾波核、首層自發學出邊緣偵測器);若一開始就學YOLO，雖能快速做出成果，卻只是呼叫他人函式，遇到問題無從診斷。可能的代價：前期較缺乏「炫目」的成果、學習動機需靠Lab 與叫獸開講維持;此為刻意的取舍，重視長期理解勝於短期成就感。

10. 請說出至少三個貫穿本書的核心思想，並預測它們可能會在哪些章節出現。

提示與參考解答：可任選三個：一、換一個域看問題(第6 章頻率域、第11 章小波)。二、多解析度由粗到細(第11 章小波金字塔、第13 章SIFT、第15 章CNN 層級特徵)。三、投票對抗雜訊(第9 章霍夫轉換、第13 章RANSAC)。四、正則化對抗病態(第7 章維納濾波、第14 章對抗過度擬合)。五、精確率與召回率取舍(第9、10、14、16 章)。六、適合勝於最強(第10、11、16、17 章)。七、技術背後的责任(第1、7、14、16 章)。

11. 本課程為什麼採用 ESP32-CAM 與 Jetson 雙平台?兩者各適合什麼樣的任務?

提示與參考解答：因為它們對應不同的AI 辨識架構與資源條件，培養「選對工具」的判斷力。ESP32-CAM：極低成本與功耗的視覺感測節點，適合簡單偵測、觸發、影像串流與大規模佈署，但無法執行深度模型。Jetson：內建GPU 的邊緣AI 電腦，能即時執行深度學習與ROS 2，適合機器人主控、即時偵測、SLAM。第17 章會完整比較，實務上兩者也常搭配(多個ESP32-CAM 當眼睛、Jetson 當大腦)。

12. (反思題)承叫獸開講與 1.4.3 節：一個專業社群的日常習慣如何傳遞「誰屬於這裡」的訊息?在人工智慧時代，這個問題為什麼變得更加重要?

提示與參考解答：本題無標準答案，重點在論證品質。可能論點：當一個領域的教材、投影片、示範程式中反覆出現某類影像或範例時，它就在無聲地定義這個領域的文化樣貌與預設的參與者形象，影響誰覺得自己受歡迎。AI 時代更重要的理由：現在我們不只選擇測試影像，而是選擇訓練資料，而模型會忠實學習資料中的一切偏誤並大規模複製(第14 章肺炎X 光的捷徑學習、第15 章人臉超解析度的族裔偏誤)。工程師的每一個資料選擇，都在形塑技術的樣貌與它服務的對象。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 1(緒論與發展史)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 1.
- IEEE Computer Society, 停止接受含 Lena 影像投稿之公告, 2024 年 3 月。(本章叫獸開講之史料來源)

- Losing Lena(紀錄短片),2019.(萊娜圖爭議之文化討論)
- USC-SIPI Image Database:<https://sipi.usc.edu/database/>(標準測試影像庫)
- Kodak Lossless True Color Image Suite、DIV2K、BSDS500 等公開測試資料集。
- Resnick, M., Lifelong Kindergarten, MIT Press, 2017.(4P 創意學習理念, 第 18 章詳述)
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 2 章 開發環境與程式基礎

本章學習目標

- 建立 Python 與 OpenCV 的開發環境，理解虛擬環境與套件管理的必要性。
- 掌握 NumPy 陣列運算，理解「影像就是陣列」這個貫穿全書的核心觀念。
- 熟練 OpenCV 的影像讀取、顯示、儲存與基本操作，並避開 BGR 等常見陷阱。
- 運用 ROI、切片與遮罩對影像的局部區域進行操作。
- 比較端上、近端邊緣、雲端三種 AI 辨識架構，並建立 ESP32-CAM 與 Jetson 雙平台環境。
- 養成除錯、查閱文件與版本管理的工程習慣。

叫獸開講|OpenCV 的誕生：一個「不要再重造輪子」的決定

1998 年，英特爾研究院的一位研究員 Gary Bradski 有了一個念頭。當時他觀察到一個奇怪的現象：電腦視覺這個領域，每一個實驗室、每一間公司，都在重複寫著同樣的東西——讀取影像、做高斯模糊、找邊緣、算直方圖。這些基礎功能人人都需要，卻沒有人共用；研究生入學的第一年，往往就耗在重寫前人早已寫過無數次的程式碼上。整個領域的能量，大量浪費在重造輪子。

Bradski 的構想是：打造一套共用的、經過最佳化的電腦視覺函式庫，讓研究者能站在同一個基礎上，把時間花在真正的創新上。他在英特爾內部組建了團隊，由 Vadim Pisarevsky 擔任技術主管，並得到英特爾俄羅斯團隊與效能函式庫團隊的大力支援。

但真正關鍵的轉折點在 1999 年。英特爾原本的計畫是做一套閉源的商業產品名字叫 CVL(Computer Vision Library)。是 Bradski 力主開源——他認為只有開放，才能真正達成「不要再重造輪子」的目標，也才能讓整個領域一起前進。對一家以販售技術為業的晶片巨頭來說，把辛苦開發的程式碼免費送出去，是個相當大膽的決定，而英特爾接受了。專案也因此改名為 OpenCV，靈感來自當時已相當流行的 OpenGL。

2000 年 6 月，OpenCV 的第一個公開版本在電腦視覺頂級會議 CVPR 上發表，獲得與會者高度讚譽。它的目標在早期文件中寫得很清楚：提供不只是開放、而且是經過最佳化的基礎視覺程式碼，讓大家不必再重造輪子；透過共用的基礎設施傳播視覺知識；並以不強制開源的授權條款，讓商業應用也能自由採用。這最後一點尤其重要——正是這個寬鬆的授權，讓 OpenCV 得以進入無數的商業產品(對照第 13 章 SIFT 專利的故事，你會更能體會授權條款的份量)。

接下來的故事你已經身在其中。2005 年，史丹佛大學的無人車 Stanley 使用 OpenCV，贏得了美國國防高等研究計畫署的無人車大挑戰，在莫哈維沙漠中自主行駛超過兩百公里；2006 年 1.0 正式版問世；此後歷經 Willow Garage、Itseez

的接手維護，加入了 C++ 與 Python 介面、GPU 加速、深度學習模組。今天，OpenCV 收錄超過兩千五百個演算法，累積數千萬次下載，是全世界電腦視覺工作者共同的工具箱——包括你即將在本章安裝的這一套。

叫獸想從這個故事裡點出兩件事。第一，你在這門課裡寫的每一行 OpenCV 程式，背後都站著二十多年來成千上萬人的貢獻。這正是第 18 章 4P 中「同儕」(Peers)在全球尺度上的展現——開源社群本身，就是人類最大規模的協作學習實驗。第二，Bradski 當年那個「不要再重造輪子」的判斷，今天依然適用於你：寫程式時，先問問「這個功能是不是已經有人寫得比我好了？」把時間留給真正需要你創造的那一部分。

思考題：英特爾把辛苦開發的程式碼免費開源，短期看是「損失」，長期卻讓公司在電腦視覺生態系中佔據了核心位置。你能想到其他「開放反而獲益」的例子嗎？這對你思考「知識該不該分享」有什麼啟發？

2.1 開發環境的建立

2.1.1 為什麼選 Python 與 OpenCV

本書選用 Python 搭配 OpenCV 作為主要工具，理由有四：其一，Python 語法簡潔，讓你能專注在影像處理的觀念而非語言細節；其二，OpenCV 是業界標準，學會它等於學會了一項可直接用於職場的技能；其三，Python 擁有最完整的科學運算與深度學習生態系(NumPy、Matplotlib、PyTorch、Ultralytics)，第 14 至 16 章會大量使用；其四，同樣的程式碼稍加調整就能在 Jetson 等邊緣裝置上執行(第 17 章)。

需要提醒的是：OpenCV 的核心是以 C++ 撰寫並高度最佳化，Python 只是它的介面。這代表你享受了 Python 的易用性，卻仍能獲得接近 C++ 的執行速度——前提是你使用 OpenCV 或 NumPy 的向量化運算，而非用 Python 的迴圈逐像素處理(2.3.3 節會用實測數據說明這個差異有多大)。

2.1.2 虛擬環境：為什麼不要直接安裝

初學者常直接用系統的 Python 安裝套件，這會在專案變多時造成災難：A 專案需要某套件的舊版本、B 專案需要新版本，兩者無法共存。虛擬環境(virtual environment)為每個專案建立獨立的套件空間，彼此互不干擾。這是專業開發的基本習慣，請從第一天就養成。

程式 2-1 建立虛擬環境與安裝套件

```
# --- 建立虛擬環境(以 Python 內建的 venv 為例) ---
python -m venv cv_env

# 啟動(Windows)
cv_env\Scripts\activate
# 啟動(macOS / Linux)
source cv_env/bin/activate

# --- 安裝本課程所需套件 ---
pip install opencv-python          # OpenCV 主套件
pip install opencv-contrib-python # 含額外模組(第 12 章追蹤器需要)
```



```

pip install numpy matplotlib      # 陣列運算與繪圖

# --- 記錄環境,方便他人重現(團隊協作與第 18 章專題必備) ---
pip freeze > requirements.txt
# 他人只需:pip install -r requirements.txt

# --- 離開虛擬環境 ---
deactivate

```

請注意 requirements.txt 這個檔案。它記錄了專案所需的所有套件與版本，讓別人(或三個月後的你)能完整重現你的環境。第 18 章期末專題的技術文件要求「另一個人照著做能重現你的成果」，這個檔案就是關鍵的一環。

2.1.3 驗證安裝與開發工具

程式 2-2 驗證環境安裝

```

import cv2
import numpy as np
import sys

print("Python 版本:", sys.version)
print("OpenCV 版本:", cv2.__version__)
print("NumPy 版本 :", np.__version__)

# 測試:建立一張純色影像並顯示
img = np.zeros((300, 400, 3), dtype=np.uint8)      # 全黑影像
img[:] = (0, 185, 118)                             # BGR:綠色
cv2.putText(img, "Hello OpenCV!", (60, 160),
             cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
cv2.imshow("test", img)
cv2.waitKey(0)                                       # 按任意鍵關閉
cv2.destroyAllWindows()

```

開發工具方面，建議三選一：Jupyter Notebook 適合探索與教學(能逐格執行、即時看結果，本課程的 Lab 很適合);VS Code 適合開發完整專案(有除錯器、Git 整合);Google Colab 則適合沒有本機環境或需要免費 GPU 的場合(第 15、16 章訓練模型時特別有用)。三者可以並用，依任務選擇。

2.2 影像就是陣列：NumPy 基礎

這一節是全書最重要的觀念之一。如果你只記得本章的一句話，請記得這一句：對電腦而言，一張影像就是一個數字陣列。

2.2.1 影像的資料結構

OpenCV 讀進來的影像，型別是 NumPy 的 ndarray。灰階影像是二維陣列，形狀為(高度，寬度);彩色影像是三維陣列，形狀為(高度，寬度，通道數)。每個元素的資料型別通常是 uint8(無號八位元整數)，數值範圍 0 到 255——這正是第 3 章要深入探討的量化與位元深度。

請特別留意兩個容易搞混的順序。第一，陣列的索引順序是「先列後行」，也就是 `img[y, x]`，與數學上習慣的 (x, y) 相反；第二，OpenCV 的彩色通道順序是 BGR 而非 RGB(2.3.2 節會說明這個歷史包袱)。這兩點是初學者最常見的錯誤來源。

程式 2-3 影像的陣列結構

```
import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
color = cv2.imread("photo.jpg")           # 預設讀成彩色(BGR)

print("灰階形狀:", gray.shape)           # (高, 寬)
print("彩色形狀:", color.shape)          # (高, 寬, 3)
print("資料型別:", color.dtype)          # uint8
print("總像素數:", color.size)           # 高 × 寬 × 3

# 存取單一像素(注意:先 y 後 x!)
y, x = 100, 250
print("該點灰階值:", gray[y, x])
b, g, r = color[y, x]                    # BGR 順序,不是 RGB
print(f"該點色彩:B={b}, G={g}, R={r}")

# 修改像素
color[y, x] = (0, 0, 255)                # 設為紅色(BGR)
```

2.2.2 建立與初始化影像

程式 2-4 以 NumPy 建立測試影像

```
import numpy as np

black = np.zeros((300, 400, 3), dtype=np.uint8)      # 全黑
white = np.ones((300, 400, 3), dtype=np.uint8) * 255 # 全白
gray = np.full((300, 400), 128, dtype=np.uint8)      # 中灰(單通道)
noise = np.random.randint(0, 256, (300, 400),
                           dtype=np.uint8)            # 隨機雜訊

# 漸層(用於測試第 5 章的灰階轉換)
ramp = np.tile(np.arange(256, dtype=np.uint8), (100, 1))

# 棋盤格(用於測試第 4 章相機校正與第 6 章頻譜)
board = np.indices((320, 320)).sum(axis=0) // 40 % 2
board = (board * 255).astype(np.uint8)
```

自己合成測試影像是一個非常實用的技巧：當演算法出錯時，用一張你完全知道內容的簡單影像（純色塊、漸層、棋盤格）去測試，遠比用複雜的自然照片容易找出問題。第 6 章 Lab 的「頻譜偵探」就大量使用這個技巧。

2.2.3 向量化運算：為什麼不要用迴圈

NumPy 的威力在於向量化——對整個陣列一次運算，而非逐元素迴圈。這不只是寫法簡潔的問題，而是效能的天壤之別：NumPy 的運算底層以 C 實作並最佳化，而 Python 的迴圈每次疊代都有直譯器的額外開銷。

程式 2-5 向量化 vs. 迴圈的效能比較

```

import cv2, numpy as np, time

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
h, w = img.shape

# --- 壞方法:Python 迴圈逐像素(慢) ---
t0 = time.time()
out1 = img.copy()
for y in range(h):
    for x in range(w):
        out1[y, x] = 255 - img[y, x]      # 負片
print(f"迴圈:{time.time()-t0:.3f} 秒")

# --- 好方法:向量化(快數百倍) ---
t0 = time.time()
out2 = 255 - img                          # 一行搞定
print(f"向量化:{time.time()-t0:.5f} 秒")

print("結果相同:", np.array_equal(out1, out2))

# 常見向量化操作
brighter = np.clip(img.astype(np.int16) + 50, 0, 255).astype(np.uint8)
mask = img > 128                          # 布林陣列(第 9 章門檻分割的基礎)
print("亮部像素比例:", mask.mean())

```

請務必實際跑一次程式 2-5，親眼看看那個倍數差距。這個習慣在第 17 章邊緣部署時格外重要——在算力有限的 Jetson 或樹莓派上，一個寫成迴圈的前處理步驟，就足以讓整個即時系統掉影格。

另一個常見陷阱是溢位：uint8 的範圍是 0 到 255，若直接對它做加法，超過 255 會「繞回」變成小數字(例如 250 + 50 得到 44)，而非停在 255。正確做法是先轉成較大的型別、運算後再用 np.clip 限制範圍——程式 2-5 的 brighter 就是示範。第 5 章的亮度調整會反覆用到這個技巧。

2.3 OpenCV 基本操作

2.3.1 讀取、顯示與儲存

程式 2-6 影像的讀取、顯示與儲存

```

import cv2

# --- 讀取 ---
img = cv2.imread("photo.jpg")          # 彩色(BGR)
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
raw = cv2.imread("photo.png", cv2.IMREAD_UNCHANGED) # 含 alpha 通道

# 務必檢查是否讀取成功(路徑錯誤時 imread 回傳 None 而不報錯!)
if img is None:
    raise FileNotFoundError("找不到影像,請檢查路徑")

```

```
# --- 顯示 ---
cv2.imshow("window title", img)
key = cv2.waitKey(0)          # 0 = 無限等待; 數字 = 等待毫秒數
cv2.destroyAllWindows()

# --- 儲存(副檔名決定格式與壓縮方式, 第 11 章詳述) ---
cv2.imwrite("output.png", img)          # 無損
cv2.imwrite("output.jpg", img,
            [cv2.IMWRITE_JPEG_QUALITY, 95])  # 有損, 可設品質
```

三個實務提醒：第一，`imread` 找不到檔案時不會拋出例外，而是安靜地回傳 `None`，若不檢查會在後續操作時出現難以理解的錯誤訊息——請養成檢查的習慣。第二，中文路徑在某些系統上會導致讀取失敗，建議檔名與資料夾都用英文。第三，`waitKey` 的回傳值是按鍵碼，可用於互動控制(第 8、12 章的 Lab 會用到)。

2.3.2 BGR 之謎：一個歷史包袱

OpenCV 使用 BGR 而非 RGB 的通道順序，是初學者最常踩的坑——用 Matplotlib 顯示 OpenCV 讀進來的影像時，紅色和藍色會對調。這個設計來自 OpenCV 早期(2000 年代初)的歷史背景：當時許多相機硬體與 Windows 的點陣圖格式採用 BGR 排列，為了效能而直接沿用。等到 RGB 成為主流，OpenCV 已有大量既有程式碼依賴 BGR，改動的代價太高，只好一路保留至今。

這個小小的歷史包袱其實是一堂很好的工程課：早期的設計決定會長期影響一個系統，即使當初的理由早已消失。你在第 18 章的專題中做架構決定時，不妨想想這件事。

程式 2-7 BGR 與 RGB 的轉換

```
import cv2, matplotlib.pyplot as plt

img = cv2.imread("photo.jpg")          # BGR

# 用 Matplotlib 顯示時務必轉換, 否則紅藍顛倒
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off"); plt.show()

# 通道的分離與合併
b, g, r = cv2.split(img)
merged = cv2.merge([b, g, r])

# 用 NumPy 切片更快(向量化思維)
blue_channel = img[:, :, 0]
rgb = img[:, :, ::-1]                  # 反轉通道順序即 BGR→RGB
```

2.3.3 ROI、切片與遮罩

ROI(Region of Interest, 感興趣區域)是只對影像的某個區域進行操作的技巧，在 NumPy 中用切片即可完成。它在整本書中反覆出現：第 9 章車道線偵測只處理路面梯形區、第 10 章人臉偵測後只在臉部區域找眼睛、第 13 章裁切車牌區域做字元分割。

程式 2-8 ROI、切片與遮罩

```
import cv2, numpy as np

img = cv2.imread("photo.jpg")

# --- ROI:用切片取出區域(注意:先 y 後 x) ---
roi = img[100:300, 200:500]          # y 從 100 到 300,x 從 200 到 500
cv2.imwrite("roi.jpg", roi)

# 對 ROI 做處理後貼回(注意:切片是「檢視」不是複製!)
img[100:300, 200:500] = cv2.GaussianBlur(roi, (21, 21), 0)

# 若不想影響原圖,務必用 copy()
roi_safe = img[100:300, 200:500].copy()

# --- 遮罩:對不規則區域操作 ---
mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.circle(mask, (320, 240), 120, 255, -1)      # 白色實心圓
masked = cv2.bitwise_and(img, img, mask=mask)    # 只保留圓內

# 遮罩也可用布林索引(NumPy 風格)
img[mask == 255] = (0, 255, 0)                  # 圓內塗綠
```

程式中「切片是檢視不是複製」這一點請特別留意：NumPy 的切片回傳的是原陣列的一個視圖，修改它會直接改到原影像。這既是效能優勢(不必複製資料)，也是常見的 bug 來源。需要獨立副本時，務必呼叫 `copy()`。

2.4 三種 AI 辨識架構與雙平台

本節建立本書實作的硬體框架。這三種架構會在第 10、16、17 章反覆出現，是理解「演算法該跑在哪裡」的關鍵。

2.4.1 三種架構

1. 端上運算(On-device)：在感測裝置本身完成運算，例如 ESP32-CAM 直接執行輕量的人臉偵測。優點是延遲最低、不需網路、資料不出裝置(隱私最佳)、功耗低;限制是算力極為有限。
2. 近端邊緣運算(Edge)：在鄰近的較強裝置上運算，例如產線旁的 Jetson 或工業電腦處理多台相機的影像。它在算力與即時性之間取得平衡，資料不出廠區，是機器人與工業視覺的主力架構。
3. 雲端運算(Cloud)：把資料上傳到遠端資料中心處理。算力最強、適合訓練大模型與複雜分析，但延遲高、需要網路、且涉及資料外流的隱私考量。

關鍵觀念是三者可以分層協作。第 10 章的門禁案例就是典型：ESP32-CAM 在端上判斷「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把影像送到雲端做「是誰」的辨識(低頻、高算力)。第 17 章會用完整的比較表深入這個主題。

2.4.2 雙平台介紹

本書所有實作圍繞兩個平台展開，對應上述架構：

比較面向	ESP32-CAM(入門)	NVIDIA Jetson(進階)
定位	會看的感測節點	機器人的視覺大腦
對應架構	端上運算	近端邊緣運算
開發方式	Arduino IDE 或 ESP-IDF(C/C++)	Linux + Python + CUDA(同桌機)
可跑的任務	移動偵測、顏色追蹤、Haar 人臉偵測、影像串流	YOLO 即時偵測、分割、姿態、SLAM、ROS 2
本書出現章節	第 8、10、12、17 章 Lab	第 15-17 章與期末專題

表 2-1 雙平台比較

學習策略上，建議的流程是：先在個人電腦上用 Python 與 OpenCV 把演算法邏輯驗證清楚，再移植到目標平台。這個「先驗證邏輯、再移植硬體」的工作方式，是嵌入式與機器人開發的標準做法——因為在電腦上除錯遠比在裝置上容易。

2.4.3 ESP32-CAM 快速上手

ESP32-CAM 最常見的用法是作為網路攝影機：燒錄官方的 CameraWebServer 範例後，它會提供一個串流網址，而 OpenCV 可以直接讀取這個網址，把它當成一台普通的相機。這讓你能在電腦上用 Python 開發，卻使用 ESP32-CAM 作為影像來源——本書多章的 Lab 都採用這個模式。

程式 2-9 以 OpenCV 讀取 ESP32-CAM 串流

```
import cv2

# ESP32-CAM 燒錄 CameraWebServer 範例後取得的串流網址
STREAM_URL = "http://192.168.1.50:81/stream"

cap = cv2.VideoCapture(STREAM_URL)      # 與讀取本機相機的用法完全相同
if not cap.isOpened():
    raise ConnectionError("無法連線,請確認 ESP32-CAM 與電腦在同一網路")

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # === 此處可套用本書任何一章的處理 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow("ESP32-CAM", frame)
    if cv2.waitKey(1) == 27:            # ESC 離開
        break

cap.release()
cv2.destroyAllWindows()
```

2.5 工程習慣的養成

這一節談的不是影像處理，而是讓你在往後十六章(以及日後職涯)少走彎路的習慣。它們看似瑣碎，卻是區分「會寫程式」與「會做專案」的關鍵。

2.5.1 除錯的基本功

影像處理的除錯有其特殊性：錯誤往往不會拋出例外，而是「結果看起來怪怪的」。三個實用技巧：其一，隨時印出陣列的 shape 與 dtype——大多數錯誤都源於形狀不符或型別溢位；其二，把中間結果存檔或顯示出來，不要一路算到底才看結果(所謂「視覺化除錯」)；其三，用自己合成的簡單影像(2.2.2 節)測試，你知道正確答案應該長什麼樣。

程式 2-10 除錯輔助函式

```
import cv2, numpy as np

def debug_info(name, arr):
    """印出陣列的關鍵資訊,除錯必備"""
    print(f"{name:12s} shape={arr.shape} dtype={arr.dtype} "
          f"min={arr.min()} max={arr.max()} mean={arr.mean():.1f}")

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
debug_info("原圖", img)

blur = cv2.GaussianBlur(img, (5, 5), 0)
debug_info("模糊後", blur)

edges = cv2.Canny(blur, 80, 160)
debug_info("邊緣", edges)

# 視覺化除錯:把各階段並排顯示
stacked = np.hstack([img, blur, edges])
cv2.imwrite("debug_stages.jpg", stacked)
```

2.5.2 查閱文件與求助

沒有人記得住 OpenCV 兩千多個函式的所有參數。專業工程師的能力不在於「記得多少」，而在於「多快能找到並理解」。建議的順序：先看官方文件(docs.opencv.org，有完整的參數說明與範例)、再看官方教學(tutorial 章節)、最後才是搜尋引擎與問答網站。使用 AI 助手時也請記得驗證——它可能給你舊版本的 API 或看似合理但錯誤的參數。

一個實用技巧：在 Python 中直接用 help(cv2.GaussianBlur) 或在 Jupyter 中用問號查詢函式簽名，比切換到瀏覽器更快。

2.5.3 版本管理與程式碼組織

從第一個 Lab 就開始用 Git。它的價值不只是備份，更在於「讓你敢於嘗試」——知道隨時能回復到能用的版本，你才會放手實驗(這正是第 18 章 Play 精神的技術基礎)。程式碼組織上，建議把常用的處理函式抽成獨立模組(例如把第 5 章的前處理

管線寫成 preprocess.py)，往後各章直接引用——這既是良好習慣，也呼應叫獸開講「不要重造輪子」的精神。

2.6 應用案例

2.6.1 案例一：批次影像處理工具

情境：你有一資料夾的照片需要統一處理——調整尺寸、轉灰階、加浮水印、重新命名。這是影像處理最基本也最常見的實務需求。做法：用 Python 的 glob 或 pathlib 列出所有檔案，逐一讀取、處理、儲存到輸出資料夾。

這個看似簡單的案例，實際上是所有後續工作的基礎：第 14 至 16 章訓練模型前你都需要先把原始影像批次處理成統一的尺寸與格式。及早寫好一個可重複使用的批次處理腳本，會替你省下大量時間。

程式 2-11 批次影像處理

```
import cv2, glob, os

IN_DIR, OUT_DIR = "raw/", "processed/"
os.makedirs(OUT_DIR, exist_ok=True)

for i, path in enumerate(sorted(glob.glob(IN_DIR + "*.jpg"))):
    img = cv2.imread(path)
    if img is None:
        print(f"跳過無法讀取的檔案:{path}")
        continue

    img = cv2.resize(img, (640, 480))
    cv2.putText(img, "YunTech", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 185, 118), 2)

    out_path = os.path.join(OUT_DIR, f"img_{i:04d}.jpg")
    cv2.imwrite(out_path, img)

print("批次處理完成")
```

2.6.2 案例二：即時攝影機處理管線

情境：打開筆電相機，即時顯示原始畫面與處理後的結果並排對照。這個「讀取—影格、處理、顯示」的迴圈，是第 12 章視訊處理、第 16 章 YOLO 即時偵測、第 17 章邊緣部署的共同骨架——本書後半的所有即時應用，結構都與它相同。

建議在這個案例中加入按鍵切換功能(例如按 1 顯示原圖、按 2 顯示灰階、按 3 顯示邊緣)，它會成為你整學期最常用的實驗工具：每學一個新演算法，就把它加進這個管線，即時看效果。

2.6.3 案例三：ESP32-CAM 遠端監看

情境：把 ESP32-CAM 放在實驗室或家中，從電腦即時監看，並在偵測到變化時自動存檔。這個案例整合了本章的 2.4.3 節(串流讀取)與後續第 12 章的移動偵測。它

的價值在於讓你及早體驗「感測器在遠端、運算在本機」的分散式架構——這正是 2.4.1 節三種架構中「端上取像、近端運算」的實例，也是第 17 章分散式監測網案例的雛形。

2.6.4 案例四：建立你的測試影像庫

情境：承第 1 章 Lab 1-3 的討論，建立一組屬於你自己的標準測試影像，供整學期使用。做法：挑選或拍攝六到十張涵蓋不同特性的影像——平滑漸層、銳利邊緣、細緻紋理、高對比、低光雜訊、含文字、含人臉(需取得同意)、規律圖案。統一尺寸與命名，建立資料夾與說明文件。

為什麼這值得花時間？因為當你在第 5 章調對比、第 6 章看頻譜、第 7 章去雜訊、第 9 章找邊緣時，都使用同一組影像，你就能跨章比較不同演算法在同一素材上的表現，建立起連貫的直覺。這也呼應第 1 章叫獸開講的反思——選擇測試素材本身，就是一個值得思考的決定。

2.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
影像即陣列	全書所有章節	所有演算法都是對陣列的運算
uint8 與溢位	第 3、5 章	位元深度、量化;亮度調整需 clip
向量化運算	第 5、6、17 章	濾波效能;邊緣裝置的即時性
BGR 通道順序	第 8 章	色彩空間轉換與 HSV 範圍陷阱
ROI 與遮罩	第 9、10、13 章	車道 ROI、人臉內找眼睛、車牌裁切
合成測試影像	第 4、6、7 章	棋盤格校正、頻譜驗證、雜訊模擬
VideoCapture 迴圈	第 12、16、17 章	視訊處理、即時偵測、邊緣部署的骨架
三種辨識架構	第 10、16、17 章	端上預篩加雲端精算的混合設計
雙平台	第 8、10、12、15-17 章	ESP32-CAM 感測、Jetson 推論與 ROS 2
工程習慣	第 18 章	版本管理、工程筆記、可重現的文件

表 2-2 第 2 章與全書其他章節的觀念連結

程式實作 Lab 2

Lab 2-1 環境建立與驗證

目標：建立可用一整個學期的開發環境。(a)依程式 2-1 建立虛擬環境並安裝套件;(b)執行程式 2-2 驗證安裝，記下三個套件的版本號;(c)產出 requirements.txt 並確認內容;(d)三種開發工具(Jupyter、VS Code、Colab)各試用一次，寫下你覺得各自

適合什麼場合;(e)建立本課程的專案資料夾結構(建議: images/ 放素材、labs/ 放各章程式、outputs/ 放結果、utils/ 放共用函式), 並用 git init 開始版本管理。

Lab 2-2 陣列即影像

目標: 徹底內化「影像就是陣列」。(a)讀取一張照片, 印出 shape、dtype、size, 並解釋每個數字的意義;(b)不使用任何影像檔, 純以 NumPy 合成: 純色塊、水平漸層、垂直漸層、棋盤格、隨機雜訊、以及一個中心亮四周暗的圓形漸層;(c)只用陣列運算(不用 OpenCV 函式)實作: 負片、亮度加減、左右翻轉、上下翻轉、裁切中央區域;(d)執行程式 2-5, 實測迴圈與向量化的效能差距並記錄倍數;(e)刻意製造一次 uint8 溢位(如對亮部加 100 而不 clip), 觀察結果並解釋為什麼會出現詭異的暗塊。

Lab 2-3 ROI 與遮罩實作

目標: 掌握局部操作。(a)用切片取出照片中的一塊區域並存檔;(b)對該區域做模糊後貼回原圖(製造「馬賽克隱私保護」的效果);(c)驗證「切片是檢視不是複製」: 先不加 copy() 修改 ROI, 觀察原圖是否被改變, 再加上 copy() 重試;(d)建立圓形與多邊形遮罩, 只保留遮罩內的影像;(e)進階: 用滑鼠事件(cv2.setMouseCallback)讓使用者框選 ROI——這個工具在第 4 章文件掃描與第 12 章物件追蹤都會用到。

Lab 2-4 即時處理管線與 ESP32-CAM

目標: 建立整學期最常用的實驗工具。(a)寫一支程式, 讀取筆電相機並即時顯示;(b)加入按鍵切換模式(1 原圖、2 灰階、3 邊緣、4 模糊), 並在畫面上顯示目前模式與 FPS;(c)加入按鍵存檔功能(按 s 儲存目前影格);(d)若有 ESP32-CAM, 燒錄 CameraWebServer 範例, 依程式 2-9 改用串流網址作為輸入, 比較兩者的畫質與延遲;(e)把這支程式存成可重複使用的工具(如 live_demo.py), 往後每學一個新演算法就把它加進去——這將是你這門課最有價值的個人資產。

本章重點整理

- OpenCV 源於 Gary Bradski 1998 年「不要再重造輪子」的構想, 1999 年英特爾決定開源(原計畫為閉源的 CVL), 2000 年 6 月於 CVPR 首次公開發表; 其寬鬆授權讓商業應用得以自由採用, 是它普及的關鍵。
- 虛擬環境為每個專案建立獨立套件空間, requirements.txt 讓環境可被他人重現; 開發工具 Jupyter(探索)、VS Code(專案)、Colab(免費 GPU)可依任務並用。
- 影像就是 NumPy 陣列: 灰階為二維、彩色為三維(高, 寬, 通道); 索引順序為 img[y, x](先列後行), 資料型別多為 uint8(0 至 255), 超出範圍會溢位需以 clip 處理。
- 向量化運算比 Python 迴圈快數百倍, 因 NumPy 與 OpenCV 底層以最佳化的 C/C++ 實作; 這個習慣在第 17 章邊緣部署的即時系統中尤其關鍵。
- OpenCV 使用 BGR 而非 RGB(早期硬體與 Windows 點陣圖格式的歷史包袱), 與 Matplotlib 互通時須轉換; imread 失敗時安靜回傳 None, 務必檢查。

- ROI 以切片實作，但切片是「檢視」非複製，修改會影響原圖，需獨立副本時應用 `copy()`；不規則區域則以遮罩配合 `bitwise` 運算或布林索引處理。
- 三種 AI 辨識架構：端上(ESP32-CAM，低延遲、保護隱私、算力有限)、近端邊緣(Jetson，算力與即時性平衡)、雲端(算力最強但延遲高)；三者可分層協作。學習策略為先在電腦驗證邏輯、再移植目標平台。

習題

1. 承叫獸開講：Gary Bradski 為什麼要創立 OpenCV？英特爾在 1999 年做了什麼關鍵決定，這個決定為何重要？

提示與參考解答：因為他觀察到電腦視覺領域每個實驗室都在重複實作同樣的基礎功能(讀影像、模糊、找邊緣)，整個領域的能量浪費在「重造輪子」上。1999 年的關鍵決定是：原本計畫做閉源商業產品 CVL，經 Bradski 力主改為開源，並更名 OpenCV(靈感來自 OpenGL)。重要性在於只有開放才能真正達成共用基礎、讓領域一起前進；而不強制開源的寬鬆授權，更讓它得以進入無數商業產品。

2. 為什麼要使用虛擬環境？requirements.txt 的用途是什麼？它與第 18 章的哪個要求相關？

提示與參考解答：虛擬環境為每個專案建立獨立的套件空間，避免不同專案需要同一套件的不同版本而衝突。requirements.txt 記錄專案所需的所有套件與版本，讓他人(或日後的自己)能以一行指令完整重現環境。它與第 18 章期末專題技術文件的要求「另一個人照著做能重現你的成果」直接相關。

3. 請說明灰階影像與彩色影像的陣列形狀差異。存取像素時為什麼是 `img[y, x]` 而非 `img[x, y]`？

提示與參考解答：灰階為二維陣列(高度，寬度)；彩色為三維陣列(高度，寬度，通道數)。索引為 `img[y, x]` 是因為 NumPy 陣列的索引順序是「先列(row)後行(column)」，而列對應影像的垂直方向(y)、行對應水平方向(x)，與數學習慣的(x, y)相反——這是初學者最常見的錯誤來源之一。

4. 為什麼要避免用 Python 迴圈逐像素處理影像？請估算一張 1920×1080 彩色影像用迴圈與向量化處理的差距量級，並說明原因。

提示與參考解答：因為向量化比迴圈快數百倍。一張 $1920 \times 1080 \times 3$ 約六百二十萬個元素，Python 迴圈每次疊代都有直譯器的額外開銷；而 NumPy 與 OpenCV 的運算底層以最佳化的 C/C++ 實作、且能利用 SIMD 指令與快取。實測差距常達數百倍。這個習慣在第 17 章邊緣裝置的即時系統上格外關鍵——一個寫成迴圈的前處理就足以讓系統掉影格。

5. 什麼是 uint8 溢位？對一個值為 250 的像素加上 50 會得到什麼？正確做法是什麼？

提示與參考解答：uint8 範圍是 0 至 255，超出會「繞回」(模 256)。250 + 50 = 300，溢位後得到 44——一個原本很亮的像素反而變暗，產生詭異的暗塊。正確做法：先轉成較大的型別(如 int16 或 float)運算，再用 `np.clip(結果, 0, 255)` 限制範圍，最後轉回 uint8。第 5 章的亮度調整會反覆用到。

6. OpenCV 為什麼使用 BGR 而非 RGB？這個歷史包袱會在什麼情況下造成問題？這給了我們什麼工程啟示？

提示與參考解答：因為 OpenCV 早期(2000 年代初)許多相機硬體與 Windows 點陣圖格式採用 BGR 排列，為效能而沿用；待 RGB 成為主流時，已有大量既有程式碼依賴 BGR，改動代價太高。問題常出現在與 Matplotlib、PIL 等使用 RGB 的函式庫互通時，紅藍會顛倒。工程啟示：早期的設計決定會長期影響一個系統，即使當初的理由早已消失——做架構決定時需考慮長遠影響。

7. imread 讀取失敗時會發生什麼？為什麼這個行為特別容易造成困擾？應如何防範？

提示與參考解答：它不會拋出例外，而是安靜地回傳 None。困擾在於程式不會在出錯的地方停下，而是在後續操作(如

8. 什麼是 ROI？為什麼說「切片是檢視不是複製」？這個特性何時是優點、何時是陷阱？

提示與參考解答：ROI 是感興趣區域，以 NumPy 切片取出影像的局部(如

9. 請說明端上、近端邊緣、雲端三種 AI 辨識架構的差異，並舉一個「分層協作」的實例。

提示與參考解答：端上：在感測裝置本身運算(如 ESP32-CAM)，延遲最低、不需網路、隱私最佳，但算力極有限。近端邊緣：在鄰近的較強裝置運算(如 Jetson)，算力與即時性平衡、資料不出廠區。雲端：遠端資料中心，算力最強但延遲高、需網路、有隱私考量。分層協作實例(第 10 章門禁)：ESP32-CAM 端上判斷「有沒有人臉」(高頻低算力)，偵測到才上傳雲端做「是誰」的辨識(低頻高算力)。

10. 本課程為什麼建議「先在電腦上驗證邏輯、再移植到目標平台」？

提示與參考解答：因為在個人電腦上開發與除錯遠比在嵌入式裝置上容易：有完整的除錯器、可即時顯示中間結果、修改後立即執行不需重新燒錄、且不受裝置算力與記憶體限制。先確認演算法邏輯正確，移植時就只需處理硬體相關的問題(效能、記憶體、介面)，能大幅縮短開發時間。這是嵌入式與機器人開發的標準工作方式。

11. 影像處理的除錯有什麼特殊性？請說明三個實用的除錯技巧。

提示與參考解答：特殊性：錯誤往往不拋出例外，而是「結果看起來怪怪的」，不易定位。三個技巧：一、隨時印出陣列的 shape 與 dtype(大多數錯誤源於形狀不符或型別溢位)，可寫成 debug_info 輔助函式；二、視覺化除錯——把中間結果存檔或並排顯示，不要一路算到底才看；三、用自己合成的簡單影像(純色塊、漸層、棋盤格)測試，因為你知道正確答案該長什麼樣。

12. (綜合題)承叫獸開講「不要再重造輪子」：請說明這個原則在你的學習與專題中該如何實踐，以及它有沒有需要注意的界線。

提示與參考解答：實踐：寫程式前先查是否已有成熟的函式庫或實作(OpenCV 兩千多個演算法、PyPI 生態系)，把時間留給真正需要創造的部分；把自己常用的處理抽成共用模組供各章重複使用；善用開源社群的成果並回饋。界線：一、學習階段有時值得「重造一次輪子」以理解原理(如本書多章要求手動實作演算法再與函式庫對照)；

二、需確認授權條款是否允許你的用途(對照第13章SIFT專利的故事);三、直接使用而不理解,遇到問題時無從診斷——理解與效率需要平衡。

參考資料與延伸閱讀

- OpenCV 官方文件與教學: <https://docs.opencv.org>(函式參考與 tutorial)
- OpenCV Anniversary — 專案發展歷程:
<https://opencv.org/anniversary/>(本章叫獸開講之史料來源)
- Bradski, G., & Kaehler, A., Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly, 2008.(創始者親撰的經典入門書)
- NumPy 官方文件: <https://numpy.org/doc/>(陣列運算與廣播機制)
- Espressif ESP32-CAM 官方範例與文件: <https://www.espressif.com>
- NVIDIA Jetson 開發者資源: <https://developer.nvidia.com/embedded-computing>
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 3 章 數位影像基礎

本章學習目標

- 說明影像數位化的兩個步驟——取樣與量化，並理解像素與解析度的意義。
- 陳述 Nyquist 取樣定理，解釋混疊現象的成因與防範方式。
- 說明位元深度與動態範圍的關係，理解量化不足造成的假輪廓現象。
- 區分空間解析度、灰階解析度與時間解析度，並判斷各自的取捨。
- 描述影像感測器的成像原理與色彩濾鏡陣列，理解 RAW 與 JPEG 的差異。
- 運用 MSE、PSNR、SSIM 客觀評估影像品質，並理解其侷限。
- 說明像素鄰域、連通性與距離度量，為後續分割與型態學打底。

叫獸開講|第一張數位影像：一位父親、一台巨型電腦與 176 個像素

1957 年初的某一天，一位二十七歲的年輕科學家把一張兒子的照片帶到辦公室。這件事聽起來平凡無奇——多少新手爸媽都做過同樣的事。但他的辦公室不太一樣：那是美國國家標準局(NBS，今日的 NIST)，而他是少數獲准操作 SEAC 的人之一。SEAC 是美國第一台可程式化的儲存程式電腦，大到能塞滿一整個房間，平時的工作是社會安全帳務、空軍後勤運算，以及氫彈相關的計算。

這位科學家名叫 Russell Kirsch。他心裡有一個當時幾乎沒人問過的問題：「如果電腦能夠看見圖片，會發生什麼事？」為了回答它，他和同事打造了一台滾筒式掃描器——照片貼在旋轉的滾筒上，一端是馬達、另一端是閃頻圓盤，搭配一支能偵測極微弱光線的光電倍增管，隨著滾筒緩緩轉動，透過箱壁上一個方形的觀察孔逐點掃描影像的明暗變化，再轉換成電腦看得懂的二進位數字。

他選來測試的，是三個月大的兒子 Walden 的頭肩照。原本那是一張他與兒子的合影，但因為當時的設備最多只能處理 176×176 (共 30,976 個) 像素，他把照片裁成了 5 公分見方的小方塊——順帶讓兒子的臉，比他自己更有名。掃出來的影像模糊、顆粒粗大、灰階層次極少，像個幽靈，卻是人類史上第一張數位影像。Kirsch 後來自嘲說，他「坦承曾竊取原本該用於更有用產品(比如熱核武器計算)的機器時間」。

這 30,976 個像素，大約只有 2010 年代數位相機一張照片的千分之一資訊量，卻成了往後一切的源頭：衛星影像、電腦斷層掃描、商品條碼、桌上排版、數位攝影，全都從這口井裡湧出。2003 年，《生活》雜誌的編輯把它列入「改變世界的一百張照片」。

故事還有一個耐人尋味的尾聲。Kirsch 晚年回顧時，對自己當年的一個選擇感到不滿意：他把像素做成了方形。他說那是為了實作方便而做的權宜，卻讓全世界沿用了半世紀——方形的網格在表現曲線與斜線時天生笨拙(這也是你在放大影像時看到鋸齒的根本原因，第 4 章的內插正是為了緩解它)。晚年他甚至還在研

究以三角形等其他形狀來組成影像。這是一位工程師難得的誠實：承認自己的一個「暫時的決定」變成了整個領域的預設，而它未必是最好的答案。

叫獸把這個故事放在第 3 章開頭，是因為本章要談的正是那個「把連續的世界切成一格一格數字」的過程——取樣與量化。Kirsch 當年受限於設備只能取 176×176 、只能表現極少的灰階；今天你的手機能拍下數千萬像素、每個通道 12 位元以上。但本質完全相同：我們永遠是在用有限的格子，去逼近一個無限細緻的真實世界。理解這個「逼近」會遺失什麼、又該怎麼把遺失降到最低，就是本章的全部功課。

思考題：Kirsch 為了實作方便選了方形像素，結果影響了半世紀。回想第 2 章 OpenCV 的 BGR 順序，也是類似的故事。你認為工程師在做「暫時的權宜決定」時，該如何判斷它會不會變成難以更改的長期標準？

3.1 從連續到離散：取樣與量化

3.1.1 影像數位化的兩個步驟

真實世界的影像是連續的：在空間上，兩個點之間永遠可以再找到一個點；在亮度上，兩個灰階之間永遠可以再分出更細的差異。但電腦只能處理有限的數字，因此必須把連續的影像轉換成離散的數值陣列。這個轉換分成兩個獨立的步驟：

1. 取樣(Sampling)：在空間上離散化。把影像切成一格一格的小方塊，每一格取一個代表值——這就是像素(pixel, picture element 的縮寫)。取樣的密度決定了空間解析度，也就是我們常說的「幾百萬畫素」。
2. 量化(Quantization)：在亮度上離散化。把每個像素的連續亮度值，對應到有限個階層中的一個。階層數由位元深度決定：8 位元有 256 階(0 至 255)，這正是第 2 章 uint8 的由來。

用一個比喻幫助理解：取樣像是決定「用多細的網格去描摹一幅畫」，量化則是決定「調色盤上有幾種顏色可用」。兩者都會造成資訊損失，而且是不可逆的——一旦取樣或量化完成，失去的細節就永遠回不來了(這也預告了第 7 章影像還原的物理上限)。承叫獸開講，Kirsch 當年在兩個維度上都受到極大限制： 176×176 的取樣、極少的量化階層。

3.1.2 像素、解析度與尺寸的混淆

這裡有一組經常被混用、實則不同的概念，值得釐清：

- 影像尺寸(Image Size)：影像的像素數，例如 1920×1080 。這是陣列的形狀(第 2 章的 shape)。
- 空間解析度(Spatial Resolution)：單位長度內的像素數，例如掃描器的 300 dpi(每英寸點數)、感測器的每毫米像素數。它才真正描述「細節的精細程度」。
- 顯示尺寸(Display Size)：影像在螢幕或紙上實際呈現的大小，取決於顯示裝置的密度。

為什麼要區分？因為「像素多」不等於「細節多」。把一張模糊的照片放大四倍存檔，像素數變成十六倍，細節卻一點也沒增加——第 4 章的內插只是在既有資訊之間

「猜」出新像素。真正決定細節的，是拍攝當下的光學品質與取樣密度。這也是為什麼手機廠商的「億級畫素」宣傳需要打個問號：感測器再多像素，若鏡頭的解析力跟不上，多出來的像素只是在記錄模糊。

3.1.3 三種解析度

完整地說，數位影像其實有三個維度的解析度，各有取捨：

解析度類型	離散化的維度	不足時的現象	相關章節
空間解析度	空間(取樣)	鋸齒、細節喪失、混疊	第 4 章內插、第 6 章混疊
灰階解析度	亮度(量化)	假輪廓、色階斷層	第 5 章灰階轉換、第 11 章壓縮
時間解析度	時間(影格率)	運動模糊、時間混疊	第 7 章運動模糊、第 12 章視訊

表 3-1 三種解析度與其不足時的現象

三者之間常有取捨關係。以攝影為例：要提高時間解析度(縮短曝光以凍結動作)，進光量就變少，雜訊相對變大；要提高空間解析度(更多像素)，每個像素接收的光就變少，同樣影響訊噪比。這種「此消彼長」的張力，是感測器設計的核心課題，也會在第 5 章夜景模式、第 7 章運動模糊、第 17 章產線取像中反覆出現。

3.2 取樣定理與混疊

3.2.1 Nyquist 取樣定理

取樣要多密才夠？這個問題有一個精確的答案，由 Nyquist 與 Shannon 給出：取樣頻率必須至少是訊號中最高頻率成分的兩倍，才能完整重建原訊號。用影像的語言說：若影像中最細的紋理是每毫米五個明暗週期，取樣密度就必須至少每毫米十個像素。

為什麼是兩倍？直覺上可以這樣理解：要記錄一個完整的波形週期，至少需要兩個取樣點——一個記錄波峰、一個記錄波谷。若每個週期只取到一個點，你完全無從得知這個波的存在，甚至會誤以為它是別的東西。這個「別的東西」，就是下一節的混疊。

這條定理的完整意義，要到第 6 章的頻率域才說得清楚：取樣的動作在頻域上會使原訊號的頻譜以取樣頻率為間隔無限複製；取樣夠密時複製品彼此分離、可用低通濾波取回原訊號，不夠密則相鄰複製品重疊、高頻成分「摺疊」進低頻區。第 6.6.1 節會完整展開這個解釋。這裡先記住結論：取樣不足，高頻會偽裝成低頻。

3.2.2 混疊的三張面孔

混疊(Aliasing)是取樣不足的直接後果，它在不同場合有不同的樣貌，但成因完全相同：

3. 空間混疊——摩爾紋：拍攝細密的條紋(格子襯衫、螢幕、紗窗)時出現的詭異波紋。這是螢幕像素網格與相機感測器網格相互干涉，高頻紋理摺疊成了低頻的粗大波紋。第 6.7.1 節會示範如何以陷波濾波移除它。
4. 鋸齒(Jaggies)：斜線與曲線在低解析度下呈現階梯狀。這是叫獸開講中方形像素的直接後果，也是抗鋸齒(anti-aliasing)技術要對付的問題——其原理正是先做低通模糊，再取樣。
5. 時間混疊——車輪倒轉：影片中高速旋轉的車輪或電風扇看似靜止或倒轉。這是影格率(時間取樣)不足，高頻的旋轉運動被摺疊成了低頻甚至反向的運動。第 12 章叫獸開講的思考題正是這個現象。

3.2.3 如何防範

防範混疊的方法只有一個原則：在取樣之前，先把超過取樣率一半的高頻成分濾掉。這稱為抗混疊濾波(anti-aliasing filter)，硬體上以光學低通濾鏡實現，軟體上則以模糊處理實現。

這解釋了一個常令初學者困惑的建議：縮小影像前應先做模糊。因為縮小就是降低取樣率，若不先把高頻濾掉，它們就會摺疊成假訊號。OpenCV 的 INTER_AREA 內插之所以適合縮小，正是因為它內建了區域平均(即低通)的動作——第 4.3 節的口訣「縮小用 AREA」，根源就在這裡。

程式 3-1 混疊的產生與防範

```
import cv2, numpy as np

# --- 製造混疊：合成高頻條紋後直接降取樣 ---
N = 512
x = np.arange(N)
stripes = ((np.sin(2 * np.pi * x / 4) + 1) * 127).astype(np.uint8)
img = np.tile(stripes, (N, 1))          # 每 4 像素一個週期(高頻)

# 錯誤做法：直接跳點取樣 → 混疊
bad = img[::4, ::4]                    # 條紋消失或變成假圖案

# 正確做法：先低通模糊，再降取樣
blurred = cv2.GaussianBlur(img, (0, 0), sigmaX=2)
good = blurred[::4, ::4]

# OpenCV 的 INTER_AREA 內建低通，是縮小的建議選項(第 4 章)
area = cv2.resize(img, (N//4, N//4), interpolation=cv2.INTER_AREA)

cv2.imwrite("alias_bad.png", bad)
cv2.imwrite("alias_good.png", good)
cv2.imwrite("alias_area.png", area)
# 並排比較：bad 會出現原圖沒有的假圖案
```

3.3 量化與位元深度

3.3.1 位元深度與動態範圍

位元深度(bit depth)決定每個像素能表示多少個灰階層次：1 位元只有黑與白兩階(二值影像，第 10 章的主角)；8 位元有 256 階，是最常見的規格；10、12、14 位元則常見於專業相機的 RAW 檔與醫學影像。彩色影像的位元深度通常以「每通道」計算，8 位元的 RGB 影像共有約一千六百萬種顏色組合。

動態範圍(Dynamic Range)則描述影像能同時記錄的最亮與最暗之間的比值。人眼在單一場景中約可分辨十多個 f-stop 的亮度差異，而一般 8 位元 JPEG 只能記錄較窄的範圍——這就是為什麼逆光拍攝時，若曝光在人臉，背景的天空就會過曝成一片白。高動態範圍(HDR)技術以不同曝光的多張影像合成，正是為了突破這個限制，與第 5 章夜景模式的多影格合併是同源的思路。

3.3.2 量化不足：假輪廓

當位元深度不足時，最明顯的現象是假輪廓(false contouring，或稱色階斷層、posterization)：在本該平滑漸變的區域(晴朗的天空、柔和的陰影、漸層背景)出現一圈圈生硬的階梯狀邊界。原因很單純：真實的漸變是連續的，但量化後只能用有限的幾階去表示，相鄰階層之間就出現了肉眼可見的跳躍。

有趣的是，人眼對「平滑區域的微小跳躍」特別敏感，對「細節豐富區域的量化誤差」反而遲鈍。這個特性有兩個實務意義：其一，漸層區域是判斷位元深度是否足夠的最佳測試素材；其二，刻意加入極微量的雜訊(稱為抖動，dithering)反而能打散階梯邊界、讓漸變看起來更平滑——這是「用雜訊改善觀感」的少見例子。

程式 3-2 量化與假輪廓

```
import cv2, numpy as np

# 建立平滑漸層(最容易看出量化不足的素材)
ramp = np.tile(np.linspace(0, 255, 512, dtype=np.uint8), (200, 1))

# 模擬不同位元深度的量化
for bits in [8, 5, 4, 3, 2, 1]:
    levels = 2 ** bits
    step = 256 // levels
    q = (ramp // step) * step          # 量化
    cv2.imwrite(f"quant_{bits}bit.png", q)
    print(f"{bits} 位元 = {levels} 階")
    # 觀察:位元數愈少,漸層上的階梯(假輪廓)愈明顯

# 抖動:加入微量雜訊打散階梯邊界
noise = np.random.normal(0, 8, ramp.shape)
dithered = np.clip(ramp + noise, 0, 255).astype(np.uint8)
q_dither = (dithered // 32) * 32      # 同樣量化成 3 位元
cv2.imwrite("quant_3bit_dither.png", q_dither)
# 與 quant_3bit.png 比較:加了雜訊反而看起來更平滑
```

3.3.3 量化的一體多面

「用有限的階層逼近連續值」這個思想，會在本書中以三種面貌出現，值得在此先建立連結：本章談的是感測器把光強度量化成灰階；第 11 章 JPEG 壓縮的量化步驟，是把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻；第 17 章的模型量化，則是把神經網路的權重從 32 位元浮點降到 8 位元整數以加速推論。三者的數學核心完全相同——都是在可接受的品質損失下，換取儲存、傳輸或運算的大幅節省。當你在第 11、17 章再次遇到「量化」時，請記得回到本章。

3.4 影像的產生：感測器與色彩

3.4.1 感測器如何看見

數位相機的感測器(CCD 或 CMOS)由數百萬個微小的光電二極體組成，每一個對應一個像素。曝光期間，光子撞擊感光元件產生電子，累積的電荷量正比於接收到的光量；曝光結束後，電荷被讀出、放大、經類比數位轉換器量化成數字。

這個過程解釋了幾個重要現象。其一，光子到達是隨機的，因此即使拍攝完全均勻的畫面，各像素的數值也會有隨機起伏——這就是第 7 章的 Poisson 雜訊(散粒雜訊)，它在低光時特別明顯，也是第 5 章夜景模式要對付的主角。其二，讀出與放大電路本身也會引入雜訊(讀取雜訊)。其三，像素若接收過多光子會「滿溢」，數值卡在最大值——這就是過曝，一旦發生，該區域的資訊就徹底喪失，任何後製都救不回來。

3.4.2 色彩濾鏡陣列與去馬賽克

這裡有一個常被忽略的事實：感光元件本身只能感測光的強度，不能分辨顏色。那彩色照片是怎麼來的？答案是在感測器表面覆蓋一層色彩濾鏡陣列(CFA)，讓每個像素只接收特定顏色的光。最常見的是拜爾陣列(Bayer pattern)，它以「一紅、兩綠、一藍」為單位重複排列——綠色佔一半，是因為人眼對綠光最敏感(這個設計思想與第 8 章 YCbCr 對色度降取樣、第 11 章 JPEG 的 4:2:0 完全一致：把資源分配給人眼最在意的地方)。

於是每個像素其實只記錄了三個顏色中的一個，另外兩個必須由鄰近像素「猜」出來——這個過程稱為去馬賽克(demosaicing)，本質上是一種內插(第 4 章)。這也解釋了為什麼相機的彩色細節解析力，實際上低於它的像素數所暗示的水準，以及為什麼在高對比的細節邊緣容易出現偽色。

3.4.3 RAW 與 JPEG：一個關鍵的取捨

相機可以輸出兩種檔案。RAW 保存的是感測器讀出的原始資料——高位元深度(通常 12 至 14 位元)、線性(數值正比於光量)、未經壓縮或僅無損壓縮、未套用白平衡與銳化。JPEG 則是相機內部已經完成去馬賽克、白平衡、色調曲線、Gamma 編碼、銳化，並以有損壓縮(第 11 章)存成 8 位元的成品。

對影像處理而言，這個差異至關重要，有三個理由：第一，位元深度——12 位元的 RAW 有 4096 階，遠多於 JPEG 的 256 階，後製調整曝光時不易產生假輪廓。第二，線性——RAW 的數值正比於實際光量，而 JPEG 經過 Gamma 編碼後並非線性

(第 5.1.2 節詳述)，做精確的光度量測時必須留意。第三，未經破壞——JPEG 的壓縮與銳化都是不可逆的，且反覆存檔會累積失真(第 11.4.2 節)。

因此本書的建議是：做嚴肅的影像處理與科學量測時，盡可能從 RAW 出發；第 7 章的還原、第 14 至 16 章的模型訓練資料蒐集，都會因為輸入品質而受益。當然，RAW 檔案大、處理慢，一般應用用 JPEG 就夠——這又是一個依需求判斷的工程取捨。

3.5 影像品質的客觀評估

當你用某個演算法處理影像後，如何判斷它「好不好」？主觀的目視評估重要但不可量化、不可重現；因此我們需要客觀指標。這一節建立的工具，將在第 5、6、7、11 章的每一個 Lab 中反覆使用。

3.5.1 MSE 與 PSNR

均方誤差(MSE)是最基本的指標：把處理後影像與參考影像逐像素相減、取平方、再平均。它直接反映「差多少」，值愈小愈好，但它的數值大小與影像的亮度範圍有關，不同影像之間難以比較。

峰值訊噪比(PSNR)以對數形式改良了這一點，單位是分貝(dB)，值愈大愈好。實務上的經驗參考值：超過 40 dB 表示品質極佳、幾乎看不出差異；30 至 40 dB 為良好；20 至 30 dB 可見明顯劣化；低於 20 dB 則品質很差。它是影像壓縮與還原領域最廣泛使用的指標(第 7、11 章)。

3.5.2 SSIM 與 PSNR 的盲點

但 PSNR 有一個根本盲點：它逐像素比較，完全不考慮人類視覺的特性。舉一個經典的例子——把整張影像的亮度平均提高一點點，PSNR 會顯著下降，但人眼幾乎看不出差異；反之，在平坦區域加入少數幾個突兀的雜點，PSNR 下降有限，人眼卻一眼就看到刺眼的瑕疵。換言之，PSNR 高不代表「看起來好」。

結構相似性指標(SSIM)為此而生。它不逐像素比較，而是分別衡量兩張影像在亮度、對比度、結構三個面向的相似程度，再綜合成一個 0 到 1 之間的分數(1 代表完全相同)。因為它著眼於「結構」，更貼近人類的感知。實務建議是兩者並用：PSNR 反映數值誤差，SSIM 反映感知品質，兩者一致時結論可靠，不一致時就值得深究原因。

還有一個更根本的提醒，將在第 7.6.1 節的 MRI 案例中詳述：全域指標衡量的是平均表現，而許多應用關心的是最壞情況。一個把微小病灶抹掉的去噪演算法，PSNR 可能不降反升(因為背景更乾淨)，但在醫學上是災難。指標是工具，不是目的——這個判斷力，是本書希望你建立的核心素養之一。

程式 3-3 MSE、PSNR 與 SSIM

```
import cv2, numpy as np

clean = cv2.imread("original.png", cv2.IMREAD_GRAYSCALE)
noisy = cv2.imread("processed.png", cv2.IMREAD_GRAYSCALE)

# --- MSE ---
```



```

mse = np.mean((clean.astype(np.float64) - noisy.astype(np.float64)) **
2)
print(f"MSE = {mse:.2f}")

# --- PSNR(OpenCV 內建) ---
print(f"PSNR = {cv2.PSNR(clean, noisy):.2f} dB")

# --- SSIM(需安裝 scikit-image: pip install scikit-image) ---
from skimage.metrics import structural_similarity as ssim
score, diff_map = ssim(clean, noisy, full=True)
print(f"SSIM = {score:.4f}")

# SSIM 差異圖:可視化「哪裡不像」,除錯很有用
diff_vis = (diff_map * 255).astype(np.uint8)
cv2.imwrite("ssim_map.png", diff_vis)

# --- 示範 PSNR 的盲點 ---
shifted = np.clip(clean.astype(np.int16) + 5, 0, 255).astype(np.uint8)
print(f"整體加亮 5:PSNR={cv2.PSNR(clean, shifted):.2f} dB,"
      f" SSIM={ssim(clean, shifted):.4f}")
# 人眼幾乎看不出差異,但 PSNR 顯著下降 → 指標未必等於觀感

```

3.6 像素的鄰域、連通性與距離

本節建立的三個概念看似瑣碎，卻是第 9 章分割、第 10 章型態學與連通元件標記的直接基礎。

3.6.1 鄰域與連通性

一個像素的「鄰居」有兩種定義方式：4-鄰域只算上下左右四個像素；8-鄰域則加上四個對角，共八個。由此衍生出兩種連通性，而選擇哪一種會直接影響結果——這在第 10 章計算「畫面中有幾個物件」時尤其關鍵。

舉一個經典的困境：兩個只在對角相接的白色區塊，在 8-連通下算一個物件，在 4-連通下算兩個；而一條斜向的細線，在 4-連通下會被判定為斷裂的一群孤立點。更麻煩的是所謂的連通性悖論——若前景與背景採用同一種連通性，會出現「一條封閉的曲線卻沒有把背景分成內外兩部分」這類矛盾。實務上的慣例是前景與背景採用互補的連通性(例如前景 8-連通、背景 4-連通)。

3.6.2 距離度量

兩個像素有多遠？三種常見的定義各有用途：歐氏距離(直線距離)最符合直覺、也最常用於量測；城市街區距離(D4，只能上下左右移動的步數)與棋盤距離(D8，可斜走的步數)則計算快速，且與 4-連通、8-連通對應。

這些度量在第 10.5.2 節的距離轉換中扮演核心角色：為每個前景像素計算它到最近背景像素的距離，得到的「距離圖」可用於求物件骨架，更是第 9 章分水嶺分割「分開相黏物件」的關鍵——每個相黏圓形物件的中心會在距離圖上形成一個局部極大值，正好作為分水嶺的標記。屆時請記得回到本節。

程式 3-4 連通性與距離度量

```

import cv2, numpy as np

# 建立一個測試二值影像:兩個對角相接的方塊
img = np.zeros((100, 100), np.uint8)
img[20:50, 20:50] = 255
img[50:80, 50:80] = 255          # 與前者僅在一個角相接

# 連通性的影響:同一張圖,不同答案
n4, _ = cv2.connectedComponents(img, connectivity=4)
n8, _ = cv2.connectedComponents(img, connectivity=8)
print(f"4-連通:{n4-1} 個物件")    # 2(視為分開)
print(f"8-連通:{n8-1} 個物件")    # 1(視為相連)

# 三種距離度量的距離轉換(第 10 章)
for name, dist_type in [("歐氏", cv2.DIST_L2),
                        ("城市街區", cv2.DIST_L1),
                        ("棋盤", cv2.DIST_C)]:
    d = cv2.distanceTransform(img, dist_type, 5)
    print(f"{name}距離:最大值 = {d.max():.1f}")
    vis = cv2.normalize(d, None, 0, 255, cv2.NORM_MINMAX)
    cv2.imwrite(f"dist_{dist_type}.png", vis.astype(np.uint8))

```

3.7 應用案例

3.7.1 案例一：掃描解析度該設多少

情境：要把一批紙本文件或老照片數位化典藏，掃描解析度該設 150、300 還是 600 dpi?這是取樣定理最貼近生活的應用。判斷依據是「你要保留的最細特徵有多細」：一般文字文件的筆畫寬度約 0.2 至 0.3 毫米，依取樣定理需要每毫米至少八到十個取樣點，對應約 200 至 300 dpi;若要保留印刷網點或做後續的文字辨識(OCR)，則建議 300 至 600 dpi;而老照片的細節(如髮絲、織品紋理)則可能需要更高。

反過來說，盲目提高解析度也有代價：檔案大小以平方成長(600 dpi 是 300 dpi 的四倍大)、掃描時間拉長、且若原稿本身的細節有限，多出來的像素只是在記錄底片顆粒或紙張纖維。這個「依需求決定規格」的思考，正是第 17 章延遲預算與第 18 章專題規劃的同一種能力。

3.7.2 案例二：醫學影像為什麼需要高位元深度

情境：CT 與 MRI 影像通常以 12 至 16 位元儲存，遠高於一般照片的 8 位元。為什麼?因為人體組織之間的密度差異可能極小——一個早期的腫瘤與周圍正常組織的訊號差，可能只有幾個灰階單位。若量化成 256 階，這個差異可能被歸進同一階而完全消失。

但這帶來一個顯示的難題：一般螢幕只能顯示 256 階灰度，人眼也分辨不了那麼多階。解法是「視窗化」(windowing)：醫師依觀察目標，把 12 位元資料中的某一段範圍拉伸到完整的顯示範圍——看肺部用一組視窗、看骨骼用另一組、看軟組織再一組。同一份資料，不同的視窗設定會看到截然不同的內容。這其實就是第 5 章灰階

轉換的醫學版本，也說明了為什麼高位元深度的原始資料必須完整保存：它承載了顯示時尚未取用的資訊。

3.7.3 案例三：工業檢測的取像規格計算

情境：要檢測一個工件表面上寬度 0.1 毫米的刮痕，相機該選多少像素？這是產線設計的第一道題，而它是可以精確計算的。

計算方式：依取樣定理，要可靠地偵測 0.1 毫米的特徵，至少需要每 0.05 毫米一個像素(兩倍取樣)；實務上為了穩健，常要求三到五個像素涵蓋最小特徵，即每像素約 0.02 至 0.03 毫米。若工件的視野寬度是 60 毫米，則所需的水平像素數約為 60 除以 0.025，即 2400 像素。加上安全餘裕，選一台 3000 萬像素等級的相機或縮小視野分次拍攝。

這個案例展示了工程與「調參數」的差別：規格不是試出來的，而是從需求推算出來的。同樣的推算思維會出現在第 7.6.3 節(由產線速度與曝光時間推算運動模糊長度)與第 17.4.2 節(由機器人速度推算延遲預算)。這種「從需求反推規格」的能力，是叫獸最希望你從這門課帶走的東西之一。

3.7.4 案例四：影像壓縮的品質決策

情境：網站要上傳數千張商品照片，JPEG 品質參數該設多少？這是 3.5 節評估指標的實戰。做法：取一組代表性的照片，分別以品質 95、85、75、60、40 存檔，記錄每個設定的檔案大小、PSNR 與 SSIM，繪製「檔案大小對品質」的曲線(即第 11 章的率失真曲線)，找出「再降就明顯劣化」的轉折點。

經驗上，品質 85 附近往往是甜蜜點——檔案大小比 95 小得多，而肉眼幾乎看不出差異。但這個結論會因影像內容而異：含大面積平滑漸層的照片對壓縮較敏感(容易出現第 11 章的方塊效應)，而紋理豐富的照片則較耐壓。這也再次呼應 3.5.2 節：指標要看，眼睛也要看。

3.8 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
取樣與內插	第 4 章	縮放的內插法選擇；縮小用 INTER_AREA 防混疊
取樣定理與混疊	第 6、12 章	頻域解釋(頻譜複製與摺疊)；摩爾紋移除；時間混疊
量化	第 11、17 章	JPEG 的 DCT 係數量化；神經網路的模型量化
位元深度與動態範圍	第 5、7 章	Gamma 與對數轉換；HDR；RAW 為還原的理想輸入
感測器雜訊	第 5、7 章	Poisson 與讀取雜訊；夜景多影格平均；去雜訊方法

拜爾陣列與去馬賽克	第 4、8、11 章	內插;人眼對綠光敏感;色度降取樣的同源思想
MSE、PSNR、SSIM	第 5-7、11、14 章	所有處理成效的客觀驗收;全域指標的盲點
鄰域與連通性	第 9、10 章	連通元件標記的計數依據;型態學的結構元素
距離度量	第 9、10 章	距離轉換;分水嶺標記;骨架化

表3-2 第3章與全書其他章節的觀念連結

程式實作 Lab 3

Lab 3-1 取樣、混疊與抗混疊

目標：親眼見證取樣定理。(a)以 `np.sin` 合成一組不同疏密的條紋影像(每 2、4、8、16 像素一個週期);(b)對每張直接跳點降取樣四倍，觀察哪些出現了「原圖沒有的假圖案」，並用取樣定理解釋為什麼;(c)改為先高斯模糊再降取樣，比較結果;(d)用 OpenCV 的四種內插旗標(NEAREST、LINEAR、CUBIC、AREA)各縮小一次，比較混疊程度，驗證第 4 章「縮小用 AREA」的建議;(e)實拍：用手機拍攝格子襯衫、紗窗或電腦螢幕，重現摩爾紋，並保存這張影像——第 6 章 Lab 會用它來做陷波濾波移除。

Lab 3-2 量化與假輪廓

目標：理解位元深度的意義。(a)以程式 3-2 對一張平滑漸層與一張真實照片分別做 8、5、4、3、2、1 位元的量化，並排比較;(b)記錄「從幾位元開始肉眼可見假輪廓」，並比較漸層與紋理豐富的照片，何者較早出現、為什麼;(c)實作抖動：量化前加入微量高斯雜訊，比較有無抖動的視覺差異;(d)量化計算 PSNR 與 SSIM 隨位元數的變化，繪成曲線;(e)討論：為什麼醫學影像需要 12 位元以上?(對照 3.7.2 節)

Lab 3-3 品質評估工具箱

目標：建立整學期都會用到的評估工具。(a)寫一個函式 `evaluate(original, processed)`，一次回傳 MSE、PSNR、SSIM 三個指標，並輸出 SSIM 差異圖;(b)驗證 PSNR 的盲點：對同一張影像做兩種處理——整體亮度加 5、以及在平坦區加入十個突兀的雜點，比較兩者的 PSNR 與 SSIM，並用肉眼判斷哪個比較難看;(c)討論結果與你的視覺判斷是否一致，不一致的原因是什麼;(d)把這個函式存成共用模組(如 `utils/metrics.py`)，往後第 5、6、7、11 章的 Lab 直接引用——呼應第 2 章「不要重造輪子」。

Lab 3-4 連通性與距離轉換

目標：為第 9、10 章打底。(a)以程式 3-4 建立「對角相接的兩個方塊」，分別用 4-連通與 8-連通做連通元件標記，確認計數結果不同;(b)畫一條斜向的一像素細線，觀察 4-連通下它是否被判為斷裂;(c)對一張含多個圓形物件(可用 `cv2.circle` 合成，刻

意讓兩個相接)的二值影像做距離轉換,把結果以第 8 章的虛擬色彩顯示;(d)找出距離轉換的局部極大值,觀察它們是否落在每個圓的中心——這正是第 9 章分水嶺分割的標記來源;(e)比較歐氏、城市街區、棋盤三種距離度量的結果差異。

本章重點整理

- 影像數位化分為取樣(空間離散化,決定像素與空間解析度)與量化(亮度離散化,決定位元深度);兩者皆造成不可逆的資訊損失。
- 影像尺寸、空間解析度、顯示尺寸三者不同;像素多不等於細節多——放大既有影像只是內插猜值,並未增加資訊。
- Nyquist 取樣定理要求取樣頻率至少為最高頻率的兩倍;不足則產生混疊,表現為摩爾紋(空間)、鋸齒、車輪倒轉(時間)。防範之道是取樣前先低通濾波,故縮小影像應先模糊或用 INTER_AREA。
- 位元深度決定灰階層次(8 位元 256 階),不足時在平滑區產生假輪廓;動態範圍描述可同時記錄的明暗比值;加入微量雜訊(抖動)反而能打散階梯。量化思想在第 11 章 JPEG 與第 17 章模型量化中重現。
- 感測器以光電效應成像,帶來 Poisson 與讀取雜訊、以及過曝的不可逆資訊喪失;拜爾陣列使每像素僅記錄一個顏色,需經去馬賽克內插;RAW 保有高位元深度與線性數值,是嚴肅影像處理的理想輸入。
- 客觀品質評估: MSE(誤差平方均值)、PSNR(對數形式, 40 dB 以上極佳)、SSIM(衡量亮度、對比、結構,更貼近感知);PSNR 有逐像素比較的盲點,全域指標亦掩蓋最壞情況,須與目視並用。
- 鄰域分 4-鄰域與 8-鄰域,連通性的選擇直接影響物件計數;距離度量有歐氏、城市街區、棋盤三種,為第 10 章距離轉換與第 9 章分水嶺的基礎。

習題

1. 影像數位化的兩個步驟是什麼?各自離散化了哪一個維度?為什麼說這個過程不可逆?

提示與參考解答: 取樣: 在空間上離散化,把影像切成一格格像素,決定空間解析度。量化: 在亮度上離散化,把連續亮度對應到有限階層,由位元深度決定。不可逆是因為兩者都丟棄了原始的連續資訊——落在同一格內的空間細節、落在同一階層內的亮度差異,都永遠無法還原。這也是第 7 章影像還原的物理上限。

2. 請區分影像尺寸、空間解析度與顯示尺寸。為什麼說「像素多不等於細節多」?

提示與參考解答: 影像尺寸是像素數(如 1920×1080,即陣列的 shape);空間解析度是單位長度內的像素數(如 300 dpi),才真正描述細節精細程度;顯示尺寸是實際呈現的大小,取決於顯示裝置密度。像素多不等於細節多: 把模糊照片放大四倍,像素變十六倍但細節未增(內插只是猜值);感測器像素再多,若鏡頭解析力不足,多出的像素只是記錄模糊。

3. 請陳述 Nyquist 取樣定理,並用「記錄一個波形週期」的直覺解釋為什麼是兩倍。

提示與參考解答: 取樣頻率必須至少是訊號中最高頻率成分的兩倍,才能完整重建原訊號。直覺解釋: 要記錄一個完整的波形週期,至少需要兩個取樣點——一個抓

波峰、一個抓波谷；若每週期只取到一個點，就無從得知這個波的存在，甚至會誤判成別的低頻訊號（即混疊）。

4. 混疊在空間與時間上分別表現為什麼現象？請各舉一例並說明成因相同之處。

提示與參考解答：空間混疊：摩爾紋（拍格子襯衫或螢幕時的詭異波紋）、鋸齒（斜線呈階梯狀）。時間混疊：車輪或電風扇在影片中看似靜止或倒轉。成因相同：取樣率（空間的像素密度或時間的影格率）不足以滿足取樣定理，高頻成分被「摺疊」進低頻區，偽裝成原本不存在的低頻訊號。

5. 為什麼縮小影像前應先做模糊？這與 OpenCV 的 INTER_AREA 有何關聯？

提示與參考解答：縮小即降低取樣率，依取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊（假圖案）。先低通模糊把這些高頻濾掉，就無物可摺疊。INTER_AREA 內插的原理是區域平均，本身就內建了低通動作，因此是縮小的建議選項（第4.3 節「縮小用 AREA」的根源）。

6. 什麼是假輪廓？它為什麼最容易出現在平滑漸層區域？為什麼加入微量雜訊反而能改善？

提示與參考解答：假輪廓是位元深度不足時，在本該平滑漸變的區域出現生硬的階梯狀邊界。最容易出現在漸層區，是因為真實漸變是連續的，量化後只能用有限階層表示，相鄰階層間出現肉眼可見的跳躍；而人眼對平滑區的微小跳躍特別敏感。加入微量雜訊（抖動）能打散規則的階梯邊界，使量化誤差在視覺上呈隨機分布而非整齊的界線，反而看起來更平滑。

7. 本書中「量化」這個思想出現在哪三個地方？它們的共通核心是什麼？

提示與參考解答：一、本章：感測器把連續的光強度量化成有限灰階。二、第11章：JPEG 把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻。三、第17章：模型量化把神經網路權重從32 位元浮點降到8 位元整數。共通核心：在可接受的品質損失下，用有限的階層逼近連續值，換取儲存、傳輸或運算的大幅節省。

8. 感光元件本身能分辨顏色嗎？彩色照片是如何產生的？拜爾陣列為什麼綠色佔一半？

提示與參考解答：不能，感光元件只能感測光的強度。彩色是靠感測器表面的色彩濾鏡陣列(CFA)，讓每像素只接收特定顏色的光，再以去馬賽克內插補出另外兩色。拜爾陣列以「一紅兩綠一藍」重複排列，綠色佔一半是因為人眼對綠光最敏感——這與第8 章 YCbCr 對色度降取樣、第11 章 JPEG 的 4:2:0 是同一個設計思想：把資源分配給人眼最在意的地方。

9. RAW 與 JPEG 有哪三個關鍵差異？為什麼做嚴肅的影像處理時建議從 RAW 出發？

提示與參考解答：一、位元深度：RAW 通常12 至14 位元(4096 階以上)，JPEG 僅8 位元(256 階)，後製調整不易產生假輪廓。二、線性：RAW 數值正比於實際光量，JPEG 經 Gamma 編碼後非線性（第5.1.2 節），精確光度量測須留意。三、未經破壞：JPEG 已套用白平衡、銳化並有損壓縮，皆不可逆且反覆存檔會累積失真。因此第7 章還原與模型訓練資料蒐集都會因 RAW 的輸入品質而受益。

10. 請說明 PSNR 的盲點，並解釋 SSIM 如何改善。為什麼說「指標是工具，不是目的」？

提示與參考解答：PSNR 逐像素比較、不考慮人類視覺特性：整張影像亮度微幅提高會使 PSNR 顯著下降但人眼幾乎無感；平坦區的少數突兀雜點對 PSNR 影響有限卻極為刺眼。SSIM 改為衡量亮度、對比度、結構三面向的相似度，更貼近感知。

「指標是工具不是目的」：全域指標衡量平均表現，而許多應用關心最壞情況——一個抹掉微小病灶的去噪演算法，PSNR 可能不降反升(第 7.6.1 節 MRI 案例)，但在醫學上是災難。

11. 4-連通與 8-連通有何差異?請舉一個「同一張影像、不同連通性得到不同物件數」的例子，並說明為什麼實務上前景與背景要用互補的連通性。

提示與參考解答：4-鄰域只算上下左右，8-鄰域再加四個對角。例子：兩個僅在對角相接的白色方塊，8-連通視為一個物件、4-連通視為兩個；一條斜向細線在 4-連通下會被判為斷裂的孤立點。前景與背景用互補連通性(如前景 8、背景 4)是為了避免連通性悖論——若兩者同用一種，會出現「封閉曲線卻沒把背景分成內外」這類矛盾。

12. (綜合題)承 3.7.3 節：要檢測工件上寬度 0.2 毫米的裂紋，視野寬度為 100 毫米，請推算所需的相機水平像素數，並說明你的假設與安全餘裕。

提示與參考解答：依取樣定理，偵測 0.2 毫米特徵至少需每 0.1 毫米一像素(兩倍取樣)；但實務上為穩健，常要求三到五像素涵蓋最小特徵，取四像素則每像素約 0.05 毫米。所需水平像素數 = $100 \div 0.05 = 2000$ 像素。加上安全餘裕(考慮鏡頭邊緣解析力下降、對焦誤差、工件擺放偏移)，建議選 2500 至 3000 像素以上的相機，或縮小視野分次拍攝。關鍵在於說明假設(每特徵幾個像素)與餘裕的理由，而非單一數字。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2(數位影像基礎)。
- NIST, First Digital Image — Russell Kirsch, 1957:<https://www.nist.gov/mathematics-statistics/first-digital-image>(本章叫獸開講之史料來源)
- Wang, Z., Bovik, A. C., et al., Image Quality Assessment: From Error Visibility to Structural Similarity, IEEE TIP, 2004.(SSIM 原始論文)
- Shannon, C. E., Communication in the Presence of Noise, Proc. IRE, 1949.(取樣定理)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2(影像形成與感測器)。
- scikit-image 官方文件(SSIM 等品質指標):<https://scikit-image.org>
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 4 章 幾何轉換

本章學習目標

- 以矩陣表示平移、旋轉、縮放等基本幾何轉換，並說明齊次座標存在的理由。
- 區分剛體、相似、仿射與透視轉換的自由度、不變性質與適用場景。
- 解釋反向映射為何需要內插，比較四種內插法的品質、速度與適用時機。
- 描述針孔相機模型的內部參數與外部參數，辨識桶狀與枕狀鏡頭畸變。
- 以張氏標定法完成相機校正與去畸變，並以重投影誤差驗收成果。
- 運用透視轉換完成文件拉正與機器人導航用的鳥瞰圖。
- 理解幾何轉換在資料增強、視訊穩定、影像拼接與機器人座標系中的角色。

叫獸開講|張正友與一張棋盤格：讓相機校正走進每個人口袋

要讓相機從「拍照的機器」升級成「測量的儀器」，第一步是校正 (Calibration)：精確求出它的焦距、光心位置與鏡頭畸變參數。在 1990 年代，這是一件昂貴的苦差事。經典方法需要精密加工的三維標定物——例如兩三個互相垂直、貼滿精確標記點的金屬平面，加工精度直接決定校正精度，一般實驗室根本玩不起。另一派「自我校正」方法不需標定物，數學卻極其艱深且不穩定。相機校正，就這樣卡在「太貴」與「太難」之間。

1998 年 12 月 2 日，微軟研究院的張正友(Zhengyou Zhang)發表了技術報告 MSR-TR-98-71 《A Flexible New Technique for Camera Calibration》，給出了第三條路。他的方法簡單到令人難以置信：只需要把一張棋盤格圖案印在紙上、貼到一塊平板上，讓相機從至少兩個不同方位觀察它就行了——而且相機動或棋盤動都可以，更關鍵的是，你完全不需要知道它是怎麼動的。

技術上，這個方法先利用「平面與其影像之間的單應性」導出對內部參數的約束，以封閉解求出初始估計，再以最大概似準則做非線性最佳化，同時把徑向畸變一併建模。論文於 1999 年在頂級會議 ICCV 發表，並於 2000 年正式刊登在頂級期刊 IEEE TPAMI 上。張正友在論文中寫下的一句話，精準地概括了它的意義相較於需要兩三個正交平面等昂貴設備的傳統方法，這個技術簡單而靈活，讓三維電腦視覺從實驗室環境往真實世界前進了一步。

「一張紙取代精密機械」的威力是巨大的。任何人花五分鐘列印一張棋盤格，就能完成過去要專業設備才能做的事。這個方法後來成為 OpenCV 內建的標準校正流程(cv2.calibrateCamera 的核心正是張氏標定法)，也是全球手機、無人機、機器人出廠校正管線的基礎，論文引用數以萬計，是電腦視覺史上影響力最大的工作之一。張正友後來持續在工業界深耕電腦視覺與機器人研究——而你今天在 Lab 4-2 印出的那張棋盤格，正是這段歷史的延續。

叫獸想從這裡點出一個貫穿全書的觀察。回想第 2 章 OpenCV 的開源、第 13 章 ORB 對 SIFT 專利的回應、第 17 章五美元的 ESP32——這些改變領域的突破，往往不是「做出更強大的東西」，而是「大幅降低了做這件事的門檻」。把一項只有少數人玩得起的技術，變成每個人都能用的工具，其影響力常常遠超過在精度上多推進幾個百分點。這是工程創新中最容易被低估、卻最有力量的一種。

思考題：張氏標定法要求棋盤格「至少兩個不同方位」，但實務上大家都拍 10 到 20 張。想一想：多拍的那些照片，是在對抗什麼？(提示：雜訊、角點定位誤差，以及涵蓋畫面各個角落的畸變。張正友原始論文的實驗也顯示，估計結果的不確定度會隨影像張數增加而下降)

4.1 基本幾何轉換與齊次座標

4.1.1 幾何轉換改變什麼

先釐清一件事：幾何轉換改變的是像素的「位置」，而非像素的「數值」。第 5 章的灰階轉換是「位置不變、數值改變」，本章的幾何轉換則恰好相反——把影像中的內容搬到別的地方去，而每個點的亮度或顏色原則上跟著它一起搬。這個對照有助於你建立整體樣貌：影像處理的操作，不是在動數值，就是在動位置，或兩者兼有。

4.1.2 三種基本轉換

三種最基本的轉換是：平移把每個點 (x, y) 移到 $(x + tx, y + ty)$ ；縮放把座標乘上倍率，得到 $(sx \cdot x, sy \cdot y)$ ；旋轉將點繞原點轉 θ 角，新座標為 $x' = x \cdot \cos\theta - y \cdot \sin\theta, y' = x \cdot \sin\theta + y \cdot \cos\theta$ 。

縮放與旋轉都能寫成 2×2 矩陣乘法，唯獨平移是「加法」，格格不入。這個小小的不和諧看似無關痛癢，實際上很麻煩：當你要串接多個轉換時，乘法與加法混在一起，無法合成為單一運算。這正是齊次座標登場的理由。

4.1.3 齊次座標：讓所有轉換說同一種語言

齊次座標的做法很簡單：替每個點多加一維，把 (x, y) 寫成 $(x, y, 1)$ 。如此一來，平移也能表示成 3×3 矩陣乘法，三種轉換全部統一成「矩陣乘以向量」的形式。

統一的好處立刻顯現。第一，多個轉換的串接，等於把矩陣依序相乘成一個矩陣——「先旋轉 30 度、再放大 1.5 倍、再平移」三個動作，可以合成一個 3×3 矩陣一次做完。第二，這不只是寫法漂亮：一次完成比分三次做更快，而且更重要的是避免了多次取樣造成的品質劣化(每做一次轉換就要內插一次，而每次內插都會損失一點細節，第 4.3 節詳述)。

請特別留意矩陣相乘的順序：轉換是從右往左作用的。若寫成 $M = T \cdot S \cdot R$ ，代表先旋轉、再縮放、最後平移。順序不可交換——先旋轉再平移，與先平移再旋轉，結果完全不同(想像繞原點旋轉的圓心位置)。這是初學者最常見的錯誤之一。

「升一維換取統一性」這個技巧，其影響遠超出本章。第 4.4 節的相機模型、第 17 章 ROS 2 的座標轉換系統(tf2)，背後都是齊次座標。一台機器人身上有相機座標

系、機械臂座標系、底盤座標系、世界座標系，而 tf2 就是以齊次座標矩陣管理這些座標系之間的關係——當你在第 17 章看到 tf2 時，請記得它的根就在這裡。

程式 4-1 平移、旋轉、縮放與串接

```
import cv2
import numpy as np

img = cv2.imread("robot.jpg")
h, w = img.shape[:2]

# --- 平移:x 方向 +100,y 方向 +50 ---
M_t = np.float32([[1, 0, 100],
                  [0, 1, 50]]) # 2x3 仿射矩陣
shifted = cv2.warpAffine(img, M_t, (w, h))

# --- 旋轉:繞影像中心轉 30 度,順便縮放 0.8 倍 ---
M_r = cv2.getRotationMatrix2D(center=(w/2, h/2), angle=30, scale=0.8)
rotated = cv2.warpAffine(img, M_r, (w, h))

# --- 縮放:resize 是最常用的幾何轉換 ---
half = cv2.resize(img, None, fx=0.5, fy=0.5,
                  interpolation=cv2.INTER_AREA) # 縮小用 AREA(第 3 章)

# --- 串接:合成單一矩陣,只取樣一次 ---
R = np.vstack([M_r, [0, 0, 1]]) # 補成 3x3 齊次矩陣
T = np.float32([[1, 0, 100], [0, 1, 50], [0, 0, 1]])
M = T @ R # 先旋轉、再平移(從右往左)
combined = cv2.warpAffine(img, M[:2], (w, h))
# 比較:分兩次做 vs. 合成一次做,後者更快且細節損失更少
```

4.2 轉換家族：從剛體到透視

幾何轉換構成一個層層包含的家族，自由度愈高、能表達的變形愈多、但保持的性質愈少。理解這個家族譜，你才知道面對一個問題該選哪一種。

1. 剛體轉換(Rigid / Euclidean)：只有旋轉與平移，3 個自由度。保持長度與角度——物體不變形，只是換了位置與朝向。機器人的移動、物件的擺放姿態屬於此類。
2. 相似轉換(Similarity)：剛體再加上等比例縮放，4 個自由度。保持角度與形狀比例，但大小可變。照片的等比例縮放、無人機拉高後看到的地面屬於此類。
3. 仿射轉換(Affine)：再加上剪切與非等比例縮放， 2×3 矩陣、6 個自由度、需 3 組對應點。保持平行性——轉換前平行的線，轉換後仍平行(長度與角度可變)。
4. 透視轉換(Perspective / Homography): 3×3 矩陣、8 個自由度、需 4 組對應點。不保平行但保直線——平行線可以相交(這正是近大遠小的視覺效果)，而直線轉換後仍是直線。

4.2.1 仿射轉換：平行線的守護者

仿射轉換是平移、旋轉、縮放、剪切的任意組合。因為 6 個自由度對應 3 個點的對應關係，`cv2.getAffineTransform()` 只需要「轉換前後各三個點」就能求出矩陣。

典型應用有二。其一是影像對齊：把兩張有輕微位移旋轉的影像疊合(第 12 章的視訊穩定即屬此類)。其二是資料增強——第 14、15、16 章訓練神經網路時的隨機旋轉、平移、翻轉、剪切，本質上全是仿射轉換。這個連結值得記住：當你在第 15 章看到 `RandomRotation` 這類增強參數時，它呼叫的就是本章的技術。

4.2.2 透視轉換：拍歪的世界拉回正面

用相機斜拍一張 A4 紙，紙的影像不再是矩形而是梯形——原本平行的紙邊，在影像中相交了。要描述這種「近大遠小」的投影，就需要透視轉換，又稱單應性(Homography)。它需要 4 組對應點，由 `cv2.getPerspectiveTransform()` 求解。

透視轉換是本章最實用的工具，它的應用橫跨全書：文件掃描 App 的「拉正」功能、第 13 章車牌影像的正面化、機器人導航的鳥瞰圖、第 13 章影像拼接求兩張照片間的關係、以及第 13.4.3 節 RANSAC 求出的正是這個矩陣。

一個重要的觀念補充：單應性成立有其前提——要嘛拍攝的是一個平面(如紙張、地面、牆面)，要嘛相機只做純旋轉而不平移。若場景有明顯的深度變化而相機又平移了，單一的單應性就無法描述兩張影像的關係(這也是為什麼手持平移拍攝的照片較難拼接，而原地旋轉拍攝的全景圖很容易)。理解這個限制，能幫你判斷什麼時候該用透視轉換、什麼時候需要更複雜的三維方法。

轉換	自由度	對應點數	保持的性質	典型應用
剛體	3	2	長度、角度	物件姿態、機器人位姿
相似	4	2	角度、形狀比例	等比縮放、尺度變化
仿射	6	3	平行性、面積比	影像對齊、資料增強
透視	8	4	直線性、交比	文件拉正、鳥瞰圖、拼接

表 4-1 幾何轉換家族比較

程式 4-2 透視轉換：文件拉正

```
import cv2
import numpy as np

img = cv2.imread("tilted_doc.jpg")

# 斜拍文件的四個角(左上、右上、右下、左下)
src = np.float32([[168, 92], [583, 128], [612, 707], [95, 664]])

# 自動計算合理的長寬比，避免文件被不自然拉伸
def dist(a, b): return np.linalg.norm(a - b)
W = int((dist(src[0], src[1]) + dist(src[3], src[2])) / 2)
H = int((dist(src[0], src[3]) + dist(src[1], src[2])) / 2)
```

```

dst = np.float32([[0, 0], [W, 0], [W, H], [0, H]])

M = cv2.getPerspectiveTransform(src, dst)    # 3x3 透視矩陣
flat = cv2.warpPerspective(img, M, (W, H))
cv2.imwrite("document_flat.jpg", flat)      # 文件掃描 App 的核心

# 反向:也可把矩陣反轉,把正面圖貼回原始視角(擴增實境的原理)
M_inv = np.linalg.inv(M)

```

4.3 影像內插：轉換後的像素從哪裡來

4.3.1 為什麼要反向映射

直覺上，執行幾何轉換似乎應該「把每個輸入像素搬到它該去的位置」——這稱為正向映射。但這樣做會產生兩個嚴重問題：某些輸出位置沒有任何輸入像素落上去，形成破洞；而某些位置則有多個輸入像素重疊，不知該用哪一個。

因此 OpenCV 實際採用反向映射(Backward Mapping)：對輸出影像的每一個像素位置，用轉換矩陣的反矩陣反算「它來自輸入影像的哪裡」。這樣每個輸出像素都保證有值、且只有一個來源，沒有破洞也沒有衝突。

但新問題來了：反算出的來源座標幾乎都不是整數——像 (217.3, 154.8) 這樣的位置根本沒有現成的像素值。從鄰近的整數像素「估」出這個值，就是內插(Interpolation)。這也解釋了為什麼幾何轉換必然伴隨品質損失：每一次轉換都要內插一次，而內插本質上是猜測。

4.3.2 四種內插法

5. 最近鄰內插(INTER_NEAREST)：直接取最近的那個像素值。最快，但放大後鋸齒與方塊感明顯。它唯一但關鍵的優點是不會產生「新的數值」——因此處理標籤圖、分割遮罩(第 9、16 章)時必須用它。若對一張只有 0 與 1 的遮罩用雙線性內插，會產生 0.5 這種無意義的中間值。
6. 雙線性內插(INTER_LINEAR, OpenCV 預設)：取周圍 2×2 共 4 個像素，依距離加權平均。速度與品質的甜蜜點，絕大多數場合的首選。
7. 雙三次內插(INTER_CUBIC)：取周圍 4×4 共 16 個像素，以三次多項式加權。邊緣更平滑銳利，代價是約數倍的運算量，適合離線的高品質放大。
8. 區域內插(INTER_AREA)：以區域平均進行降取樣。它是縮小影像的建議選項——原因在第 3 章已經說明：縮小就是降取樣，不先做低通就會產生混疊，而 INTER_AREA 的區域平均正是內建的低通濾波。

口訣：放大用 LINEAR 或 CUBIC，縮小用 AREA，遮罩用 NEAREST。這個口訣背後串起了第 3 章(取樣定理與混疊)、第 5 章(平滑濾波)、第 9 章(分割遮罩)三章的觀念。

程式 4-3 四種內插法與多次轉換的代價

```
import cv2, numpy as np
```

```

img = cv2.imread("small_photo.jpg")

# --- 放大:比較三種內插的品質 ---
for name, flag in [("nearest", cv2.INTER_NEAREST),
                  ("linear", cv2.INTER_LINEAR),
                  ("cubic", cv2.INTER_CUBIC)]:
    big = cv2.resize(img, None, fx=4, fy=4, interpolation=flag)
    cv2.imwrite(f"up_{name}.png", big)
# 放大 4 倍後檢視:nearest 滿是方塊,cubic 邊緣最平滑

# --- 遮罩必須用 NEAREST,否則會產生無意義的中間值 ---
mask = np.zeros((100, 100), np.uint8)
mask[30:70, 30:70] = 1 # 標籤圖:只有 0 與 1

bad = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_LINEAR)
good = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_NEAREST)
print("雙線性後的唯一值:", np.unique(bad)) # 出現 0 與 1 以外的值!
print("最近鄰後的唯一值:", np.unique(good)) # 仍只有 0 與 1

# --- 多次轉換 vs. 合成一次:品質差異 ---
a = img.copy()
for _ in range(10): # 連續旋轉 10 次,每次 36 度
    M = cv2.getRotationMatrix2D((img.shape[1]/2, img.shape[0]/2), 36,
    1)
    a = cv2.warpAffine(a, M, img.shape[1::-1])
print("轉一圈後與原圖的 PSNR:", cv2.PSNR(img, a))
# 理論上轉回原位,實際上因十次內插而明顯劣化 → 應合成矩陣一次做

```

4.4 相機模型、鏡頭畸變與張氏標定

4.4.1 針孔相機模型

針孔相機模型把成像抽象成一句話：三維空間中的點沿直線投影穿過光心，落在成像平面上。以齊次座標表示，整個投影過程可寫成「內參矩陣 K 乘以外參 $[R|t]$ 乘以世界座標點」——注意這裡再次用到了 4.1.3 節的齊次座標，它讓三維到二維的投影也能寫成單一的矩陣乘法。

參數分成兩組，區分它們是理解校正的關鍵：

- 內部參數(Intrinsics)：屬於相機本身，包括焦距 f_x 、 f_y (以像素為單位)與主點 c_x 、 c_y (光軸與成像面的交點，理想上在影像中心)。只要相機與鏡頭組合不變，這組參數就固定，與相機放在哪裡無關。
- 外部參數(Extrinsics)：描述相機在世界中的位置與朝向，包括旋轉矩陣 R 與平移向量 t 。相機一移動，這組參數就改變。

這個區分有實務意義：換一顆鏡頭或調整變焦，必須重新校正內參；而把相機搬到另一個腳架上，內參不變、只有外參改變。在機器人系統中，內參通常在出廠時校正一次並存檔，外參則由第 17 章的 `tf2` 系統即時追蹤。

4.4.2 鏡頭畸變

真實鏡頭不是理想的針孔，光線經過鏡片組會產生畸變(Distortion)。徑向畸變沿半徑方向作用，以係數 k_1 、 k_2 、 k_3 建模：係數為負時影像向外鼓、直線變成外凸的弧線，稱為桶狀畸變(廣角鏡、GoPro、ESP32-CAM 的魚眼感都是它);係數為正時向內縮，稱為枕狀畸變，常見於望遠端。切向畸變(p_1 、 p_2)則來自鏡片與感光元件未能完全平行。

畸變對機器人與量測應用是致命的，因為它破壞了「直線成像仍是直線」這個假設：第 9 章的霍夫轉換車道偵測會把彎曲的車道線判為多段短線、第 13 章的特徵匹配位置會系統性偏移、第 17 章以像素換算實際距離的計算會失準——而且愈靠近畫面邊緣誤差愈大。因此本書的原則是：先校正，再談視覺量測。

4.4.3 張氏標定法實作

依叫獸開講的原理，OpenCV 的標定流程分四步：拍攝棋盤格多張、偵測角點、求解參數、去畸變。

程式 4-4 張氏標定法完整流程

```
import cv2
import numpy as np
import glob

CB = (9, 6)    # 棋盤格「內角點」數:10x7 格的板子內角點為 9x6

# 棋盤格角點的世界座標(z=0 平面,單位:格)
objp = np.zeros((CB[0]*CB[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CB[0], 0:CB[1]].T.reshape(-1, 2)

objpoints, imgpoints = [], []
for fname in glob.glob("calib/*.jpg"):    # 10~20 張不同方位
    gray = cv2.imread(fname, cv2.IMREAD_GRAYSCALE)
    ok, corners = cv2.findChessboardCorners(gray, CB, None)
    if ok:
        # 次像素精化:把角點定位精度提升到小數點以下
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
                                   (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
0.001))
        objpoints.append(objp)
        imgpoints.append(corners)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None)

print("重投影誤差(愈小愈好,<0.5 佳):", ret)
print("內參矩陣 K:\n", K)                # fx, fy, cx, cy
print("畸變係數:", dist.ravel())         # k1 k2 p1 p2 k3

# 存檔:整學期所有實驗共用(第 12、17 章都會用到)
np.savez("camera_params.npz", K=K, dist=dist)
```

```
# 去畸變:彎掉的直線拉回來
img = cv2.imread("calib/test.jpg")
undist = cv2.undistort(img, K, dist)
cv2.imwrite("undistorted.jpg", undist)
```

驗收標準是重投影誤差：把求得的參數反過來把世界點投影回影像，與實際偵測到的角點位置比較，取平均距離。一般以低於 0.5 像素為佳。若誤差偏大，常見原因有三：棋盤格不平整(貼在軟紙板上會微微彎曲)、照片模糊或動態模糊、方位變化不足(所有照片都從相似角度拍)。這正好回應了叫獸開講的思考題——多拍不同方位的照片，就是在對抗這些誤差來源。

4.4.4 從像素到公尺

校正的最終價值，是讓相機成為量測工具。一旦知道內參，加上一個已知的參考(例如物體在已知距離的平面上、或畫面中有已知尺寸的參考物)，就能把「像素」換算成「公釐」。這個換算是後續多章的基礎：第 7.6.3 節由產線速度與像素尺寸推算運動模糊長度、第 12.7.2 節由車輛的像素位移換算實際車速、第 17 章機器人以視覺估計障礙物距離——它們全都建立在本節的校正之上。

4.5 幾何轉換在全書中的角色

幾何轉換是本書中出現頻率最高的工具之一，值得在此把散落各章的應用收攏起來：

- 資料增強(第 14、15、16 章)：訓練模型時的隨機旋轉、平移、翻轉、縮放，是對抗過度擬合最有效的手段之一，而它們全是本章的仿射轉換。第 16 章 YOLO 訓練參數中的 `degrees`、`fliplr` 正是如此。
- 視訊穩定(第 12 章)：估計相鄰影格之間的抖動(以第 6 章相位相關或第 13 章特徵匹配)，再用本章的幾何轉換反向補償，把每一影格「拉回」穩定位置。
- 影像拼接(第 13 章)：求出兩張照片間的透視轉換，把一張拉到與另一張對齊後融合，即成全景圖。
- 物件定位(第 13 章)：特徵匹配加 RANSAC 求出的單應性矩陣，能把參考影像的四個角投影到場景中，框出物件的位置與姿態。
- 車牌辨識(第 13 章)：斜拍的車牌先以透視轉換拉正，字元分割與辨識才能可靠進行。
- 鳥瞰圖與導航(本章 Lab、第 12 章)：把地面透視轉換成俯視圖，是循跡機器人與車道偵測的標準前處理。
- 機器人座標系(第 17 章)：ROS 2 的 `tf2` 以齊次座標矩陣管理相機、機械臂、底盤、世界之間的座標轉換關係。

這份清單說明了一件事：本章不是一個「學完就過去」的獨立章節，而是往後每一章都會回頭取用的工具箱。建議你把程式 4-4 產出的相機參數檔妥善保存，它會陪你走完整個學期。

4.6 應用案例

4.6.1 案例一：文件掃描 App

情境：用手機斜拍一張紙本文件，自動偵測紙張邊界並拉正成正面的掃描檔。這是透視轉換最貼近生活的應用，完整流程串起本書多章：轉灰階與去雜訊(第 5 章)→ Canny 邊緣偵測(第 9 章)→ 找出最大的四邊形輪廓(第 9、10 章)→ 取四個角以透視轉換拉正(本章)→ 自適應門檻二值化成清晰的黑白掃描檔(第 9 章)。

工程細節值得留意：目標矩形的長寬比不能隨便設，否則文件會被不自然拉伸——程式 4-2 用「量測對邊長度取平均」的方式估計合理比例。若已知是 A4 紙，也可直接指定 1:1.414 的比例。這種「利用先驗知識提升結果」的思路，在工程上總是划算的。

4.6.2 案例二：機器人導航的鳥瞰圖

情境：循跡機器人或自駕車的前視相機，看到的地面是有透視變形的——遠處的車道線看起來會匯聚。若直接在原始影像上判斷「線在左邊還是右邊、彎曲程度多少」，會因透視而失真。

解法是逆透視映射(IPM)：選取地面上的一塊梯形區域，把它透視轉換成矩形，得到俯視的鳥瞰圖。在鳥瞰圖上，平行的車道線真的是平行的、彎道的曲率也能正確量測，後續的第 9 章霍夫轉換或曲線擬合都變得單純許多。這是車道保持與循跡機器人的標準前處理，也是本章 Lab 4-2 的目標。

前提條件必須說清楚：IPM 假設所有內容都在同一個平面(地面)上。因此地面上的車道線會被正確攤平，但立體的物件(其他車輛、行人)在鳥瞰圖中會被拉成怪異的長條——因為它們違反了平面假設。理解這個限制，你就知道鳥瞰圖適合用來分析路面標線，而不適合用來判斷障礙物的形狀。

4.6.3 案例三：資料增強的幾何操作

情境：訓練一個工件瑕疵偵測模型，但標註樣本只有兩百張。資料增強能以幾何轉換人工擴充資料的多樣性：隨機旋轉正負十五度、隨機平移百分之十、水平翻轉、隨機縮放。每一張原始影像因此能衍生出許多變化，逼迫模型學習「不管工件怎麼擺放都要認得」的穩健特徵。

但增強必須合理，這是最容易被忽略的一點：若你的工件永遠以固定方向進料，那麼旋轉一百八十度的增強反而是在教模型學習不會發生的情況，可能稀釋了有效的學習。同理，數字辨識任務不該做垂直翻轉(6 會變成 9 的樣子)、醫學影像不該做左右翻轉(器官有左右之分)。增強要模擬「真實世界可能出現的變化」，而非隨意變形。這個判斷力，是第 15 章訓練模型時的必修課。

4.6.4 案例四：相機校正在產線量測的價值

情境：以視覺量測工件尺寸，要求精度達到零點一毫米。此時鏡頭畸變絕不能忽略——未校正的廣角鏡頭在畫面邊緣可能有數個像素的位置偏差，換算成實際尺寸就是零點幾毫米的誤差，直接讓量測失效。

標準做法：出廠時以張氏標定法校正相機並存檔內參與畸變係數(程式 4-4)→ 每次取像後先去畸變 → 以已知尺寸的標準件或標定板建立「像素對實際尺寸」的換算比例 → 定期(如每季或更換鏡頭後)重新校正驗證。

這個案例呼應了叫獸開講的核心：在張氏標定法問世之前，要做到這件事需要昂貴的三維標定物，只有大企業玩得起；而現在，一張列印的棋盤格加上開源的 OpenCV，任何一間中小企業、任何一個學生專題都能達成工業級的量測校正。降低門檻的力量，莫過於此。

4.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
齊次座標	第 17 章	ROS 2 的 tf2 座標框架管理
仿射轉換	第 12、14-16 章	視訊穩定的運動補償;訓練時的資料增強
透視轉換/單應性	第 13 章	RANSAC 求單應性做物件定位與影像拼接
內插與 INTER_AREA	第 3、9、16 章	取樣定理與防混疊;遮罩須用 NEAREST
相機校正與內參	第 7、12、17 章	推算運動模糊長度;像素換算車速;機器人測距
鏡頭畸變	第 9、13、17 章	直線偵測失準;特徵位置偏移;量測誤差
鳥瞰圖(IPM)	第 9、12 章	車道偵測與循跡機器人的前處理

表 4-2 第 4 章與全書其他章節的觀念連結

程式實作 Lab 4

Lab 4-1 轉換家族與內插實驗

目標：建立對轉換與內插的直覺。(a)以程式 4-1 對一張照片分別做平移、旋轉、縮放，並驗證「先旋轉再平移」與「先平移再旋轉」結果不同;(b)實作矩陣串接：比較「分三次做」與「合成一次做」的執行時間與 PSNR;(c)執行程式 4-3 的「連續旋轉十次轉回原位」實驗，記錄 PSNR 的劣化程度，說明為什麼理論上該回到原圖卻沒有;(d)對一張分割遮罩分別用 LINEAR 與 NEAREST 放大，用 np.unique 驗證前者產生了無意義的中間值;(e)整理出一張「什麼情況用哪種內插」的決策表。

Lab 4-2 相機校正與機器人鳥瞰圖

目標：替課程相機建立專屬校正檔，並產生導航鳥瞰圖。(a)列印棋盤格 (OpenCV 官網可下載)，固定於硬板上——務必平整，不可貼在會彎曲的軟紙板;(b)拍攝 15 張涵蓋畫面四角、遠近與傾斜各異的照片;(c)執行程式 4-4，記錄重投影誤差與內參，將 K 與 dist 存成 npz 檔供整學期所有實驗使用;(d)去畸變後，在地面貼四張

定位貼紙圍成已知尺寸的矩形，以透視轉換產生俯視鳥瞰圖;(e)量化實驗：刻意只用 3 張照片重新校正，比較重投影誤差與 15 張時的差異，驗證叫獸開講思考題的結論;(f)ESP32-CAM 的廣角鏡畸變特別明顯，校正前後直線的彎曲程度差多少?請量化它。

Lab 4-3 文件掃描器

目標：做出手機掃描 App 的核心功能。(a)以程式 4-2 為基礎，用 `cv2.setMouseCallback` 讓使用者在影像上依序點四個角，取代寫死的座標;(b)依四點自動計算目標矩形的長寬比，避免文件被不自然拉伸;(c)串接第 3 章的成果——輸出前將文件轉灰階並提高對比，存成清晰的掃描檔;(d)挑戰題：第 9 章學會輪廓偵測後，回來把「手動點四角」升級成「自動找四角」，做成全自動掃描器;(e)延伸：實作反向轉換，把一張圖片貼到原始影像中那張紙的位置上——這正是擴增實境的基本原理。

Lab 4-4 資料增強管線(第 15 章前導)

目標：提前建立資料增強的觀念與工具。(a)寫一個 `augment(img)` 函式，隨機套用旋轉(正負 15 度)、平移(10%)、縮放(0.9 到 1.1 倍)、水平翻轉;(b)對一張影像產生 20 個增強版本並拼貼顯示，檢查是否有出現不合理的變形(如內容被轉出畫面外);(c)思辨題：針對三種不同任務——工件瑕疵偵測、手寫數字辨識、胸部 X 光判讀——各自判斷哪些增強該用、哪些不該用，並說明理由;(d)這個函式將在第 15、16 章訓練模型時直接使用，請存成共用模組。

本章重點整理

- 幾何轉換改變像素位置而非數值;齊次座標以「升一維」把平移、旋轉、縮放統一為矩陣乘法，多重轉換可預先相乘合成單一矩陣，一次取樣完成(更快且品質更好);矩陣相乘順序不可交換，從右往左作用。
- 轉換家族層層包含：剛體(3 自由度，保長度角度)、相似(4)、仿射(6 自由度、3 組對應點、保平行)、透視(8 自由度、4 組對應點、保直線)。單應性成立的前提是拍攝平面或相機純旋轉。
- 反向映射避免破洞與衝突，但來源座標非整數故需內插;NEAREST 最快且不產生新值(遮罩專用)、LINEAR 為預設甜蜜點、CUBIC 高品質放大、AREA 為縮小首選(內建低通防混疊，呼應第 3 章)。
- 針孔模型的內部參數(f_x 、 f_y 、 c_x 、 c_y)屬於相機本身，外部參數(R 、 t)描述相機位姿;換鏡頭需重新校正內參，搬動相機只改外參。
- 徑向畸變分桶狀(係數負，廣角鏡)與枕狀(係數正，望遠端)，另有切向畸變;畸變破壞「直線成像仍為直線」的假設，使邊緣偵測、特徵匹配、視覺量測全部失準，故須先校正再量測。
- 張氏標定法(1998 MSR-TR-98-71,2000 TPAMI)只需印一張棋盤格、從至少兩個未知方位拍攝即可校正，為 OpenCV `calibrateCamera` 之核心;以重投影誤差驗收，一般小於 0.5 像素為佳。

- 幾何轉換是全書最常取用的工具：資料增強(第 14-16 章)、視訊穩定(第 12 章)、影像拼接與物件定位(第 13 章)、車牌拉正(第 13 章)、鳥瞰圖(第 9、12 章)、ROS 2 座標系(第 17 章)。

習題

- 為什麼平移無法用 2×2 矩陣表示?齊次座標如何解決這個問題?統一之後帶來哪兩個好處?

提示與參考解答：因為平移是「加法」而非線性的矩陣乘法，縮放與旋轉可寫成 2×2 矩陣乘法，平移不行。齊次座標替每個點多加一維(x, y, 1)，使平移也能寫成 3×3 矩陣乘法。兩個好處：一、多個轉換可預先相乘合成單一矩陣，一次做完；二、只需取樣內插一次，避免多次內插造成的品質劣化。

- 寫出「先繞原點旋轉 90 度、再平移 (10, 20)」的合成 3×3 矩陣，並驗算點 (1, 0) 的落點。若順序顛倒，結果會相同嗎?

提示與參考解答：旋轉 90 度矩陣 $R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ ；平移 $T = \begin{bmatrix} 1 & 0 & 10 \\ 0 & 1 & 20 \\ 0 & 0 & 1 \end{bmatrix}$ 。合成 $M = T \cdot R$ (從右往左：先旋轉後平移)。點 (1, 0, 1) 經 R 得 (0, 1, 1)，再經 T 得 (10, 21)。順序顛倒 ($M = R \cdot T$) 結果不同：(1, 0) 先平移得 (11, 20)，再旋轉得 (-20, 11)。矩陣乘法不可交換，順序必須明確。

- 請比較剛體、相似、仿射、透視四種轉換的自由度、所需對應點數與保持的性質。

提示與參考解答：剛體：3 自由度、2 組對應點、保長度與角度。相似：4 自由度、2 組點、保角度與形狀比例。仿射：6 自由度、3 組點、保平行性與面積比。透視：8 自由度、4 組點、保直線性與交比(不保平行，故能表現近大遠小)。可對照表 4-1。

- 斜拍的棋盤，格線在影像中不再平行。應以仿射還是透視轉換拉正?為什麼?單應性成立的前提是什麼?

提示與參考解答：應用透視轉換。因為仿射保持平行性，無法描述「平行線相交」的透視效果；透視轉換不保平行但保直線，正好對應近大遠小的投影。單應性成立的前提：拍攝的是一個平面(如紙張、地面、牆面)，或相機只做純旋轉而不平移。若場景有深度變化且相機平移，單一單應性無法描述兩影像的關係。

- 為什麼幾何轉換採用反向映射而非正向映射?反向映射帶來什麼新問題?

提示與參考解答：正向映射(把輸入像素搬到目標位置)會產生兩個問題：某些輸出位置沒有任何輸入落上去而形成破洞，某些位置則多個輸入重疊而衝突。反向映射對每個輸出像素反算來源，保證每個輸出都有值且唯一。新問題是：反算出的來源座標幾乎都不是整數，必須從鄰近整數像素估值——這就是內插，也是幾何轉換必然伴隨品質損失的原因。

- 放大分割遮罩(數值只有 0 與 1)時，為什麼必須用最近鄰內插?用雙線性會發生什麼事?

提示與參考解答：因為最近鄰直接取最近像素的值，不會產生新數值，遮罩仍只有 0 與 1。雙線性是加權平均，會在邊界產生 0.3、0.5 這類中間值，而標籤圖的「0.5 類」毫無意義，後續處理會出錯。這也適用於第 9、16 章的分割結果與第 14 章的標籤資料。

- 縮小影像為什麼建議用 INTER_AREA?這與第 3 章的哪個定理有關?

提示與參考解答：縮小就是降低取樣率，依第3章的Nyquist取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊(假圖案)。INTER_AREA以區域平均進行降取樣，本身內建了低通濾波的動作，先把高頻濾掉再取樣，因此能有效抑制混疊，是縮小的建議選項。

8. 說明內部參數與外部參數的差別。換一顆鏡頭後，哪一組需要重新校正?把相機搬到另一個腳架上呢?

提示與參考解答：內部參數(f_x 、 f_y 、 c_x 、 c_y)屬於相機與鏡頭本身，描述成像的幾何特性，與相機放在哪裡無關。外部參數(R 、 t)描述相機在世界中的位置與朝向。換鏡頭或調整變焦會改變成像特性，必須重新校正內參;搬到另一個腳架只改變位置與朝向，內參不變、只有外參改變(在機器人中由第17章的tf2即時追蹤)。

9. GoPro 影像中筆直的地平線變成弧形，這是哪種畸變?對應的係數正負為何?為什麼畸變對視覺量測是致命的?

提示與參考解答：這是桶狀畸變(barrel distortion)，徑向畸變係數為負，影像向外鼓、直線變成外凸弧線，常見於廣角鏡、GoPro、ESP32-CAM。致命原因：它破壞了「直線成像仍為直線」的假設，使第9章霍夫直線偵測失準、第13章特徵位置系統性偏移、第17章像素換算實際距離產生誤差，且愈靠畫面邊緣誤差愈大。故須先校正再量測。

10. 張氏標定法相較「精密三維標定物」方法的核心優勢是什麼?實務上為何要拍10張以上?

提示與參考解答：核心優勢：只需一張列印的平面棋盤格，從至少兩個不同方位觀察即可，且相機或棋盤皆可自由移動、運動不需已知;傳統方法需要精密加工的兩個正交平面，昂貴且門檻高。張正友論文稱它「讓三維電腦視覺從實驗室往真實世界前進一步」。拍10張以上是為了對抗影像雜訊與角點定位誤差、並涵蓋畫面各角落以準確估計畸變;原論文實驗亦顯示估計的不確定度隨影像張數增加而下降。

11. 重投影誤差1.8像素算好嗎?請列出三個可能原因與對應的改善做法。

提示與參考解答：不好，一般以低於0.5像素為佳，1.8表示校正品質不佳。三個可能原因與對策：一、棋盤格不平整(貼在會彎曲的軟紙板上)——改貼在硬質平板;二、照片模糊或有動態模糊，角點偵測不準——用腳架、加強打光、縮短曝光;三、方位變化不足，所有照片角度相似——刻意涵蓋畫面四角、不同傾斜與遠近。另可檢查是否誤設棋盤格內角點數。

12. (綜合題)承4.6.3節：針對「工件瑕疵偵測」、「手寫數字辨識」、「胸部X光判讀」三種任務，請各自判斷旋轉180度與左右翻轉這兩種增強是否適用，並說明理由。

提示與參考解答：工件瑕疵偵測：若工件以固定方向進料，旋轉180度是在教模型學習不會發生的情況，可能稀釋有效學習，不建議;若工件會任意擺放則適用。左右翻轉通常可行(瑕疵無左右之分)。手寫數字辨識：旋轉180度不適用(6與9會混淆);左右翻轉不適用(鏡像數字不存在於真實資料)。胸部X光：旋轉180度不適用(解剖方向固定);左右翻轉不適用，因人體器官有左右之分(心臟偏左)，翻轉會產生醫學上錯誤的影像。原則：增強應模擬真實世界可能出現的變化。

參考資料與延伸閱讀

- Zhang, Z., A Flexible New Technique for Camera Calibration, IEEE TPAMI, 22(11), 1330-1334, 2000.(技術報告版 MSR-TR-98-71, 1998 年 12 月;本章叫獸開講之史料來源)
- Zhang, Z., Flexible Camera Calibration by Viewing a Plane from Unknown Orientations, ICCV, 1999.
- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2.4(幾何轉換與內插)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2、11(投影幾何與立體視覺)。
- Hartley, R., & Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge, 2004.(進階必讀)
- OpenCV 官方教學：Camera Calibration(含可下載之棋盤格圖檔)。
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 5 章 空間域影像強化

本章學習目標

- 區分點運算與鄰域運算，說明空間域強化的基本框架與它在全書中的定位。
- 運用負片、對數與 Gamma 校正等灰階轉換調整影像明暗與對比，並理解 LUT 最佳化。
- 讀懂直方圖，並以直方圖等化與 CLAHE 改善對比不足的影像。
- 正確定義卷積與相關，說明濾波器核、邊界處理與可分離濾波的運算量差異。
- 比較均值、高斯、中值、雙邊四種平滑濾波的特性與適用雜訊型態。
- 以 Laplacian、Unsharp Masking 與 High-Boost 進行影像銳化，並理解其代價。
- 說明多影格合成的原理，並理解計算攝影如何以軟體突破硬體限制。

叫獸開講|手機如何在黑夜裡看見：夜景模式的祕密

2018 年 11 月，Google 為 Pixel 手機推出 Night Sight(夜視模式)。它的能力在當時令人難以置信：在照度低至 0.3 勒克斯的環境——那是連人類視覺都已退化成模糊單色的暗度——不用腳架、不用閃光燈，手持就能拍出乾淨、銳利、色彩正確的照片。負責這項技術的是 Marc Levoy，史丹佛大學計算攝影的先驅、後來成為 Google 的傑出工程師。

低光攝影為什麼困難？Levoy 團隊在技術文件中解釋得很清楚：手機的鏡頭與感光元件都很小，收集到的光子本來就少；而主要的雜訊來源是進入鏡頭的光子數量本身的隨機波動(散粒雜訊，第 3、7 章)，加上電荷轉數字時的讀取雜訊，兩者共同拉低訊噪比。物理學給的解方很單純：訊噪比隨曝光時間的平方根上升，曝光愈久畫面愈乾淨。但問題是——手持拍照，你會抖；被拍的人，也在動。長曝光換來的是一張明亮的糊照。

Google 的答案不是換更大的鏡頭，而是換一種思路：連拍一疊短曝光影像，在軟體中對齊後合併。一張張「不夠亮但夠銳利」的照片疊起來，隨機的雜訊互相抵消，亮度與訊噪比同時提升。這個「以多換好」的想法，正是本章 5.6 節要展開的多影格合成。

Night Sight 把這套做法推到極致，其中幾個設計極具巧思。其一是運動測光(motion metering)：在你按下快門之前，系統就已經在量測手抖程度與場景中的運動量，據此預測接下來的運動並自動決定每張的曝光時間——場景夠靜時，單張曝光甚至可超過 300 毫秒。其二是影格數的動態調整：若偵測到手機放在腳架上就拍 6 張，手持則最多 15 張，並限制總擷取時間以免使用者失去耐心。其三是穩健合併：採用專為高雜訊影像設計的分塊對齊策略，能丟棄運動過大的區塊，

避免鬼影。其四是學習式白平衡——因為暗處的光源色溫難以判斷，他們訓練了專門針對夜景的演算法(第 8 章會回到這個主題)。

最後一個設計尤其耐人尋味：色調映射刻意受到繪畫的啟發。團隊發現，若只是單純把夜景調亮，結果會像大白天拍的，失去夜的氛圍。因此他們仿效畫家描繪夜景的手法——提高對比、把陰影壓到近乎全黑、讓畫面周圍沉入黑暗。這是一個很好的提醒：影像強化的目標，未必是「還原真實」，有時是「傳達感受」。這正是本章「強化」與第 7 章「還原」的根本差異。

拆開來看，Night Sight 用到的技術幾乎全在本章：多張平均去雜訊(5.6 節)、對比拉伸與色調曲線(5.1 節)、局部對比增強(5.2 節)，而對齊則用到第 6 章的相位相關與第 13 章的特徵匹配。學完這一章，你就有能力自己寫一個「窮人版夜景模式」——這正是 Lab 5-4 的任務。

思考題：「連拍多張再平均」為什麼能降低雜訊？如果雜訊是隨機且零均值的，平均 N 張後雜訊的標準差會變成原來的幾分之一？(提示：與叫獸開講中「訊噪比隨曝光時間平方根上升」是同一個數學原理)

5.1 空間域處理的框架與灰階轉換

5.1.1 本章的定位

空間域處理是指直接對像素數值進行運算，相對於第 6 章在頻率域的處理。它可寫成 $g(x, y) = T[f(x, y)]$ ，其中 f 是輸入影像、 g 是輸出影像、 T 是作用在 (x, y) 鄰域上的運算子。當鄰域只有一個像素時， T 退化為灰階轉換函數，稱為點運算(Point Operation)：輸出只取決於該像素自己的值，與鄰居無關。當鄰域擴大到 3×3 、 5×5 時，就成為 5.3 節的空間濾波(鄰域運算)。

這裡也要釐清本章與第 7 章的關係，因為兩者常被混淆。本章談的是影像強化(Enhancement)：目標是讓影像「看起來更好」或「更適合後續處理」，判準相當主觀——就像叫獸開講中 Night Sight 刻意把陰影壓黑以營造夜的氛圍，那並非還原真實，卻是更好的照片。第 7 章的影像還原(Restoration)則是客觀的逆問題：先建立劣化的數學模型，再反推最接近原始的估計。一句話總結：強化是化妝，還原是手術。

5.1.2 負片與對數轉換

負片轉換 $s = L - 1 - r$ (8 位元時為 $255 - r$) 把黑白顛倒。它在醫學影像中特別有用，X 光片上細小的白色病灶，反轉成黑色後在淺色背景中反而更醒目——這利用了人眼對暗背景上的亮物與亮背景上的暗物，敏感度並不不同的特性。

對數轉換 $s = c \cdot \log(1 + r)$ 則大幅拉伸暗部、壓縮亮部，適合處理動態範圍極大的影像。最經典的例子是第 6 章的傅立葉頻譜：直流分量的數值往往比其他成分大好幾個數量級，不取對數幾乎只看得到中央一個亮點。當你在第 6 章看到「顯示頻譜前要取對數」時，用的就是這裡的工具。

5.1.3 Gamma 校正與幕次法則

幕次法則轉換 $s = c \cdot r^\gamma$ 是實務上最常用的明暗調整工具。 γ 小於 1 時，暗部被拉亮(整體變亮、暗部細節浮現); γ 大於 1 時，暗部被壓暗(整體變暗、亮部細節保留)。

它的重要性不只在美觀，還有一段硬體的歷史淵源：早期映像管螢幕的亮度響應天生近似 $\gamma = 2.2$ 的幕次曲線，因此影像在儲存時預先做了反向的 Gamma 編碼來抵消。這就是為什麼 8 位元 JPEG 的像素值並非線性正比於光強度，而第 3 章提到的 RAW 檔保留的才是線性感測值。

這個差別在做精確的光度量測時會咬人：若你要判斷「這個區域的實際亮度是那個區域的幾倍」，直接用 JPEG 的像素值相除會得到錯誤答案，必須先做反 Gamma 轉換回到線性空間。這也是第 7 章影像還原、第 8 章白平衡計算時要留意的細節——許多演算法在推導時都假設輸入是線性的。

程式 5-1 灰階轉換：負片、Gamma、對數

```
import cv2
import numpy as np

img = cv2.imread("dark_room.jpg")

# --- 負片 ---
negative = 255 - img

# --- Gamma 校正:用查表(LUT)一次算好 256 個對應值,最快 ---
def gamma_correct(img, gamma):
    inv = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv) * 255
                      for i in range(256)]).astype("uint8")
    return cv2.LUT(img, table)

brighter = gamma_correct(img, 2.2) # gamma>1 → 提亮暗部
darker   = gamma_correct(img, 0.45) # gamma<1 → 壓暗

# --- 對數轉換(常用於顯示頻譜,第 6 章) ---
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
log_img = 255 * np.log1p(gray) / np.log1p(255)
log_img = log_img.astype(np.uint8)

# --- 反 Gamma:回到線性空間再做光度計算(第 7、8 章需要) ---
linear = np.power(img.astype(np.float32) / 255.0, 2.2)
```

請注意程式中的 `cv2.LUT()`：灰階轉換只有 256 種可能的輸入，先建好查表再一次對映，遠比對每個像素重算幕次快——這正是第 2 章「向量化思維」的延伸，也是影像處理最常見的最佳化手法之一。在第 17 章的 Jetson 或 ESP32 等邊緣裝置上，這類最佳化尤其值得。

5.2 直方圖處理

5.2.1 讀懂直方圖

影像直方圖統計每個灰階值出現的像素個數，是影像的「亮度身分證」。曝光不足的影像，直方圖擠在左側；過曝擠在右側；對比不足則集中在中間一小段；對比良好的影像，分布通常較為寬廣。

直方圖是影像分析的第一個診斷工具，它的用途貫穿全書：攝影師看它判斷曝光；工程師據此決定該做什麼處理；第 7 章用平坦區域的直方圖形狀來辨識雜訊型態（鐘形是高斯、兩端尖峰是椒鹽）；第 9 章的 Otsu 門檻分割更是直接建立在直方圖之上——它假設前景與背景各形成一個峰，並自動找出最佳的分界點。養成「拿到影像先看直方圖」的習慣，你會少走很多彎路。

5.2.2 直方圖等化

直方圖等化(Histogram Equalization)的目標是讓灰階分布盡可能均勻，以擴展對比。其核心是把累積分布函數(CDF)當作轉換函數：先統計各灰階的機率 $p(r_k) = nk / MN$ ，再計算累積機率並乘上 $L-1$ 得到映射值 $s_k = (L-1) \cdot \sum p(r_j)$ 。

直觀理解：某個灰階若聚集大量像素，CDF 在該處陡升，映射後鄰近灰階被拉開，對比自然增強；像素稀少的區段則被壓縮。換句話說，等化是把「有限的灰階資源」分配給「像素多的地方」——這與第 8 章色度降取樣、第 11 章 JPEG 量化把資源分配給人眼在意之處，是同一種資源分配的思維。

手算範例 ($L = 8$ ，共 64 像素)：設灰階 0 至 7 的像素數為 8、10、16、14、8、5、2、1。累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000；乘以 7 並四捨五入，得到映射 0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。原本擠在中低灰階的像素被推展到整個範圍，對比明顯提升——這就是等化的全部祕密。

但等化也有代價：它是「全自動、不可控」的，而且會放大平坦區域的雜訊（因為雜訊造成的微小灰階差異也被拉開了）。此外，映射後某些原本不同的灰階可能合併成同一階（如上例的 5、6、7 都映射到 7），造成細節喪失。這解釋了為什麼實務上很少直接用全域等化，而多採用下一節的 CLAHE。

5.2.3 CLAHE：局部才是王道

全域等化有個致命弱點：它用整張影像的統計量決定映射。若影像同時有明亮窗戶與陰暗室內，等化結果往往顧此失彼——這正是第 8 章「洋裝效應」與第 9 章打光不均問題在灰階世界的翻版。

CLAHE(對比度受限的自適應直方圖等化)的解法有二：其一，把影像切成小格(例如 8×8)，各自等化後再以雙線性內插消除格線接縫，如此亮區與暗區各自依自己的統計量調整；其二，設定對比限制(clipLimit)，把直方圖中過高的長條「削頂」並把超出的部分均攤到其他灰階，避免雜訊被暴力放大——這個「削頂」正是 CLAHE 名稱中 Contrast Limited 的由來，也是它優於一般自適應等化的關鍵。

實務上，CLAHE 幾乎總是比全域等化好用，是內視鏡影像、AOI 檢測、夜間監控的常客。本書後續多章的前處理(第 9 章分割前、第 16 章訓練資料準備)都會用到它。

程式 5-2 直方圖診斷、等化與 CLAHE

```
import cv2, numpy as np

gray = cv2.imread("lowcontrast.jpg", cv2.IMREAD_GRAYSCALE)

# 先看直方圖:診斷是第一步
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
print("最亮/最暗像素值:", gray.max(), gray.min())
print("集中區間佔比:", (hist[80:160].sum() / gray.size))

eq = cv2.equalizeHist(gray)          # 全域等化

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
cl = clahe.apply(gray)              # 局部等化(建議)

# --- 彩色影像的正確做法:只等化亮度通道,別碰色彩 ---
bgr = cv2.imread("lowcontrast_color.jpg")
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)    # L=亮度, a,b=色彩
l, a, b = cv2.split(lab)
l = clahe.apply(l)
result = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

# 錯誤示範:對 BGR 三通道分別等化 → 色偏
wrong = cv2.merge([cv2.equalizeHist(c) for c in cv2.split(bgr)])
```

程式最後兩段是常見考點：若對 BGR 三個通道分別等化，各通道的映射函數不同色彩比例會被破壞，產生詭異的色偏。正確做法是轉到亮度與色彩分離的色彩空間 (LAB 或 YCbCr)，只處理亮度通道——第 8 章會完整說明這些色彩空間的設計理由。

5.3 空間濾波：卷積與相關

5.3.1 定義與差異

空間濾波把一個小矩陣(稱為濾波器核、遮罩或 kernel)滑過影像，在每個位置與鄰域對應相乘後加總，作為輸出像素值。

這裡有兩個容易混淆的名詞：相關(Correlation)是直接對應相乘相加；卷積(Convolution)則先把核旋轉 180 度再做相同運算。對於對稱核(如高斯)，兩者結果完全相同，所以日常常被混用；但對非對稱核就會差一個翻轉。理論上要求卷積的原因是它滿足交換律與結合律，並對應第 6 章的卷積定理。實務提醒：OpenCV 的 cv2.filter2D 執行的其實是相關，需要真正卷積時得自己先翻轉核。

一個值得先建立的連結：第 15 章的卷積神經網路，其「卷積層」做的正是這件事——差別在於，本章的核是人設計的(高斯核、Sobel 核)，而 CNN 的核是從資料中學出來的。當你在第 15 章看到 Conv2D 時，請記得它的數學核心就在這一節。

5.3.2 邊界處理

核滑到影像邊緣時，一部分會伸出界外，必須決定「界外的值是什麼」。常見策略：補零 (BORDER_CONSTANT，簡單但邊緣會變暗)、複製邊緣像素 (BORDER_REPLICATE)、鏡射 (BORDER_REFLECT, OpenCV 預設之一，通常視覺效果最自然)、環繞 (BORDER_WRAP)。

這個選擇在單張照片上不痛不癢，但在三種情況下會造成問題：拼接時 (第 13 章) 接縫處會出現亮度不連續；批次分塊處理大圖時 (第 17 章 AOI 大圖分割檢測) 每塊的邊緣都有瑕疵；以及第 6 章的頻率域處理——DFT 隱含「影像週期性延拓」的假設，等同於 BORDER_WRAP，這會在頻譜上產生十字狀的假亮線。理解邊界處理，能幫你診斷許多莫名其妙的邊緣瑕疵。

5.3.3 可分離濾波與運算量

若一個 $m \times m$ 的核可以拆成一個垂直向量與一個水平向量的外積，就稱為可分離 (Separable)。此時先做一次水平一維卷積、再做一次垂直一維卷積，結果與二維卷積相同，但每個像素的乘法次數從 m^2 降為 $2m$ 。

以 15×15 的高斯核為例：225 次乘法降為 30 次，快了七倍以上；若是 51×51 ，則從 2601 降為 102，快了二十五倍。高斯核與均值核都是可分離的，OpenCV 的 GaussianBlur 內部就這麼做。

這個技巧是即時影像處理能在 Jetson 或手機上跑得動的關鍵之一，也預告了第 6.5.2 節的工程判準：當核大到連可分離都不划算時，就該轉到頻率域了。三者 (空間域直接卷積、可分離卷積、頻率域相乘) 構成一個依核大小選擇的完整光譜。

5.4 平滑濾波：對付雜訊

1. 均值濾波 (Box Filter)：鄰域取算術平均。最快最簡單，但邊緣與細節同樣被抹平，且方形核容易產生方向性偽影 (從第 6 章的觀點看，它的頻率響應是有振盪旁瓣的 sinc 型低通，這正是偽影的來源)。
2. 高斯濾波 (Gaussian Blur)：以高斯函數加權，中心權重最大、離得愈遠愈小。平滑效果自然，是最常用的平滑濾波；參數為核大小與標準差 sigma，sigma 愈大愈模糊。它同時是第 9 章 Canny 邊緣偵測的第一步、第 11 章小波多尺度與第 13 章 SIFT 尺度空間的基礎——高斯是整本書出現最頻繁的濾波器。
3. 中值濾波 (Median Blur)：取鄰域的中位數而非平均。對椒鹽雜訊 (隨機黑白點) 有奇效——因為極端值不會影響中位數；且比均值更能保留邊緣銳利度。缺點是運算較慢，且大核心會讓影像產生塊狀感。它是非線性運算，因此沒有對應的頻率響應 (第 6.6.2 節會說明這個限制的意義)。
4. 雙邊濾波 (Bilateral Filter)：同時考慮「空間距離」與「像素值差異」兩個權重，只平均那些「離得近且顏色也相近」的鄰居，因此能在去雜訊的同時保住邊緣。代價是運算量大；它正是手機美顏「磨皮但不糊掉五官」的原理，也是第 13 章車牌偵測前處理的常用選擇。

濾波器	擅長的雜訊	邊緣保留	速度	OpenCV 函式
-----	-------	------	----	-----------

均值	高斯雜訊(普通)	差	最快	cv2.blur
高斯	高斯雜訊(佳)	差	快	cv2.GaussianBlur
中值	椒鹽雜訊(極佳)	中等	中等	cv2.medianBlur
雙邊	高斯雜訊(佳)	優異	慢	cv2.bilateralFilter

表5-1 四種平滑濾波比較

程式 5-2 四種平滑濾波與效能比較

```
import cv2, numpy as np

img = cv2.imread("noisy.jpg")

mean = cv2.blur(img, (5, 5))
gauss = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)
median = cv2.medianBlur(img, 5) # 核大小須為奇數
bil = cv2.bilateralFilter(img, d=9,
                          sigmaColor=75, sigmaSpace=75)

# 實驗建議:在原圖上分別加高斯雜訊與椒鹽雜訊,
# 用第 3 章的 cv2.PSNR 量化四種濾波的還原成效
clean = cv2.imread("clean.jpg")
for name, out in [("均值", mean), ("高斯", gauss),
                  ("中值", median), ("雙邊", bil)]:
    print(f"{name}: PSNR = {cv2.PSNR(clean, out):.2f} dB")

# 驗證可分離濾波的效能(5.3.3 節)
import time
k = 31
t0 = time.time(); cv2.GaussianBlur(img, (k, k), 0)
print(f"高斯(可分離){time.time()-t0:.4f} 秒")
kernel = np.ones((k, k), np.float32) / (k * k)
t0 = time.time(); cv2.filter2D(img, -1, kernel)
print(f"filter2D(二維){time.time()-t0:.4f} 秒")
```

5.5 銳化濾波：強調細節

平滑靠平均，銳化則靠微分：影像中的邊緣就是灰階劇烈變化之處，而微分對這種變化最敏感。這也預告了第 9 章——邊緣偵測用的是同一套數學，只是目的從「強調邊緣」變成「找出邊緣的位置」。

5.5.1 Laplacian 銳化

拉普拉斯運算子(Laplacian)是二階微分的離散近似，常見核為中心 -4(或 -8)、四鄰為 1 的 3×3 矩陣。單獨使用 Laplacian 得到的是「邊緣圖」，把它加回原圖才是銳化： $g = f - c \cdot \nabla^2 f$ 。

二階微分的特性是各向同性(不分方向)、定位精準，但對雜訊極度敏感——因為微分兩次會把雜訊放大得更兇。這正是第 9.3.2 節 LoG(高斯拉普拉斯)要先做高斯平滑的原因，也是本節結尾「先去雜訊再銳化」原則的根據。

5.5.2 Unsharp Masking 與 High-Boost

更常用的路線是遮罩銳化(Unsharp Masking)，名字有點反直覺——它其實來自傳統暗房技術：先把影像模糊化，用原圖減去模糊版得到「細節層」(即被模糊抹掉的高頻)，再把細節層加權加回原圖。公式為 $g = f + k \cdot (f - \text{blur}(f))$ 。

當 $k = 1$ 稱為 Unsharp Masking, $k > 1$ 則稱為 High-Boost Filtering，能同時提亮並強化細節。從第 6 章的頻率觀點看，這正是「高頻強調濾波」——本章與第 6 章做的是同一件事，只是一個用空間域的減法實現、一個用頻率域有加權實現。這種「同一操作的兩張面孔」是理解影像處理的關鍵心法。

5.5.3 銳化的代價

銳化有三個必須知道的代價。第一，過衝(overshoot): k 太大時邊緣兩側會出現白邊與黑邊，俗稱光暈，看起來不自然——這與第 6 章的振鈴效應同源。第二，雜訊放大：高頻成分中雜訊佔了很大比重，銳化會一併放大它們。第三，無中生有的錯覺：銳化並沒有把模糊掉的資訊找回來，它只是把邊緣兩側的對比拉大，製造出「更清楚」的視覺印象——真正的還原是第 7 章的反卷積。

因此有一條實務鐵則：銳化之前要先適度去雜訊，順序不能顛倒。若先銳化再去雜訊，雜訊已經被放大，再平滑只會連同細節一起抹掉。這個「順序決定成敗」的原則，在第 9 章的 Canny(先平滑再求梯度)中會再次出現。

程式 5-3 銳化與正確的處理順序

```
import cv2, numpy as np

img = cv2.imread("soft.jpg")

# --- 方法一:Laplacian 銳化 ---
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
sharp1 = cv2.convertScaleAbs(img - 0.5 * lap)

# --- 方法二:Unsharp Masking / High-Boost(實務首選) ---
blur = cv2.GaussianBlur(img, (0, 0), sigmaX=3)
k = 1.5 # k>1 即 High-Boost
sharp2 = cv2.addWeighted(img, 1 + k, blur, -k, 0)

# --- 正確順序:先去雜訊,再銳化 ---
denoised = cv2.bilateralFilter(img, 9, 50, 50) # 保邊去雜訊
blur2 = cv2.GaussianBlur(denoised, (0, 0), 3)
good = cv2.addWeighted(denoised, 2.0, blur2, -1.0, 0)

# --- 錯誤順序:先銳化再去雜訊(雜訊已被放大) ---
bad = cv2.bilateralFilter(sharp2, 9, 50, 50)

cv2.imwrite("sharp_correct.jpg", good)
cv2.imwrite("sharp_wrong.jpg", bad)
# 並排比較:錯誤順序的結果細節較糊、雜點殘留較多
```

5.6 多影格合成：以軟體突破硬體限制

本節把叫獸開講的核心技術展開，它也是計算攝影 (Computational Photography) 這個領域的縮影。

5.6.1 為什麼平均能去雜訊

假設雜訊是隨機且零均值的(高斯雜訊符合此假設)，那麼對同一場景拍 N 張、逐像素平均後：訊號部分因為每張都相同，平均後不變；雜訊部分因為隨機且正負相消，其標準差降為原來的 N 平方根分之一。平均 4 張，雜訊減半；平均 16 張，雜訊剩四分之一；平均 100 張，剩十分之一。

這正是叫獸開講思考題的答案，也解釋了「訊噪比隨曝光時間平方根上升」的物理直覺——更長的曝光就是收集更多光子，而更多的獨立樣本平均起來，隨機波動自然變小。天文攝影界稱這個技術為「曝光堆疊」，已使用數十年。

5.6.2 多影格平均與空間平均的關鍵差異

這裡有一個極重要的對照，值得細細體會：5.4 節的空間平滑，平均的是「同一時間的不同像素」——但鄰近像素的訊號未必相同(邊緣兩側就完全不同)，因此平均會模糊邊緣。而多影格平均，平均的是「不同時間的同一個像素」——訊號理論上完全相同，只有雜訊在變。

結論是：多影格平均能在幾乎不損失任何細節的前提下降低雜訊，這是單一影格去雜訊永遠做不到的。這也是為什麼所有嚴肅的低光攝影、天文攝影、以及第 7 章的醫學影像，都盡可能採用多影格方案。

5.6.3 對齊：多影格方案的真正難點

但前提是「同一個像素」——手持拍攝必然有位移與旋轉，若不對齊就直接平均結果是一張模糊的照片。因此對齊才是多影格合成的真正技術核心，也是 Night Sight 所謂 robust merge 的重點。

對齊的方法本書後續都會學到：第 6.7.2 節的相位相關法(利用傅立葉的平移性質，快速且對亮度變化免疫)、第 13 章的特徵匹配加 RANSAC(能處理旋轉與透視變化)、以及第 12 章的光流。而 Night Sight 更進一步採用分塊對齊，並丟棄運動過大的區塊——因為場景中可能有移動的人或車，強行對齊會產生鬼影。

這條技術鏈完整地展示了本書各章如何協作：第 5 章提供合成的原理、第 6 或 13 章提供對齊的工具、第 4 章提供轉換的數學、第 3 章提供品質的評估。一個看似單純的「夜景模式」，背後是整本書的知識。

程式 5-4 窮人版夜景模式

```
import cv2, numpy as np, glob

# --- 窮人版夜景模式:多影格平均 ---
files = sorted(glob.glob("burst/*.jpg"))      # 連拍的一疊暗景
ref = cv2.imread(files[0])
acc = np.zeros_like(ref, dtype=np.float32)
```



```

n = 0

ref_gray = cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY).astype(np.float32)
win = cv2.createHanningWindow(ref_gray.shape[::-1], cv2.CV_32F)

for f in files:
    img = cv2.imread(f)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # 對齊:相位相關法求位移(第 6 章)
    (dx, dy), _ = cv2.phaseCorrelate(ref_gray, gray, win)
    M = np.float32([[1, 0, -dx], [0, 1, -dy]]) # 第 4 章仿射
    aligned = cv2.warpAffine(img, M, img.shape[1::-1])

    acc += aligned.astype(np.float32)
    n += 1

merged = (acc / n).astype(np.uint8)

# 後處理:提亮 + 局部對比 + 補回銳利度
merged = gamma_correct(merged, 2.0) # 5.1 節
lab = cv2.cvtColor(merged, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
l = cv2.createCLAHE(2.0, (8, 8)).apply(l) # 5.2 節
merged = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

blur = cv2.GaussianBlur(merged, (0, 0), 2)
merged = cv2.addWeighted(merged, 1.5, blur, -0.5, 0) # 5.5 節
cv2.imwrite("night_sight_poor_man.jpg", merged)

print(f"合併 {n} 張,理論雜訊降為 1/{np.sqrt(n):.1f}")

```

5.7 應用案例

5.7.1 案例一：工業影像前處理管線

情境：AOI 檢測系統要在打光不均的工件表面找出刮痕。若直接做第 9 章的門檻分割，亮區的背景會被誤判為前景、暗區的瑕疵則被漏掉。

標準管線：轉灰階 → 用 CLAHE 拉平局部對比(5.2.3 節)→ 用雙邊濾波或中值濾波去除雜點但保住瑕疵邊緣(5.4 節)→ 適度銳化強化瑕疵的邊緣對比(5.5 節)→ 交給第 9 章分割。

這個案例的關鍵教訓是：前處理的目標不是「讓影像好看」，而是「讓後續演算法好做」。有時候一張經過強化的影像在人眼看來反而怪異(對比過強、色彩失真)，但只要它讓分割與量測更準確，就是成功的前處理。這也是第 5 章與第 7 章在判準上的又一個差異——強化的成敗，取決於下游任務的表現。

5.7.2 案例二：醫學影像的視窗化與強化

情境：承第 3.7.2 節，CT 影像以 12 位元儲存(4096 階)，但螢幕只能顯示 256 階。醫師需要依觀察目標選擇「視窗」：看肺部用一組視窗中心與寬度、看骨骼用另一組、看軟組織再一組。

視窗化本質上就是本章的灰階轉換——把 12 位元中的某一段線性拉伸到完整的顯示範圍，超出範圍的一律設為全黑或全白。這說明了一個重要觀念：同一份資料，不同的轉換會揭示不同的資訊。這也是為什麼高位元深度的原始資料必須完整保存——它承載了顯示時尚未取用的資訊。

醫學應用也有其特殊的謹慎：過度的強化(尤其是銳化)可能製造出不存在的邊緣、或讓微小病灶的邊界看起來比實際清楚，誤導判讀。這與第 7 章「還原 vs. 腦補」、第 16 章的技術倫理是同一個主題：在高風險領域，「看起來更清楚」與「更接近真實」必須嚴格區分。

5.7.3 案例三：計算攝影與手機夜景

情境：承叫獸開講，現代手機的相機模組在物理上很受限——鏡頭小、感光元件小、進光量少。但拍出來的照片卻常常勝過某些專業相機的單張直出，原因就在計算攝影：用軟體與運算補上硬體的不足。

這個案例最能說明本章的價值定位。回顧全書的三種思路：硬體不足時，可以換更好的硬體(貴)、可以用數學還原(第 7 章，但受物理上限所限)、也可以「多拍幾張再合成」(本章 5.6 節)。第三條路往往最務實——它把「空間上的限制」轉換成「時間上的成本」，而時間通常比硬體便宜。這個「換一種資源來解決問題」的思維，值得學生反覆體會。

5.7.4 案例四：訓練資料的前處理與正規化

情境：承第 14 至 16 章，訓練深度學習模型前，所有影像都要經過統一的前處理這裡本章的技術扮演兩個角色。

其一是正規化：把像素值從 0 到 255 縮放到 0 到 1 或標準化為零均值單位變異數這在數學上是最單純的點運算(本章 5.1 節)，卻對訓練的收斂速度有顯著影響——因為神經網路的梯度下降對輸入尺度相當敏感。

其二是條件統一：若訓練資料來自不同的光照條件(有些在晴天拍、有些在陰天拍)，用 CLAHE 或白平衡統一它們，能減少模型需要學習的變異、提升泛化能力。但這裡有個微妙的取舍——若推論時的影像也會有同樣的光照變化，那麼「保留變化並用資料增強讓模型適應」可能比「強行統一」更好。這正是第 7.7.3 節與第 15 章討論過的「還原 vs. 資料增強」抉擇，在前處理階段的體現。

5.8 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
對數轉換	第 6 章	顯示傅立葉頻譜前的動態範圍壓縮

Gamma 與線性空間	第 3、7、8 章	RAW 為線性;還原與白平衡須留意非線性
直方圖	第 7、9 章	雜訊型態診斷;Otsu 自動門檻的基礎
CLAHE	第 9、10、16 章	分割前處理;AOI 打光不均;訓練資料統一
卷積與濾波核	第 6、15 章	卷積定理;CNN 的卷積層即可學習的核
高斯濾波	第 9、11、13 章	Canny 首步;小波多尺度;SIFT 尺度空間
中值與雙邊濾波	第 6、7、13 章	非線性故無頻率響應;去雜訊工具箱;車牌前處理
可分離濾波	第 6、17 章	運算量取舍;邊緣裝置的即時性
Unsharp / High-Boost	第 6、7 章	等同頻率域高頻強調;與反卷積的根本差異
多影格合成	第 6、12、13 章	相位相關與特徵匹配對齊;視訊的時間維度

表5-2 第5章與全書其他章節的觀念連結

程式實作 Lab 5

Lab 5-1 灰階轉換與直方圖診斷

目標：建立「先診斷、再開藥」的習慣。(a)準備三張影像：曝光不足、過曝、對比不足，各自繪製直方圖並描述其形狀特徵;(b)對每張套用適當的處理(Gamma、對數、等化)，說明你的選擇理由;(c)實作程式 5-1 的 LUT 版 Gamma 校正，與「逐像素用 np.power 計算」比較執行時間(呼應第 2 章向量化);(d)驗證 Gamma 編碼的非線性：對一張 RAW(或以 $\gamma=1/2.2$ 模擬的線性影像)與其 JPEG 版本，比較「兩個區域的亮度比值」是否相同;(e)結論：寫出一張決策流程圖——看到什麼樣的直方圖，該用哪種轉換？

Lab 5-2 等化、CLAHE 與彩色陷阱

目標：掌握對比強化的正確做法。(a)手算驗證 5.2.2 節的範例($L=8$ 、64 像素)，再用程式驗證你的答案;(b)對一張「同時有明亮窗戶與陰暗室內」的影像，比較全域等化與 CLAHE 的結果，說明前者為何顧此失彼;(c)調整 CLAHE 的 clipLimit(1、2、4、10)與 tileGridSize(4×4 、 8×8 、 16×16)，觀察各自的影響並找出最佳組合;(d)彩色陷阱實驗：對同一張彩色影像，分別用「BGR 三通道各自等化」與「轉 LAB 只等化 L 通道」，並排比較色偏程度;(e)觀察等化在平坦區域放大雜訊的現象，並思考如何緩解。

Lab 5-3 濾波器擂台

目標：為第 7 章的去雜訊工具箱打底。(a)對一張乾淨影像分別加入高斯雜訊與椒鹽雜訊;(b)用均值、高斯、中值、雙邊四種濾波各處理一次，製作 2×4 的 PSNR 與 SSIM 比較表(用第 3 章 Lab 3-3 建立的評估函式);(c)寫出結論：哪種雜訊該配哪種濾波器?為什麼中值對椒鹽近乎完美、對高斯卻無特別優勢?(d)實測可分離濾波：比較 GaussianBlur 與同尺寸 filter2D 的執行時間，核大小從 3 到 51，繪製曲線;(e)銳化順序實驗：比較「先去雜訊再銳化」與「先銳化再去雜訊」的結果，以 PSNR 與目視雙重驗證。

Lab 5-4 窮人版夜景模式

目標：重現叫獸開講的核心技術。(a)用手機或 USB 相機以固定姿勢連拍同一暗景 16 張;(b)先不對齊直接平均，計算雜訊標準差並驗證思考題：平均 16 張後是否約降為四分之一?(c)手持連拍(必然有晃動)，比較「直接平均」與「先用相位相關對齊再平均」的清晰度差異;(d)以程式 5-4 完成完整管線：對齊平均 → Gamma 提亮 → CLAHE 局部對比 → Unsharp 補銳度;(e)以第 3 章的 PSNR/SSIM 比較「單張」、「多張平均」、「完整管線」三者相對於長曝光參考照的差距;(f)進階挑戰：在畫面中放一個移動的物體，觀察直接平均會產生什麼鬼影，並思考 Night Sight 的「分塊對齊並丟棄運動區塊」如何解決它。

本章重點整理

- 空間域處理框架 $g = T[f]$;鄰域為單一像素時稱點運算(灰階轉換)，擴大鄰域即空間濾波。本章屬影像強化(主觀、讓影像好看或好用)，與第 7 章的還原(客觀、逼近原始)判準不同。
- 灰階轉換：負片(醫學影像)、對數(壓縮動態範圍、顯示頻譜)、Gamma 冪次法則($\gamma > 1$ 提亮暗部);螢幕 $\gamma = 2.2$ 的歷史使 JPEG 非線性，精確光度計算須先轉回線性;以 LUT 查表實作最有效率。
- 直方圖是診斷工具，也是第 7 章雜訊辨識與第 9 章 Otsu 門檻的基礎;等化以累積分布函數為映射，但會放大雜訊且不可控;CLAHE 分格局部等化並以 clipLimit 削頂抑制雜訊，實務上優於全域等化;彩色影像應只等化亮度通道。
- 卷積需將核旋轉 180 度，相關則否(OpenCV filter2D 實為相關);CNN 的卷積層即可學習的濾波核。邊界處理影響拼接、分塊與頻域結果;可分離濾波將 m^2 次乘法降為 $2m$ 次，是即時處理的關鍵。
- 平滑濾波：均值(快但有偽影)、高斯(自然，全書最常用)、中值(專治椒鹽、非線性)、雙邊(保邊緣、磨皮原理)。
- 銳化基於微分：Laplacian 為二階微分(對雜訊極敏感);Unsharp Masking 以原圖減模糊得細節層再加回， $k > 1$ 即 High-Boost，等同頻率域的高頻強調。銳化會過衝與放大雜訊，且未真正找回資訊，必須先去雜訊再銳化。
- 多影格平均使雜訊標準差降為 N 平方根分之一;相較空間平均(同時刻不同像素，會模糊邊緣)，多影格平均(不同時刻同一像素)幾乎不損失細節，但需先對齊(第 6、12、13 章的技術)。

習題

1. 點運算與鄰域運算的差別是什麼?各舉兩個例子。本章的「強化」與第 7 章的「還原」在判準上有何根本不同?

提示與參考解答：點運算的輸出只取決於該像素自己的值(如負片、Gamma 校正、直方圖等化);鄰域運算則取決於周圍一片像素(如高斯模糊、中值濾波、銳化)。強化追求主觀的「看起來更好或更適合後續處理」,如Night Sight 刻意壓黑陰影以營造夜的氛圍;還原則是客觀的逆問題,先建立劣化模型再反推最接近原始的估計,可用PSNR、SSIM 量化驗收。一句話：強化是化妝,還原是手術。

2. Gamma 校正中, $\gamma = 2.2$ 與 $\gamma = 0.45$ 分別使影像變亮或變暗?為什麼 JPEG 的像素值不是線性正比於光強度?這在什麼情況下會造成問題?

提示與參考解答：公式 $s = c \cdot r^\gamma$ 中,實作上常以 $1/\gamma$ 為指數(如程式 5-1);依課文定義, $\gamma > 1$ 提亮暗部、 $\gamma < 1$ 壓暗。JPEG 非線性的原因：早期映像管螢幕的亮度響應近似 $\gamma=2.2$ 的冪次曲線,影像儲存時預先做了反向的Gamma 編碼來抵消。問題發生在精確光度量測時——直接用JPEG 像素值計算「亮度比值」會得到錯誤答案,須先反Gamma 轉回線性空間(第 7 章還原、第 8 章白平衡皆須留意)。

3. 為什麼顯示傅立葉頻譜前通常要取對數?這利用了對數轉換的什麼特性?

提示與參考解答：因為頻譜的直流分量(整張影像的平均亮度)數值往往比其他成分大好幾個數量級,不取對數的話整張頻譜圖只看得到中央一個亮點、其餘全黑。對數轉換大幅拉伸暗部、壓縮亮部,能壓縮極大的動態範圍,讓微弱的高頻成分也看得見。這是對數轉換「適合處理動態範圍極大影像」的最佳實例。

4. 某 $L = 8$ 的影像共 64 像素,各灰階像素數為 8、10、16、14、8、5、2、1。請計算直方圖等化後的映射表,並說明會發生什麼細節損失。

提示與參考解答：累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000;乘以 $7(L-1)$ 並四捨五入得映射：0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。細節損失：原本不同的灰階 5、6、7 都映射到 7,合併成同一階,這三階之間的差異永久消失。這說明等化雖擴展了整體對比,卻可能犧牲高灰階區的細節。

5. 說明 CLAHE 相較全域等化的兩項改良,並解釋 clipLimit 的作用與名稱由來。

提示與參考解答：改良一：分格局部等化(如 8×8)，各格依自己的統計量調整,再以雙線性內插消除接縫,解決「同時有明亮窗戶與陰暗室內時全域等化顧此失彼」的問題。改良二：對比限制——設定 clipLimit 把直方圖中過高的長條削頂,並把超出部分均攤到其他灰階,避免平坦區的雜訊被暴力放大。名稱 Contrast Limited 正來自這個削頂機制。

6. 對彩色影像的 B、G、R 三通道分別做直方圖等化,會產生什麼問題?正確做法為何?

提示與參考解答：三個通道的映射函數各不相同,原本的色彩比例會被破壞,產生詭異的色偏。正確做法：轉到亮度與色彩分離的色彩空間(LAB 或 YCbCr),只對亮度通道(L 或 Y)做等化或 CLAHE,色彩通道保持不變,再轉回 BGR。第 8 章會說明這些色彩空間的設計理由。

7. 卷積與相關的差別是什麼?什麼情況下兩者結果相同?CNN 的卷積層與本章的空間濾波有何異同?

提示與參考解答：卷積先把核旋轉180度再做對應相乘相加，相關則直接相乘相加。對稱核(如高斯)兩者結果相同。理論上要求卷積是因為它滿足交換律與結合律，並對應第6章的卷積定理；OpenCV的filter2D實際執行的是相關。CNN卷積層的數學運算與本章相同，關鍵差異在於：本章的核是人設計的(高斯、Sobel)，CNN的核是從資料中學出來的(第15章)。

8. 一個 11×11 的可分離高斯核，相對於直接二維卷積，每個像素可省下多少次乘法？這個技巧為什麼對第17章重要？

提示與參考解答：二維卷積每像素需 $11 \times 11 = 121$ 次乘法；可分離則先水平再垂直各做一維，需 $11 + 11 = 22$ 次，省下99次，約快5.5倍。核愈大差距愈懸殊(51×51 時從2601降為102)。對第17章重要，是因為邊緣裝置(Jetson、ESP32)算力有限，即時處理必須錙銖必較；而當核大到連可分離都不划算時，就該考慮轉到第6章的頻率域。

9. 影像同時受到椒鹽雜訊污染且需保留邊緣，你會選哪一種平滑濾波？為什麼中值濾波對椒鹽近乎完美，對高斯雜訊卻無特別優勢？

提示與參考解答：選中值濾波。因為椒鹽雜訊是極端值(0或255)，排序後落在最前或最後，永遠不會被選為中位數，故能完全去除且不模糊邊緣。對高斯雜訊則無特別優勢：高斯雜訊是圍繞真值的小幅隨機起伏，並非極端值，取中位數與取平均的效果相近，而均值濾波在此情況下反而是統計上的最佳線性估計。

10. 說明Unsharp Masking的公式與步驟。為什麼銳化應該在去雜訊之後執行？過度銳化會出現什麼現象？銳化有沒有真正「找回」被模糊掉的資訊？

提示與參考解答：公式 $g = f + k \cdot (f - \text{blur}(f))$ ：先模糊、原圖減模糊得細節層(高頻)、再加權加回。順序理由：高頻成分中雜訊佔比重，銳化會放大雜訊；若先銳化再去雜訊，雜訊已被放大，再平滑只會連同細節一起抹掉。過度銳化出現過衝(邊緣兩側的白邊黑邊，俗稱光暈，與第6章振鈴同源)。銳化並未找回資訊，它只是把邊緣兩側的對比拉大製造「更清楚」的視覺印象；真正的還原是第7章的反卷積。

11. 為什麼多影格平均能降低雜訊？平均25張後雜訊標準差降為原來的幾分之一？它與空間平滑相比有什麼根本優勢？

提示與參考解答：若雜訊隨機且零均值，平均 N 張後訊號不變、雜訊正負相消，標準差降為原來的 N 平方根分之一；平均25張則降為五分之一。根本優勢：空間平滑平均的是「同一時刻的不同像素」，而鄰近像素的訊號未必相同(邊緣兩側就不同)，故會模糊細節；多影格平均的是「不同時刻的同一像素」，訊號理論上完全相同，因此能在幾乎不損失細節的前提下降噪。

12. (綜合題)承叫獸開講：請說明Night Sight用到了本章與本書哪些技術，並解釋為什麼「對齊」是多影格方案的真正難點。

提示與參考解答：本章技術：多影格平均去雜訊(5.6節)、Gamma與色調曲線提亮(5.1節)、局部對比增強(5.2節)、銳化補回細節(5.5節)。其他章節：對齊用第6.7.2節相位相關法或第13章特徵匹配、第4章的幾何轉換做補償、第3章的PSNR/SSIM評估、第8章的白平衡(Night Sight用學習式演算法)。對齊是難點的原因：手持必有位移旋轉，不對齊直接平均會得到模糊照片；而場景中可能有移動的人車，強行整體對齊會產生鬼影，故Night Sight採用分塊對齊並丟棄運動過大的區塊(robust merge)。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 3(空間域強化之標準參考)。
- Levoy, M., & Pritch, Y., Night Sight: Seeing in the Dark on Pixel Phones, Google Research Blog, Nov. 2018.(本章叫獸開講之技術來源)
- Liba, O., et al., Handheld Mobile Photography in Very Low Light, ACM TOG, 2019.(Night Sight 完整論文;運動測光與學習式白平衡)
- Hasinoff, S. W., et al., Burst Photography for High Dynamic Range and Low-Light Imaging on Mobile Cameras, ACM TOG, 2016.(HDR+ 原始論文)
- Zuiderveld, K., Contrast Limited Adaptive Histogram Equalization, Graphics Gems IV, 1994.(CLAHE 原始文獻)
- Tomasi, C., & Manduchi, R., Bilateral Filtering for Gray and Color Images, ICCV, 1998.
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 6 章 頻率域影像處理

本章學習目標

- 以「訊號是多個弦波疊加」的觀點理解傅立葉轉換，說明空間頻率在影像中的意義。
- 計算與顯示二維離散傅立葉轉換(DFT)的頻譜，並正確執行頻譜中心化與對數壓縮。
- 解讀頻譜圖：由亮線方向、亮點位置反推影像中的紋理、週期干擾與方向性結構。
- 解釋振幅譜與相位譜各自攜帶的資訊，說明為何相位對影像結構更關鍵。
- 設計並實作理想、Butterworth 與高斯的低通、高通、帶通與帶阻濾波器，理解振鈴效應的成因。
- 以陷波濾波去除週期性雜訊與摩爾紋，並以相位相關法完成影像對齊。
- 陳述卷積定理，並據以判斷同一個濾波工作該在空間域或頻率域執行。
- 說明頻率域觀點與取樣定理、空間濾波、影像還原、特徵擷取、影像壓縮之間的連結。

叫獸開講|傅立葉：一篇被退稿的論文，兩百年後撐起你的每一張 JPEG

1807 年 12 月，一位法國數學家把一份關於熱傳導的長篇手稿送進巴黎的法蘭西學院。他叫 Jean-Baptiste Joseph Fourier(傅立葉)，履歷相當奇特：革命年代差點上斷頭台，曾隨拿破崙遠征埃及，回國後被派去當省長，而這篇改變世界的論文，是他在管理行政事務之餘寫出來的。手稿的核心主張大膽到近乎狂妄：任何函數，哪怕是不連續的、有稜有角的，都可以表示成一系列正弦與餘弦函數的疊加。

審查委員的陣容堪稱豪華：拉普拉斯(Laplace)、拉格朗日(Lagrange)、蒙日(Monge)、拉克魯瓦(Lacroix)——三位都是傅立葉在高等師範學院的老師，蒙日還是他遠征埃及的同袍。然而，論文被擋下了。史料記載，雖然拉普拉斯、拉克魯瓦與蒙日傾向支持出版，拉格朗日堅決反對，因為傅立葉處理三角級數的方式與他自己在 1750 年代所主張的大不相同；審查者也批評證明不夠嚴謹、不夠一般化。1811 年，科學院以熱傳導為題舉辦論文獎，傅立葉提交修訂版並奪得首獎，卻仍附帶「缺乏嚴謹性」的評語。傅立葉認為批評並不公平，繼續修改；完整的《熱的解析理論》直到 1822 年才出版——那時拉格朗日已經過世。

兩百年後回看這場爭論：拉格朗日的質疑在數學上並非全無道理(級數收斂的嚴格條件確實要等後人補上)，但傅立葉的直覺是對的，而且對得驚人。今天，傅立葉轉換是 JPEG 影像壓縮、MP3 音訊、MRI 醫學造影、5G 通訊、地震波分析共

同的數學心臟。你手機裡每一張照片、每一通電話，都在替這篇被退稿的論文作證。

這一章要教的觀念，說穿了就是換一個角度看世界：空間域是「實體」——你看到的是一個個像素的明暗；頻率域是「思想」——你看到的是這張影像由哪些「變化的快慢」組成。有些問題在空間域怎麼看都無解（例如布滿全圖的週期性條紋雜訊），換到頻率域卻一眼就能揪出元凶，伸手把它拔掉。學會在兩個域之間自由切換，是影像工程師的成年禮。

而且這一章的影響力遠不止於本章：第 3 章的取樣定理、第 5 章每一個濾波器、第 7 章的影像還原、第 11 章的 JPEG 壓縮與小波、第 13 章的特徵擷取，甚至第 15 章卷積神經網路為什麼有效——背後都站著傅立葉。6.6 節會把這些線索一次串起來。

思考題：如果拉格朗日當年沒有擋下這篇論文，傅立葉分析會更早成熟嗎？從這段歷史看，學術審查的「保守」是阻力還是必要的品質關卡？

6.1 從弦波到影像：傅立葉的基本觀念

6.1.1 一維直覺：從聲音開始

先想像聲音：一個和弦聽起來是單一的聲響，但它其實是好幾個不同頻率的音同時響起。訓練有素的耳朵能「聽出」其中有 Do、Mi、Sol 三個音，以及各自的響度——傅立葉分析做的就是這件事的數學版本。任何週期訊號都可以分解成不同頻率的正弦與餘弦之和；非週期訊號則以積分形式的傅立葉轉換處理。轉換的結果不再以「時間」或「位置」為軸，而是以「頻率」為軸，稱為頻譜(Spectrum)。

這個「同一個東西，兩種表示法」的觀念極為重要，值得用一個比喻鞏固：同一首曲子，可以用錄音波形表示(時域)，也可以用五線譜表示(頻域)。兩者資訊完全等價、可以互相轉換，但適合的用途不同——想剪掉開頭三秒，看波形最方便；想把整首曲子升一個八度，看樂譜最方便。影像處理也是如此：想裁切一塊區域，用空間域；想去除一組固定頻率的干擾，用頻率域。

6.1.2 影像中的「頻率」是什麼

影像沒有時間軸，它的頻率指的是空間頻率——灰階隨位置變化的快慢，單位可以想成「每單位距離有幾個明暗週期」。低頻對應緩慢變化的成分：平坦的牆面、漸層的天空、物體的大塊區域、整體亮度；高頻對應劇烈變化的成分：邊緣、細節紋理、文字筆畫，以及雜訊。

這個對應關係是整章的鑰匙，也是所有頻率域操作的設計邏輯：想模糊影像，就削弱高頻(低通濾波)；想強調邊緣，就保留或放大高頻(高通濾波)；想抓出特定粗細的紋理，就只留下某個頻帶(帶通濾波)；想去除規律的條紋干擾，就在頻譜上找到那幾個異常亮點，精準消除(帶阻或陷波濾波)。四種濾波器，四種目的，全部都是同一句話的不同版本：決定哪些「變化的快慢」可以留下來。

6.1.3 基底影像：每個頻率成分長什麼樣子

二維傅立葉轉換把影像分解成一組「基底影像」的加權和，而每一張基底影像，都是一張斜條紋圖。頻率座標 (u, v) 決定這張條紋圖的三個性質：條紋的疏密由 u 與 v 的合成大小(離頻譜中心的距離)決定，離中心愈遠、條紋愈密；條紋的方向由 u 與 v 的比例(相對於中心的角度)決定，且條紋方向與頻譜上該點的方位垂直；而係數的振幅與相位，則決定這張條紋圖在原圖中占多少比重、以及擺放的位移。

一張影像，就是無數張這種條紋圖疊加而成——這正是傅立葉當年主張的「任何函數都能用弦波疊加表示」在二維的版本。建議學生務必在 Lab 6-1 中親手驗證：先用 `np.sin` 合成一張純條紋圖，看它的頻譜是否真的只有兩個對稱亮點；再把條紋轉 45 度，看亮點是否跟著轉。這個實驗做過一次，對頻譜的直覺就永遠建立起來了。

順帶一提「兩個對稱亮點」的由來：實數影像的傅立葉轉換具有共軛對稱性， $F(-u, -v)$ 是 $F(u, v)$ 的共軛複數，因此頻譜圖必然以中心點對稱。這個性質不只是數學細節——6.4 節設計陷波濾波器時，忘了處理對稱點是最常見的錯誤。

6.1.4 本章在全書中的位置

頻率域是本書的樞紐觀念之一，它向前解釋了第 3、5 章的許多「為什麼」，向後支撐了第 7、11、13、15 章的核心方法。閱讀時建議隨時對照：

- 與第 3 章(取樣與量化):Nyquist 取樣定理本身就是頻域敘述——取樣頻率不足時，高頻成分在頻譜上「摺疊」回低頻區偽裝成假訊號，這就是混疊。摩爾紋是它最常見的視覺表現。
- 與第 5 章(空間域強化)：每一個空間濾波核都有一張「頻率面孔」。均值濾波是粗糙的低通、高斯濾波是平滑的低通、拉普拉斯是高通、Unsharp Masking 是高頻強調。理解這一點，調參就不再是盲目試誤。
- 與第 7 章(影像還原)：劣化模型 $g = h * f + n$ 在頻率域是 $G = H \cdot F + N$ ，卷積變成乘法，還原變成除法；而退化函數 H 本身就是一個低通濾波器。
- 與第 9 章(影像分割):Canny 的第一步高斯平滑是低通(壓制雜訊)，第二步梯度運算是高通(找邊緣)——一個演算法內同時用到兩種濾波，順序不能顛倒。
- 與第 11 章(小波與正交轉換):DCT 是 JPEG 的核心，與 DFT 同宗；小波則補上傅立葉最大的弱點——它只告訴你「有哪些頻率」，不告訴你「頻率出現在哪裡」。
- 與第 13 章(特徵擷取):Gabor 濾波器是帶通加方向選擇的紋理特徵萃取器；SIFT 的高斯尺度空間本質上是一組低通濾波器；相位相關法則直接用頻域相位做影像對齊。
- 與第 15 章(深度學習):CNN 的卷積核在訓練後，常自發演化成邊緣偵測器與方向性帶通濾波器——與人工設計的濾波器驚人地相似。
- 與第 17 章(邊緣部署):FFT 的運算成本、記憶體需求與 GPU 加速可行性，是即時系統設計必須評估的項目。

6.2 二維離散傅立葉轉換(DFT)與 FFT

6.2.1 定義與快速演算法

對 $M \times N$ 的數位影像 $f(x, y)$ ，二維離散傅立葉轉換定義為 $F(u, v) = \sum_x \sum_y f(x, y) \cdot \exp[-j2\pi(ux/M + vy/N)]$ ，反轉換則加上 $1/(MN)$ 的係數與正號指數。直接依定義計算的複雜度是 $O(M^2N^2)$ ，對一張 1024×1024 的影像大約需要一兆次運算，根本跑不動；1965 年 Cooley 與 Tukey 提出的快速傅立葉轉換(FFT)利用了指數項的對稱性與週期性，以分而治之的方式把複雜度降到 $O(MN \cdot \log(MN))$ ——同樣一張影像降到約兩千萬次運算，快了五萬倍。這個演算法讓頻率域處理從理論走進實用，被公認為二十世紀最重要的演算法之一。

實作上還有一個效率細節：FFT 在長度為 2 的冪次時效率最高，因此有些函式庫會建議把影像補零到 2 的冪次尺寸。OpenCV 提供 `cv2.getOptimalDFTSize()` 來查詢最佳尺寸，對即時系統而言值得使用。

6.2.2 頻譜的中心化與顯示

直接轉換得到的頻譜，零頻率(直流分量，代表整張影像的平均亮度)位於左上角，四個角落各占一部分低頻，不易觀察。慣例是做頻譜中心化(`fftshift`)，把零頻率移到影像正中央，如此一來「愈靠近中心愈低頻、愈往外愈高頻」，直覺得多。

另一個必要步驟是取對數。這裡正好呼應第 5 章的對數轉換：直流分量的數值往往比其他成分大上好幾個數量級(因為自然影像的能量絕大部分集中在低頻)，不取對數的話，整張頻譜圖只會看到中央一個亮點，其餘全黑，什麼資訊都讀不到。取對數壓縮了動態範圍，讓微弱的高頻成分也看得見——這正是第 5 章所說「對數轉換適合處理動態範圍極大的影像」的最佳實例。

程式 6-1 計算與顯示二維頻譜

```
import cv2
import numpy as np

gray = cv2.imread("building.jpg", cv2.IMREAD_GRAYSCALE)

F = np.fft.fft2(gray)          # 二維 FFT
Fshift = np.fft.fftshift(F)    # 零頻率移到中央

magnitude = np.abs(Fshift)     # 振幅譜
spectrum = 20 * np.log(magnitude + 1)  # 取對數才看得見
spectrum = cv2.normalize(spectrum, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("spectrum.png", spectrum)

# 反轉換:確認可以完整還原
back = np.fft.ifft2(np.fft.ifftshift(Fshift))
back = np.abs(back).astype(np.uint8)
print("還原誤差:", np.max(cv2.absdiff(gray, back)))  # 應接近 0
```

6.2.3 頻譜判讀指南

會算頻譜只是第一步，會「讀」頻譜才是真功夫。以下是常見的對應關係，建議做成隨身小抄：

影像中的內容	頻譜上的表現	相關章節
整體亮度(平均值)	中心的直流分量，通常最亮	第 5 章 灰階轉換
大面積平坦區、漸層	能量集中在中心附近的低頻	第 5 章 平滑濾波
銳利邊緣、細節、文字	高頻區有明顯能量	第 9 章 邊緣偵測
垂直方向的條紋或欄杆	沿水平方向延伸的亮線	本章 6.3.4 方向濾波
規律的週期性干擾	對稱的孤立亮點	本章 6.4 陷波濾波
隨機雜訊	全頻域均勻分布的底噪	第 7 章 影像還原
失焦或運動模糊	高頻能量被壓低，或出現暗紋	第 7 章 反卷積
影像被旋轉	整個頻譜同步旋轉相同角度	第 4 章 幾何轉換
影像被平移	振幅譜不變，只有相位改變	本章 6.7.2 相位相關

表 6-1 頻譜判讀對照表

最後兩列特別值得玩味，它們是傅立葉轉換的兩個重要性質：旋轉性質(影像旋轉，頻譜同步旋轉)與平移性質(影像平移，振幅譜完全不變，資訊全部藏在相位裡)。這兩個性質不只是理論趣味——6.7.2 節的相位相關對齊法，正是直接建立在平移性質上的實用演算法。

6.2.4 振幅與相位：哪個更重要？

傅立葉轉換的結果是複數，可拆成振幅譜(每個頻率成分的強度)與相位譜(每個成分的位移)。有一個經典實驗能揭示兩者的分工：取 A、B 兩張照片，用 A 的振幅配 B 的相位重建影像，結果看起來像什麼？答案是——像 B。

這說明相位攜帶了影像的結構資訊(邊緣在哪裡、輪廓如何排列、物體位於何處)，而振幅主要反映各頻率的能量分布(整體的「質感」與對比強弱)。直覺上可以這樣理解：振幅告訴你「有哪些條紋圖參與疊加、各占多少比重」，相位告訴你「這些條紋圖要對齊在哪裡」——而讓邊緣在正確位置浮現的，正是眾多條紋圖在該處剛好同相疊加的結果。

這個發現有深遠的實務意義：許多影像處理與辨識演算法因此特別重視相位資訊；第 13 章的相位相關法、以及某些對光照變化強健的特徵描述子，都是這個觀念的產物。

程式 6-2 振幅與相位的經典交換實驗

```
import cv2, numpy as np

a = cv2.imread("cat.jpg", 0).astype(np.float32)
```

```

b = cv2.imread("car.jpg", 0).astype(np.float32)
b = cv2.resize(b, (a.shape[1], a.shape[0]))

Fa, Fb = np.fft.fft2(a), np.fft.fft2(b)

# A 的振幅 + B 的相位
mixed = np.abs(Fa) * np.exp(1j * np.angle(Fb))
out = np.abs(np.fft.ifft2(mixed))
cv2.imwrite("amp_a_phase_b.png",
            cv2.normalize(out, None, 0, 255, cv2.NORM_MINMAX))

# 對照組:只保留相位(振幅全設為 1)
phase_only = np.exp(1j * np.angle(Fa))
po = np.abs(np.fft.ifft2(phase_only))
cv2.imwrite("phase_only.png",
            cv2.normalize(po, None, 0, 255, cv2.NORM_MINMAX))
# 觀察:只有相位仍能看出輪廓,只有振幅則什麼都看不出來

```

6.2.5 DFT 的隱藏假設：週期性與邊界效應

這是一個容易被忽略、卻經常在實作中咬人的細節。DFT 隱含假設影像是週期性延拓的：它認為影像的右邊界緊接著左邊界、下邊界緊接著上邊界，像貼壁紙一樣無限重複。但真實照片的左右邊界通常亮度差異很大，拼接處就出現一道人造的劇烈跳變——這道假邊緣會在頻譜上產生沿水平與垂直方向的十字亮線，經常被初學者誤讀為「影像中有強烈的橫豎紋理」。

解決方式有二：一是加窗(Windowing)，在做 FFT 前把影像乘上一個邊緣衰減到零的視窗函數(如 Hanning 窗)，消除邊界跳變；二是補零與鏡射填充，把影像擴大後再轉換。在做精確的頻譜分析或第 6.7.2 節的相位相關對齊時，加窗幾乎是必要步驟。

6.3 頻率域濾波：低通、高通、帶通

6.3.1 濾波的基本流程

頻率域濾波的標準流程是五個步驟：對影像做 FFT；將頻譜中心化；把頻譜乘上一個濾波器轉移函數 $H(u, v)$ ；反中心化；做反 FFT 取實部。整個過程可寫成 $g = \text{IFFT}[H \cdot \text{FFT}(f)]$ 。設計濾波器，就是設計 H 這張「遮罩」——它決定哪些頻率通過、哪些被削減。

請特別注意這裡的「乘上」二字：在頻率域，濾波是逐點相乘，這是最單純的運算；而同一件事在空間域是卷積，要滑動視窗、逐點相乘再加總。同一個操作，在兩個域的複雜度天差地別——這正是 6.5 節卷積定理的精髓，也是頻率域方法最重要的工程價值。

6.3.2 低通濾波器的三種設計

1. 理想低通濾波器(Ideal)：在截止頻率 D_0 內完全通過、之外完全阻斷，像用剪刀直接剪掉。看似完美，卻會產生嚴重的振鈴效應(Ringing)：影像邊緣附近出現一圈圈同心漣漪。原因是「頻率域的圓形方波」對應「空間域的類 sinc 函

數」，而 sinc 有無限延伸的振盪旁瓣——這是卷積定理的直接後果，也是本章最重要的反面教材：數學上最乾脆的做法，工程上往往最糟糕。

2. Butterworth 低通濾波器：轉移函數 $H(u,v) = 1 / [1 + (D(u,v)/D_0)^{2n}]$ ，以階數 n 控制過渡的陡峭程度。 $n = 1$ 時極為平滑、幾乎無振鈴； n 增大則逐漸逼近理想濾波器，振鈴隨之出現。它提供了「銳利度」與「振鈴」之間的連續旋鈕，實務上很受歡迎； $n = 2$ 是常用的預設值。
3. 高斯低通濾波器： $H(u,v) = \exp[-D^2(u,v) / (2D_0^2)]$ 。高斯函數的傅立葉轉換仍是高斯函數，沒有負值旁瓣，因此完全不產生振鈴，過渡最平滑。代價是截止不夠俐落——想擋掉的高頻總會漏一點過去。但在多數應用中這反而是優點，因為自然影像本來就沒有明確的頻率邊界。

三者的取舍可以歸納成一句話：頻率域截得愈乾脆，空間域的副作用愈嚴重。這是不確定性原理在影像處理中的體現——一個函數不可能在兩個域同時都很「窄」。第 11 章談小波時，我們會再次遇到這個根本限制。

6.3.3 高通濾波與高頻強調

高通濾波器的轉移函數是低通的補集： $H_{hp}(u,v) = 1 - H_{lp}(u,v)$ 。單純的高通濾波會把直流分量也濾掉，結果是一張平均亮度為零的影像——邊緣浮現但整體呈灰黑，大部分區域資訊喪失。

實務上更常用的是高頻強調濾波(High-Frequency Emphasis): $H(u,v) = a + b \cdot H_{hp}(u,v)$ ，其中 a 保留部分低頻(通常取 0.5 至 1), b 控制高頻的增益(通常大於 1)。這樣既強調了邊緣，又保留了原本的亮度層次。請把它與第 5 章的 High-Boost 銳化 $g = f + k \cdot (f - \text{blur}(f))$ 並列比較：兩者做的是完全相同的事，只是一個在空間域用減法實現、一個在頻率域用加權實現。這種「同一個操作的兩張面孔」在本章反覆出現，是理解影像處理的關鍵心法。

另一個經典應用是同態濾波(Homomorphic Filtering)，用於同時壓縮動態範圍並增強對比。它的理論基礎是：影像可視為「照度」與「反射率」的乘積，照度變化緩慢(低頻，對應打光不均)，反射率變化劇烈(高頻，對應物體本身的紋理)。做法是先取對數把乘法變成加法，再用一個「低頻壓抑、高頻放大」的濾波器分別處理，最後取指數還原。這對第 17 章 AOI 中「打光不均勻的工件表面」特別有效——它能在拉平打光差異的同時，強化瑕疵的紋理對比。

程式 6-3 低通、高通與高頻強調濾波

```
import cv2, numpy as np

gray = cv2.imread("building.jpg", 0).astype(np.float32)
M, N = gray.shape
u = np.arange(M).reshape(-1, 1) - M // 2
v = np.arange(N).reshape(1, -1) - N // 2
D = np.sqrt(u**2 + v**2)          # 到頻譜中心的距離

D0 = 50                          # 截止頻率

H_ideal = (D <= D0).astype(np.float32)          # 理想低通
```

```

H_butter = 1 / (1 + (D / D0)**(2 * 2))          # n=2
Butterworth
H_gauss = np.exp(-(D**2) / (2 * D0**2))        # 高斯低通

Fshift = np.fft.fftshift(np.fft.fft2(gray))

def apply_filter(Fshift, H):
    out = np.fft.ifft2(np.fft.ifftshift(Fshift * H))
    return np.clip(np.abs(out), 0, 255).astype(np.uint8)

for name, H in [("ideal", H_ideal), ("butter", H_butter),
                ("gauss", H_gauss)]:
    cv2.imwrite(f"lowpass_{name}.png", apply_filter(Fshift, H))
    # 比較 ideal 邊緣的同心漣漪(振鈴)與 gauss 的平順

# --- 高通:取補集 ---
cv2.imwrite("highpass.png", apply_filter(Fshift, 1 - H_gauss))

# --- 高頻強調:a + b*Hhp,對照第 5 章的 High-Boost ---
a, b = 0.5, 2.0
H_emph = a + b * (1 - H_gauss)
cv2.imwrite("emphasis.png", apply_filter(Fshift, H_emph))

```

比較項目	理想	Butterworth	高斯
過渡帶	無(垂直截斷)	可調(階數 n)	平滑
振鈴效應	嚴重	n 大時出現	無
截止俐落度	最佳	中等偏佳	較差
空間域對應	sinc(有振盪旁瓣)	介於兩者之間	高斯(無旁瓣)
適用場合	教學示範	一般工程應用	要求無偽影者

表 6-2 三種頻率域濾波器比較

6.3.4 帶通與方向性濾波：通往特徵擷取的橋

低通與高通都是以「距離中心多遠」為唯一標準，屬於各向同性(isotropic)濾波器——不管方向，只管頻率高低。但影像中的資訊往往是有方向性的：布料的織紋、木材的紋理、晶圓上的線路、道路上的車道線，都有明確的走向。這時就需要更精緻的工具。

帶通濾波器(Bandpass)只讓某個頻率區間通過，以 $H_{bp} = H_{lp}(D_0 \text{ 大}) - H_{lp}(D_0 \text{ 小})$ 的方式構造。它的用途是「挑出特定粗細的結構」：想找出寬度約十像素的刮痕，就設計一個中心頻率對應該尺度的帶通濾波器，比單純的高通乾淨得多，因為它同時排除了更細的雜訊與更粗的背景起伏。

方向性濾波器則進一步在頻譜上只保留某個角度扇形區域的成分。因為「頻譜上的方位」對應「影像中條紋的垂直方向」，只要在頻譜上留下水平方向的扇形，就能萃取出影像中的垂直條紋。這正是織品瑕疵檢測、晶圓線路檢查的常用手法——把規律的背景紋理當成「已知頻率」濾掉，剩下的異常就是瑕疵(見 6.7.3 節案例)。

把「帶通」與「方向選擇」兩者結合，再加上空間域的局部化(用高斯視窗限制作用範圍)，就得到了著名的 Gabor 濾波器——第 13 章特徵擷取的重要工具，也是早期人臉辨識與紋理分類的主力。有趣的是，神經科學研究發現哺乳動物視覺皮層 V1 區的簡單細胞，其感受野特性與 Gabor 濾波器高度相似；而第 15 章會看到，CNN 第一層的卷積核在訓練後，也常自發演化出類似的方向性帶通結構。人腦、人工設計、機器學習，三條路走到了同一個答案——這是本書最耐人尋味的觀察之一。

程式 6-4 帶通與方向性濾波

```
import cv2, numpy as np

M, N = gray.shape
u = np.arange(M).reshape(-1, 1) - M // 2
v = np.arange(N).reshape(1, -1) - N // 2
D = np.sqrt(u**2 + v**2)
theta = np.arctan2(u, v) # 頻譜上每點的方位角

# --- 帶通:保留 D0_low ~ D0_high 之間的頻率 ---
D_low, D_high = 20, 60
H_bp = (np.exp(-(D**2) / (2 * D_high**2)) -
        np.exp(-(D**2) / (2 * D_low**2)))

# --- 方向性濾波:只保留 ±15 度扇形(萃取垂直條紋) ---
ang = np.abs(np.degrees(theta))
H_dir = ((ang < 15) | (ang > 165)).astype(np.float32)
H_dir = cv2.GaussianBlur(H_dir, (0, 0), 3) # 平滑邊界避免振鈴

Fshift = np.fft.fftshift(np.fft.fft2(gray))
cv2.imwrite("bandpass.png", apply_filter(Fshift, H_bp))
cv2.imwrite("directional.png", apply_filter(Fshift, H_bp * H_dir))
```

6.4 帶阻與陷波濾波：狙擊週期性雜訊

這是頻率域最無可取代的應用場景。當影像受到規律性干擾——掃描舊報紙時的網點、翻拍螢幕的摩爾紋、電源干擾造成的橫紋、感光元件的固定圖案雜訊、機械振動造成的條紋——這些干擾在空間域遍布全圖，用第 5 章的平滑濾波去除必然連細節一起犧牲。但在頻率域中，週期性干擾只集中在少數幾個孤立的亮點上(因為它本身就是幾個特定頻率的弦波)，把那幾點壓下去，反轉換回來，干擾消失而細節完好無損。

工具有二：帶阻濾波器(Bandreject)阻擋以中心為圓心、特定半徑範圍的環狀頻帶，適合干擾頻率已知且呈環狀分布者；陷波濾波器(Notch)則只在頻譜上指定的幾個小圓區域歸零，精準狙擊特定的干擾點。

三個實作要點必須牢記：第一，務必同時消除共軛對稱點(承 6.1.3 節)，否則反轉換會出現虛部殘留與明顯偽影；第二，陷波的半徑不宜過大，否則會連帶損失鄰近的正常頻率成分，在影像上表現為局部模糊；第三，陷波區域的邊界應做平滑處理(如乘上高斯剖面)，避免硬截斷造成振鈴——這與 6.3.2 節理想濾波器的教訓完全一致。

程式 6-5 陷波濾波去除摩爾紋(含自動偵測)

```
import cv2, numpy as np
```

```

gray = cv2.imread("moire.jpg", 0).astype(np.float32)
M, N = gray.shape
Fshift = np.fft.fftshift(np.fft.fft2(gray))

# 步驟一:先看頻譜,用肉眼(或程式)找出異常亮點座標
cy, cx = M // 2, N // 2
notches = [(cy - 40, cx + 55)]      # 依實際觀察填入

# 步驟二:在每個亮點及其對稱點畫上平滑的抑制區
H = np.ones((M, N), np.float32)
r = 8
for (py, px) in notches:
    cv2.circle(H, (px, py), r, 0, -1)
    cv2.circle(H, (2 * cx - px, 2 * cy - py), r, 0, -1) # 對稱點!
H = cv2.GaussianBlur(H, (0, 0), 2) # 平滑邊界,抑制振鈴

out = np.abs(np.fft.ifft2(np.fft.ifftshift(Fshift * H)))
cv2.imwrite("moire_removed.png", out.astype(np.uint8))

# 進階:自動偵測干擾亮點
mag = 20 * np.log(np.abs(Fshift) + 1)
local_max = cv2.dilate(mag, np.ones((9, 9), np.uint8))
peaks = (mag == local_max) & (mag > mag.mean() + 3 * mag.std())
peaks[cy-30:cy+30, cx-30:cx+30] = False      # 排除中心低頻
print("偵測到的干擾點:", np.argwhere(peaks)[:10])

```

6.5 卷積定理：兩個域的橋梁

6.5.1 定理與推論

卷積定理是連結第 5 章與本章的核心命題：空間域的卷積，等於頻率域的逐點相乘；反過來，空間域的相乘等於頻率域的卷積。以符號表示： $f * h$ 的傅立葉轉換等於 $F \cdot H$ 。

這條定理解釋了本章許多現象，值得逐條對照：為什麼理想低通會振鈴（頻域的圓形方波，其空間域對應是振盪的類 sinc 函數）；為什麼高斯模糊如此溫和（高斯的轉換仍是高斯，無旁瓣）；為什麼第 7 章的模糊還原可以寫成頻率域的除法（卷積退化為乘法，反運算自然是除法）；為什麼第 5 章可分離濾波能省下大量運算（因為二維高斯可拆成兩個一維高斯的外積，對應頻域也可拆解）。

6.5.2 該用哪個域？一個工程判準

卷積定理帶來一個實務問題：同一件事（卷積）有兩條路可走，該走哪一條？答案取決於核的大小。空間域卷積的成本正比於核的面積（ m^2 ）；而頻率域路線的成本主要是兩次 FFT，約為 $O(MN \cdot \log(MN))$ ，與核大小完全無關。

因此判準是：小核在空間域做比較快，大核在頻率域做比較划算。以一張 1024×1024 的影像為例， 3×3 卷積在空間域約需九百萬次乘法，而走頻率域要付出

約兩千萬次運算的固定成本——顯然不划算。但若核放大到 51×51 ，空間域需要約二十七億次乘法，頻率域的成本卻分毫未增，差距超過百倍。

這就是為什麼 OpenCV 的 GaussianBlur、filter2D 對小核直接在空間域運算，而像第 7 章維納濾波這類「退化函數與影像同尺寸」的操作，只能在頻率域進行。實務上還有第三個考量：GPU 對空間域卷積的平行化極為友善(第 15 章的 CNN 正是靠這一點)，因此在 Jetson 等有 GPU 的平台上，空間域的優勢區間會比純 CPU 更寬——這是第 17 章部署時需要實測的工程細節。

6.6 頻率域觀點與全書的觀念連結

本節把散落在各章的線索一次串起來。建議讀者在讀完全書後回頭重讀一次——屆時每一條連結都會有更深的體會。

6.6.1 與第 3 章：取樣定理與混疊的頻域解釋

第 3 章介紹 Nyquist 取樣定理時，說「取樣頻率必須至少是訊號最高頻率的兩倍」。這句話的完整意義只有在頻率域才說得清楚：取樣的動作在頻域上會使原訊號的頻譜以取樣頻率為間隔無限複製。若取樣頻率夠高，這些複製品彼此分離，可以用低通濾波器完整取回原訊號；若取樣頻率不足，相鄰的複製品就會重疊，高頻成分「摺疊」進低頻區，偽裝成原本不存在的低頻訊號——這就是混疊(Aliasing)。

摩爾紋正是它最常見的視覺表現：細密的條紋(高頻)在取樣不足時，摺疊成了粗大的波紋(低頻)。這也解釋了為什麼「縮小影像前應先做低通模糊」——先把超過新取樣率一半的高頻成分濾掉，就不會有東西可以摺疊回來。OpenCV 的 INTER_AREA 內插(第 4 章)之所以適合縮小，正是因為它內建了這個平均(低通)的動作。

6.6.2 與第 5 章：每個空間濾波器的頻率面孔

第 5 章的所有濾波器，在本章都能看到它們的另一張臉。這個對照表值得學生自己動手驗證一遍(見 Lab 6-4)：

空間域(第 5 章)	頻率域面孔	觀察重點
均值濾波(方形核)	二維 sinc 型低通	有振盪旁瓣，故平滑效果不佳且有方向性偽影
高斯濾波	高斯型低通	無旁瓣，sigma 愈大則頻域愈窄、影像愈模糊
拉普拉斯運算子	高通	響應隨頻率平方增長，故對雜訊極敏感
Sobel 梯度(第 9 章)	方向性高通	水平 Sobel 對垂直邊緣響應強，即方向選擇性
Unsharp / High-Boost	高頻強調	等同 6.3.3 節的 $a + b \cdot H_{hp,k}$ 對應 b
中值濾波	無對應(非線性)	不滿足線性與位置不變性，無法以轉移函數

		描述
--	--	----

表 6-3 空間域濾波器與其頻率域面孔

最後一列特別重要：卷積定理只適用於線性位置不變系統。中值濾波、雙邊濾波非局部平均都是非線性的，沒有轉移函數可言——這也解釋了它們為何能做到線性濾波做不到的事(例如中值濾波能去除椒鹽雜訊而不模糊邊緣)。線性系統的分析工具強大，但也劃下了能力的上限。

6.6.3 與第 7 章：模糊是低通，還原是除法

第 7 章的劣化模型 $g = h * f + n$ ，在本章的語言中就是 $G = H \cdot F + N$ 。所有形式的模糊——失焦、運動、大氣擾動、鏡頭像差——在頻域上做的都是同一件事：把高頻振幅壓低。因此退化函數 H 本質上就是一個低通濾波器，而影像還原就是「受控的高通增強」。

這個觀點也預告了還原的物理上限：運動模糊的 H 是 sinc 函數，在某些頻率上恰好等於零，那些頻率的資訊被乘以零而徹底消失，再高明的演算法也救不回來。理解頻率域，才能分辨「演算法不夠好」與「資訊已經死亡」的差別。

6.6.4 與第 9 章：分割演算法裡的雙重濾波

第 9 章的 Canny 邊緣偵測是一個絕佳的教學案例，因為它在一個演算法內同時用到兩種相反的濾波：第一步的高斯平滑是低通(壓制雜訊，避免微分放大雜訊);第二步的梯度運算是高通(找出灰階劇烈變化處)。順序不能顛倒——先微分再平滑，雜訊已經被放大，再平滑也救不回來。

這也解釋了 Canny 的 σ 參數為何如此關鍵： σ 太小，低通不足，雜訊產生大量假邊緣; σ 太大，低通過度，弱邊緣連同雜訊一起被抹平。在頻率域的語言中， σ 就是在決定「訊號與雜訊的分界線畫在哪個頻率」。

6.6.5 與第 11 章：傅立葉的極限與小波的誕生

傅立葉轉換有一個根本的弱點：它是全域的。頻譜告訴你「這張影像含有哪些頻率」，卻不告訴你「這些頻率出現在影像的哪個位置」。一張左半邊平坦、右半邊佈滿細紋的影像，與一張細紋均勻分布的影像，可能有相似的振幅譜。

這個弱點正是第 11 章小波轉換的存在理由：小波以「有限長度的小波形」為基底，同時提供頻率與位置的資訊，這就是多解析度分析。而 JPEG 使用的 DCT 則採取另一種折衷：把影像切成 8×8 小區塊，各自做轉換——用「分區塊」的方式勉強取得局部性，代價就是高壓縮率下惱人的方塊效應。從 DFT 到 DCT 到小波的演進，是一部「如何同時掌握頻率與位置」的奮鬥史。

6.6.6 與第 13 章：頻率域就是特徵

特徵擷取與頻率域的關係比多數人想像的更深。三個具體連結：

- Gabor 濾波器(承 6.3.4 節)是帶通加方向選擇加空間局部化的組合，直接用於紋理分類與早期人臉辨識，其響應強度就是一種特徵。

- SIFT 的尺度空間是由一系列不同 σ 的高斯模糊構成，本質上是一組截止頻率遞減的低通濾波器；而相鄰兩層相減得到的高斯差(DoG)，正是一個帶通濾波器。SIFT 尋找極值的動作，就是在「頻率與位置」的三維空間中找特徵點。
- 相位相關法(6.7.2 節)利用傅立葉的平移性質做影像對齊，是一種完全在頻域運作的匹配方法，對亮度變化與雜訊比空間域的模板匹配更強健。

6.6.7 與第 15 章：CNN 學到了什麼濾波器

卷積神經網路的名字就洩露了它與本章的血緣。訓練完成後把第一層的卷積核視覺化，會發現它們自發演化成兩類：一類是不同顏色的低頻色塊(相當於低通)，另一類是不同方向與粗細的條紋偵測器(相當於方向性帶通，與 Gabor 濾波器高度相似)。人工設計了數十年的濾波器，被梯度下降重新發明了一次。

近年也有研究從頻譜角度分析神經網路的行為，例如觀察到網路傾向先學會低頻成分再逐漸擬合高頻細節的現象；也有研究指出對抗樣本的擾動常集中在高頻。這些發現提醒我們：即使進入深度學習時代，頻率域仍是理解影像模型的重要透鏡。

6.7 應用案例

6.7.1 案例一：摩爾紋與週期性雜訊移除

這是本章最經典的案例。情境：用手機翻拍電腦螢幕或電視，拍出的照片佈滿波浪狀的彩色紋路。成因是螢幕的像素網格(高頻週期結構)與相機感光元件的像素網格(取樣格點)相互干涉，產生了低頻的拍頻——正是 6.6.1 節所說的混疊。

處理流程：計算頻譜 → 目視或程式找出干擾造成的孤立亮點 → 設計陷波濾波器(記得對稱點與平滑邊界) → 反轉換。成效驗收建議用第 3 章的 PSNR 與 SSIM，與「單純高斯模糊」的做法對照——數據會清楚顯示：對付週期雜訊，頻率域完勝空間域，因為前者只移除了那幾個頻率，後者則是全圖細節陪葬。

延伸應用相同：老報紙掃描的網點(印刷半色調網屏)、電源干擾造成的橫紋、CMOS 感光元件的固定圖案雜訊、掃描器機構震動造成的週期條紋，都適用同一套流程。這也是第 7 章影像還原中，唯一不需要動用反卷積就能完美解決的劣化類型。

6.7.2 案例二：相位相關法做影像對齊與運動估計

這個案例展示了頻率域的另一種威力：不做濾波，而是做「比對」。理論基礎是 6.2.3 節提到的平移性質——影像平移時，振幅譜完全不變，所有位移資訊都編碼在相位中。

做法是：把兩張影像各自做 FFT，計算它們的交叉功率譜(一張的頻譜乘上另一張的共軛，再除以其大小做正規化，只保留相位差)，再做反 FFT。若兩張影像之間只有平移關係，結果會是一個尖銳的脈衝，脈衝的位置就直接給出位移量，精度可達像素等級。

這個方法的優勢在於：對亮度整體變化免疫(因為只用相位)、對雜訊相當強健、而且是全域搜尋一次算完，不像空間域的模板匹配需要逐位置滑動比對。應用場合極

廣：第 5 章夜景模式的多影格對齊、影像穩定(防手震)、第 12 章的運動估計、衛星影像的時序配準、以及第 13 章影像拼接的初始位移估計。OpenCV 直接提供 `cv2.phaseCorrelate` 函式。

實作提醒：務必先加窗(6.2.5 節)。因為 DFT 假設影像週期性延拓，未加窗時邊界跳變會在相關圖上產生強烈的十字狀假響應，可能蓋過真正的峰值。

程式 6-6 相位相關法對齊影像

```
import cv2, numpy as np

a = cv2.imread("frame1.png", 0).astype(np.float32)
b = cv2.imread("frame2.png", 0).astype(np.float32)

# 加窗:消除 DFT 週期性延拓造成的邊界假響應
win = cv2.createHanningWindow(a.shape[:::-1], cv2.CV_32F)

# OpenCV 內建相位相關,回傳次像素精度的位移與峰值信心值
(dx, dy), response = cv2.phaseCorrelate(a, b, win)
print(f"位移量:dx={dx:.3f}, dy={dy:.3f} 像素,信心值={response:.3f}")

# --- 手動實作,理解原理 ---
Fa = np.fft.fft2(a * win)
Fb = np.fft.fft2(b * win)
R = Fa * np.conj(Fb)
R /= (np.abs(R) + 1e-8) # 正規化:只保留相位差
corr = np.fft.fftshift(np.abs(np.fft.ifft2(R)))
peak = np.unravel_index(np.argmax(corr), corr.shape)
print("峰值位置(相對中心即為位移):", peak)

# 應用:多影格對齊後平均去雜訊(第 5 章夜景模式)
M = np.float32([[1, 0, dx], [0, 1, dy]]) # 第 4 章仿射矩陣
b_aligned = cv2.warpAffine(b, M, b.shape[:::-1])
merged = ((a + b_aligned) / 2).astype(np.uint8)
```

6.7.3 案例三：AOI 紋理瑕疵檢測

第三個案例最貼近本學程的產業場域，也是頻率域在工業界最重要的應用之一。情境：檢測織品、金屬網、晶圓線路、印刷電路板等「具有規律紋理」的產品表面瑕疵。

傳統的空間域做法很棘手：背景本身就佈滿高對比的規律紋理，任何門檻分割或邊緣偵測都會被背景淹沒。但換到頻率域，問題突然變得優雅：規律的背景紋理是週期性的，因此在頻譜上表現為少數幾個明確的亮點(位置由紋理的疏密與方向決定);而瑕疵——刮痕、缺線、汙點、破洞——是打破規律的異常，能量散布在其他頻率上。

於是策略反轉了：不是「找出瑕疵」，而是「濾掉正常」。設計一個陷波濾波器把代表正常紋理的那幾個亮點移除，反轉換後，規律的背景消失殆盡，剩下的殘差影像中，瑕疵便成為畫面上唯一顯著的區域，再用第 9 章的門檻分割與第 10 章的連通元件標記即可定位與計數。

這個方法的工程優點是：不需要大量瑕疵樣本訓練(相對於第 16 章的深度學習方案)，而且對「沒見過的新瑕疵類型」天生有效——因為它偵測的是「異常」而非「特定瑕疵」。缺點是隻適用於背景高度規律的產品，且對取像品質要求高(打光不均會產生額外的低頻成分干擾判讀，此時可先用 6.3.3 節的同態濾波處理)。實務上，它常與深度學習方案互補使用。

程式 6-7 頻率域紋理瑕疵檢測

```
import cv2, numpy as np

fabric = cv2.imread("fabric_defect.png", 0).astype(np.float32)
M, N = fabric.shape
cy, cx = M // 2, N // 2

F = np.fft.fftshift(np.fft.fft2(fabric))
mag = 20 * np.log(np.abs(F) + 1)

# 步驟一：自動找出代表「正常紋理」的週期性亮點
local_max = cv2.dilate(mag, np.ones((11, 11), np.uint8))
peaks = (mag == local_max) & (mag > mag.mean() + 2.5 * mag.std())
peaks[cy-15:cy+15, cx-15:cx+15] = False          # 保留直流與低頻

# 步驟二：陷波濾掉這些「正常」頻率
H = np.ones((M, N), np.float32)
for (py, px) in np.argwhere(peaks):
    cv2.circle(H, (int(px), int(py)), 10, 0, -1)
H = cv2.GaussianBlur(H, (0, 0), 3)

# 步驟三：反轉換，殘差中只剩異常
residual = np.abs(np.fft.ifft2(np.fft.ifftshift(F * H)))
residual = cv2.normalize(residual, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)

# 步驟四：門檻分割定位瑕疵(第 9、10 章)
_, defect = cv2.threshold(residual, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
defect = cv2.morphologyEx(defect, cv2.MORPH_OPEN,
                          np.ones((3, 3), np.uint8))
n, labels, stats, _ = cv2.connectedComponentsWithStats(defect)
print(f"偵測到 {n - 1} 處疑似瑕疵")
```

6.7.4 案例四：JPEG 壓縮中的頻率思維

最後一個案例，說明頻率域如何撐起你手機裡的每一張照片。JPEG 的核心邏輯是人眼對低頻(整體形狀與亮度)敏感，對高頻(細微紋理)遲鈍，因此可以大膽丟棄高頻資訊。

流程是：先轉到 YCbCr 色彩空間(第 8 章)並對色度降取樣 → 切成 8×8 區塊 → 對每個區塊做離散餘弦轉換(DCT，傅立葉家族的近親，只用餘弦項因此結果為實數) → 用量化表把高頻係數除以較大的數再取整數，許多高頻係數因此變成零 → 以熵編碼壓縮這些零。品質參數控制的正是量化表的縮放倍率。

這也解釋了 JPEG 的典型瑕疵：高壓縮率時， 8×8 區塊各自量化的差異在邊界處顯現，形成方塊效應；而銳利邊緣附近的高頻被截斷，產生類似振鈴的蚊狀雜訊——沒錯，又是 6.3.2 節那個「頻域硬截斷造成空間域振盪」的老問題。第 11 章會完整拆解 DCT 與量化表，並比較小波為何能避免方塊效應。

6.8 本章與全書的觀念連結總表

相關章節	共用的觀念	頻率域提供的洞見
第 3 章 數位影像基礎	取樣定理、混疊、摩爾紋	取樣使頻譜週期性複製，不足則重疊摺疊成混疊
第 4 章 幾何轉換	旋轉、平移；縮小前的低通	影像旋轉則頻譜同步旋轉；平移只改相位
第 5 章 空間域強化	平滑、銳化、可分離濾波	每個空間核都有頻率面孔；High-Boost 即高頻強調
第 7 章 影像還原	劣化模型、反卷積	模糊即低通、還原即除法；H 的零點代表資訊死亡
第 9 章 影像分割	Canny 的平滑與梯度	低通與高通在同一演算法中依序作用，順序不可顛倒
第 11 章 小波轉換	DCT、多解析度分析	傅立葉無位置資訊，催生 DCT 分區塊與小波
第 13 章 特徵擷取	Gabor、SIFT 尺度空間、匹配	DoG 即帶通；相位相關法以平移性質做對齊
第 15 章 深度學習	卷積核、頻譜偏差	CNN 首層自發學成方向性帶通，近似 Gabor
第 17 章 邊緣部署	運算成本、GPU 加速	小核用空間域、大核用頻率域；GPU 會改變分界點

表 6-4 第 6 章與全書其他章節的觀念連結

程式實作 Lab 6

Lab 6-1 頻譜偵探

目標：培養「看頻譜說故事」的能力。(a)分別對下列影像計算並顯示頻譜：純色塊、單一方向的條紋、旋轉 45 度的條紋、棋盤格、自然風景照、文字掃描頁，對照表 6-1 說明每張頻譜的特徵與影像內容的關係；(b)自己合成一張影像：用 `np.sin` 產生單一頻率的條紋圖，計算其頻譜，驗證是否只有兩個對稱亮點；(c)改變條紋的疏密與角度，觀察亮點如何在頻譜上移動，驗證「疏密對應距中心距離、方向對應方位角」；(d)把一張照片旋轉 30 度後重算頻譜，驗證旋轉性質；把同一張照片平移 50 像素，比較振幅譜是否不變——這就是 6.7.2 節相位相關法的理論基礎；(e)挑戰題：對同一張照片分別「加窗」與「不加窗」計算頻譜，觀察十字亮線的有無，理解 6.2.5 節的邊界效應。

Lab 6-2 摩爾紋清除工作坊

目標：完成一次真實的週期雜訊移除。(a)用手機翻拍電腦螢幕或電視，取得帶有明顯摩爾紋的照片；(b)計算頻譜，找出干擾造成的孤立亮點並記錄座標；(c)以程式 6-5 設計陷波濾波器移除干擾，記得處理對稱點並平滑邊界；(d)以第 3 章的 PSNR 與 SSIM 比較「陷波濾波」與「單純高斯模糊」兩種做法相對於直接截圖(乾淨參考影像)的品質——用數據證明：對付週期雜訊，頻率域完勝空間域；(e)分別測試陷波半徑 $r = 3$ 、8、20 三種設定，說明過大的半徑為何造成局部模糊；(f)挑戰題：完成程式 6-5 中的自動偵測函式，讓程式自己找出所有干擾亮點，不需人工輸入座標。

Lab 6-3 相位相關對齊與多影格去雜訊

目標：實作 6.7.2 節的案例，並與第 5 章的夜景模式串接。(a)拍攝或合成一組僅有平移差異的影像對，以程式 6-6 求出位移量，與真值比較誤差(應可達次像素等級)；(b)刻意調整其中一張的整體亮度(乘上 0.7 或加上 30)，重新計算——驗證相位相關法對亮度變化的免疫力，並與 `cv2.matchTemplate` 的結果對照；(c)加入不同強度的高斯雜訊，繪製「雜訊強度對定位誤差」曲線，比較兩種方法的強健度；(d)實戰：手持連拍同一暗景 10 張(必然有輕微晃動)，以相位相關逐張對齊後平均，與「未對齊直接平均」比較清晰度與 PSNR——這正是 Google Night Sight 所謂 robust alignment 的簡化版；(e)討論：若影像之間除了平移還有旋轉與縮放，此法還適用嗎？可以如何擴充？

Lab 6-4 濾波器的兩張臉

目標：親手驗證表 6-3，把第 5 章與第 6 章徹底打通。(a)取 3×3 均值核、 5×5 均值核、 $\sigma=1$ 與 $\sigma=3$ 的高斯核、拉普拉斯核、水平 Sobel 核，分別補零到 256×256 後做 FFT，顯示各自的頻率響應；(b)對照觀察：哪些是低通？哪些是高通？哪些有方向性？均值核的旁瓣在哪裡？(c)驗證「 σ 愈大、頻域愈窄」的反比關係，並解釋這如何對應到「影像愈模糊」；(d)實測卷積定理：用同一個 15×15 高斯核，分別以 `cv2.filter2D`(空間域)與 FFT 相乘(頻率域)處理同一張影像，比較兩者結果的最大差異值(應接近零)與執行時間；(e)把核尺寸從 3×3 逐步加大到 101×101 ，繪製「核大小對兩種方法執行時間」的曲線，找出交叉點——用你自己的數據驗證 6.5.2 節的工程判準。

本章重點整理

- 影像的空間頻率描述灰階變化的快慢：低頻對應平坦區域與整體亮度，高頻對應邊緣、細節與雜訊；二維傅立葉的每個基底都是一張斜條紋圖，疏密對應距中心距離、方向對應方位角。
- 二維 DFT 以 FFT 計算，複雜度由 $O(M^2N^2)$ 降為 $O(MN \cdot \log MN)$ ；顯示頻譜必須先 `fftshift` 中心化並取對數壓縮動態範圍；實數影像的頻譜具共軛對稱性。
- 相位譜攜帶影像的結構資訊，振幅譜反映能量分布；影像平移時振幅譜不變、資訊全在相位，此性質即相位相關對齊法的基礎；影像旋轉則頻譜同步旋轉。
- DFT 隱含週期性延拓假設，邊界跳變會造成十字狀假亮線，精確分析前應加窗處理。

- 理想濾波器截止俐落但有嚴重振鈴(頻域硬截斷對應空間域 sinc 振盪); Butterworth 以階數 n 調控; 高斯無振鈴。高通轉移函數為 1 減低通, 實務多用高頻強調 $a + b \cdot \text{Hhp}$, 等價於第 5 章的 High-Boost。
- 帶通濾波挑出特定尺度的結構, 方向性濾波挑出特定走向的紋理; 兩者結合並加上空間局部化即 Gabor 濾波器, 是第 13 章紋理特徵的基礎。
- 週期性雜訊在頻譜上呈孤立亮點, 以帶阻或陷波濾波精準移除; 務必同時處理共軛對稱點、控制陷波半徑、平滑陷波邊界。
- 卷積定理: 空間域卷積等於頻率域相乘。小核在空間域計算較快, 大核則轉頻率域較划算; 非線性濾波(中值、雙邊、非局部平均)無轉移函數可言。
- 頻率域是全書的樞紐觀念: 解釋取樣定理與混疊(第 3 章)、賦予空間濾波器另一張面孔(第 5 章)、界定影像還原的物理上限(第 7 章)、催生 DCT 與小波(第 11 章)、支撐 Gabor 與 SIFT(第 13 章)、並在 CNN 的卷積核中重現(第 15 章)。

習題

- 請以自己的話說明「影像的高頻成分」代表什麼, 並舉出三種會產生高頻的影像內容與兩種只含低頻的影像內容。

提示與參考解答: 高頻代表灰階隨位置劇烈變化的成分。高頻內容: 銳利邊緣、細密紋理(織品、髮絲)、文字筆畫、雜訊。低頻內容: 漸層的天空、大面積平坦的牆面、失焦的背景。可對照表 6-1。

- 為什麼顯示頻譜前需要 `fftshift` 與取對數? 各解決什麼問題? 取對數這個動作與第 5 章的哪個概念相同?

提示與參考解答: `fftshift` 把零頻率從左上角移到中央, 使「愈近中心愈低頻」符合直覺。取對數壓縮動態範圍: 自然影像能量集中於低頻, 直流分量比其他成分大數個數量級, 不取對數則只看得到中央一個亮點。這正是第 5 章對數轉換「拉伸暗部、壓縮亮部」的應用。

- 一張影像若含有水平的規律條紋, 其頻譜上的亮點會沿哪個方向分布? 為什麼? 若條紋變得更細密, 亮點會如何移動?

提示與參考解答: 沿垂直方向分布。因為頻譜上的方位對應影像中條紋的垂直方向: 水平條紋代表灰階沿垂直方向週期變化, 故頻率分量沿垂直軸。條紋愈細密代表頻率愈高, 亮點會愈往外(離中心愈遠)移動。

- 說明傅立葉轉換的旋轉性質與平移性質, 並各舉一個實際應用。

提示與參考解答: 旋轉性質: 影像旋轉 θ 角, 頻譜同步旋轉 θ 角。應用: 由頻譜上亮線的方向估計文件掃描的傾斜角以自動校正。平移性質: 影像平移時振幅譜完全不變, 位移資訊全部編碼在相位。應用: 相位相關法做影像對齊與運動估計(6.7.2 節)。

- 用 A 的振幅搭配 B 的相位重建影像, 結果較像哪一張? 這說明了什麼? 若只保留相位(振幅全設為 1)會看到什麼?

提示與參考解答: 較像 B。說明相位攜帶影像的結構資訊(邊緣位置、輪廓排列), 振幅主要反映各頻率的能量分布(整體質感與對比)。只保留相位仍能看出輪廓與物體形狀; 只保留振幅則幾乎什麼都認不出來。

6. 什麼是 DFT 的週期性延拓假設?它會在頻譜上造成什麼假象?如何避免?

提示與參考解答: DFT 假設影像左右、上下邊界相接、無限重複。真實照片邊界亮度不連續,產生人造的劇烈跳變,在頻譜上形成沿水平與垂直方向的十字亮線,易被誤讀為影像有強烈橫豎紋理。避免方式:做 FFT 前乘上 Hanning 等視窗函數(加窗),或以鏡射填充擴大影像。

7. 說明振鈴效應的成因,並解釋為何高斯濾波器不會產生振鈴。這個現象與「不確定性原理」有何關聯?

提示與參考解答: 理想濾波器在頻域是硬截斷的圓形方波,依卷積定理其空間域對應是類 sinc 函數,具有無限延伸的振盪旁瓣,與影像卷積後在邊緣附近產生同心漣漪。高斯函數的傅立葉轉換仍是高斯,無負值旁瓣,故無振鈴。關聯: 函數不可能在空間域與頻率域同時都很窄——頻域截得愈乾脆,空間域的擴散與振盪愈嚴重。

8. 寫出 Butterworth 低通濾波器的轉移函數,說明階數 n 增大時的行為變化,並指出 n 趨近無限大時會退化成什麼。

提示與參考解答: $H(u,v) = 1 / [1 + (D/D_0)^{2n}]$ 。 n 小時過渡平滑、幾乎無振鈴但截止不俐落; n 增大則過渡帶變陡、截止更俐落,但振鈴開始出現。 n 趨近無限大時退化為理想低通濾波器(在 D_0 處垂直截斷)。

9. 高通濾波與高頻強調濾波有何差異?後者與第 5 章的 High-Boost 銳化在數學上是什麼關係?

提示與參考解答: 純高通會濾掉直流分量,結果平均亮度為零、整體灰黑。高頻強調 $H = a + b \cdot H_{hp}$ 保留部分低頻(a)並放大高頻(b),兼顧亮度層次與邊緣強調。它與第 5 章 $g = f + k \cdot (f - \text{blur}(f))$ 做的是完全相同的事: 一個在空間域用減法實現,一個在頻率域用加權實現, k 對應 b 。

10. 什麼是帶通濾波?若要偵測影像中寬度約 10 像素的刮痕,為什麼帶通比單純的高通更合適?

提示與參考解答: 帶通只讓某個頻率區間通過,構造方式為兩個不同截止頻率的低通相減。針對特定寬度的刮痕,對應的是特定尺度(頻率),帶通能同時排除更細的雜訊(更高頻)與更粗的背景起伏(更低頻),訊噪比遠優於單純高通——高通會把所有雜訊一併留下。

11. Gabor 濾波器由哪三個要素組合而成?它與人類視覺系統及 CNN 有什麼耐人尋味的關聯?

提示與參考解答: 三要素: 帶通(選擇尺度)、方向選擇(選擇走向)、空間局部化(以高斯視窗限制作用範圍)。關聯: 神經科學研究發現哺乳動物視覺皮層 V1 區簡單細胞的感受野特性與 Gabor 高度相似;而 CNN 第一層卷積核經訓練後也常自發演化出類似的方向性帶通結構——人腦、人工設計、機器學習三條路走到同一個答案。

12. 為什麼設計陷波濾波器時必須同時消除共軛對稱點?若忘記會發生什麼?陷波半徑過大又會有什麼副作用?

提示與參考解答: 實數影像的頻譜滿足共軛對稱 $F(-u,-v) = F^*(u,v)$,只消除單邊會破壞對稱性,反轉換後出現虛部殘留與明顯偽影。半徑過大會連帶移除鄰近的正常頻率成分,在影像上表現為該區域的局部模糊或細節損失。此外邊界應平滑處理,否則硬截斷會造成振鈴。

13. 摩爾紋為什麼適合用頻率域處理而非空間域平滑?請從兩者的作用範圍說明,並解釋摩爾紋與第 3 章混疊的關係。

提示與參考解答：摩爾紋是週期性干擾，在頻譜上只佔少數幾個孤立點，陷波濾波只移除那幾個頻率，其餘細節完好；空間域平滑則是無差別壓制所有高頻，細節必然陪葬。與混疊的關係：螢幕的像素網格（高頻週期結構）與相機感光元件的取樣格點相互干涉，高頻成分摺疊回低頻區，形成肉眼可見的粗大波紋。

14. 陳述卷積定理。一張 1024×1024 的影像要做 51×51 的模糊，你會選空間域還是頻率域？請估算兩者的運算量並說明理由。

提示與參考解答：卷積定理： $f * h$ 的傅立葉轉換等於 $F \cdot H$ ，即空間域卷積等於頻率域逐點相乘。空間域成本約 $1024 \times 1024 \times 51 \times 51 \approx 27$ 億次乘法；頻率域主要成本為兩次 FFT，約 $O(MN \log MN) \approx 2$ 千萬次運算，與核大小無關。應選頻率域，差距超過百倍。若核為 3×3 則反之，空間域較快。

15. 為什麼中值濾波在表 6-3 中沒有對應的「頻率面孔」？這說明了卷積定理的適用範圍為何？

提示與參考解答：中值濾波是非線性運算，不滿足線性與位置不變性，無法以轉移函數 $H(u, v)$ 描述。卷積定理只適用於線性位置不變 (LSI) 系統。這也解釋了非線性濾波為何能做到線性濾波做不到的事——例如中值濾波去除椒鹽雜訊而不模糊邊緣。

16. 傅立葉轉換的根本弱點是什麼？第 11 章的 DCT 與小波分別如何回應這個弱點？

提示與參考解答：弱點是全域性：頻譜只告訴你「有哪些頻率」，不告訴你「頻率出現在哪個位置」。DCT 的回應是把影像切成 8×8 區塊各自轉換，以分區塊勉強取得局部性，代價是高壓縮率下的方塊效應。小波的回應是以「有限長度的小波形」為基底，同時提供頻率與位置資訊，即多解析度分析。

17. 承 6.7.3 節的 AOI 案例：為什麼檢測規律紋理產品的瑕疵時，策略是「濾掉正常」而非「找出瑕疵」？這個方法相對深度學習方案有何優缺點？

提示與參考解答：因為規律背景紋理是週期性的，在頻譜上僅佔少數明確亮點，容易精準移除；而瑕疵是打破規律的異常，能量散布於其他頻率。濾掉正常後，殘差影像中瑕疵成為唯一顯著區域。優點：不需大量瑕疵樣本訓練，對未見過的新瑕疵類型天生有效。缺點：僅適用於背景高度規律的產品，且對取像品質與打光均勻度要求高（可先用同態濾波改善）。

18. 說明相位相關法的原理與三項優勢，並解釋為何實作時務必先加窗。

提示與參考解答：原理：依平移性質，兩張只有平移差異的影像其相位差編碼了位移量；計算正規化交叉功率譜後反轉換，峰值位置即為位移，可達次像素精度。三項優勢：對整體亮度變化免疫（只用相位）、對雜訊強健、一次算完不需逐位置滑動比對。加窗原因：DFT 的週期性延拓假設使邊界跳變在相關圖上產生十字狀假響應，可能蓋過真正的峰值。

19. (綜合題) 請說明頻率域觀點如何解釋以下三件事：(a) 縮小影像前為什麼要先模糊；(b) Canny 為什麼要先高斯平滑再算梯度；(c) 運動模糊為什麼有時完全救不回來。

提示與參考解答：(a) 縮小即降取樣，依取樣定理，超過新取樣率一半的高頻會摺疊成混疊；先低通模糊濾掉這些成分就無物可摺疊，故 `INTER_AREA` 內建平均動作。(b) 梯度是高通運算會放大雜訊，高斯平滑是低通先壓制雜訊；順序顛倒則雜訊已被放大，再平滑也無法還原。(c) 運動模糊的退化函數是 sinc 函數，在某些頻率恰好等於零，該頻率的資訊被乘以零而徹底消失，屬於資訊死亡而非演算法不足。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 4(頻率域處理之標準參考)。
- Fourier, J. B. J., Théorie analytique de la chaleur, Paris, 1822.(1807年手稿遭拉格朗日反對，本章叫獸開講之史料)
- MacTutor History of Mathematics, Biography: Joseph Fourier, University of St Andrews.
- Cooley, J. W., & Tukey, J. W., An Algorithm for the Machine Calculation of Complex Fourier Series, Math. Comp., 1965.(FFT 原始論文)
- Oppenheim, A. V., & Lim, J. S., The Importance of Phase in Signals, Proc. IEEE, 1981.(相位重要性之經典文獻)
- Kuglin, C. D., & Hines, D. C., The Phase Correlation Image Alignment Method, Proc. IEEE Int. Conf. Cybernetics and Society, 1975.(相位相關法原始論文)
- Daugman, J. G., Uncertainty Relation for Resolution in Space, Spatial Frequency, and Orientation Optimized by Two-Dimensional Visual Cortical Filters, JOSA A, 1985.(Gabor 與視覺皮層)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 3(影像處理與頻域分析)。
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 7 章 影像還原

本章學習目標

- 區分影像強化與影像還原的目標差異，寫出影像劣化的數學模型並說明其病態性。
- 說明影像還原與第 5 章空間濾波、第 6 章低通/高通濾波在觀念上的一致性與差異。
- 辨識高斯、椒鹽、均勻、Poisson、週期性等雜訊型態，並以直方圖與平坦區統計量估計雜訊參數。
- 運用排序統計濾波、自適應局部降噪濾波與非局部平均處理不同型態的雜訊。
- 說明逆濾波為何失敗，推導維納濾波與約束最小平方濾波如何加以修正。
- 以維納濾波還原模糊與含雜訊的影像，並以 PSNR 與 SSIM 量化成效。
- 評估影像還原對後續邊緣偵測、特徵擷取與物件偵測的影響。
- 認識反覆式反卷積與 AI 影像修復的原理，並思辨「還原」與「腦補」的界線。

叫獸開講|馬蓋先重生記與哈伯望遠鏡的太空救援

先說一個溫暖的故事。1985 年開播的影集《百戰天龍》(MacGyver)，陪伴了一整個世代的台灣觀眾——那位不用槍、只靠一把瑞士刀和滿腦子物理化學知識就能脫困的馬蓋先，大概是許多工程師最早的偶像。然而，三十多年前的類比訊號母帶畫質模糊、雜訊滿佈，以今天的螢幕觀看幾乎不忍卒睹。近年來，透過 AI 影像修復技術，那段經典的片頭曲被重製成 4K 畫質：雜訊被抹去、邊緣重新銳利、色彩重獲飽和，馬蓋先以前所未有的清晰度重返螢幕。這是「影像還原」最貼近生活的示範。

再說一個驚心動魄的故事。1990 年 4 月，人類史上最昂貴的科學儀器之一——哈伯太空望遠鏡升空。所有人屏息等待它傳回宇宙深處的照片，結果收到的卻是一張張模糊的影像。工程師追查後發現一個令人心碎的錯誤：主鏡的研磨形狀有偏差，產生球面像差，使得點光源超過八成的光線被散射成半徑二至三角秒的光暈。一個微小到以微米計的加工誤差，讓十幾億美元的望遠鏡近乎失明。

修復需要太空人上去動手術，但那是三年後(1993 年 12 月)的事。在那之前的三年，天文學家怎麼辦？他們選擇了另一條路：既然模糊的成因(點擴散函數，PSF)可以被測量與建模，那就用數學把影像「解模糊」回來。維納濾波、Richardson-Lucy 反覆演算法、最大熵法等一系列反卷積技術被大規模投入，竟然真的從模糊資料中救回了大量可用的科學成果。1993 年太空人裝上 COSTAR 校正光學系統與新的 WFPC2 相機後，硬體問題才根本解決——但那三年間，是影像還原技術替人類守住了通往宇宙的窗口。

兩個故事，一今一古，正好對應本章的兩條路線：傳統的數學還原(第 7.5 節的逆濾波、維納濾波與約束最小平方)靠的是「理解劣化的物理過程，再反推回去」；現代的 AI 修復(第 7.8 節)靠的是「從海量影像中學會世界長什麼樣子，然後補上」。前者誠實但受限，後者驚豔但危險——因為它補出來的細節，可能從來不存在。

思考題：如果法庭上的監視器畫面經過 AI 超解析度處理，把模糊的車牌「還原」成清晰的號碼，這個證據可以採信嗎？請從本章兩條路線的差異思考。

7.1 影像還原的定位、模型與病態性

7.1.1 還原與強化的差別

第 5、6 章談的是影像強化(Enhancement)：目標是讓影像「看起來更好」，判斷相當主觀——把夜景調亮、把邊緣變銳利，好不好看你說了算。本章的影像還原(Restoration)則是客觀的逆問題：先建立劣化的數學模型，再設法反推出最接近原始影像的估計。它問的不是「怎樣比較好看」，而是「原本長什麼樣子」，因此可以用第 3 章介紹的 PSNR、SSIM 等指標客觀驗收。

這個差別在工程實務上非常關鍵。舉例來說：同樣是讓模糊的影像變清楚，第 5 章的 Unsharp Masking 只是「把邊緣兩側的對比拉大」，製造出銳利的視覺印象，原本被模糊掉的細節並沒有回來；而本章的反卷積則是「依據模糊的數學模型，把被抹平的高頻成分算回去」。前者是化妝，後者是手術。也因此，還原方法需要更多的先驗知識(退化函數、雜訊統計量)，而且在資訊確實已經完全消失的情況下，它也無能為力——這是所有還原技術的物理上限。

比較項目	影像強化(第 5、6 章)	影像還原(本章)
目標	看起來更好、更適合後續處理	逼近原始未劣化的影像
判準	主觀(人眼、下游任務表現)	客觀(MSE、PSNR、SSIM)
需要的知識	幾乎不需要先驗知識	退化函數 h 、雜訊統計特性
代表方法	直方圖等化、Gamma、平滑與銳化	維納濾波、約束最小平方、反覆式反卷積
數學性質	正問題(直接運算)	逆問題(常為病態問題)

表 7-1 影像強化與影像還原的比較

7.1.2 劣化模型

標準的劣化模型是： $g(x, y) = h(x, y) * f(x, y) + n(x, y)$ ，其中 f 是原始影像， h 是退化函數(以卷積作用，例如失焦模糊、運動模糊、光學系統的點擴散函數 PSF)， n 是加性雜訊， g 是我們實際拿到的影像。依第 6 章的卷積定理，在頻率域可寫成 $G = H \cdot F + N$ ——空間域的卷積變成乘法，還原問題於是變成「已知 G 與 H ，如何解出 F 」的代數問題。這正是頻率域方法在本章大放異彩的原因，也再次印證第 6 章的核心訊息：換一個域，難題就換一副面孔。

請留意這個模型隱含的三個假設，以及它們在真實系統中可能失效的情形：第一退化是線性且位置不變的(整張影像用同一個 h)。實務上鏡頭邊緣的模糊往往比中心嚴重，哈伯望遠鏡的 PSF 甚至隨時間與位置變動，這時就需要分區塊處理或更複雜的模型。第二，雜訊是加性的且與訊號獨立。這對高斯雜訊成立，但對 Poisson 雜訊(第 7.2 節)並不成立，因為它的強度隨訊號變化。第三，雜訊是零均值的。若感測器有直流偏移(dark current)，必須先做暗電流校正再進入還原流程。

依 h 是否已知，還原問題分為兩類難度：若 h 已知(例如哈伯望遠鏡的 PSF 可由觀測標準星測得、運動模糊的方向與長度可由拍攝參數推算)，稱為非盲反卷積(Non-blind Deconvolution); 若 h 未知，必須同時估計 h 與 f ，稱為盲反卷積(Blind Deconvolution)。後者困難得多——因為「模糊的影像 = 清楚的影像卷積大模糊核」與「稍微模糊的影像卷積小模糊核」可以得到完全相同的觀測結果，解不唯一，必須靠額外的先驗假設(例如自然影像的梯度分布特性)才能收斂到合理解。

7.1.3 為什麼是病態問題

數學上，一個問題被稱為適定問題(Well-posed)，必須滿足三個條件：解存在、解唯一、解對資料連續依賴(輸入的微小擾動只造成輸出的微小變化)。影像還原問題最常違反的是第三條：退化函數 h 是低通性質的，它把高頻成分壓到接近零;還原時要把這些成分放大回來，等於乘上一個極大的倍數，而觀測資料中不可避免的微小雜訊也會被同等放大。輸入端萬分之一的擾動，可能在輸出端變成鋪天蓋地的雪花——這就是病態(ill-posed)。

解決病態問題的通用策略叫正則化(Regularization)：在「忠實還原資料」之外，額外加上對解的約束，例如「影像應該是平滑的」、「影像的梯度應該是稀疏的」。維納濾波中的雜訊訊號比項、約束最小平方法中的平滑度項、以及深度學習模型從資料學到的先驗，本質上都是正則化的不同形式。理解這一點，你就抓住了整章的骨架：所有還原方法的差別，只在於「用什麼方式告訴演算法：合理的影像應該長什麼樣子」。

7.1.4 本章在全書中的位置

影像還原是一個承先啟後的樞紐章節，它幾乎與本書每一篇都有連結，建議讀者在閱讀本章時隨時回顧與前瞻：

- 與第 3 章(數位影像基礎)：還原成效的驗收工具 MSE、PSNR、SSIM 來自第 3 章;而 RAW 檔保留的高位元深度線性資料，是還原演算法最理想的輸入——8 位元 JPEG 已經過壓縮與 Gamma 編碼，細節與線性關係都已受損。
- 與第 5 章(空間域強化)：中值、高斯、雙邊濾波在本章重新登場，但角色從「讓影像好看」變成「依雜訊模型抑制雜訊」;而夜景模式的多張平均，正是本章「以增加觀測次數對抗隨機雜訊」的實例。
- 與第 6 章(頻率域處理)：劣化模型在頻率域是乘法，還原則是除法;退化函數是低通濾波器，還原則相當於受控的高通增強;週期性雜訊的移除直接沿用第 6 章的陷波濾波。7.4 節會完整展開這條主線。
- 與第 9 章(影像分割)：雜訊會讓 Canny 產生大量假邊緣，模糊則會讓邊緣定位偏移。還原品質直接決定分割品質——7.7 節有量化實驗。

- 與第 13 章(特徵擷取):Harris、SIFT、ORB 都依賴局部灰階變化。模糊會抹平角點的響應值、雜訊會製造大量假特徵點，兩者都使匹配率下降。
- 與第 15、16 章(深度學習與 YOLO)：訓練資料若乾淨、測試場景卻模糊含噪，模型準確率會顯著下滑。還原可作為推論前的前處理，或反過來——把劣化當成資料增強來訓練更強健的模型。
- 與第 17 章(邊緣部署)：還原演算法的運算成本必須納入即時系統的預算。維納濾波需要兩次 FFT, Richardson-Lucy 需要數十次反覆，在 Jetson 上是否跑得動?這是工程決策。

7.2 雜訊模型、辨識與參數估計

7.2.1 常見雜訊型態

1. 高斯雜訊(Gaussian)：機率密度為常態分布，由平均值與標準差兩個參數決定。來源包括感測器熱雜訊、電子電路雜訊、放大器雜訊。它是最常見也最常被假設的雜訊模型，一方面因為它確實普遍，另一方面因為中央極限定理——大量獨立小擾動的總和趨近常態分布。直方圖呈鐘形。
2. 椒鹽雜訊(Salt-and-Pepper)：又稱脈衝雜訊，像素以某機率隨機變成全黑(0，胡椒)或全白(255，鹽)。來源是訊號傳輸位元錯誤、類比數位轉換失誤、感測器壞點。直方圖在兩端出現孤立尖峰，中間分布不受影響——這是它與高斯雜訊最容易區分的特徵。
3. 均勻雜訊(Uniform)：在某區間內等機率分布。自然界較少見，但量化誤差(第 3 章)的統計特性接近均勻分布，因此在分析位元深度不足造成的失真時會用到。
4. Poisson 雜訊(散粒雜訊, Shot Noise)：源自光子到達感測器的隨機性，其變異數等於平均值——換言之，強度與訊號本身相關：亮處雜訊的絕對值大、暗處小，但暗處的「相對雜訊」(訊噪比)反而更差。這正是第 5 章夜景模式要對付的主角，也是低光攝影、醫學影像(X 光光子計數)、天文攝影的主要雜訊來源。
5. 乘性雜訊與斑點雜訊(Speckle)：雜訊與訊號相乘而非相加，常見於超音波影像、合成孔徑雷達(SAR)。處理策略通常是先取對數把乘法變成加法，再套用加性雜訊的方法，最後取指數還原。
6. 週期性雜訊(Periodic)：來自電源干擾、機械振動、掃描機構的規律誤差。它是唯一「不隨機」的雜訊，因此不該用平滑濾波處理——正確做法是回到第 6 章的頻率域，以陷波濾波精準移除，詳見 7.4.3 節。

7.2.2 雜訊辨識與參數估計

還原工作的第一步永遠是診斷。實務技巧是：找一塊本該平坦均勻的區域(例如純色背景、牆面、天空、工件的平整表面)，把該區域的像素值取出來分析。因為原始訊號在該區近乎常數，所觀察到的變異幾乎全部來自雜訊。

依直方圖形狀判讀：鐘形對稱分布指向高斯雜訊；兩端出現孤立尖峰、中央分布正常指向椒鹽雜訊；平坦的矩形分布指向均勻雜訊；若變異數隨區域平均亮度成正比增加(需比較亮區與暗區兩塊平坦區)，則指向 Poisson 雜訊。判斷正確，才能選對濾波器

——用中值濾波去對付高斯雜訊，或用均值濾波去對付椒鹽雜訊，都是事倍功半的常見錯誤。

參數估計則更進一步：高斯雜訊的標準差可直接由平坦區的樣本標準差估得，這個數值是設定維納濾波 K 值、非局部平均 h 參數的重要依據。另一個不需要人工選區的實用技巧，是利用高通濾波的殘差來估計雜訊：對影像做拉普拉斯運算，由於自然影像的高頻能量遠小於雜訊，殘差的統計量可作為雜訊強度的粗估。

程式 7-1 產生雜訊影像與估計雜訊參數

```
import cv2, numpy as np

img = cv2.imread("clean.jpg", 0).astype(np.float32)

# --- 加入高斯雜訊 ---
sigma = 20
gauss = np.random.normal(0, sigma, img.shape)
noisy_g = np.clip(img + gauss, 0, 255).astype(np.uint8)

# --- 加入椒鹽雜訊(各 2%) ---
noisy_sp = img.copy().astype(np.uint8)
p = 0.02
rnd = np.random.rand(*img.shape)
noisy_sp[rnd < p] = 0          # 胡椒(黑)
noisy_sp[rnd > 1 - p] = 255    # 鹽(白)

# --- 加入 Poisson 雜訊(強度隨訊號變化) ---
noisy_po = np.random.poisson(img).clip(0, 255).astype(np.uint8)

# --- 雜訊參數估計 ---
# 方法一:取平坦 ROI 直接算標準差
roi = noisy_g[10:60, 10:60].astype(np.float32)  # 選一塊平坦背景
print("ROI 估計 sigma:", roi.std())

# 方法二:以拉普拉斯殘差估計(免選區)
lap = cv2.Laplacian(noisy_g.astype(np.float32), cv2.CV_32F)
sigma_est = np.sqrt(np.pi / 2) * np.abs(lap).mean() / 6
print("拉普拉斯法估計 sigma:", sigma_est)
```

7.3 空間域去雜訊：從排序統計到非局部平均

當劣化只有雜訊、沒有模糊(即 h 為單位脈衝)時，問題退化為純粹的去雜訊，在空間域處理往往最直接有效。第 5 章介紹的均值、高斯、中值濾波在此重新登場，但這次我們關心的不是「好不好看」，而是「哪一種在對應的雜訊模型下，統計上最接近原始值」。

7.3.1 均值濾波家族

算術平均是高斯雜訊的最佳線性估計：若鄰域內的原始訊號近乎常數，平均 m 個獨立同分布的雜訊樣本，雜訊標準差降為原來的 m 的平方根分之一。這正是第 5 章夜景模式「連拍平均」的數學根據，差別只在於：夜景模式平均的是「不同時間的同一

個像素」(訊號完全相同,理想),而空間平均的是「同一時間的不同像素」(訊號未必相同,因此會模糊邊緣)。這個對照值得學生細細體會——它說明了為什麼多影格去雜訊優於單一影格去雜訊。

幾何平均能在去雜訊的同時比算術平均保留更多細節;諧波平均對鹽雜訊(白點)與高斯雜訊有效,但無法處理胡椒雜訊;反諧波平均則以階數 Q 控制: Q 為正時消除胡椒雜訊, Q 為負時消除鹽雜訊, Q 為零時退化為算術平均——但用錯正負號會使結果更糟,實務上必須先完成 7.2 節的診斷。

7.3.2 排序統計濾波家族

這一族不做算術,而是把鄰域內的像素排序後挑選特定名次的值:中值濾波取中位數,對椒鹽雜訊近乎完美,因為極端值排在最前或最後,永遠不會被選中;最大值濾波取最大值,適合去除胡椒雜訊(黑點),但會讓亮區膨脹;最小值濾波取最小值,適合去除鹽雜訊(白點);中點濾波取最大與最小的平均,對高斯與均勻雜訊表現不錯;alpha 修剪均值濾波先刪去最大與最小的若干個值再平均,能同時應付混合了高斯與脈衝的雜訊,是實務上很實用的折衷。

這裡也埋著一個與第 10 章的伏筆:最大值與最小值濾波,其實就是型態學的膨脹(Dilation)與侵蝕(Erosion)運算。同一個運算,在本章是「去雜訊工具」,在第 10 章是「形狀操作工具」——影像處理中許多概念都有這種一體多面的性格。

7.3.3 自適應濾波:讓濾波器懂得看場合

固定核濾波的根本缺陷是「一視同仁」:它在平坦區與邊緣區施加同樣強度的平滑,結果不是雜訊沒去乾淨,就是細節被抹掉。自適應濾波的核心思想是:先看看局部的統計特性,再決定要不要動手、動多重的手。

7. 自適應局部降噪濾波器:比較「全域雜訊變異數」與「局部變異數」。若局部變異數接近雜訊變異數,代表這裡是平坦區,雜訊佔主導,應大幅平滑;若局部變異數遠大於雜訊變異數,代表這裡有邊緣或紋理,應盡量保留原值。它的行為可以理解成:平坦處當均值濾波用,邊緣處幾乎不作用。
8. 自適應中值濾波:固定核中值濾波在雜訊密度高時會失效(鄰域內超過一半都是雜訊,中位數本身就是雜訊),且會抹平細節。自適應版本分兩階段:第一階段判斷「中位數本身是不是雜訊(等於 0 或 255)」,若是則擴大視窗重試,直到找到有效的中位數或達到視窗上限;第二階段判斷「中心像素是不是雜訊」,若不是就原值保留、完全不動。如此一來,乾淨的像素受到完整保護,只有雜訊點被替換,細節保留能力遠優於固定中值濾波,在雜訊密度達 20% 以上時差距尤其懸殊。

7.3.4 非局部平均:跳脫鄰近的假設

以上所有方法都建立在同一個假設上:相鄰的像素值應該相似。非局部平均(Non-Local Means,NLM)徹底顛覆了這個假設,改為在整張影像中尋找「周圍紋理相似的區塊」來加權平均——不管它離得多遠。它利用的是自然影像的自相似性:磚牆上的每一塊磚、毛衣上的每一道織紋、草地上的每一叢草,在影像各處反覆出現。

具體做法：對每個待處理像素，取一個小的比對視窗(template window，例如 7×7)，在一個較大的搜尋範圍(search window，例如 21×21)內尋找相似的視窗，依相似度給權重再加權平均。因為參與平均的是「內容相同」的像素而非「位置相鄰」的像素，邊緣與紋理得以完整保留，去雜訊能力顯著優於傳統濾波，是深度學習出現前的最強方法之一。代價是運算量大——這也是它在即時系統中較少使用、卻在離線修復(如老照片、醫學影像)中大放異彩的原因。

程式 7-2 五種去雜訊方法的客觀比較

```
import cv2, numpy as np

clean = cv2.imread("clean.jpg", 0)
noisy = cv2.imread("noisy_gaussian.png", 0)

# --- 傳統濾波 ---
mean_f = cv2.blur(noisy, (5, 5))
gauss_f = cv2.GaussianBlur(noisy, (5, 5), 1.5)
med_f = cv2.medianBlur(noisy, 5)
bil_f = cv2.bilateralFilter(noisy, 9, 75, 75)

# --- 非局部平均(h 應依估得的雜訊 sigma 設定) ---
nlm = cv2.fastNlMeansDenoising(noisy, None, h=20,
                                templateWindowSize=7,
                                searchWindowSize=21)

# --- 客觀比較(第 3 章工具) ---
for name, out in [("原始含噪", noisy), ("均值", mean_f),
                  ("高斯", gauss_f), ("中值", med_f),
                  ("雙邊", bil_f), ("NLM", nlm)]:
    print(f"{name:8s} PSNR = {cv2.PSNR(clean, out):.2f} dB")
```

7.4 頻率域觀點：還原與濾波的血緣關係

這一節是本章與第 6 章的接榫，也是理解所有反卷積方法的鑰匙。請先記住一句話：模糊就是低通濾波，還原就是受控的高通增強。

7.4.1 退化函數是低通濾波器

回想第 6 章：影像的高頻成分對應邊緣與細節，低頻對應平坦區域與整體亮度。任何形式的模糊——失焦、運動、大氣擾動、鏡頭像差——在頻率域上做的事都一樣：把高頻成分的振幅壓低。以高斯模糊為例，它的轉移函數 H 本身也是高斯形狀，中心(低頻)為 1，愈往外(高頻)愈趨近於零。運動模糊的 H 則更兇殘：它是一個 sinc 函數，在某些特定頻率上直接等於零——那些頻率的資訊被完全抹除，再高明的演算法也救不回來。

這解釋了一個常令初學者困惑的現象：為什麼有些模糊影像還原後很漂亮，有些卻無論如何都救不回來？答案就在 H 的零點：資訊被乘以零的地方，是真正的資訊死亡，不是演算法不夠好。

7.4.2 還原是受控的高通增強

既然模糊是把高頻乘上一個小數，還原自然就是把它除回來——也就是對高頻施加巨大的增益。這在概念上與第 6 章的高通濾波、第 5 章的 High-Boost 銳化是同一件事：三者都在提升高頻。差別在於：

- 第 5 章的 Unsharp Masking：增益由使用者主觀設定的 k 決定，與造成模糊的物理過程無關。
- 第 6 章的高頻強調濾波：增益由濾波器形狀決定，同樣不管實際的退化函數是什麼。
- 本章的反卷積：增益由退化函數 H 精確決定—— H 在哪個頻率壓得多，就在哪個頻率補得多。這是「量身訂做」的高通增強。

而三者共享同一個宿命：雜訊主要分布在高頻，因此任何提升高頻的動作，都必然同時放大雜訊。這是影像處理中最根本的矛盾之一——銳利與乾淨不可兼得。第 5 章用「先去雜訊再銳化」的順序來緩解，第 6 章用平滑的濾波器形狀避免振鈴，而本章的維納濾波與約束最小平方，則是把這個矛盾寫成數學式，直接求出最佳的折衷點。

7.4.3 週期性雜訊：直接沿用第 6 章

週期性雜訊是本章唯一「不需要動用反卷積」的情況。因為它在頻率域上表現為少數幾個孤立亮點，只要以第 6 章的陷波濾波(記得處理共軛對稱點)把那幾點壓下，反轉換即可乾淨移除，且細節毫髮無傷。反之，若誤用空間域的平滑濾波去對付它，必然是全圖細節陪葬。這個例子最能說明「選對域比選對演算法更重要」。

程式 7-3 觀察退化函數的頻率響應

```
import cv2, numpy as np

# 觀察三種退化函數在頻率域的樣貌(理解還原難度)
N = 256

# 高斯模糊 PSF
k = cv2.getGaussianKernel(15, 3); psf_gauss = k @ k.T

# 水平運動模糊 PSF(長度 15)
psf_motion = np.zeros((15, 15), np.float32)
psf_motion[7, :] = 1.0 / 15

for name, psf in [("gauss", psf_gauss), ("motion", psf_motion)]:
    pad = np.zeros((N, N), np.float32)
    pad[:psf.shape[0], :psf.shape[1]] = psf
    H = np.fft.fftshift(np.fft.fft2(pad))
    mag = np.abs(H)
    print(f"{name}: |H| 最小值 = {mag.min():.6f}")
    # 運動模糊的最小值接近 0 → 該頻率資訊完全喪失, 無法還原
    spec = 20 * np.log(mag + 1e-8)
    cv2.imwrite(f"H_{name}.png",
                cv2.normalize(spec, None, 0, 255,
                             cv2.NORM_MINMAX).astype(np.uint8))
```

7.5 反卷積：逆濾波、維納濾波與約束最小平方

7.5.1 逆濾波為何失敗

既然頻率域中 $G = H \cdot F + N$ ，最直覺的解法就是兩邊除以 H ： $\hat{F} = G / H$ 。若完全沒有雜訊($N = 0$)，這個逆濾波確實能完美還原。但真實世界中 N 從不為零，展開後得到：

$$\hat{F}(u,v) = F(u,v) + N(u,v) / H(u,v)$$

問題就出在第二項。承 7.4.1 節，退化函數 H 在高頻通常趨近於零，當 H 很小時 N/H 會被放大到天文數字，雜訊完全淹沒訊號；若 H 恰好等於零，更是直接除以零。這正是 7.1.3 節所說病態性的具體表現：除法在數學上合法，在工程上災難。

有一個簡單的補救叫半徑受限逆濾波：既然災難發生在高頻，那就只在以頻譜中心為圓心、半徑 D_0 之內執行除法，半徑之外一律不動(或設為零)。這相當於「逆濾波 + 低通濾波」的組合，能得到堪用的結果，但 D_0 的選擇完全靠試誤，而且截斷會帶來第 6 章的振鈴效應。這個粗糙的做法提示了正確方向：我們需要一個「隨頻率自動調整還原力道」的機制——那就是維納濾波。

7.5.2 維納濾波

維納濾波(Wiener Filter)由 Norbert Wiener 提出，以最小均方誤差為準則，在「還原銳利度」與「抑制雜訊放大」之間取得最佳平衡。其轉移函數為：

$$\hat{F}(u,v) = [H^*(u,v) / (|H(u,v)|^2 + S_n(u,v)/S_f(u,v))] \cdot G(u,v)$$

其中 H^* 是 H 的共軛複數， S_n 與 S_f 分別是雜訊與原始影像的功率譜。關鍵在分母多出來的 S_n/S_f 項(常以常數 K 近似，即雜訊對訊號的功率比)：當某頻率的訊號夠強(S_f 大、 K 相對小)時，該項趨近 0，公式退化為逆濾波，盡力還原；當訊號微弱而雜訊相對強時，該項變大，壓抑該頻率的還原力道，避免雜訊爆炸。一句話總結：維納濾波是「知道何時該放棄」的逆濾波。

把它與 7.4.2 節對照更能看出巧妙之處：維納濾波在低頻(訊噪比高)做接近完全的高通增強，在高頻(訊噪比低)自動收手，等於內建了一個「依訊噪比自動調整的低通保護」。這也解釋了為什麼維納濾波的結果通常比原圖銳利、卻又不像逆濾波那樣佈滿雪花。

實務上 S_n 與 S_f 很難精確得知，因此常把 S_n/S_f 直接當成可調參數 K ： K 太小則雜訊被放大、影像顆粒粗糙； K 太大則還原不足、影像仍然模糊。7.2.2 節估計出的雜訊標準差，是設定 K 的重要參考起點。調參的過程，就是在銳利與乾淨之間找甜蜜點——這個尋找的過程，正是 Lab 7-2 的核心。

程式 7-4 維納濾波反卷積實作

```
import cv2, numpy as np

def wiener_filter(img, psf, K=0.01):
    """維納濾波反卷積。img:退化影像;psf:退化函數;K:雜訊訊號比"""
    img = img.astype(np.float32)
```

```

# 把 PSF 填補到與影像同尺寸並轉到頻率域
psf_pad = np.zeros_like(img)
kh, kw = psf.shape
psf_pad[:kh, :kw] = psf
psf_pad = np.roll(psf_pad, -kh // 2, axis=0)
psf_pad = np.roll(psf_pad, -kw // 2, axis=1)

H = np.fft.fft2(psf_pad)
G = np.fft.fft2(img)

# 維納公式:  $H^* / (|H|^2 + K)$ 
W = np.conj(H) / (np.abs(H) ** 2 + K)
F_hat = W * G

out = np.abs(np.fft.ifft2(F_hat))
return np.clip(out, 0, 255).astype(np.uint8)

# --- 實驗:製造模糊 + 雜訊,再還原 ---
clean = cv2.imread("clean.jpg", 0)
psf = cv2.getGaussianKernel(15, 3)
psf = psf @ psf.T                                # 15x15 高斯 PSF

blurred = cv2.filter2D(clean, -1, psf)
noisy = np.clip(blurred + np.random.normal(0, 5, clean.shape), 0, 255)

print("未還原(模糊+雜訊)PSNR:",
      cv2.PSNR(clean, noisy.astype(np.uint8)))

for K in [0.0001, 0.001, 0.01, 0.1]:
    restored = wiener_filter(noisy, psf, K)
    print(f"K={K}: PSNR = {cv2.PSNR(clean, restored):.2f} dB")
    cv2.imwrite(f"wiener_K{K}.png", restored)

```

7.5.3 約束最小平方濾波

維納濾波有個實務門檻：它需要知道 S_n/S_f ，而這兩個功率譜通常都不知道。約束最小平方濾波(Constrained Least Squares)提供了另一條路：它只需要知道雜訊的平均值與變異數(比功率譜容易估計得多)，並把「還原結果應該平滑」這件事直接寫進最佳化目標。其轉移函數為：

$$\hat{F}(u,v) = [H^*(u,v) / (|H(u,v)|^2 + \gamma \cdot |P(u,v)|^2)] \cdot G(u,v)$$

其中 P 是拉普拉斯運算子的頻率響應(第 5 章的二階微分核), γ 是調節參數。比較兩式可以發現：維納濾波在分母加的是常數 K ，約束最小平方加的則是隨頻率變化的 $\gamma|P|^2$ 。因為拉普拉斯在高頻的響應大，這等於「頻率愈高、抑制愈強」——把「不要產生劇烈震盪的結果」這個先驗知識，寫成了數學。這正是 7.1.3 節所說正則化的教科書範例： γ 就是正則化強度，調大則結果平滑但模糊，調小則銳利但可能出現雜訊與振鈴。

7.5.4 反覆式反卷積：Richardson-Lucy

以上都是一次算完的線性方法，另有一類反覆逼近的非線性方法，代表是 Richardson-Lucy 演算法(Richardson 1972、Lucy 1974 分別在光學與天文領域獨立提出)。它假設 Poisson 雜訊模型(因此特別適合光子計數的天文與醫學影像)，從一個初始估計出發，每次反覆用「觀測影像除以目前估計的模糊結果」來修正估計，並保證每次結果都非負(符合光強度的物理意義，這是線性方法做不到的優點——維納濾波的輸出可能出現負值)。

這正是叫獸開講中拯救哈伯望遠鏡的主力演算法之一。實務上有個關鍵抉擇：反覆次數。次數太少還原不足；次數太多則開始放大雜訊，影像出現不自然的顆粒與塊狀結構。天文學界處理哈伯早期影像時，經驗上約 50 次反覆能明顯改善影像品質、削減散射光暈又保留原始結構；超過 100 次後，物體的形狀反而開始崩壞、出現人造的塊狀質感。「知道何時停手」是所有反覆式還原演算法的共同課題，學界也發展出以廣義交叉驗證等準則自動決定停止時機的方法。

7.6 應用案例

7.6.1 案例一：MRI 醫學影像去噪

維納濾波在醫學影像領域有長期的應用傳統。以磁共振造影(MRI)為例：掃描時間愈長，訊號累積愈多、訊噪比愈好，但病人必須維持不動、費用也愈高；工程上因此常以較快的掃描搭配後處理去噪。將維納濾波套用於含雜訊的 MRI 切片影像後，以 PSNR 量化比較處理前後的差異，是評估演算法成效的標準做法——處理後 PSNR 若提升數個 dB，即代表雜訊被有效抑制而結構資訊得以保留。

醫學影像的特殊要求也值得注意，而且是本案例最重要的教學點：過度平滑可能抹去微小的病灶，把腫瘤的早期徵兆一起「濾掉」。更麻煩的是，這種錯誤在 PSNR 上幾乎看不出來——一個直徑十個像素的病灶，佔整張影像的比例微乎其微，把它抹平對全域 MSE 的影響小到可以忽略，PSNR 甚至可能因為背景更乾淨而上升。這說明了全域指標的根本盲點：它衡量的是平均表現，而醫療關心的是最壞情況。因此醫療應用的去噪往往寧可保守，並且必須經過臨床驗證與放射科醫師的視覺評估，不能只看 PSNR 好看就上線。

7.6.2 案例二：哈伯太空望遠鏡的 PSF 反卷積

叫獸開講中的哈伯救援，是「非盲反卷積」的教科書級案例，值得從工程角度再拆解一次。它成功的關鍵有三：

9. 退化函數可測量：天文學家可以觀測遙遠的恆星——那是完美的點光源，它在感光元件上留下的光斑，就是點擴散函數 PSF 的直接量測值。此外也可從光學設計反推理論 PSF，理論 PSF 的優點是不含雜訊。有了 h ，問題從困難的盲反卷積降級為可解的非盲反卷積。
10. 雜訊模型明確：天文影像的主要雜訊是光子計數的 Poisson 雜訊，正好符合 Richardson-Lucy 的假設前提，演算法與物理模型高度契合。

11. 願意接受不完美：天文學家的目標不是「拍出好看的照片」，而是「量出正確的亮度與位置」。即使還原後的影像仍有殘餘偽影，只要科學量測的誤差在可接受範圍內，任務就算成功。

這個案例也暴露了實務的複雜性：哈伯的 PSF 並非位置不變(視野不同區域的 PSF 不同)，也非時間不變(太空船抖動、望遠鏡結構受熱脹冷縮影響，PSF 會在數十分鐘的尺度上緩慢改變)。這直接違反 7.1.2 節劣化模型的第一個假設，因此實務上必須分區塊、分時段建模——理論模型與工程現實之間的落差，永遠是工程師的主戰場。

7.6.3 案例三：產線運動模糊還原與下游任務

第三個案例最貼近本學程的場域。想像一條輸送帶，工件以每秒 0.5 公尺的速度通過檢測站，相機曝光時間 2 毫秒——工件在曝光期間移動了 1 毫米。若影像的空間解析度是每像素 0.05 毫米，這代表每個點被抹成了 20 個像素長的線段：典型的水平運動模糊。結果是：第 9 章的邊緣偵測找不到清楚的工件輪廓，第 13 章的特徵匹配比對不上模板，第 16 章的 YOLO 信心值大幅下降，整條檢測線的誤判率飆升。

這個問題有三個層次的解法，依序是工程師應該考慮的優先順序：

12. 光學與機構層(最優先)：縮短曝光時間並加強打光(第 17 章的 AOI 光源設計)，或改用全域快門(Global Shutter)相機、加裝頻閃光源在極短時間內凍結畫面。這是治本之道——沒有模糊，就不需要還原。
13. 演算法層(次之)：若模糊無法避免，則運動模糊的退化函數其實是已知的！模糊方向由輸送帶方向決定，模糊長度等於「速度乘以曝光時間再除以像素尺寸」。編碼器的速度訊號與相機的曝光參數都是現成的資料，可以即時計算出 PSF，再用維納濾波還原。這是非盲反卷積在工業現場的漂亮應用。
14. 學習層(最後)：若前兩者都受限，可考慮把運動模糊當成資料增強加入訓練集(第 15 章)，讓 YOLO 直接學會辨識模糊的工件。這是「與其還原資料以配合模型，不如訓練模型以適應資料」的思路轉換。

本案例的教學價值在於：影像還原不是孤立的技術練習，而是整個系統設計中的一環。一個好的視覺工程師，會先問「這個模糊能不能從源頭消除」，而不是一頭栽進演算法。

程式 7-5 由產線參數推算 PSF 並還原運動模糊

```
import cv2, numpy as np

def motion_psf(length, angle_deg):
    """產生運動模糊 PSF: length 為模糊長度(像素), angle 為方向"""
    psf = np.zeros((length, length), np.float32)
    psf[length // 2, :] = 1.0
    M = cv2.getRotationMatrix2D((length / 2 - 0.5, length / 2 - 0.5),
                                angle_deg, 1.0) # 第 4 章的旋轉
    psf = cv2.warpAffine(psf, M, (length, length))
    return psf / psf.sum()

# --- 由產線參數推算 PSF (非盲反卷積的關鍵) ---
speed_mm_s = 500.0 # 輸送帶速度(mm/s), 來自編碼器
exposure_s = 0.002 # 曝光時間(s), 來自相機設定
pixel_mm = 0.05 # 每像素代表的實際尺寸(mm), 來自第 4 章相機校正
```



```

blur_len = int(round(speed_mm_s * exposure_s / pixel_mm))
print("推算模糊長度(像素):", blur_len)          # → 20

psf = motion_psf(blur_len, angle_deg=0)          # 水平運動

blurred = cv2.imread("part_blurred.png", 0)
restored = wiener_filter(blurred, psf, K=0.005)    # 沿用程式 7-4
cv2.imwrite("part_restored.png", restored)

# --- 驗證還原對下游任務的效益(第 9、13 章) ---
for name, im in [("模糊", blurred), ("還原", restored)]:
    edges = cv2.Canny(im, 80, 160)
    orb = cv2.ORB_create(1000)
    kps = orb.detect(im, None)
    print(f"{name}: 邊緣像素數 = {int(edges.sum() / 255)}, "
          f"ORB 特徵點數 = {len(kps)}")

```

7.7 影像還原對後續處理的影響

影像還原很少是最終目的，它幾乎總是某個更大任務的前處理步驟。因此評估還原的價值，除了看 PSNR，更應該看它讓下游任務進步了多少。以下三個連結值得學生實際動手量化：

7.7.1 對邊緣偵測的影響(第 9 章)

Canny 邊緣偵測的第一步就是高斯平滑，原因正是「梯度運算會放大雜訊」——雜訊造成的局部劇烈變化，在微分後比真實邊緣還強。若影像雜訊嚴重，Canny 會產生大量細碎的假邊緣；若影像模糊，則邊緣的梯度峰值變得平坦，非極大值抑制難以精確定位，邊緣位置會產生偏移。良好的還原能同時改善這兩個問題，但過度平滑的還原(K 設太大)則會讓弱邊緣完全消失。這裡有一個實務準則：先想清楚下游要的是「完整的邊緣」還是「乾淨的邊緣」，再回頭調還原參數。

7.7.2 對特徵擷取的影響(第 13 章)

Harris 角點偵測依賴局部灰階在兩個方向上的變化量，模糊會直接降低角點響應值，使得許多真實角點落到門檻之下而被漏掉；雜訊則會在平坦區製造大量虛假的高響應點。SIFT 的情況更微妙：它本身就是多尺度的高斯模糊金字塔上找極值，因此對輕微模糊有一定的耐受性，但嚴重模糊會使得特徵被偵測在錯誤的尺度上，描述子跟著失真，匹配率大幅下降。ORB 使用的 FAST 角點偵測依賴中心像素與圓周上像素的亮度比較，對雜訊特別敏感——這也是程式 7-5 用 ORB 特徵點數作為還原成效指標的原因。

7.7.3 對深度學習模型的影響(第 15、16 章)

這是近年研究相當熱門的題目，結論可以歸納成三點。第一，分布偏移是主因：模型在乾淨影像上訓練，遇到模糊或含噪的測試影像時，準確率往往顯著下滑，因為輸入的統計特性已經偏離訓練分布。第二，還原不一定總是有幫助：如果還原演算法引入了訓練時未曾見過的偽影(如振鈴、過度銳化的邊緣光暈)，模型的表現可能不升

反降——這是一個違反直覺但務必記住的陷阱。第三，資料增強常是更穩健的解法：在訓練時就加入模糊、雜訊、低光等劣化樣本，讓模型自己學會不變性，通常比在推論時做還原更可靠，運算成本也更低(不必為每一影格影像付出還原的代價)。

這三點合起來給出一個重要的工程判斷原則：若你能控制訓練流程，優先用資料增強；若你面對的是已經訓練好、無法重訓的模型，才考慮在前端加上還原模組，而且務必實測它是否真的提升了端到端的準確率，而不是想當然爾。

7.8 AI 影像修復：從還原到生成

近年來，深度學習徹底改寫了影像還原的版圖。以卷積神經網路或 Transformer 訓練的去噪、去模糊、超解析度模型(如 SRCNN、ESRGAN、Real-ESRGAN 及各式擴散模型)，在視覺品質上遠勝傳統方法，老照片修復、影片重製、手機的超解析變焦都已是成熟商品——叫獸開講中馬蓋先片頭的 4K 重生，正是這類技術的成果。

從 7.1.3 節正則化的觀點看，深度學習的突破其實很好理解：傳統方法的先驗知識是人工設計的(「影像應該平滑」、「梯度應該稀疏」)，而深度模型是從數百萬張自然影像中「學」出先驗——它知道人臉長什麼樣、磚牆的紋理如何、樹葉的邊緣有多銳利。當這個先驗遠比人工設計的更貼近真實世界，還原品質自然大幅躍升。

影像補繪(Inpainting)則處理「缺一塊」的問題：移除照片中的路人、修補老照片的裂痕、去除浮水印。傳統方法(OpenCV 的 cv2.inpaint，提供 Telea 與 Navier-Stokes 兩種演算法)從破損區域邊緣向內推進，以周圍的紋理與梯度填補，適合小面積修補；生成式 AI 則能「想像」出整片合理的內容，面積再大也填得出來。

7.8.1 還原，還是腦補？

這裡必須劃下一條重要的界線。傳統反卷積是在「解方程式」：它只使用觀測資料與已知的物理模型，還原出的每一個細節，理論上都來自原始影像中殘存的資訊。生成式 AI 則是在「猜答案」：它根據訓練資料學到的先驗知識，產生一個「看起來最合理」的高解析度版本。兩者的差異在娛樂用途上無關緊要——馬蓋先的臉多了幾根不存在的頭髮不影響懷舊；但在司法鑑識、醫學診斷、科學量測上就性命攸關：AI 放大的車牌號碼可能是憑空編造的，AI 補全的病灶邊界可能誤導手術。

曾有廣為流傳的案例顯示，某些人臉超解析度模型在放大低解析度的人臉時，會產生與原本族裔特徵不符的結果——因為模型輸出的是「訓練資料中最常見的臉」，而不是「這個人的臉」。這提醒我們：生成模型的偏誤會忠實地反映訓練資料的偏誤，而使用者往往因為結果「看起來很清楚」就誤以為它很準確。清晰度不等於正確性，這是 AI 時代影像工程師必須內化的第一戒律。

身為工程師，你必須永遠能回答一個問題：這個像素的資訊，是「找回來的」還是「編出來的」？在交付任何經 AI 處理的影像時，誠實標註處理流程，是專業倫理的一部分。這也是叫獸開講思考題的核心。

7.9 本章與全書的觀念連結總表

相關章節	共用的觀念	本章的延伸或差異
第 3 章 數位影像基礎	MSE、PSNR、SSIM;RAW 高位元深度	作為還原成效的客觀驗收工具;RAW 為最佳還原輸入
第 5 章 空間域強化	平滑濾波、銳化、多影格平均	由主觀好看轉為依雜訊模型的統計最佳估計
第 6 章 頻率域處理	低通、高通、陷波濾波;卷積定理	模糊即低通、還原即受控高通;週期雜訊直接沿用陷波
第 9 章 影像分割	Canny 的高斯平滑、梯度運算	雜訊生假邊緣、模糊使邊緣定位偏移
第 10 章 二值影像處理	最大值/最小值濾波	即型態學的膨脹與侵蝕，一體多面
第 13 章 特徵擷取	Harris、SIFT、ORB 的局部灰階變化	模糊降低角點響應、雜訊製造假特徵
第 15、16 章 深度學習	資料分布、資料增強	還原作為前處理 vs. 劣化作為增強的取舍
第 17 章 邊緣部署	運算成本、即時性	FFT 與反覆式演算法的算力預算評估

表 7-2 第 7 章與全書其他章節的觀念連結

程式實作 Lab 7

Lab 7-1 雜訊診斷與濾波器配對

目標：建立「先診斷、再開藥」的工作習慣。(a)以程式 7-1 對同一張乾淨影像分別加上高斯、椒鹽與 Poisson 雜訊;(b)取平坦區域繪製直方圖，驗證三種雜訊的分布特徵，並用兩種方法估計高斯雜訊的標準差，比較估計值與真值;(c)對三張雜訊影像分別套用均值、高斯、中值、雙邊、非局部平均五種濾波，製作 3×5 的 PSNR 與 SSIM 比較表;(d)寫出結論：哪種雜訊該配哪種濾波器?為什麼中值濾波對高斯雜訊沒有特別優勢，對椒鹽卻近乎完美?(e)挑戰題：自行實作自適應中值濾波，在椒鹽雜訊密度 5%、20%、40% 三種情況下與固定核中值濾波比較，繪製「雜訊密度對 PSNR」曲線。

Lab 7-2 維納濾波與醫學影像去噪

目標：重現 7.6.1 節的 MRI 去噪實驗流程。(a)取公開的 MRI 或 CT 切片影像(或以任意灰階醫學影像替代)，加入已知強度的高斯雜訊作為模擬退化影像;(b)以程式 7-4 的維納濾波處理，掃描參數 K 從 0.0001 到 0.5，繪製「K 對 PSNR」的曲線，找出最佳 K 值;(c)將最佳結果與非局部平均去噪比較 PSNR 與 SSIM，並附上局部放大截圖;(d)在原圖中人工加入一個直徑約十像素的低對比小圓點模擬病灶，重跑一次流程，觀

察 PSNR 最佳的參數是否會把它抹掉——驗證 7.6.1 節提到的「全域指標盲點」;(e) 討論題：PSNR 最高的結果，在醫師眼中是否就是最好的影像？為什麼醫學影像處理不能只看數字？

Lab 7-3 產線運動模糊還原與下游任務評估

目標：完成 7.6.3 節的完整工程流程，並量化還原對下游任務的效益。(a)取一張清晰的工件或零件照片，以程式 7-5 的 motion_psf 製造已知長度與角度的運動模糊，再加入少量高斯雜訊;(b)假裝你不知道真實參數，改由「模擬的產線參數」(自訂速度、曝光時間，並用第 4 章相機校正得到的像素尺寸)推算 PSF，以維納濾波還原;(c)比較三種影像——原始、模糊、還原——在下列三項下游任務的表現：Canny 邊緣像素數與視覺完整度、ORB 特徵點數與對原圖的匹配成功率、以及(若已學到第 16 章)YOLO 偵測的信心值;(d)刻意把推算的模糊長度設錯正負 30%，觀察還原品質如何惡化，體會「PSF 估計準確度」的重要性;(e)分組討論題：承 7.6.3 節的三個層次，若你是這條產線的工程師，實際會怎麼做？請列出成本、可行性與效益的評估。

本章重點整理

- 影像強化追求主觀好看且不需先驗知識，影像還原則基於劣化模型 $g = h * f + n$ 客觀反推原始影像;頻率域形式為 $G = H \cdot F + N$ 。
- 還原是逆問題，常為病態問題(解不連續依賴資料);解法的共通策略是正則化——以先驗知識約束解的形態。
- 模糊即低通濾波，還原即受控的高通增強;與第 5 章 High-Boost、第 6 章高頻強調同屬提升高頻，差別在於增益是否依退化函數量身訂做。提升高頻必然放大雜訊，是本領域的根本矛盾。
- 雜訊型態：高斯(鐘形)、椒鹽(兩端尖峰)、均勻、Poisson(變異數等於平均值)、乘性/斑點、週期性;以平坦區直方圖辨識，以樣本標準差或拉普拉斯殘差估計參數。
- 去雜訊工具：均值家族(算術、幾何、諧波、反諧波)、排序統計家族(中值、最大、最小、alpha 修剪)、自適應濾波(局部降噪、自適應中值)、非局部平均(利用自相似性)。
- 逆濾波 $\hat{f} = G/H$ 因 H 趨近零時放大雜訊而失敗;維納濾波在分母加入常數 K (雜訊訊號比)，約束最小平方則加入 $\gamma|P|^2$ (拉普拉斯正則化)，兩者皆為病態問題的正則化解。
- Richardson-Lucy 為反覆式非線性反卷積，假設 Poisson 雜訊並保證非負;反覆次數過多會放大雜訊，哈伯影像經驗上約 50 次為佳，超過 100 次影像開始崩壞。
- 還原品質直接影響下游：雜訊使 Canny 產生假邊緣、模糊使邊緣定位偏移與特徵點漏檢;對深度學習模型，資料增強常比推論時還原更穩健，且還原引入的偽影可能反而降低準確率。
- AI 修復與生成式補繪效果驚人，但屬於「依學到的先驗猜測」而非「依觀測資料還原」;清晰度不等於正確性，司法、醫療、科學量測場合必須嚴格區分並誠實標註處理流程。

習題

1. 影像強化與影像還原的目標與驗收方式有何不同?各舉一例,並說明兩者對先驗知識的需求差異。

提示與參考解答:強化追求主觀視覺效果(如直方圖等化讓夜景變亮),不需知道劣化原因;還原追求逼近原始影像(如維納濾波去除已知的運動模糊),需要退化函數與雜訊統計量。驗收上強化多靠人眼或下游任務表現,還原則可用PSNR、SSIM 客觀量化。可參考表7-1。

2. 寫出空間域與頻率域的影像劣化模型,說明各符號的物理意義,並列出這個模型的三個隱含假設。

提示與參考解答:空間域 $g = h * f + n$; 頻率域 $G = H \cdot F + N$ 。f 為原始影像、h 為退化函數(PSF)、n 為加性雜訊、g 為觀測影像。三個假設為:退化線性且位置不變、雜訊為加性且與訊號獨立、雜訊零均值。實務上鏡頭邊緣模糊較嚴重違反第一項, Poisson 雜訊違反第二項,感測器暗電流違反第三項。

3. 什麼是適定問題的三個條件?影像還原最常違反哪一個?這在工程上造成什麼後果?

提示與參考解答:解存在、解唯一、解對資料連續依賴。還原最常違反第三條:退化函數在高頻趨近零,還原時要乘上極大倍數,觀測資料中的微小雜訊被同等放大,造成輸出佈滿雪花。應對策略為正則化。

4. 什麼是盲反卷積?為什麼它比非盲反卷積困難?請從解的唯一性說明。

提示與參考解答:盲反卷積指退化函數h未知,需同時估計h與f。困難在於解不唯一:「清楚影像配大模糊核」與「稍模糊影像配小模糊核」可產生相同的觀測結果,必須靠額外先驗(如自然影像梯度分布的稀疏性)才能收斂到合理解。

5. 如何從直方圖判斷一張影像受到的是高斯雜訊、椒鹽雜訊還是均勻雜訊?請描述完整的診斷步驟。

提示與參考解答:步驟:先選取一塊本該平坦均勻的區域(純色背景、天空、工件平整面),取出該區像素繪製直方圖。鐘形對稱分布為高斯;中央分布正常但兩端出現孤立尖峰為椒鹽;矩形平坦分布為均勻。若比較亮區與暗區,變異數隨平均亮度成正比增加則為Poisson。

6. Poisson 雜訊與高斯雜訊最大的差別是什麼?這個特性對低光攝影有何影響?與第5章的夜景模式有何關聯?

提示與參考解答:Poisson 雜訊的變異數等於平均值,強度隨訊號變化(訊號相關),不符合「加性且獨立」的假設。低光時訊號弱,雖然雜訊絕對值小,但訊噪比更差。第5章夜景模式以多影格平均提升訊噪比,平均N張後雜訊標準差降為原來的N平方根分之一,正是對抗此類隨機雜訊的標準做法。

7. 自適應中值濾波相較固定核中值濾波的兩項改良是什麼?在什麼情況下差距最明顯?

提示與參考解答:第一,若中位數本身是雜訊(0或255)則擴大視窗重試,解決高密度雜訊下中位數失效的問題;第二,若中心像素本身不是雜訊則原值保留,完全保護乾淨像素與細節。雜訊密度愈高(20%以上)差距愈明顯,因為固定核在此時中位數常常也是雜訊。

8. 非局部平均利用了自然影像的什麼特性?請舉出兩種特別適合它發揮的影像內容,以及一種不適合的情況。

提示與參考解答:利用自相似性:相似的紋理區塊在影像各處反覆出現。適合磚牆、織品紋理、規律的建築立面、草地等重複性高的內容。不適合的情況:內容高度獨特無重複結構的影像,或需要即時處理的場合(運算量大)。

9. 推導 $\hat{F} = G/H$ 在含雜訊時的展開式,說明逆濾波失敗的數學原因,並說明「半徑受限逆濾波」如何補救及其副作用。

提示與參考解答:將 $G = H \cdot F + N$ 代入得 $\hat{F} = F + N/H$ 。H 在高頻趨近零時 N/H 發散。半徑受限逆濾波只在頻譜中心半徑 D_0 內執行除法,相當於逆濾波加低通,但 D_0 靠試誤決定,且頻域的硬截斷會造成第 6 章所述的振鈴效應。

10. 比較維納濾波與約束最小平方濾波的轉移函數,說明兩者分母的差異代表什麼樣的先驗知識。

提示與參考解答:維納分母為 $|H|^2 + K$ (常數,代表雜訊訊號功率比);約束最小平方分母為 $|H|^2 + \gamma|P|^2$, P 為拉普拉斯的頻率響應,高頻響應大,等於「頻率愈高抑制愈強」,把「還原結果不應劇烈震盪」的平滑先驗寫進數學。兩者都是正則化,差別在正則化項是否隨頻率變化。

11. 維納濾波公式中的 K 值調大或調小,分別對還原結果造成什麼影響?應如何決定起始值?

提示與參考解答: K 太小接近逆濾波,還原銳利但雜訊被放大、影像顆粒粗糙; K 太大則過度抑制高頻,還原不足、影像仍模糊。起始值可由 7.2.2 節估得的雜訊標準差推估雜訊功率,再對照影像本身的功率;實務上以掃描 K 值並觀察 PSNR 曲線找極大值(見 Lab 7-2)。

12. Richardson-Lucy 演算法有哪兩項相對於線性方法的優點?為什麼反覆次數不是愈多愈好?

提示與參考解答:優點:假設 Poisson 雜訊模型,適合光子計數影像(天文、醫學);每次反覆保證結果非負,符合光強度物理意義(維納濾波輸出可能為負)。反覆次數過多會放大雜訊,產生不自然的顆粒與塊狀結構;哈伯經驗約 50 次為佳,超過 100 次形狀開始崩壞。

13. 為什麼說「模糊是低通濾波、還原是高通增強»?本章的反卷積與第 5 章的 Unsharp Masking、第 6 章的高頻強調濾波,三者有何異同?

提示與參考解答:模糊把高頻振幅壓低即低通;還原要把高頻補回即高通增強。三者相同處:都提升高頻、都會放大雜訊。相異處:Unsharp Masking 的增益由使用者主觀設定的 k 決定;高頻強調由濾波器形狀決定;反卷積的增益由退化函數 H 精確決定,是量身訂做的補償。可參考 7.4.2 節與表 7-2。

14. 影像還原對第 9 章的 Canny 邊緣偵測與第 13 章的 ORB 特徵擷取分別有何影響?請說明雜訊與模糊各自造成的問題。

提示與參考解答:雜訊:梯度運算放大雜訊, Canny 產生大量細碎假邊緣;ORB 的 FAST 角點依賴中心與圓周像素亮度比較,雜訊會製造大量虛假特徵點。模糊:梯度峰值變平坦,非極大值抑制難以定位,邊緣位置偏移;角點響應值下降,真實角點落到門檻之下被漏檢,匹配率下降。

15. 對一個已訓練好的 YOLO 模型,在推論前加上影像還原模組,是否一定會提升準確率?請說明三種可能情況。

提示與參考解答：不一定。情況一：還原減輕分布偏移，準確率提升。情況二：還原引入訓練時未見過的偽影(振鈴、過度銳化光暈)，反而使輸入更偏離訓練分布，準確率下降。情況三：還原的運算成本使系統無法即時處理，實際部署效益為負。較穩健的做法是在訓練階段以劣化樣本做資料增強，見 7.7.3 節。

16. 承 7.6.3 節，若產線工件的運動模糊長度為 20 像素，身為工程師你會依什麼順序考慮解法？請說明每個層次的具體做法。

提示與參考解答：順序為光學機構層、演算法層、學習層。光學機構層(治本)：縮短曝光時間並加強打光、改用全域快門相機、加裝頻閃光源凍結畫面。演算法層：由編碼器速度、曝光時間與像素尺寸推算 PSF，以維納濾波做非盲反卷積。學習層：將運動模糊作為資料增強加入訓練集，讓模型直接適應。

17. (思辨題)監視器畫面經 AI 超解析度處理後在法庭上作為證據，你贊成嗎？請以本章「還原 vs. 腦補」的區分，提出至少三點論證。

提示與參考解答：本題無標準答案，評分重點在論證品質。可能論點包括：傳統反卷積只使用觀測資料與物理模型，細節來自殘存資訊；生成式模型輸出「最合理的猜測」，可能產生原本不存在的細節；模型的訓練資料偏誤會反映在輸出上(如人臉超解析度改變族裔特徵)；清晰度不等於正確性，而清晰的影像對陪審團有過度的說服力；可主張區分「可採信的還原」與「僅供參考的重建」，並要求揭露完整處理流程與演算法。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 5(影像還原與重建)。
- ESA/Hubble, History: The Spherical Aberration Problem.(本章叫獸開講之史料來源)
- Hanisch, R. J., & White, R. L., The Restoration of HST Images and Spectra II, STScI, 1994.
- Richardson, W. H., Bayesian-Based Iterative Method of Image Restoration, JOSA, 1972; Lucy, L. B., An Iterative Technique for the Rectification of Observed Distributions, AJ, 1974.
- Buades, A., Coll, B., & Morel, J.-M., A Non-Local Algorithm for Image Denoising, CVPR, 2005.(非局部平均原始論文)
- Hunt, B. R., The Application of Constrained Least Squares Estimation to Image Restoration by Digital Computer, IEEE Trans. Computers, 1973.
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 10(計算攝影與影像還原)。
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 8 章 色彩影像處理

本章學習目標

- 說明三原色理論與人眼感色機制，理解 RGB 加色與 CMY 減色系統的差異。
- 描述 HSL 與 HSV 色彩模型的圓柱、圓錐與雙圓錐幾何詮釋，並說明兩者的差異。
- 比較 YCbCr 與 CIE Lab 色彩空間的設計目的與應用場合。
- 正確使用 OpenCV 進行色彩空間轉換，避開 BGR 與 HSV 數值範圍的陷阱。
- 運用虛擬色彩將灰階或非可見光資料轉為易於判讀的彩色影像，並理解色階選擇的專業考量。
- 以灰階世界法等演算法實作白平衡，並完成基於 HSV 的顏色分割與物件追蹤。
- 理解色彩恆常性對機器視覺的啟示，並據以設計不受光源干擾的檢測系統。

叫獸開講|藍黑還是白金?一件洋裝如何撕裂全世界

2015 年 2 月，一張洋裝照片被貼上社群網站，附帶一句求助：「這件洋裝到底是白金色還是藍黑色？」接下來的 48 小時，這個問題席捲全球，朋友吵架、情侶決裂、名人與政治人物紛紛表態，連視覺科學家都被捲進戰局。最詭異的是：雙方都不是在說謊，也不是視力有問題——他們看著同一堆像素，大腦卻給出截然不同的答案。而這件洋裝的真實顏色是藍色與黑色。

科學界的反應非常迅速。同年，麻省理工學院的 Rosa Lafer-Sousa、Katherine Hermann 與 Bevil Conway 在《Current Biology》發表了研究：他們對 1401 人做了自由回答(而非二選一)的調查，確認多數人確實只看到白金或藍黑兩種答案，少數人看到藍棕;而且回報白金的比例在年長者與女性中較高。這證實了「分歧是真實的」，不是社群媒體提問方式造成的假象。

關鍵詞叫做色彩恆常性(Color Constancy)：同一顆蘋果，在正午陽光下、在黃昏夕照下、在螢光燈下，反射到你眼睛的光譜其實差異極大，但你始終覺得它是紅色的。這是因為大腦會自動推測「現在的光源是什麼顏色」，再把光源的影響扣掉，還原出物體本身的顏色——這就是人腦內建的自動白平衡。

問題在於，這張洋裝的照片曝光過度、背景線索模糊，光源資訊變得極度曖昧，大腦只能仰賴各自的「先驗假設」來猜。研究團隊做了一個漂亮的驗證：把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就會趨於一致，並且與被暗示的光源方向吻合。換句話說——看到白金的人，大腦假設場景被偏藍的日光或陰影照亮，於是扣掉藍色成分;看到藍黑的人，大腦假設是偏黃的室內人造光，於是扣掉黃色成分。

那為什麼不同人的假設不同?一個廣受討論的假說是生活作息：紐約大學的 Pascal Wallisch 在 2017 年的《Journal of Vision》發表了一項超過一萬三千人

的大規模研究，發現自認是「雲雀」(早起、白天活動)的人較常看到白金，而「夜貓子」(習慣人造光下活動)的人較常看到藍黑，且呈現劑量反應的關係。不過也要誠實說明：Lafer-Sousa 團隊自己的資料對這個假說只提供了微弱的支持，學界至今仍在討論確切的機制。這也是科學該有的樣子——一個引人入勝的假說，需要更多證據來檢驗。

這個全民事件對影像處理學生有三重啟示。第一，你「看到」的顏色不是物理量，而是大腦處理過的結果——同一組 RGB 數值，可以被解讀成完全不同的顏色。第二，電腦視覺系統必須做同一件事：本章 8.5 節的白平衡演算法，就是在教機器扣掉光源的影響，而 Night Sight 的學習式白平衡(第 5 章)正是這條路線的現代版本。第三，也是最重要的：在工業檢測與機器人視覺中，如果光源不穩定，顏色判讀就會像這件洋裝一樣不可靠。這正是第 17 章 AOI 系統要不惜血本控制打光的原因。

思考題：如果要設計一台「絕不會被光源騙倒」的顏色檢測系統，你會採取什麼策略?(提示一：控制光源;提示二：在畫面中放一張已知顏色的參考色卡;提示三：改用對光源較不敏感的特徵)

8.1 色彩的基礎：從人眼到 RGB

8.1.1 三原色理論

人眼視網膜有三種錐狀細胞，分別對長波(紅)、中波(綠)、短波(藍)敏感。任何顏色的光進入眼睛，都被化約成三個數值的組合送進大腦——這就是三原色理論的生理基礎，也是為什麼數位影像用三個通道就能表現幾乎所有顏色。

值得深思的是同色異譜(Metamerism)現象：兩束光譜完全不同的光，只要刺激三種錐狀細胞的比例相同，人眼就會覺得它們是同一個顏色。這既是彩色顯示技術得以成立的原因(螢幕只用紅綠藍三種光源，就能模擬幾乎所有顏色)，也是色彩管理困難的根源——在某個光源下看起來相同的兩塊布料，換到另一個光源下可能顏色明顯不同，這在紡織與印刷業是實際的品管難題。

這也對機器視覺有一個重要啟示：一般 RGB 相機與人眼一樣，只記錄三個數值，因此無法分辨同色異譜的物體。若要真正區分材質(例如判斷兩塊看似同色的塑膠是否為同一種材料)，就需要記錄完整光譜的多光譜或高光譜相機——這在農產品分級、材料檢測、遙測中相當常見。

8.1.2 加色與減色系統

RGB 是加色系統：光源相加，紅加綠得黃，三色全開得白，適用於自發光的裝置——螢幕、投影機、LED。CMY(青、洋紅、黃)是減色系統：顏料吸收特定波長，疊愈多愈暗，三色相加理論上得黑(實務上因顏料不純而加入黑色 K，成為印刷用的 CMYK)。

同一張照片在螢幕上鮮豔、印出來卻黯淡，正是因為兩個系統的色域(可表現的顏色範圍)不同。這也解釋了為什麼專業的印刷流程需要色彩管理與打樣——這正是 8.3.2 節 CIE Lab 存在的理由。

8.1.3 RGB 的侷限

RGB 對機器友善(直接對應感光元件與螢幕)，卻對人類不直觀。兩個具體的痛點：

第一，調整困難。請問「把這個顏色變深一點但色相不變」該怎麼改 RGB 三個數值？答案是三個都要按比例調整，很不方便；若只改其中一個，色相就跑掉了。

第二，也是對機器視覺更致命的：同一個物體在陰影中與陽光下，RGB 三個值全部大幅改變，難以判斷「還是不是同一個顏色」。舉例來說，一顆紅色蘋果在陽光下可能是 (220, 40, 40)，在陰影中變成 (90, 15, 15)——數值差了兩倍以上，但它顯然是同一顆蘋果。這使得「用 RGB 數值範圍來偵測特定顏色」在光線變化的環境中極不可靠。

這兩個痛點，催生了下一節的 HSL 與 HSV。

8.2 HSL 與 HSV：符合人類直覺的色彩模型

HSV(色相 Hue、飽和度 Saturation、明度 Value)與 HSL(色相、飽和度、亮度 Lightness)把「是什麼顏色」與「有多亮、多鮮豔」分離開來，更貼近人類描述顏色的方式：一個「深一點的紅」只需要調降 V 或 L，色相 H 保持不變。

對機器視覺而言，這個分離帶來一個關鍵好處：承 8.1.3 節，陽光下與陰影中的紅蘋果，雖然 RGB 差異巨大，但它們的 H(色相)卻相當接近——變的主要是 V(明度)。因此「以 H 為主要判準、對 V 放寬門檻」的顏色分割，遠比在 RGB 空間設範圍來得穩健。這正是 8.6 節顏色追蹤的核心策略。

8.2.1 幾何詮釋：圓柱、圓錐與雙圓錐

這兩個模型有三種常見的幾何視覺化方式，各自對應不同的理解角度：

1. 圓柱模型：最直接的表示法。繞著圓周一圈是色相 H(0 度紅、120 度綠、240 度藍)；離中心軸的半徑是飽和度 S(中心為灰，愈外圈愈鮮豔)；沿著軸的高度是 V 或 L。缺點是它「浪費」了很多空間：當 $V=0$ 時整層都是黑色，無論 H 與 S 為何，但圓柱仍然畫出一整個圓面。
2. 圓錐模型(HSV)：把圓柱底部收縮成一個點，正好對應「明度為零時只有黑色」的事實。頂面圓心為白色，頂面外緣是最飽和的純色。這是 HSV 最貼切的幾何。
3. 雙圓錐模型(HSL)：上下各一個圓錐，尖端分別是白($L=1$)與黑($L=0$)，最飽和的顏色出現在中間高度 $L=0.5$ 的赤道圓周上。這反映了 HSL 的設計：亮度 L 是對稱的，從黑經過純色到白。

HSV 與 HSL 的關鍵差異就在這裡：HSV 的 $V=1$ 頂面同時包含白色與所有純色(把白色和純紅都視為「最亮」)；HSL 則把純色放在中間，只有 $L=1$ 才是白色。

實務上的選擇原則：做顏色分割與追蹤時 HSV 較常用，因為「色相加飽和度」的門檻設定較直覺，且 V 通道直接對應「有多亮」便於放寬；而做顏色調整介面(如繪圖軟體的調色盤)則 HSL 較符合使用者對「亮度」的期待——把滑桿拉到最上面應該是白色，而非刺眼的純色。

8.2.2 OpenCV 的 HSV 數值範圍陷阱

這是初學者的第二大地雷(第一大是第 2 章的 BGR)。理論上色相 H 的範圍是 0 至 360 度，但 OpenCV 為了塞進 8 位元的 uint8(上限 255)，把 H 除以二，範圍變成 0 至 179;S 與 V 則是 0 至 255。因此網路上查到的「紅色 H 約為 0 或 360」，在 OpenCV 中要換算成 0 或 179。

另一個必須注意的實務細節：紅色的色相跨越 0 度的接縫，因此偵測紅色時通常需要設定兩段區間再取聯集。這個「環狀資料的接縫問題」在處理角度類資料時普遍存在(第 12 章光流的方向、第 13 章 SIFT 的方向直方圖也都要處理)，值得記在心裡。

還有一個常被忽略的陷阱：當顏色很暗或很不飽和時(接近黑、白、灰),H 值變得極不穩定甚至無意義——想像圓錐的尖端，所有色相在那裡都收縮成同一點。因此顏色分割時務必同時設定 S 與 V 的下限，把這些「色相無意義」的區域排除，否則會出現大量誤判。

程式 8-1 HSV 轉換與顏色門檻

```
import cv2, numpy as np

bgr = cv2.imread("robot.jpg")
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

# OpenCV:H 為 0~179、S 與 V 為 0~255
h, s, v = cv2.split(hsv)
print("H 範圍:", h.min(), h.max())

# --- 偵測藍色:單一區間即可 ---
# 注意 S 與 V 的下限:排除過暗與過灰的區域(H 在那裡無意義)
blue = cv2.inRange(hsv, (100, 120, 60), (130, 255, 255))

# --- 偵測紅色:跨越 0 度接縫,需兩段聯集 ---
red1 = cv2.inRange(hsv, (0, 120, 60), (10, 255, 255))
red2 = cv2.inRange(hsv, (170, 120, 60), (179, 255, 255))
red = cv2.bitwise_or(red1, red2)

# --- 驗證 8.1.3 節的說法:同一物體在明暗下的 RGB 與 HSV 變化 ---
bright = np.uint8([[[40, 40, 220]]]) # BGR 的亮紅
dark = np.uint8([[[15, 15, 90]]]) # 同色但變暗
for name, px in [("亮", bright), ("暗", dark)]:
    hsv_px = cv2.cvtColor(px, cv2.COLOR_BGR2HSV)[0, 0]
    print(f"{name}: BGR={px[0,0]}, HSV={hsv_px}")
# 觀察:BGR 三值都大幅改變,但 H 幾乎不變 → 故以 H 為判準較穩健

cv2.imwrite("mask_red.png", red)
```

8.3 YCbCr 與 CIE Lab

8.3.1 YCbCr：為壓縮與傳輸而生

YCbCr 把亮度 Y 與兩個色差通道 Cb(藍色差)、Cr(紅色差)分開。它的設計動機非常務實：人眼對亮度細節的敏感度遠高於色彩細節，因此可以對 Cb、Cr 大幅降取樣(常見的 4:2:0 取樣就是把色度解析度砍成四分之一)，而人眼幾乎察覺不出差異。

這是 JPEG 壓縮與所有數位視訊標準的第一步——第 11 章談 JPEG 時會完整展開。它也是數位電視與類比電視相容的關鍵：黑白電視只讀 Y 通道即可正常顯示。

值得留意這個「把資源分配給人眼在意之處」的設計思想，它在本書中反覆出現第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、本節的色度降取樣、第 11 章 JPEG 對高頻係數用較大的量化步長。三者都是同一種智慧——不是所有資訊都同等重要，把位元花在刀口上。

8.3.2 CIE Lab：為量測色差而生

CIE Lab 由國際照明委員會制定，目標是感知均勻(Perceptually Uniform)：在這個空間中，兩個顏色的歐氏距離大致正比於人眼感受到的色差大小。

這在 RGB 中是做不到的——RGB 空間裡同樣的數值差距，在不同區域造成的視覺差異可能天差地別(人眼對綠色的細微變化極為敏感，對藍色則相對遲鈍)。Lab 的 L 為亮度(0 黑到 100 白)，a 為綠紅軸，b 為藍黃軸。

應用場合：印刷與紡織的品管(以 ΔE 色差值判定是否合格，通常 ΔE 小於 1 表示人眼幾乎無法分辨)、色彩校正、以及第 5 章介紹過的技巧——對彩色影像做 CLAHE 時，轉到 Lab 只處理 L 通道，色彩才不會失真。

色彩空間	設計目的	通道意義	典型應用
RGB	對應感光元件與螢幕	紅、綠、藍強度	顯示、儲存
HSV	符合人類對顏色的描述	色相、飽和度、明度	顏色分割與追蹤
HSL	亮度對稱、便於調色	色相、飽和度、亮度	調色盤、繪圖軟體
YCbCr	利用人眼對色度不敏感以壓縮	亮度、藍色差、紅色差	JPEG、視訊編碼
CIE Lab	感知均勻，可量測色差	亮度、綠紅軸、藍黃軸	品管色差、色彩校正

表 8-1 常用色彩空間比較

8.4 虛擬色彩：讓看不見的資訊現形

虛擬色彩(Pseudocolor，又稱偽彩色)是把灰階或單一數值的資料，依照一張色彩對照表(Colormap)映射成彩色影像。

它的目的不是「還原真實顏色」——這些資料本來就沒有顏色——而是利用人眼分辨顏色的能力遠強於分辨灰階的特性：人類大約只能分辨數十階灰度，卻能分辨數萬種顏色。把資料的細微差異映射到色彩，就能讓原本看不出來的結構一目了然。

應用場景極廣：熱影像儀把溫度映射成藍到紅的色階；醫學影像用色彩標示血流速度或代謝強度；衛星遙測把近紅外波段染成紅色以突顯植被；深度相機(第 17 章)把距離映射成彩虹色；第 12 章的稠密光流用色相表示運動方向；第 15、16 章的神經網路熱力圖也是靠虛擬色彩顯示「AI 在看哪裡」。

8.4.1 色階選擇的專業考量

經典的 JET(彩虹色)雖然搶眼，但在科學視覺化領域近年頗受批評，原因有三：其一，它的亮度變化不是單調遞增的(中間的黃綠色比兩端更亮)，容易在資料中製造出實際不存在的「邊界」，誤導判讀；其二，對色盲使用者不友善(約百分之八的男性有紅綠色盲)；其三，轉成黑白列印後資訊完全喪失。

VIRIDIS、MAGMA 等感知均勻的色階已成為新的推薦選項——它們的亮度單調遞增、色盲友善、且黑白列印後仍保有層次。在論文與報告中選擇 colormap，也是一種專業素養；這個考量與 8.3.2 節 CIE Lab 的「感知均勻」是同一種思想在視覺化上的應用。

程式 8-2 虛擬色彩與色階的檢驗

```
import cv2

gray = cv2.imread("thermal_raw.png", cv2.IMREAD_GRAYSCALE)

# 常用色彩對照表
jet      = cv2.applyColorMap(gray, cv2.COLORMAP_JET)      # 經典彩虹(慎用)
hot      = cv2.applyColorMap(gray, cv2.COLORMAP_HOT)      # 熱影像風格
viridis  = cv2.applyColorMap(gray, cv2.COLORMAP_VIRIDIS)  # 感知均勻,推薦
bone     = cv2.applyColorMap(gray, cv2.COLORMAP_BONE)     # 醫學影像風格

# 檢驗色階是否「亮度單調遞增」:轉回灰階看是否仍為漸層
import numpy as np
ramp = np.tile(np.arange(256, dtype=np.uint8), (50, 1))
for name, cm in [("jet", cv2.COLORMAP_JET),
                 ("viridis", cv2.COLORMAP_VIRIDIS)]:
    colored = cv2.applyColorMap(ramp, cm)
    back = cv2.cvtColor(colored, cv2.COLOR_BGR2GRAY)
    diff = np.diff(back[0].astype(np.int16))
    print(f"{name}: 亮度單調遞增? {np.all(diff >= 0)}")
    # JET 會出現 False → 這正是它造成假邊界的原因

cv2.imwrite("pseudo_viridis.png", viridis)
```

8.5 白平衡與色彩校正

回到叫獸開講的核心問題：如何讓機器扣掉光源的影響？這正是白平衡(White Balance)要解決的事，也是人腦色彩恆常性的工程版本。

8.5.1 灰階世界假設

最簡單的演算法是灰階世界假設(Gray World)：假設整個場景的平均顏色應該是中性灰。若實測的平均偏藍，代表光源偏藍，於是把藍通道整體調降、紅通道調升。

它在色彩豐富的場景中效果不錯，但遇到「整片草地」、「整面紅牆」、「大片藍天」這類單一色調的畫面就會失效——因為前提假設不成立，演算法會誤以為光源偏綠(或偏紅、偏藍)，硬是把草地校正成灰色。這是一個很好的提醒：所有演算法都建立在假設之上，而工程師的責任是判斷假設是否成立。

8.5.2 完美反射假設與色卡校正

第二種是完美反射假設(White Patch / Retinex)：找出影像中最亮的像素，假定它應該是純白，據此計算各通道的增益。它對「畫面中確實有白色物體」的場景很準，但遇到過曝或高光反射就會誤判——因為過曝的像素已經飽和，其色彩資訊已經喪失。

工業場合最可靠的做法則是第三種：在畫面中放置標準色卡(如 24 色的 ColorChecker)，以已知的參考色計算完整的色彩校正矩陣。這不只是白平衡，而是把整個色彩響應校正到標準——這也是叫獸開講思考題的標準答案之一，更是農產品分級、印刷打樣、醫學影像等對顏色準確度有要求的場合的標準流程。

至於現代的第四種路線：第 5 章 Night Sight 的學習式白平衡——用神經網路從大量標註資料中學習「什麼樣的影像看起來白平衡正確」。它在傳統方法失效的困難場景(如夜景、單色調畫面)表現優異，但也繼承了深度學習的侷限(第 15 章)：遇到訓練分布外的場景可能失準，且難以解釋。

程式 8-3 兩種白平衡演算法與其失效情境

```
import cv2, numpy as np

def gray_world(img):
    """灰階世界白平衡:假設場景平均為中性灰"""
    result = img.astype(np.float32)
    avg_b, avg_g, avg_r = (result[:, :, 0].mean(),
                           result[:, :, 1].mean(),
                           result[:, :, 2].mean())
    avg_gray = (avg_b + avg_g + avg_r) / 3
    result[:, :, 0] *= avg_gray / avg_b
    result[:, :, 1] *= avg_gray / avg_g
    result[:, :, 2] *= avg_gray / avg_r
    return np.clip(result, 0, 255).astype(np.uint8)

def white_patch(img, percentile=99):
    """完美反射假設:以最亮的像素作為白色參考"""
    result = img.astype(np.float32)
    for c in range(3):
        ref = np.percentile(result[:, :, c], percentile) # 取分位數較穩健
        result[:, :, c] *= 255.0 / max(ref, 1)
    return np.clip(result, 0, 255).astype(np.uint8)

img = cv2.imread("yellowish_indoor.jpg")
cv2.imwrite("wb_grayworld.jpg", gray_world(img))
```

```
cv2.imwrite("wb_whitepatch.jpg", white_patch(img))

# 驗證失效情境:對一張「整片草地」的影像套用灰階世界
grass = cv2.imread("grass_field.jpg")
cv2.imwrite("wb_grass_fail.jpg", gray_world(grass))
# 觀察:草地被硬生生校正成灰綠色 → 假設不成立時演算法失效
```

8.6 顏色分割與物件追蹤

把前面所學組合起來，就得到機器人視覺最基礎也最實用的技能：用顏色找出目標物並持續追蹤。

標準流程五步驟：轉 HSV(利用色相對光照變化較穩健)→ 以 inRange 產生二值遮罩 → 用型態學運算(第 10 章)清除雜點與補洞 → 找輪廓取最大者(第 9 章)→ 計算中心座標並輸出控制訊號。這正是循跡機器人、色球追蹤、簡易分揀系統的骨幹。

參數設定的實務心法：對 H 設窄門檻(因為色相是主要判準)、對 S 與 V 設寬門檻但要有下限(容許明暗變化，同時排除色相無意義的暗處與灰處)。這個「窄 H、寬 V」的原則，直接源自 8.1.3 與 8.2 節對 RGB 與 HSV 的比較。

程式 8-4 HSV 顏色追蹤與機器人控制訊號

```
import cv2, numpy as np

cap = cv2.VideoCapture(0)          # 或 ESP32-CAM 的串流網址(第 2 章)
kernel = np.ones((5, 5), np.uint8)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    frame = gray_world(frame)        # 白平衡前處理(8.5 節)
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # 窄 H、寬 V,但 S 與 V 有下限
    mask = cv2.inRange(hsv, (35, 100, 60), (85, 255, 255))    # 綠色物件

    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)      # 去雜點
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)     # 補破洞

    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                cv2.CHAIN_APPROX_SIMPLE)
    if cnts:
        c = max(cnts, key=cv2.contourArea)
        if cv2.contourArea(c) > 500:
            x, y, w, h = cv2.boundingRect(c)
            cx, cy = x + w // 2, y + h // 2
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)

            # 機器人控制訊號:依偏差量決定轉向
            err = cx - frame.shape[1] // 2
```



```

        cmd = "左轉" if err < -30 else ("右轉" if err > 30 else "直
行")
        cv2.putText(frame, f"{cmd} (err={err})", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        cv2.imshow("tracking", frame)
        if cv2.waitKey(1) == 27:
            break

cap.release(); cv2.destroyAllWindows()

```

8.7 應用案例

8.7.1 案例一：農產品的成熟度分級

情境：番茄、香蕉、水果的成熟度可由顏色判斷，是雲林在地農業自動化最直接的應用。做法：以 HSV 或 Lab 空間萃取顏色特徵(如色相的平均值與分布)，建立成熟度與顏色的對應關係，再以第 14 章的分類器或簡單的門檻規則分級。

這個案例最能凸顯叫獸開講的教訓：光源是成敗的關鍵。同一顆番茄在陰天與晴天、上午與傍晚拍攝，顏色可能落到不同的等級。因此實務上的必要措施是——建立封閉的檢測箱、使用穩定的 LED 光源、並在每次拍攝時納入標準色卡校正。若做不到這些(如田間直接拍攝)，就必須用第 15 章的資料增強讓模型學會適應光線變化，或改用對光源較不敏感的特徵。

8.7.2 案例二：醫學與科學的虛擬色彩

情境：熱影像儀顯示體溫分布、超音波都卜勒顯示血流方向與速度、功能性磁共振顯示腦部活動強度。這些資料本身都是單一數值(溫度、速度、訊號強度)，沒有顏色，必須透過虛擬色彩才能讓人判讀。

這裡有一個攸關判讀正確性的專業考量，呼應 8.4.1 節：色階的選擇會直接影響醫師「看到什麼」。用 JET 彩虹色階顯示的溫度分布，可能因為亮度非單調而讓人誤以為某處有明顯的邊界或異常；而選用感知均勻的色階，則能讓資料的真實漸變忠實呈現。此外，色階的數值範圍(對應第 5.7.2 節的視窗化)同樣關鍵——同一份資料用不同的範圍映射，可以讓病灶明顯或完全隱形。這再次呼應第 7、16 章的倫理主題：視覺化的選擇不是中性的。

8.7.3 案例三：工業 AOI 的光源設計

情境：檢測電子零件的顏色標記、判斷線材的色碼、辨識產品標籤。這類任務在受控環境下極為可靠，關鍵完全在於「控制變因」。

工程上的標準做法包括：使用高演色性、色溫穩定的 LED 光源(廉價 LED 的光譜不連續，會使某些顏色偏移)；設計光源的方向與擴散方式以避免高光反射(反光處會過曝，色彩資訊喪失)；以遮光罩隔絕環境光的干擾；定期以標準色卡驗證色彩響應是否漂移(LED 隨使用時間會衰減與偏色)。

這個案例是叫獸開講思考題的完整答案：與其設計一個「能對抗任何光源」的聰明演算法，不如從源頭消除光源的變異——這在可控的產線環境中，永遠是更務實、更可靠的工程選擇。這也呼應第 7.6.3 節「三個層次的解法應以光學機構層為優先」的原則。

8.7.4 案例四：色彩在深度學習中的角色

情境：訓練第 15、16 章的模型時，色彩相關的決策比想像中更多。

第一，輸入格式：大多數預訓練模型期待 RGB 順序，而 OpenCV 讀進來是 BGR(第 2 章)——這個順序錯誤是常見的 bug，會使準確率莫名下降。第二，資料增強：隨機調整色相、飽和度、亮度(第 16 章 YOLO 的 `hsv_h`、`hsv_s`、`hsv_v` 參數)是標準做法，目的正是讓模型對光源變化不敏感——這是「用學習取代白平衡」的思路。第三，是否轉灰階：若任務與顏色無關(如形狀辨識)，轉灰階能減少三分之二的計算量;但若顏色是關鍵線索(如成熟度、警示標誌)，則絕不能丟棄。

這些決策說明了一件事：即使進入深度學習時代，對色彩的理解仍然是工程師必備的判斷力來源。模型不會替你決定該用哪個色彩空間、該不該做白平衡——那是你的工作。

8.8 本章與全書的觀念連結總表

本章觀念	相關章節	如何延伸
BGR 順序	第 2、15、16 章	OpenCV 的歷史包袱;深度模型期待 RGB
拜爾陣列與去馬賽克	第 3、4 章	每像素僅記錄一色，需內插補齊
只處理亮度通道	第 5 章	彩色影像做 CLAHE 應轉 Lab 只等化 L
色度降取樣	第 11 章	JPEG 第一步的 YCbCr 與 4:2:0
虛擬色彩	第 10、12、15 章	距離轉換視覺化;稠密光流;神經網路熱力圖
白平衡與色彩恆常性	第 5、15 章	Night Sight 的學習式白平衡;資料增強取代白平衡
HSV 顏色分割	第 9、10、12 章	分割的顏色依據;型態學清理;CamShift 追蹤
光源控制	第 7、9、17 章	AOI 取像設計;打光不均的處理;治本優於治標

表 8-2 第 8 章與全書其他章節的觀念連結

程式實作 Lab 8

Lab 8-1 色彩空間探索器

目標：建立對色彩空間的直覺，並打造整學期最常用的調參工具。(a)寫一支程式，以 `cv2.createTrackbar` 建立六個滑桿(H、S、V 的上下界)，即時顯示遮罩與被遮罩後的結果;(b)用它找出教室中五種不同顏色物件的 HSV 範圍，製成表格;(c)驗證 8.1.3 節的論述：記錄同一物件在明亮處與陰影中的 RGB 與 HSV 值，比較哪些通道變化大、哪些穩定;(d)討論：為什麼顏色分割通常對 H 設窄門檻、對 V 設寬門檻?為什麼 S 與 V 必須設下限?(e)把這支工具存成共用程式，第 9、10、12 章的 Lab 都會用到。

Lab 8-2 洋裝效應實測

目標：親身驗證叫獸開講的現象。(a)取一個顏色明確的物件(如紅色馬克杯)，分別在窗邊日光、日光燈、白熾燈、以及手機的暖色螢幕光下拍攝;(b)量測同一位置的 RGB 與 HSV 值，製成表格，計算各通道的變化幅度;(c)套用灰階世界與完美反射兩種白平衡，比較校正後的數值是否收斂;(d)在畫面中放入一張白紙作為參考，以「白紙應為中性灰」為準則手動校正，比較與自動演算法的差異;(e)討論：這個實驗對「用顏色做工業檢測」有什麼啟示?

Lab 8-3 虛擬色彩與色階評估

目標：培養視覺化的專業素養。(a)取一張灰階影像(熱影像、深度圖或一般照片)，分別以 JET、HOT、VIRIDIS、BONE 四種色階顯示;(b)執行程式 8-2 的檢驗，量化各色階的亮度是否單調遞增，找出 JET 在哪個區段違反單調性;(c)把四張結果轉成黑白列印(或轉灰階)，比較哪些色階仍保有層次;(d)模擬色盲觀看：把結果的紅綠通道混合，觀察哪些色階仍可判讀;(e)結論：為一份要投稿的技術報告選擇色階，你會怎麼選?請說明理由。

Lab 8-4 循跡機器人的顏色偵測

目標：完成可上機的顏色追蹤模組。(a)以程式 8-4 為基礎，改用 ESP32-CAM 的串流網址作為輸入(第 2 章);(b)加入白平衡前處理，比較開啟與關閉時在不同光源下的穩定度;(c)輸出控制訊號：依目標物中心與畫面中心的偏差量，印出轉向指令與偏差角度;(d)量測系統的處理速度(FPS)，並思考：若改用第 16 章的 YOLO 偵測，速度與穩定度會如何變化?各適合什麼場合?(e)挑戰題：同時追蹤兩種顏色的物件並標示兩者的距離，或設計一個「顏色遙控」——用不同顏色的卡片對機器人下指令。

本章重點整理

- 人眼三種錐狀細胞是三原色理論的生理基礎;同色異譜使不同光譜可呈現相同顏色，是彩色顯示成立的原因，也使一般 RGB 相機無法區分材質(需多光譜相機)。
- RGB 為加色系統(螢幕),CMY(K) 為減色系統(印刷)，色域不同;RGB 對機器友善但對人不直觀，且同一物體在明暗下三通道全變，不利顏色偵測。

- HSV/HSL 分離「色相」與「明暗鮮豔度」;幾何上可用圓柱、圓錐(HSV)、雙圓錐(HSL)詮釋, HSL 的純色位於 $L=0.5$ 的赤道。色相對光照變化較穩健, 故顏色分割以 H 為主要判準。
- OpenCV 的 HSV 範圍為 $H:0\sim179$ 、 $S/V:0\sim255$;偵測紅色需跨 0 度接縫以兩段區間取聯集;暗處與灰處的 H 無意義, 必須設定 S 與 V 的下限。
- YCbCr 為壓縮而生(色度可降取樣),CIE Lab 為感知均勻而生(可量測色差 ΔE);「把資源分配給人眼在意之處」的思想貫穿拜爾陣列、色度降取樣與 JPEG 量化。
- 虛擬色彩利用人眼辨色優於辨灰的特性;JET 因亮度非單調而可能製造假邊界且對色盲不友善, 建議改用 VIRIDIS 等感知均勻色階。
- 白平衡是色彩恆常性的工程版本: 灰階世界(假設場景平均為灰, 單色調畫面失效)、完美反射(以最亮處為白, 過曝時誤判)、標準色卡校正(最可靠)、學習式白平衡(現代路線)。工業場合應優先從源頭控制光源。

習題

1. 什麼是同色異譜?它為什麼是彩色顯示技術得以成立的基礎?它對機器視覺有什麼限制上的啟示?

提示與參考解答: 同色異譜指兩束光譜完全不同的光, 只要刺激三種錐狀細胞的比例相同, 人眼就覺得是同一顏色。它讓螢幕只用紅綠藍三種光源就能模擬幾乎所有顏色。啟示: 一般 RGB 相機與人眼一樣只記錄三個數值, 因此無法分辨同色異譜的物體(如兩塊看似同色但材質不同的塑膠);要真正區分材質需使用記錄完整光譜的多光譜或高光譜相機。

2. 說明加色系統與減色系統的差異, 並解釋為何螢幕上鮮豔的照片印出來常變暗淡。

提示與參考解答: 加色系統(RGB)是光源相加, 三色全開得白, 用於自發光裝置(螢幕、投影機、LED)。減色系統(CMY)是顏料吸收特定波長, 疊愈多愈暗, 三色相加理論得黑(實務加K成CMYK), 用於印刷。變暗淡的原因: 兩系統的色域(可表現的顏色範圍)不同, 螢幕能顯示的某些鮮豔顏色超出印刷色域, 只能以最接近的顏色代替。

3. 為什麼說 RGB 不利於顏色偵測?請以「同一物體在陽光下與陰影中」說明, 並解釋 HSV 如何改善。

提示與參考解答: 同一顆紅蘋果在陽光下可能是 $(220, 40, 40)$ 、陰影中變成 $(90, 15, 15)$, 三個通道都大幅改變, 以 RGB 數值範圍偵測極不可靠。HSV 把「是什麼顏色」(H)與「有多亮」(V)分離: 明暗變化主要影響V, 而H相當穩定。因此以H為主要判準、對V放寬門檻的分割策略, 遠比在RGB空間設範圍穩健。

4. 請比較 HSV 的圓錐模型與 HSL 的雙圓錐模型, 說明最飽和的純色分別位於幾何體的哪個位置, 以及兩者各適合什麼應用。

提示與參考解答: HSV 圓錐: 底部尖端為黑, 頂面圓心為白、頂面外緣為最飽和的純色(白與純色同在 $V=1$)。HSL 雙圓錐: 上下尖端分別為白與黑, 最飽和的純色在中間高度 $L=0.5$ 的赤道圓周。應用: HSV 適合顏色分割與追蹤(H加S門檻直覺、V便於放寬);HSL 適合調色盤與繪圖軟體(把滑桿拉到最上應為白色, 符合使用者對「亮度」的期待)。

5. OpenCV 的色相 H 為什麼是 0 到 179 而非 0 到 359?偵測紅色時要如何處理?為什麼還必須設定 S 與 V 的下限?

提示與參考解答：因為要塞進 8 位元 uint8(上限 255)，把 0 至 360 度除以二成為 0 至 179。紅色的色相跨越 0 度接縫，須設兩段區間(如 0~10 與 170~179)再取聯集。必須設 S 與 V 下限的原因：當顏色很暗或很不飽和(接近黑白灰)時，H 值極不穩定甚至無意義(圓錐尖端所有色相收縮成一點)，不排除這些區域會造成大量誤判。

6. YCbCr 的 4:2:0 降取樣為何幾乎不影響觀感?這個設計思想在本書還出現在哪兩個地方?

提示與參考解答：因為人眼對亮度細節的敏感度遠高於色彩細節，把色度解析度砍成四分之一，人眼幾乎察覺不出。同樣的「把資源分配給人眼在意之處」思想還出現在：第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、第 11 章 JPEG 對人眼不敏感的高頻係數用較大的量化步長。

7. 什麼是「感知均勻」?CIE Lab 為何需要它?為什麼對彩色影像做 CLAHE 時應轉到 Lab 只處理 L 通道?

提示與參考解答：感知均勻指在該色彩空間中，兩顏色的歐氏距離大致正比於人眼感受到的色差大小。RGB 做不到——同樣的數值差距在不同區域造成的視覺差異天差地別。Lab 為此設計，使色差可量化(ΔE)，用於印刷紡織品管與色彩校正。做 CLAHE 只處理 L 通道的原因：若對 BGR 三通道分別等化，各通道映射函數不同，色彩比例被破壞而產生色偏;Lab 把亮度與色彩分離，只調 L 不動 a、b，色彩即可保持。

8. 虛擬色彩的目的是什麼?為什麼近年 JET 色階受到批評?請列出三個理由。

提示與參考解答：目的：把沒有顏色的資料(溫度、深度、訊號強度)映射成彩色，利用人眼辨色能力遠強於辨灰(數萬種顏色 vs. 數十階灰度)的特性，讓細微差異一目了然。JET 受批評的三個理由：一、亮度非單調遞增(中間黃綠色比兩端亮)，會製造實際不存在的假邊界而誤導判讀;二、對色盲使用者不友善(約 8% 男性有紅綠色盲);三、轉黑白列印後資訊喪失。建議改用 VIRIDIS 等感知均勻色階。

9. 說明灰階世界白平衡的假設與失效情境。完美反射假設又在什麼情況下誤判?

提示與參考解答：灰階世界假設整個場景的平均顏色應為中性灰，若實測偏藍就判定光源偏藍並反向校正。失效情境：整片草地、整面紅牆、大片藍天等單一色調畫面，假設不成立，演算法會把草地硬校正成灰綠色。完美反射假設以影像中最亮的像素為純白計算增益，在過曝或有高光反射時誤判——因為過曝像素已飽和，色彩資訊喪失，無法作為可靠的白色參考。

10. 顏色分割時，為什麼通常對 H 設定較窄的門檻，而對 V 設定較寬的門檻?

提示與參考解答：因為 H(色相)是「是什麼顏色」的主要判準，同一物體在不同光照下 H 相當穩定，設窄門檻可精確鎖定目標顏色、排除其他顏色。而 V(明度)會隨光照、陰影、角度大幅變化，設寬門檻才能容許這些變化不致漏檢。但 V(與 S)仍須設下限，以排除色相無意義的過暗與過灰區域。

11. 承叫獸開講：什麼是色彩恆常性?為什麼同一張洋裝照片會讓人看到截然不同的顏色?研究者用什麼方法驗證了這個解釋?

提示與參考解答：色彩恆常性：大腦會自動推測當下的光源顏色並扣除其影響，還原物體本身的顏色(人腦內建的自動白平衡)。洋裝照片曝光過度、背景線索模糊，光源資訊極度曖昧，大腦只能依各自的先驗假設來猜：假設偏藍日光則扣掉藍色而

看到白金，假設偏黃人造光則扣掉黃色而看到藍黑。驗證方法：研究團隊把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就趨於一致，且與被暗示的光源方向吻合。

12. (綜合設計題)請為一條農產品分級產線設計「不會被光源騙倒」的顏色檢測方案，列出至少四項具體措施並說明理由。

提示與參考解答：一、控制光源：建立封閉檢測箱、使用高演色性且色溫穩定的LED，隔絕環境光——從源頭消除變異，治本優於治標(呼應第7.6.3節的層次原則)。二、放置標準色卡：每次拍攝納入ColorChecker，計算色彩校正矩陣，補償光源漂移與感測器差異。三、選對色彩空間：以HSV的H或Lab的a、b為判準，對明度放寬，降低光照敏感度。四、光學設計：調整光源方向與擴散以避免高光反射(反光處過曝會喪失色彩資訊)。五、定期驗證：LED會隨使用時間衰減偏色，應定期以色卡檢查色彩響應是否漂移。六、備援策略：若無法完全控制光源(如田間)，則以第15章的資料增強讓模型適應光線變化。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6(彩色影像處理)。
- Lafer-Sousa, R., Hermann, K. L., & Conway, B. R., Striking Individual Differences in Color Perception Uncovered by "the Dress" Photograph, Current Biology, 25(13), 2015.(本章叫獸開講之科學來源)
- Wallisch, P., Illumination Assumptions Account for Individual Differences in the Perceptual Interpretation of a Profoundly Ambiguous Stimulus, Journal of Vision, 17(4), 2017.(逾一萬三千人的雲雀/夜貓子研究)
- Fairchild, M. D., Color Appearance Models, 3rd ed., Wiley, 2013.
- Ebner, M., Color Constancy, Wiley, 2007.(白平衡演算法之專書)
- OpenCV 官方文件：Color conversions(cv2.cvtColor 完整轉換代碼表)。
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 9 章 影像分割

本章學習目標

- 說明影像分割在視覺流程中的定位，區分基於不連續性與基於相似性的兩大分割策略。
- 運用全域、Otsu 與自適應門檻法完成灰階影像的二值化，並判斷各自的適用時機。
- 推導並實作 Sobel、Laplacian of Gaussian 與 Canny 邊緣偵測，理解其中的低通與高通運算。
- 以霍夫轉換偵測影像中的直線與圓，說明「參數空間投票」的核心思想。
- 運用區域成長與分水嶺法完成基於相似性的分割，並以標記控制避免過度分割。
- 比較傳統分割與深度學習語意分割的差異，並說明分割成效對後續量測與辨識的影響。

叫獸開講|霍夫轉換：從次原子粒子軌跡到你家的車道偏離警示

1950 年代，粒子物理學面臨一個「幸福的煩惱」。1952 年，Donald Glaser 發明了氣泡室(Bubble Chamber)——讓帶電粒子通過過熱液體時，沿途激發出一串微小氣泡，形成可拍攝的軌跡，這項發明讓他拿下 1960 年諾貝爾物理獎。問題是：氣泡室每天產生成千上萬張底片，每一張都佈滿縱橫交錯的粒子軌跡，而當年只能靠人工用眼睛一條一條描出這些軌跡、量測曲率以推算粒子動量。資料產出的速度，遠遠超過人力分析的速度。

在密西根大學與 Glaser 共事的物理學家 Paul Hough，決定讓機器來做這件苦差事。他要解決的核心難題是：如何讓電腦自動從一堆散亂的亮點中，找出「這些點排成了一條直線」？他的靈感堪稱神來一筆：與其在影像空間裡辛苦地嘗試各種直線去湊點，不如把問題翻轉到「參數空間」。影像中的每一個亮點，都對應到參數空間中的一條曲線(代表所有可能通過該點的直線)；若許多亮點共線，它們對應的曲線就會在參數空間中交會於同一點——於是「找直線」這個困難的幾何問題，變成了「找參數空間中的交會熱點」這個單純的投票統計問題。1962 年，Hough 以「辨識複雜圖樣的方法與裝置」為題取得美國專利。

這個為高能物理而生的方法，經過 Rosenfeld 在 1969 年引入主流電腦視覺、以及 Duda 與 Hart 在 1972 年改用極座標表示法(解決了垂直線斜率無限大的問題)之後，成為影像處理最經典的工具之一，至今累積超過兩千五百篇相關論文。而它最貼近你我生活的化身，就在汽車裡：當代車輛的車道偏離警示系統，核心正是用霍夫轉換即時偵測路面上的車道線；當系統發現車輛正在無預警地偏離兩條偵測到的直線，便發出警報。一個誕生於粒子加速器的數學，六十年後守護著高速公路上每一位駕駛的安全。

這個故事的啟示，是本章的核心心法：分割的本質，常常是一次「觀點的翻轉」。把「找直線」翻轉成「投票統計」，把「切割區域」翻轉成「淹水分界」，把「哪裡是邊界」翻轉成「哪裡不連續」——換一個空間、換一種提問方式，難題就迎刃而解。這與第 6 章「換到頻率域」、第 7 章「換到參數空間解逆問題」的精神一脈相承。

思考題：霍夫轉換把「找直線」變成「在參數空間找熱點」。這個「投票」的思想非常強大——你能想到生活中還有哪些問題，也可以用「每個證據都投一票、票多者當選」的方式來解決？

9.1 什麼是影像分割

9.1.1 分割的定位：承上啟下的轉捩點

影像分割(Image Segmentation)的目標，是把影像劃分成若干個有意義的區域——把前景從背景中分離、把每個物件圈出來、把不同的組織或材質區分開。它是影像處理流程中一個關鍵的轉捩點：在它之前(第 1 到 8 章)，我們處理的是「像素」，做的是強化、還原、色彩調整這類「像素進、像素出」的低階運算；從分割開始，我們的關注點從「像素」升級到「區域」與「物件」，踏入了中階視覺的領域，並為第 13 章的特徵擷取、第 16 章的物件偵測鋪路。

這個轉變的重要性怎麼強調都不為過。回顧第 1 章提過的低、中、高階視覺三層架構：分割正是連接低階(像素)與高階(語意)的橋梁。一張影像若沒有經過分割，電腦看到的永遠只是一大片數字；唯有把「這一塊像素屬於同一個物件」界定出來，後續的「這個物件是什麼」才有意義。

9.1.2 兩大策略：不連續與相似

影像分割的方法雖然繁多，但都可以歸入兩大哲學：

1. 基於不連續性(Discontinuity)：著眼於「區域之間的界線」。它假設不同區域的交界處，像素值會發生劇烈變化，因此設法找出這些劇變的位置——這就是邊緣偵測(9.3 節)與霍夫轉換(9.4 節)的思路。用一句話說：找出「哪裡變了」，變的地方就是邊界。
2. 基於相似性(Similarity)：著眼於「區域內部的一致性」。它假設同一個區域內的像素，在某種性質(灰階、顏色、紋理)上是相似的，因此設法把相似的像素聚集成群——這就是門檻法(9.2 節)、區域成長與分水嶺法(9.5 節)的思路。用一句話說：找出「哪些像素是一夥的」，一夥的就是同一區域。

這兩種策略是一體兩面：區域的邊界，正是相似性中斷之處；而相似的內部，正是被邊界包圍的範圍。實務上，強健的分割系統常常兩者並用——先用邊緣界定大致輪廓，再用相似性填補區域，或反之。理解這個二分法，就掌握了本章的骨架。

9.1.3 本章在全書中的位置

分割是承先啟後的樞紐，幾乎用到前八章的所有工具，也支撐著後續各章：

- 與第 5 章(空間域強化)：分割前的品質決定分割的成敗。對比不足的影像要先做直方圖等化或 CLAHE，否則門檻分不開前景背景；雜訊會讓邊緣偵測產生大量假邊，因此 Canny 的第一步就是第 5 章的高斯平滑。
- 與第 6 章(頻率域)：邊緣是高頻、平坦區是低頻；Canny 內含「先低通平滑、再高通求梯度」的雙重濾波，正是第 6.6.4 節詳述的觀念。分割在本質上就是在區分影像中不同頻率特性的區域。
- 與第 7 章(影像還原)：還原品質直接影響分割——雜訊使邊緣偵測產生假邊、模糊使邊緣定位偏移。第 7.7 節已量化過這個連結。
- 與第 8 章(色彩處理)：彩色影像的分割常在 HSV 空間以顏色為依據(第 8.6 節的顏色分割)，或轉到 Lab 空間計算感知色差。
- 與第 10 章(二值影像處理)：門檻分割的輸出是二值影像，緊接著就用型態學清理雜點、連通元件標記來計數與量測——第 10 章是本章的直接下游。
- 與第 13 章(特徵擷取)：分割出的輪廓可計算形狀特徵(面積、周長、圓形度、Hu 矩)，供物件辨識使用。
- 與第 16 章(物件偵測與分割)：YOLO 的實例分割、U-Net 的語意分割，是本章傳統方法的深度學習後繼者；9.6 節會說明兩者的分工與互補。
- 與第 17 章(邊緣部署)：傳統分割運算量小、不需訓練資料，在算力受限或樣本稀少的場合仍是首選——這是它在深度學習時代依然重要的原因。

9.2 門檻法：最簡單的分割

9.2.1 全域門檻

門檻法(Thresholding)是最直觀的分割：選一個灰階值 T 作為分界，大於 T 的像素歸為前景(設為白)、小於 T 的歸為背景(設為黑)。當前景與背景的灰階差異明顯、打光均勻時，一個固定的全域門檻就能漂亮地切開兩者。它的關鍵完全繫於 T 的選擇——選對了乾淨俐落，選錯了不是前景殘缺就是背景混入。

如何選 T ? 最理想的情況是影像的直方圖呈現雙峰(bimodal)：前景與背景各自形成一個峰，兩峰之間的谷底就是理想門檻。這也把我们帶回第 5 章——直方圖不只是診斷曝光的工具，更是選擇分割門檻的依據。

9.2.2 Otsu 自動門檻

人工看直方圖選門檻既主觀又難以自動化。日本學者大津展之(Nobuyuki Otsu)於 1979 年提出的方法，能自動找出最佳門檻：它遍歷所有可能的門檻值，對每一個門檻把像素分成前景與背景兩群，計算兩群的「類間變異數」(between-class variance)，選擇使類間變異數最大的那個門檻。直覺上，最好的門檻應該讓兩群「各自內部愈集中、彼此之間離得愈遠」——類間變異數最大，正對應這個目標。Otsu 法有一個漂亮的性質：最大化類間變異數，等價於最小化類內變異數，而前者的計算可以用遞推快速完成，因此效率極高。

Otsu 法的前提是直方圖確實雙峰。若前景只佔極小面積(直方圖被背景的單峰主宰)、或前景背景灰階重疊嚴重，Otsu 就會失準——這時要嘛先做前處理拉開對比，要嘛改用下一節的自適應門檻。

9.2.3 自適應門檻

全域門檻有個致命弱點：它假設整張影像的最佳門檻都一樣。但真實場景常有打光不均——一張紙的左半邊在陽光下、右半邊在陰影中，任何單一門檻都會顧此失彼(這正是第 8 章「洋裝效應」在灰階世界的翻版)。自適應門檻(Adaptive Threshold)的解法是：不用一個全域門檻，而是為每個像素、依據它周圍鄰域的局部統計量(平均值或高斯加權平均)各自計算一個門檻。如此一來，亮區用高門檻、暗區用低門檻，打光不均的問題迎刃而解。文件掃描 App 把泛黃、有陰影的紙張轉成黑白分明的掃描檔，靠的就是自適應門檻。

程式 9-1 三種門檻法

```
import cv2

gray = cv2.imread("document.jpg", cv2.IMREAD_GRAYSCALE)

# --- 全域固定門檻 ---
_, fixed = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# --- Otsu 自動門檻(回傳自動選出的 T) ---
T, otsu = cv2.threshold(gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu 選出的門檻:", T)

# --- 對付雜訊:先高斯模糊再 Otsu(第 5 章前處理) ---
blur = cv2.GaussianBlur(gray, (5, 5), 0)
_, otsu2 = cv2.threshold(blur, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 自適應門檻:對付打光不均 ---
adap = cv2.adaptiveThreshold(gray, 255,
                            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                            cv2.THRESH_BINARY, blockSize=31, C=10)

cv2.imwrite("th_fixed.png", fixed)
cv2.imwrite("th_otsu.png", otsu)
cv2.imwrite("th_adaptive.png", adap)
```

9.3 邊緣偵測

邊緣是影像中灰階劇烈變化之處，通常對應物體的輪廓、表面的轉折或深度的不連續。從第 6 章的頻率觀點看，邊緣就是高頻;而偵測高頻變化的數學工具，是微分。

9.3.1 梯度與一階微分：Sobel

影像的梯度是一個向量，由水平方向與垂直方向的偏微分組成，其大小代表灰階變化的劇烈程度、方向代表變化最快的方向(垂直於邊緣走向)。在離散的數位影像上，

微分以差分近似，並包裝成卷積核。最常用的是 Sobel 運算子：它在計算某方向差分的同時，對垂直方向做加權平均，兼具「求梯度」與「抗雜訊」兩個功能——這也呼應第 6 章的觀點：Sobel 是一個帶有平滑效果的方向性高通濾波器。取得水平梯度 G_x 與垂直梯度 G_y 後，梯度大小為兩者平方和開根號，對它做門檻化即可得到邊緣圖。

相關的運算子還有 Scharr(比 Sobel 更精確的梯度近似，適合小核)、Prewitt(較簡單的差分)。它們的共同侷限是：對雜訊敏感(微分會放大雜訊)，且邊緣往往偏粗、需要後續細化。

9.3.2 二階微分：Laplacian 與 LoG

一階微分在邊緣處產生極值，二階微分則在邊緣處產生零交越(zero-crossing)——由正轉負的那一點，正是邊緣的精確位置。拉普拉斯運算子(Laplacian)是二階微分的離散近似，定位精準且各向同性(不分方向)，但對雜訊極度敏感(第 5、6 章都提過，二階微分把雜訊放大得比一階更兇)。

解法是先平滑再微分：把高斯平滑與拉普拉斯結合，就是高斯拉普拉斯(Laplacian of Gaussian, LoG)，又因其剖面形似而俗稱「墨西哥帽」。LoG 先用高斯壓制雜訊(低通)，再用拉普拉斯找零交越(高通)——又是一個「低通加高通」的組合，與第 6 章的敘述完全一致。LoG 也是第 13 章 SIFT 特徵偵測的理論前身(SIFT 用高斯差 DoG 來近似 LoG)。

9.3.3 Canny：邊緣偵測的黃金標準

1986 年 John Canny 提出的邊緣偵測器，以「好的偵測、好的定位、單一響應」三個準則為設計目標，至今仍是最廣泛使用的邊緣偵測方法。它是一條精心設計的四步驟管線，每一步都呼應本書前面的觀念：

3. 高斯平滑：先用高斯濾波器壓制雜訊(第 5 章低通)，避免後續微分放大雜訊。 σ 的大小決定了對雜訊的容忍度與細節的保留度，是最關鍵的參數。
4. 計算梯度：以 Sobel 求出每個像素的梯度大小與方向(第 9.3.1 節)。
5. 非極大值抑制(Non-Maximum Suppression)：沿著梯度方向，只保留局部最大的像素，把粗邊緣細化成單像素寬的線。這是 Canny 邊緣俐落的關鍵。
6. 雙門檻與遲滯連接(Hysteresis)：設高、低兩個門檻。高於高門檻的必定是邊緣(強邊)；低於低門檻的必定不是；介於兩者之間的(弱邊)，只有在「連接到強邊」時才保留。這個巧妙的設計，既能避免雜訊造成的假邊，又能保住真實但較弱的邊緣不致斷裂。

Canny 的成功，正是「把多個簡單觀念(平滑、微分、抑制、連接)精心組合成一條管線」的典範——這也是叫獸開講「觀點翻轉」之外，工程上另一種創造價值的方式：巧妙的組合。

程式 9-2 Sobel、LoG 與 Canny 邊緣偵測

```
import cv2, numpy as np

gray = cv2.imread("parts.jpg", cv2.IMREAD_GRAYSCALE)

# --- Sobel 一階梯度 ---
```

```

gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
sobel = cv2.convertScaleAbs(mag)

# --- Laplacian of Gaussian(先平滑再二階微分) ---
blur = cv2.GaussianBlur(gray, (3, 3), 0)
log = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
log = cv2.convertScaleAbs(log)

# --- Canny(黃金標準) ---
edges = cv2.Canny(gray, threshold1=80, threshold2=160)

# 自動決定門檻:以中位數為基準(實務常用技巧)
v = np.median(gray)
lo = int(max(0, 0.66 * v))
hi = int(min(255, 1.33 * v))
edges_auto = cv2.Canny(gray, lo, hi)

cv2.imwrite("edge_sobel.png", sobel)
cv2.imwrite("edge_log.png", log)
cv2.imwrite("edge_canny.png", edges)

```

9.4 霍夫轉換：參數空間的投票

9.4.1 直線偵測的核心思想

邊緣偵測告訴我們「哪些像素在邊緣上」，但它給的是一堆散亂的邊緣點，並沒有告訴我們「這些點排成了一條直線」。霍夫轉換(Hough Transform)接手這個工作，而它的思想正是叫獸開講中那次精彩的觀點翻轉。

關鍵在於視角互換。在影像空間中，一條直線由通過它的所有點定義；霍夫轉換反過來問：對於影像中的每一個邊緣點，有哪些直線可能通過它？答案是無限多條，這些直線的參數在「參數空間」中構成一條曲線。於是每個邊緣點都在參數空間投下一條曲線(等於為所有可能通過它的直線各投一票)。如果影像中真的有一條直線，那麼這條線上的所有點，投出的曲線都會通過參數空間中同一個點——那個點的參數，正是這條直線的參數。於是「找直線」變成「在參數空間找得票最高的點」。

實務上不用斜截式 $y = mx + b$ ，因為垂直線的斜率是無限大無法表示。改用 Duda 與 Hart 在 1972 年引入的極座標式： $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ ，其中 ρ 是原點到直線的垂直距離、 θ 是該垂線的角度。每個邊緣點在 (ρ, θ) 參數空間中對應一條正弦曲線，共線的點其正弦曲線交會於一點。參數空間被離散化成一個個格子(稱為累加器, accumulator)，每條曲線經過的格子就加一票，最後票數超過門檻的格子就是偵測到的直線。

9.4.2 圓偵測與霍夫梯度法

同樣的投票思想可以推廣到任何能用少數參數描述的形狀。圓需要三個參數：圓心 (a, b) 與半徑 r ，因此累加器是三維的。若直接對三維空間投票，運算與記憶體成本都很高。

OpenCV 採用的霍夫梯度法(Hough Gradient Method)是一個聰明的加速：它利用一個幾何事實——圓上每一點的梯度方向都指向(或背離)圓心。因此不必盲目地為所有可能的圓心投票，而是讓每個邊緣點只沿著自己的梯度方向投票，票數自然集中到真正的圓心；先在二維空間定出圓心，再回頭決定半徑。這把三維投票拆解成兩個較低維的步驟，大幅提升效率。它廣泛用於偵測圓形物件——瞳孔、硬幣、球、儀表指針的軸心、工件上的圓孔。

程式 9-3 霍夫直線與圓偵測

```
import cv2, numpy as np

gray = cv2.imread("road.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(gray, 80, 160)          # 霍夫的輸入是邊緣圖

# --- 標準霍夫直線(回傳 rho, theta) ---
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)

# --- 機率霍夫直線(回傳線段端點, 實務更常用) ---
segments = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50,
                             minLineLength=50, maxLineGap=10)
img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
if segments is not None:
    for x1, y1, x2, y2 in segments[:, 0]:
        cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

# --- 霍夫梯度法偵測圓 ---
coins = cv2.imread("coins.jpg", cv2.IMREAD_GRAYSCALE)
coins = cv2.medianBlur(coins, 5)          # 先去雜訊很重要
circles = cv2.HoughCircles(coins, cv2.HOUGH_GRADIENT, dp=1,
                             minDist=30, param1=100, param2=30,
                             minRadius=10, maxRadius=80)
if circles is not None:
    for a, b, r in np.uint16(np.around(circles))[0]:
        cv2.circle(img, (a, b), r, (0, 0, 255), 2)

cv2.imwrite("hough_lines.png", img)
```

9.5 基於區域的分割

9.5.1 區域成長

區域成長(Region Growing)體現了「基於相似性」的策略。它從一個或多個種子點出發，檢查種子的鄰居：若鄰居與種子(或目前區域)的性質夠相似(灰階差在容許範圍內)，就把鄰居併入區域，再從新併入的像素繼續向外檢查，像墨水在紙上暈開一

樣，直到再也沒有相似的鄰居可併入。它的優點是能保證區域的連通性、對定義明確的區域效果好；缺點是結果高度依賴種子的選擇與相似度門檻，且對雜訊敏感。

9.5.2 分水嶺法

分水嶺法(Watershed)有一個極為優美的比喻。把灰階影像想像成一片地形：灰階值就是海拔，亮處是山峰、暗處是山谷(通常對梯度影像操作，則邊緣是山脊、區域內部是盆地)。現在想像從每個盆地的最低點開始注水，水位緩緩上升，不同盆地的水各自成湖；當兩個湖的水即將相遇時，在交界處築起一道堤壩——這些堤壩，就是分割的邊界。「分水嶺」之名由此而來。

分水嶺法天生擅長分開「相互接觸、黏在一起」的物件——這是門檻法最頭痛的情況(兩個相黏的細胞或零件，門檻分割會把它們當成一個)。但它有一個惡名昭彰的問題：過度分割(over-segmentation)。因為影像中每一個微小的局部極小值(常由雜訊造成)都會形成一個獨立的盆地，結果把影像切得支離破碎。

解法是標記控制的分水嶺(Marker-Controlled Watershed)：不讓水從所有局部極小值自由湧出，而是由使用者(或前處理演算法)指定少數幾個「確定是前景」與「確定是背景」的標記，只從這些標記處注水。這徹底馴服了過度分割，是實務上真正可用的版本。它的標準流程會用到第 10 章的型態學運算(侵蝕求前景標記、膨脹求背景標記、距離轉換分離相黏物件)，因此本節也是通往下一章的橋梁。

程式 9-4 標記控制的分水嶺分割

```
import cv2, numpy as np

img = cv2.imread("touching_coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Otsu 二值化(第 9.2 節)
_, thresh = cv2.threshold(gray, 0, 255,
                           cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# 2. 型態學去雜點 + 確定背景(第 10 章)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel,
                           iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# 3. 距離轉換 → 確定前景(分開相黏物件的關鍵)
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

# 4. 未知區域 = 背景 - 前景
unknown = cv2.subtract(sure_bg, sure_fg)

# 5. 標記並執行分水嶺
n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
```

```

markers = cv2.watershed(img, markers)
img[markers == -1] = [0, 0, 255]          # 分界線標紅

print("分離出的物件數:", n - 1)
cv2.imwrite("watershed.png", img)

```

9.6 從傳統分割到深度學習語意分割

本章至此介紹的都是傳統方法：它們基於人工設計的規則(門檻、梯度、投票、淹水)，不需要訓練資料，運算量小，可解釋性強。但它們有一個共同的天花板：它們分割的是「低階特徵的一致性」，而不是「語意」。門檻法能把亮的和暗的分開，卻不知道分出來的是貓還是狗；霍夫轉換能找出直線，卻不理解那是車道線還是欄杆。

深度學習帶來了語意分割(Semantic Segmentation)的躍進：為影像中的每一個像素賦予一個類別標籤(這是路面、這是行人、這是天空)。2015 年的 U-Net(原為醫學影像設計)以其對稱的編碼器-解碼器結構與跳接(skip connection)，成為語意分割的里程碑；此後的發展延伸出實例分割(Instance Segmentation，不僅分類每個像素，還區分同類的不同個體，例如畫面中的三個人各自獨立)，代表如 Mask R-CNN；近年更有 SAM(Segment Anything Model)這類基礎模型，能對幾乎任何物件進行提示式分割。第 16 章會實作 YOLO 的分割功能。

那麼傳統方法過時了嗎？並沒有。兩者是互補而非取代的關係：

比較面向	傳統分割(本章)	深度學習語意分割
依據	人工規則(灰階、梯度、形狀)	從標註資料學到的語意特徵
訓練資料	不需要	需要大量像素級標註(成本高)
運算量	小，CPU 即可即時	大，通常需要 GPU
可解釋性	高，每步驟皆可理解	低，黑箱
語意理解	無，只分低階特徵	有，能辨識類別
適用場合	受控環境、規律背景、樣本稀少	複雜場景、需要語意、資料充足

表 9-1 傳統分割與深度學習語意分割的比較

工程判斷原則：在受控的工業環境(打光穩定、背景單純)，傳統方法往往又快又準又省——一個 AOI 檢測站用 Otsu 加連通元件就能數清零件，何必動用需要標註數千張影像、還得配 GPU 的深度模型？反之，在開放世界的複雜場景(自駕車看到的街景)，語意的理解無可取代，深度學習才是正解。成熟的工程師懂得在兩者間依場景選擇，甚至混用——例如用深度模型做粗略的語意定位，再用傳統方法做精確的邊界細化與量測。

9.7 應用案例

9.7.1 案例一：零件計數與尺寸量測

這是機器視覺最經典的入門任務，也是本章前五節的綜合演練。情境：一盤散落的螺絲或墊圈，要自動數出數量並量測每一個的尺寸。標準流程串起了本書多章：先用第 5 章的前處理(灰階、CLAHE)改善對比 → 用 9.2 節的 Otsu 門檻二值化 → 用第 10 章的型態學去除雜點、填補孔洞 → 用連通元件標記或輪廓偵測數出物件並計算每個的面積、周長、最小外接矩形。若零件相互接觸，則改用 9.5 節的分水嶺分開它們。這個管線是無數工廠檢測線的骨幹。

9.7.2 案例二：車道線偵測

呼應叫獸開講，這是霍夫轉換最著名的應用。情境：從車前攝影機的影像中即時偵測車道線。完整流程：轉灰階並用第 8 章的顏色資訊強化黃白線 → 用 9.3 節的 Canny 找出邊緣 → 設定「感興趣區域」(ROI, 只保留路面梯形區域，排除天空與周邊干擾，這用到第 2 章的 ROI 概念) → 用 9.4 節的機率霍夫轉換 HoughLinesP 找出線段 → 依斜率把線段分成左右車道並各自擬合成一條線 → 判斷車輛是否偏離。

這個案例也適合討論傳統方法的極限：在標線清晰、光線良好的高速公路上，霍夫轉換又快又可靠；但遇到標線磨損、雨天反光、強烈陰影、彎道(直線模型失效)時就力有未逮——這正是為什麼當代自駕系統改用深度學習來做車道線與可行駛區域的語意分割。從霍夫轉換到深度學習，一部車道偵測的演進史，恰好就是本章 9.1 到 9.6 節的縮影。

9.7.3 案例三：醫學影像的病灶分割

分割在醫療領域至關重要：從 X 光、CT、MRI 中把腫瘤、器官、血管的範圍精確圈出，是後續量測體積、規劃手術、追蹤療效的基礎。傳統上，醫師或工程師會用區域成長(從醫師點選的種子開始擴張)、門檻法(對造影劑增強的區域)、或分水嶺法來輔助分割。

但醫學影像分割也最能凸顯分割品質的重量。一個腫瘤邊界若分割得太大，可能導致切除過多健康組織；太小則可能殘留病灶。這裡有兩個貫穿本書的呼應：其一，第 7 章談過的還原品質直接影響此處——雜訊與模糊會讓邊界偵測失準；其二，第 7 章「還原 vs. 腦補」的倫理警覺在此同樣適用——若用深度學習模型自動分割病灶，模型「腦補」出的邊界是否可信？正因如此，醫學影像的自動分割結果通常僅作為輔助，最終仍須醫師確認。U-Net 當年正是為醫學影像分割而生，如今深度學習與傳統方法在此領域廣泛並用：深度模型提供快速的初步分割，傳統方法(如主動輪廓、分水嶺)做精細的邊界調整。

9.7.4 案例四：視訊中的移動物件分割

前三個案例都是分割單張影像，第四個案例引入時間維度，銜接第 12 章。情境：固定的監視攝影機，要自動框出畫面中移動的人或車。核心思想是背景相減(Background Subtraction)：先學習一個「沒有移動物件時」的背景模型，再把每一影格與背景相減，差異顯著之處就是前景(移動物件)。

這其實是一種特殊的分割：它不靠灰階、顏色或邊緣，而是靠「時間上的變化」來區分前景與背景。得到前景遮罩後，同樣要用第 10 章的型態學清理雜點、用連通元件標記框出各個物件。OpenCV 提供 MOG2、KNN 等背景相減器，能自適應地更新背景模型(應付光線漸變、樹葉搖動等干擾)。這個案例是智慧監控、人流計數、交通流量分析的基礎，第 12 章會深入。

9.8 本章與全書的觀念連結總表

相關章節	共用的觀念	分割中的角色
第 5 章 空間域強化	直方圖、CLAHE、高斯平滑	分割前處理;Canny 首步的平滑;門檻依直方圖選
第 6 章 頻率域	低通、高通、邊緣即高頻	Canny 內含低通平滑加高通求梯度的雙重濾波
第 7 章 影像還原	雜訊與模糊的影響	雜訊生假邊、模糊使邊緣定位偏移
第 8 章 色彩處理	HSV 顏色分割、Lab 色差	彩色影像常以顏色為分割依據
第 10 章 二值影像	型態學、連通元件、距離轉換	門檻輸出的直接下游;分水嶺標記的前處理
第 12 章 視訊處理	背景相減、移動偵測	以時間變化做前景分割
第 13 章 特徵擷取	輪廓、形狀特徵、SIFT	分割出的輪廓供形狀分析;LoG 為 SIFT 前身
第 16 章 物件偵測	語意分割、實例分割	U-Net、Mask R-CNN 為傳統分割的深度學習後繼

表9-2 第9章與全書其他章節的觀念連結

程式實作 Lab 9

Lab 9-1 三種門檻法的擂台

目標：親身體會各門檻法的適用時機。(a)準備三張影像：對比良好且打光均勻的、打光不均(一側有陰影)的、以及含雜訊的;(b)對每張分別套用全域固定門檻、Otsu、自適應門檻，製成 3×3 的結果對照表;(c)畫出每張影像的直方圖，說明 Otsu 選出的門檻落在哪裡、為什麼在打光不均的影像上失準;(d)對含雜訊影像，比較「直接 Otsu」與「先高斯模糊再 Otsu」的差異;(e)結論：寫出一張決策流程圖——拿到影像後，如何依直方圖形狀與打光狀況選擇門檻法？

Lab 9-2 邊緣偵測參數探索器

目標：掌握 Canny 的參數行為。(a)以 cv2.createTrackbar 建立可即時調整 Canny 高低門檻與高斯 sigma 的介面;(b)觀察並記錄：sigma 太小、太大分別發生

什麼事(對照第 6 章「sigma 決定訊號與雜訊的分界頻率」);高低門檻的比例(常建議 1:2 或 1:3)如何影響邊緣的完整與雜訊;(c)實作程式 9-2 的「中位數自動門檻」，與手動最佳值比較;(d)把 Sobel、LoG、Canny 三者的結果並排，說明為何 Canny 的邊緣最俐落(提示：非極大值抑制與遲滯連接)。

Lab 9-3 車道線偵測器

目標：完成叫獸開講的實戰。(a)取一段行車影像(或單張路面照片)，依 9.7.2 節流程：Canny → ROI 遮罩 → HoughLinesP;(b)依斜率把線段分為左右車道，各自用最小平方法擬合成一條直線並延伸繪製;(c)在畫面標出車道中心與車輛中心的偏差，模擬車道偏離警示;(d)測試極限：找出標線磨損、陰影、彎道的影像，記錄霍夫轉換在什麼情況下失效;(e)討論題：承(d)，為什麼當代自駕系統要改用深度學習語意分割？請對照表 9-1 說明。

Lab 9-4 相黏零件的分水嶺分割

目標：解決門檻法的痛點。(a)拍攝或取得一張多個相互接觸的零件(或硬幣、豆子)影像;(b)先用 Otsu 二值化，用連通元件標記計數，觀察相黏的物件如何被錯數成一個;(c)實作程式 9-4 的標記控制分水嶺，比較計數的正確性;(d)調整距離轉換的門檻(0.5、0.7、0.9 倍最大值)，觀察對分離效果的影響;(e)挑戰題：若不用標記控制、直接對梯度影像做分水嶺，會發生什麼？請截圖說明「過度分割」現象。

本章重點整理

- 影像分割把影像劃分為有意義的區域，是從「像素」到「區域/物件」的轉捩點；兩大策略為基於不連續性(邊緣、霍夫)與基於相似性(門檻、區域成長、分水嶺)，兩者是一體兩面。
- 門檻法：全域固定門檻適合對比良好且打光均勻者；Otsu 以最大化類間變異數自動選門檻，前提是直方圖雙峰；自適應門檻依局部統計量逐像素定門檻，專治打光不均。
- 邊緣偵測基於微分：Sobel 為帶平滑的一階梯度(方向性高通)；Laplacian 為二階微分找零交越但對雜訊敏感，LoG 先平滑再微分改善之；Canny 以高斯平滑、梯度、非極大值抑制、雙門檻遲滯四步驟成為黃金標準。
- 霍夫轉換將「找形狀」翻轉為「參數空間投票」：直線用極座標 $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ 避免斜率無限大，共線點的正弦曲線交會於累加器同一格；霍夫梯度法利用梯度指向圓心的性質加速圓偵測。
- 區域成長從種子出發併入相似鄰居；分水嶺法以「地形淹水」分開相黏物件，但易過度分割，須用標記控制(結合第 10 章的型態學與距離轉換)馴服。
- 深度學習語意分割(U-Net)、實例分割(Mask R-CNN)、基礎模型(SAM)能理解語意，但需大量標註與算力；傳統方法在受控環境、樣本稀少時仍是首選，兩者互補而非取代。

習題

1. 影像分割為什麼被稱為從低階視覺到中階視覺的轉捩點?請以「像素」與「區域」的概念說明。

提示與參考解答：分割之前(第1到8章)處理的是「像素進、像素出」的低階運算(強化、還原、色彩);分割把「屬於同一物件的像素」界定出來，關注點從像素升級到區域與物件，進入中階視覺，並為高階的物件辨識(這是什麼)提供基礎。沒有分割，電腦看到的只是一片數字。

2. 請比較「基於不連續性」與「基於相似性」兩大分割策略，各舉兩種方法，並說明為何說兩者是一體兩面。

提示與參考解答：不連續性著眼區域間的界線(灰階劇變處)，方法有邊緣偵測、霍夫轉換。相似性著眼區域內的一致性(灰階/顏色/紋理相似)，方法有門檻法、區域成長、分水嶺。一體兩面：區域的邊界正是相似性中斷之處，相似的內部正是被邊界包圍的範圍;強健系統常兩者並用。

3. Otsu 法的目標函數是什麼?它有什麼前提假設?在什麼情況下會失準，該如何補救?

提示與參考解答：目標是找出使「類間變異數」最大(等價於類內變異數最小)的門檻，讓前景背景兩群各自集中、彼此遠離。前提是直方圖雙峰。失準情況：前景面積極小(直方圖被背景單峰主宰)、前景背景灰階重疊嚴重。補救：先做前處理拉開對比，或改用自適應門檻。

4. 為什麼打光不均的影像不能用單一全域門檻?自適應門檻如何解決?這與第8章的哪個現象相呼應?

提示與參考解答：打光不均使同一物體在亮區與暗區的灰階不同，任何單一門檻都會顧此失彼(亮區背景被誤判為前景，或暗區前景被誤判為背景)。自適應門檻為每個像素依其鄰域局部統計量各自計算門檻，亮區用高門檻、暗區用低門檻。這是第8章「洋裝效應/色彩恆常性」在灰階世界的翻版——局部脈絡決定判讀。

5. 說明梯度大小與方向的幾何意義。Sobel 運算子除了求梯度，還兼具什麼功能?從第6章的觀點，Sobel 是什麼濾波器?

提示與參考解答：梯度大小代表灰階變化的劇烈程度，方向指向變化最快的方向(垂直於邊緣走向)。Sobel 在求某方向差分時，對垂直方向做加權平均，兼具抗雜訊(平滑)功能。從頻率域看，Sobel 是帶有平滑效果的方向性高通濾波器。

6. 一階微分與二階微分在邊緣處各有什麼表現?為什麼 LoG 要先做高斯平滑?

提示與參考解答：一階微分在邊緣處產生極值(峰);二階微分在邊緣處產生零交越(由正轉負處即邊緣)。二階微分對雜訊比一階更敏感，故 LoG 先用高斯平滑(低通)壓制雜訊，再用拉普拉斯(高通)找零交越，是「低通加高通」的組合。

7. 請依序說明 Canny 邊緣偵測的四個步驟，並指出每一步對應本書前面的哪個觀念。

提示與參考解答：一、高斯平滑(第5章低通，壓制雜訊避免微分放大);二、Sobel 計算梯度(9.3.1 節);三、非極大值抑制(沿梯度方向只留局部最大，細化成單像素邊);四、雙門檻與遲滯連接(強邊必留、弱邊僅在連接強邊時保留，兼顧去假邊與防斷裂)。sigma 是最關鍵參數。

8. 霍夫轉換的核心思想是什麼?為什麼直線偵測要用極座標式 $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ 而非斜截式 $y = mx + b$?

提示與參考解答：核心是觀點翻轉：影像空間中每個邊緣點，對應參數空間中一條曲線(所有可能通過它的直線)，共線的點其曲線交會於一點，故「找直線」變成「參數空間找得票最高的格子」(投票/累加器)。用極座標是因為斜截式無法表示垂直線(斜率無限大)，極座標式對任何方向的直線都有界。

9. 霍夫梯度法如何加速圓偵測?它利用了什麼幾何性質?

提示與參考解答：圓需三參數(圓心 a 、 b 與半徑 r)，直接投票是三維、成本高。霍夫梯度法利用「圓上每點的梯度方向指向圓心」的性質，讓每個邊緣點只沿自己的梯度方向投票，票數集中到真正圓心；先在二維定出圓心，再回頭決定半徑，把三維投票拆成兩個低維步驟。

10. 分水嶺法的「淹水」比喻是什麼?它擅長解決什麼門檻法的痛點?又有什麼副作用，如何解決?

提示與參考解答：把灰階(或梯度)影像視為地形，灰階為海拔，從各盆地最低點注水，兩湖相遇處築壩即為分割邊界。它擅長分開「相互接觸黏在一起」的物件(門檻法會當成一個)。副作用是過度分割(每個雜訊造成的局部極小值都成一個盆地)。解決：標記控制分水嶺，只從指定的前景/背景標記注水，標記由型態學與距離轉換(第10章)產生。

11. 傳統分割與深度學習語意分割最根本的差異是什麼?請各舉一個「該用傳統方法」與「該用深度學習」的場景並說明理由。

提示與參考解答：根本差異：傳統方法分割「低階特徵的一致性」不懂語意、不需訓練、運算小、可解釋；深度學習分割「語意」能辨識類別但需大量標註與算力、可解釋性低。該用傳統：受控工業環境(打光穩定、背景單純)的AOI零件計數，又快又省。該用深度學習：自駕車的開放街景，需要理解「這是行人、這是路面」的語意。可對照表9-1。

12. (綜合題)請設計一條「散落零件自動計數與尺寸量測」的完整處理管線，列出每一步用到的技術與對應章節，並說明若零件相互接觸該如何處理。

提示與參考解答：管線：一、前處理——灰階化、CLAHE 增強對比(第5章)；二、二值化——Otsu 門檻(9.2節)，必要時先高斯模糊去雜訊；三、清理——型態學開運算去雜點、閉運算填孔(第10章)；四、標記量測——連通元件標記或輪廓偵測，計算面積、周長、最小外接矩形(第10、13章)。若零件相黏：改用標記控制分水嶺(9.5節)，以距離轉換分離，避免被錯數為一個物件。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 10(影像分割之標準參考)。
- Hough, P. V. C., Method and Means for Recognizing Complex Patterns, US Patent 3,069,654, 1962.(霍夫轉換原始專利，本章叫獸開講之史料來源)
- Duda, R. O., & Hart, P. E., Use of the Hough Transformation to Detect Lines and Curves in Pictures, Comm. ACM, 1972.(極座標表示法)

- Canny, J., A Computational Approach to Edge Detection, IEEE TPAMI, 1986.(Canny 邊緣偵測原始論文)
- Otsu, N., A Threshold Selection Method from Gray-Level Histograms, IEEE Trans. SMC, 1979.
- Ronneberger, O., Fischer, P., & Brox, T., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI, 2015.
- 補充教材投影片下載: [https://page.line.me/prof\(機器人叫獸\)](https://page.line.me/prof(機器人叫獸))

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 10 章 二值影像處理

本章學習目標

- 說明二值影像的意義與來源，理解型態學處理建立在集合論與結構元素之上。
- 推導並實作侵蝕、膨脹兩個基本運算，以及斷開、閉合兩個組合運算。
- 運用型態學梯度、頂帽與黑帽轉換解決邊緣萃取與不均勻背景校正問題。
- 操作 Hit-or-Miss、細線化、骨架化與孔洞填補等進階型態學技術。
- 以連通元件標記完成物件計數，並計算距離轉換供分割與量測使用。
- 理解 Haar 級聯分類器如何以積分影像與級聯結構達成即時人臉偵測，並在 ESP32-CAM 上部署。

叫獸開講|2001 年，相機學會了找人臉：Viola-Jones 的百萬分之一秒

在深度學習席捲電腦視覺的十一年前，有一個演算法就已經讓相機學會即時「看見」人臉了。2001 年，微軟研究院的 Paul Viola 與 Michael Jones 在頂級會議 CVPR 上發表了一篇論文《Rapid Object Detection using a Boosted Cascade of Simple Features》，提出了史上第一個能在普通電腦上即時執行的人臉偵測器。在當年一台 700MHz 的 Pentium III 處理器上，它的速度比同等準確度的其他方法快上數十甚至數百倍——這在當時是不可思議的成就。

Viola-Jones 的成功，靠的是三個環環相扣的巧思。第一是 Haar-like 特徵：它不看單一像素，而是計算「相鄰的矩形區域之間的亮度差」——例如，人臉上「眼睛區域通常比臉頰暗」，這個簡單的明暗關係就是一個特徵。第二是積分影像(Integral Image)：一張影像預先算好「每個點左上方所有像素的累加和」之後，任何一個矩形區域的像素總和，都只需要查表做三次加減法就能得到，與矩形大小完全無關——這讓成千上萬個 Haar 特徵能以驚人的速度計算。第三是級聯分類器(Cascade)：把分類器串成一連串關卡，前面的關卡用極少的特徵、極快的速度，把「明顯不是臉」的區域(佔畫面絕大部分)迅速淘汰；只有通過前關的少數候選，才交給後面更精密(也更耗時)的關卡仔細檢查。這個「快速排除、集中火力」的設計，是它能即時執行的關鍵。

把這三招放在本章的脈絡下看，你會發現它們與二值影像處理的精神一脈相承。Haar 特徵計算的是矩形區域的加總與相減，本質上就是對區域做集合式的運算；積分影像是一種「預先累加」的加速技巧，和本章連通元件標記中「一次掃描就完成統計」的思路異曲同工；而級聯「逐步淘汰、由粗到精」的策略，更是型態學處理中「先粗略清理、再精細處理」的工程智慧。Viola-Jones 內建於 OpenCV 多年(即著名的 Haar Cascade)，至今仍是資源受限裝置(如本章要部署的 ESP32-CAM)上人臉偵測的實用選擇。

當然，它也有明顯的侷限：它最擅長偵測正面、端正的人臉，遇到側臉、大角度傾斜或嚴重遮擋就力有未逮；這些正是第 16 章深度學習物件偵測器要克服的。但 Viola-Jones 的歷史地位無可撼動——它證明了「精心設計的簡單特徵 + 聰明的加速結構 + 機器學習的特徵篩選」可以創造奇蹟，這個信念直到今天仍深深影響著整個領域。

思考題：級聯分類器「用快速的關卡先淘汰明顯不對的候選，把運算集中在少數可能的目標上」——這個「由粗到精、快速排除」的思想，在生活中還有哪些應用？(想想：履歷篩選、垃圾郵件過濾、機場安檢)

10.1 二值影像與型態學的基礎

10.1.1 什麼是二值影像

二值影像(Binary Image)是每個像素只有兩種可能值的影像：前景(通常設為 1 或 255，顯示為白)與背景(通常設為 0，顯示為黑)。它是第 9 章分割的直接產物——門檻法、Otsu、邊緣偵測的輸出，幾乎都是二值影像。二值影像的資訊量雖然遠低於灰階或彩色影像，卻正是它的價值所在：當我們只關心「物件在哪裡、形狀如何、有幾個」時，把影像化約成非黑即白，能讓後續的形狀分析變得極為高效。

但分割產生的二值影像往往並不完美：前景區域中可能有小孔洞(雜訊或反光造成)、背景中散布著孤立的小白點(雜訊)、本該分開的物件黏在一起、本該相連的物件斷成幾截。本章的型態學運算，就是清理與修整這些缺陷、並從中萃取形狀資訊的工具箱——它是第 9 章分割與第 13 章特徵擷取之間承先啟後的關鍵環節。

10.1.2 型態學與結構元素

數學型態學(Mathematical Morphology)由 Georges Matheron 與 Jean Serra 在 1960 年代創立於法國，原本用於分析礦石與地質切片的形狀，其理論基礎是集合論。核心觀念是：把二值影像的前景視為一個點集合，用一個稱為結構元素(Structuring Element)的小形狀去「探測」這個集合，依據結構元素與前景的貼合關係，決定輸出。

結構元素相當於空間濾波的「核」，但作用方式是集合運算而非數值運算。它可以是任意形狀——方形、十字形、圓形、線形——形狀的選擇會直接影響處理結果：用水平線形結構元素，會強化水平結構、削弱垂直結構；用圓形結構元素，則各方向一視同仁。結構元素還有一個「原點」(通常在中心)，決定它在影像上對齊的方式。選對結構元素的形狀與大小，是型態學處理的關鍵藝術。

程式 10-1 建立結構元素

```
import cv2, numpy as np

# 建立不同形狀的結構元素
rect = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)) # 方形
cross = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5)) # 十字
ellip = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)) # 橢圓

# 也可手動指定形狀,例如水平線(專門處理水平結構)
```

```
hline = np.zeros((1, 15), np.uint8); hline[:] = 1
print(cross)    # 觀察十字結構元素的形狀
```

10.2 兩個基本運算：侵蝕與膨脹

10.2.1 侵蝕

侵蝕(Erosion)的規則是：把結構元素滑過影像，只有當結構元素完全被包含在前景之內時，其原點對應的位置才保留為前景，否則設為背景。直觀的效果是：前景物件會「縮水」、邊界向內收縮，小於結構元素的細節(細小突起、細線、孤立雜點)會被完全消滅。

侵蝕的典型用途：去除背景中的孤立雜點、斷開兩個以細頸相連的物件、縮小過度膨脹的前景。一個重要觀察(呼應第 7 章)：對灰階影像而言，侵蝕等同於在鄰域取最小值——這正是第 7 章提過的最小值濾波。同一個運算，在第 7 章是「去除鹽雜訊」，在這裡是「腐蝕形狀」，再次印證影像處理概念的一體多面。

10.2.2 膨脹

膨脹(Dilation)是侵蝕的對偶運算，規則是：只要結構元素與前景有任何交集，其原點對應的位置就設為前景。效果與侵蝕相反：前景物件會「長大」、邊界向外擴張，小的孔洞與裂縫會被填補、斷裂的部分可能重新相連。

膨脹的典型用途：填補前景內的小孔洞、連接斷裂的線段或字元、放大前景以便後續處理。同樣呼應第 7 章：對灰階影像而言，膨脹等同於在鄰域取最大值，即最大值濾波(去除胡椒雜訊)。

侵蝕與膨脹是一對對偶運算：對前景侵蝕，等於對背景膨脹。理解這個對偶性，能幫助你在「該侵蝕還是該膨脹」之間快速做出判斷——問自己是想縮前景還是縮背景即可。

程式 10-2 侵蝕與膨脹

```
import cv2, numpy as np

binary = cv2.imread("binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

eroded = cv2.erode(binary, kernel, iterations=1)    # 侵蝕:縮水
dilated = cv2.dilate(binary, kernel, iterations=1)  # 膨脹:長大

# iterations 控制重複次數,等效於放大結構元素
eroded3 = cv2.erode(binary, kernel, iterations=3)

cv2.imwrite("eroded.png", eroded)
cv2.imwrite("dilated.png", dilated)
```

10.3 兩個組合運算：斷開與閉合

把侵蝕與膨脹依不同順序組合，得到兩個極為實用的運算。它們的巧妙之處在於能在「去除雜訊/填補缺陷」的同時，大致維持物件的原始大小——因為一縮一脹相互抵消。

10.3.1 斷開

斷開(Opening)是「先侵蝕、後膨脹」。先侵蝕消滅了小於結構元素的雜點與細頸，後續的膨脹再把倖存的物件恢復到接近原本的大小。淨效果是：去除背景中的孤立小白點、平滑物件的外部輪廓、斷開細頸相連的物件——而主體幾乎不變。口訣：斷開「開」掉背景的雜訊。

10.3.2 閉合

閉合(Closing)是「先膨脹、後侵蝕」。先膨脹填補了小於結構元素的孔洞與裂縫，後續的侵蝕再把物件收回原本的大小。淨效果是：填補前景內的小孔洞、連接鄰近的斷裂、平滑物件的內部輪廓——而主體幾乎不變。口訣：閉合「閉」合前景的破洞。

斷開與閉合是實務上最常用的型態學運算，尤其在第 9 章分割之後的清理階段。標準流程往往是：Otsu 二值化 → 斷開去除背景雜點 → 閉合填補前景孔洞 → 連通元件標記計數。第 9 章程式 9-4 的分水嶺前處理，用的正是這套組合。

程式 10-3 斷開與閉合

```
import cv2, numpy as np

noisy = cv2.imread("noisy_binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

opening = cv2.morphologyEx(noisy, cv2.MORPH_OPEN, kernel) # 去背景雜點
closing = cv2.morphologyEx(noisy, cv2.MORPH_CLOSE, kernel) # 填前景孔洞

# 實務常見：先斷開再閉合，一次清乾淨
clean = cv2.morphologyEx(noisy, cv2.MORPH_OPEN, kernel)
clean = cv2.morphologyEx(clean, cv2.MORPH_CLOSE, kernel)

cv2.imwrite("opening.png", opening)
cv2.imwrite("closing.png", closing)
```

10.4 進階型態學運算

10.4.1 型態學梯度、頂帽與黑帽

1. 型態學梯度(Gradient)：膨脹減去侵蝕，得到物件的輪廓。它是另一種邊緣偵測方式，與第 9 章的 Sobel、Canny 殊途同歸，但基於形狀運算而非微分。
2. 頂帽轉換(Top Hat)：原圖減去斷開的結果，萃取出「比周圍背景亮、且小於結構元素」的細節。它的殺手級應用是校正不均勻的背景照明——這正是第 8 章「洋裝效應」與第 9 章打光不均問題的另一種解法：用一個大的結構元素做

斷開，得到的是緩慢變化的背景照明，原圖減去它，就抽出了均勻照明下的前景細節。

3. 黑帽轉換(Black Hat)：閉合的結果減去原圖，萃取出「比周圍背景暗、且小於結構元素」的細節，例如亮背景上的暗刮痕、暗字。

頂帽與黑帽在 AOI 檢測中極為實用：當工件表面因曲面或打光而有漸變的亮度背景，直接門檻分割會失敗，先做頂帽或黑帽拉平背景，瑕疵便無所遁形。這與第 6.3.3 節的同態濾波是解決同一問題(不均勻照明)的不同途徑——一個在空間域用形狀運算，一個在頻率域用對數濾波。

10.4.2 Hit-or-Miss、細線化與骨架化

Hit-or-Miss 轉換是型態學的形狀偵測工具：它同時檢查「前景該在的地方是否是前景」與「背景該在的地方是否是背景」，只有兩者都吻合才命中。它能偵測特定的局部形狀，例如角點、線的端點、孤立點，是細線化與骨架化的理論基礎。

細線化(Thinning)反覆剝除物件的邊界像素，但保留其連通性與端點，最終把物件細化成單像素寬的線。骨架化(Skeletonization)則把物件化約成其「中軸」——想像從物件的每個邊界點同時向內燃燒，火線相遇熄滅之處的軌跡就是骨架。這兩者在指紋辨識(把指紋脊線細化)、文字辨識(把筆畫細化以分析結構)、電路板走線分析、道路網路萃取中不可或缺。

10.4.3 孔洞填補與邊界清除

孔洞填補(Hole Filling)以型態學重建的方式，把前景物件內部完全封閉的背景區域(孔洞)填成前景。它在「數清楚有幾個實心物件」時很重要——一個有孔洞的墊圈，若不填補孔洞，連通元件標記可能會被孔洞干擾。邊界清除(Border Clearing)則移除接觸到影像邊界的物件，因為這些物件通常是「被畫面切到、資訊不完整」的，在量測與計數時應予排除——例如量測顯微鏡下的細胞尺寸時，只計算完整落在視野內的細胞。

10.5 連通元件標記與距離轉換

10.5.1 連通元件標記

連通元件標記(Connected Component Labeling)是把二值影像中「每一個相連的前景區域」各自賦予一個獨立編號的過程——這正式回答了「畫面中有幾個物件」這個問題。它建立在第 3 章介紹的連通性概念上：採用 4-連通或 8-連通，計數結果可能不同(第 3 章已強調過此點)。

OpenCV 的 `connectedComponentsWithStats` 不僅標記，還一併回傳每個元件的統計資訊：面積(像素數)、外接矩形(位置與大小)、質心座標。這一步直接把「一堆白色像素」轉化為「一份物件清單」，是所有計數與量測應用的核心——第 9 章的零件計數、細胞計數，骨幹都在這裡。搭配面積門檻，還能輕鬆濾除殘餘的小雜點(面積過小者不計)。

10.5.2 距離轉換

距離轉換(Distance Transform)為二值影像中的每一個前景像素，計算它到最近的背景像素的距離。結果是一張灰階圖：物件中心離背景最遠(值最大、最亮)，邊界處值最小(接近零)。它用到的正是第 3 章介紹的距離度量(歐氏、D4、D8)。

距離轉換有兩個關鍵應用。其一是求骨架：距離轉換的「山脊」(局部最大值的連線)正是物件的中軸，提供了骨架化的另一條途徑。其二是分離相黏物件：這正是第 9 章程式 9-4 分水嶺分割的核心步驟——對兩個相黏的圓形物件做距離轉換，每個圓的中心會各自形成一個亮點(局部最大)，以此作為分水嶺的「確定前景」標記，就能把黏在一起的物件正確分開。至此，第 9 章分水嶺流程中那段「距離轉換」的程式碼，其原理便完全清晰了。

程式 10-4 連通元件標記與距離轉換

```
import cv2, numpy as np

binary = cv2.imread("parts.png", cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(binary, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 連通元件標記 + 統計 ---
n, labels, stats, centroids = cv2.connectedComponentsWithStats(binary)
print("物件數(扣除背景):", n - 1)

output = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
for i in range(1, n):
    area = stats[i, cv2.CC_STAT_AREA]
    if area < 50:
        continue
    x = stats[i, cv2.CC_STAT_LEFT]
    y = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]
    cx, cy = centroids[i]
    cv2.rectangle(output, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.circle(output, (int(cx), int(cy)), 3, (0, 0, 255), -1)
    print(f"物件 {i}: 面積={area}, 質心=({cx:.1f}, {cy:.1f})")

# --- 距離轉換(分離相黏物件的關鍵) ---
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
dist_vis = cv2.normalize(dist, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("distance.png", dist_vis)
cv2.imwrite("labeled.png", output)
```

10.6 Haar 級聯分類器與人臉偵測

本節把叫獸開講的 Viola-Jones 演算法落實成可執行的技術，它也是本章「二值/區域思維」通往第 15、16 章「機器學習物件偵測」的橋梁。

10.6.1 三大支柱回顧

承叫獸開講，Viola-Jones 由三個技術支柱撐起。第一，Haar-like 特徵：計算相鄰矩形區域的亮度差，捕捉人臉的明暗規律(如眼睛比臉頰暗、鼻樑比兩側亮)。第二，積分影像：預先計算每點左上方所有像素的累加和，使任意矩形區域的總和只需查表三次加減即可求得，與區域大小無關——這是它能高速計算海量特徵的關鍵。第三 AdaBoost 與級聯：一個偵測視窗中可能的 Haar 特徵多達十八萬個，AdaBoost 從中篩選出最有鑑別力的少數；再把分類器串成級聯，前段快速淘汰非人臉區域，後段精細確認，達成即時偵測。

值得一提的是，積分影像的技巧與本章 10.5 節連通元件標記「單次掃描完成統計」的精神相通，都是「用預先計算換取查詢時的常數時間」——這是演算法設計中極重要的空間換時間思維，學生應細細體會。

10.6.2 OpenCV 實作

OpenCV 內建訓練好的 Haar 級聯模型(XML 檔)，涵蓋正面人臉、眼睛、微笑、全身等，可直接載入使用。

程式 10-5 Haar 級聯人臉與眼睛偵測

```
import cv2

# 載入 OpenCV 內建的正面人臉 Haar 級聯模型
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_eye.xml")

img = cv2.imread("group_photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # 只需灰階

# detectMultiScale:在多種尺度下滑動視窗偵測
faces = face_cascade.detectMultiScale(gray,
    scaleFactor=1.1,      # 每次縮放比例(影響速度與漏檢)
    minNeighbors=5,      # 需多少鄰近偵測才算數(抑制誤報)
    minSize=(30, 30))    # 最小人臉尺寸

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    roi_gray = gray[y:y+h, x:x+w]  # 第 2 章的 ROI
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(img[y:y+h, x:x+w], (ex, ey),
            (ex+ew, ey+eh), (255, 0, 0), 2)

print("偵測到人臉數:", len(faces))
cv2.imwrite("faces_detected.jpg", img)
```

兩個參數的調校心法：scaleFactor 愈接近 1，搜尋的尺度愈細密、愈不易漏檢，但速度愈慢；minNeighbors 愈大，愈能抑制誤報(要求同一位置有多個尺度都偵測到

才確認)，但也可能漏掉真實人臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 的雙門檻、第 16 章物件偵測的信心門檻，是同一類工程取舍。

10.6.3 三種辨識架構與 ESP32-CAM 部署

回顧第 2 章介紹的三種 AI 辨識架構——端上運算、雲端運算、近端邊緣運算——Viola-Jones 的輕量特性(不需 GPU、記憶體需求小)使它成為端上運算的理想選擇。事實上，ESP32-CAM 的官方韌體就內建了人臉偵測與辨識功能，正是基於這類輕量演算法。

這帶出一個重要的部署思考：在 ESP32-CAM 這樣算力極為有限的裝置上，深度學習的 YOLO(第 16 章)難以即時執行，而 Viola-Jones 這類傳統方法反而遊刃有餘。這再次印證第 9 章的判斷原則：方法沒有絕對優劣，只有適不適合。當任務是「在低功耗感測節點上偵測正面人臉」時，一個 2001 年的演算法可能比 2020 年代的深度模型更務實。第 17 章會深入這個部署決策。

10.7 應用案例

10.7.1 案例一：PCB 與工件的瑕疵前處理

情境：AOI 系統要檢測印刷電路板上的斷路、短路、缺件。分割(第 9 章)後得到的二值影像，線路邊緣常有毛刺、線路中有雜點造成的假斷點。處理流程：先用斷開去除毛刺與孤立雜點 → 用閉合填補線路上因反光造成的假斷點 → 用型態學梯度萃取乾淨的線路輪廓 → 用細線化把線路化約成單像素骨架，便於分析走線的連通性與交叉點。若背景照明不均，則在分割前先用頂帽轉換拉平背景。這整套流程展示了型態學作為「分割與量測之間的清理工序」的核心價值。

10.7.2 案例二：細胞與顆粒計數

情境：顯微鏡影像中要自動計數細胞或顆粒，且它們常相互接觸。這是本章與第 9 章的完整綜合：Otsu 二值化 → 斷開去雜點 → 孔洞填補(把細胞內的亮點填實) → 邊界清除(排除被視野邊緣切到的不完整細胞) → 對相黏的細胞做距離轉換，取局部極大作為標記 → 分水嶺分割分開它們 → 連通元件標記計數並量測每個細胞的面積。生物醫學、水質檢測、材料科學的顆粒分析，骨幹都是這條管線。

10.7.3 案例三：即時人臉偵測門禁

情境：用 ESP32-CAM 打造一個低成本的門禁裝置，偵測到人臉時觸發後續動作(拍照上傳、開門、記錄)。這是叫獸開講與第 2 章三架構的實戰整合：ESP32-CAM 在端上以 Haar 級聯即時偵測人臉(端上運算，低延遲、保護隱私)，偵測到後再把影像上傳雲端做進一步的身分辨識(雲端運算，借助強大算力)——這正是第 2 章「近端/雲端混合架構」的體現：輕量的偵測放在端上、耗算力的辨識放在雲端，兼顧即時性與準確度。Lab 10-3 會實作這個系統的偵測端。

10.7.4 案例四：車牌字元分割

情境：車牌辨識(第 13 章會完整處理)的前段，需要把車牌上的每個字元切開。流程：定位並裁切出車牌區域 → 轉灰階、Otsu 二值化 → 用閉合連接字元中斷裂的筆畫(如數字 8 因印刷不良而斷開) → 用斷開去除邊框雜點 → 連通元件標記，每個連通區域就是一個字元 → 依面積、長寬比濾除非字元的雜訊區域 → 依 x 座標排序，得到字元序列。這條「二值化 → 型態學清理 → 連通元件分割字元」的流程，是所有文字/字元辨識(OCR)的共通前處理，也直接銜接第 13 章的特徵擷取與第 15 章的字元分類。

10.8 本章與全書的觀念連結總表

相關章節	共用的觀念	二值處理中的角色
第 3 章 數位影像基礎	連通性、距離度量	連通元件標記與距離轉換的理論基礎
第 6 章 頻率域	不均勻照明校正	頂帽轉換與同態濾波是同一問題的兩種解法
第 7 章 影像還原	最大值/最小值濾波	灰階膨脹即最大值濾波、侵蝕即最小值濾波
第 9 章 影像分割	二值化、分水嶺	型態學是分割輸出的直接清理工序;距離轉換供分水嶺標記
第 13 章 特徵擷取	輪廓、形狀特徵、車牌	連通元件提供物件清單;字元分割接特徵擷取
第 15、16 章 深度學習	Haar 級聯、物件偵測	Viola-Jones 是深度學習偵測器的前輩
第 17 章 邊緣部署	三種辨識架構、ESP32-CAM	輕量的 Haar 級聯適合端上運算

表10-1 第10 章與全書其他章節的觀念連結

程式實作 Lab 10

Lab 10-1 型態學運算探索器

目標：建立對型態學運算的直覺。(a)取一張含雜訊的二值影像(可對乾淨二值圖加椒鹽雜訊);(b)以 `cv2.createTrackbar` 建立可調整結構元素形狀(方/十字/橢圓)、大小、運算類型(侵蝕/膨脹/斷開/閉合)與 `iterations` 的即時介面;(c)實驗並記錄：去除背景孤立雜點該用哪個運算?填補前景小孔該用哪個?斷開細頸相連的兩物件該用哪個?(d)驗證對偶性：對前景侵蝕，與對背景膨脹(先反轉、膨脹、再反轉)，結果是否相同?(e)驗證與第 7 章的關聯：對灰階影像做膨脹，結果是否等同於最大值濾波?

Lab 10-2 完整計數量測管線

目標：整合第 9、10 章完成一條可用的計數量測系統。(a)拍一張散落的螺絲、鈕扣或豆子照片;(b)依序完成：CLAHE 前處理(第 5 章)→ Otsu 二值化(第 9 章)→ 斷開去雜點 → 閉合填孔 → 孔洞填補 → 邊界清除 → 連通元件標記;(c)以面積門檻濾除雜點，框出每個物件、標示質心與編號、印出總數與各自面積;(d)進階：若物件相黏，加入距離轉換與分水嶺(第 9 章程式 9-4)正確分開，比較加入前後的計數差異;(e)量測校準：放一個已知尺寸的參考物(如硬幣)，換算出每像素代表的實際毫米數(呼應第 4 章相機校正)，輸出每個物件的實際尺寸。

Lab 10-3 ESP32-CAM 人臉偵測門禁

目標：實作叫獸開講與 10.6.3 節的端上人臉偵測。(a)先在電腦上以程式 10-5 跑通 Haar 級聯人臉偵測，調校 scaleFactor 與 minNeighbors，記錄「漏檢」與「誤報」如何隨參數變化;(b)改以 ESP32-CAM 的串流網址作為輸入來源(第 2、8 章)(c)當偵測到人臉持續數影格，觸發動作(存檔、亮 LED、印出時間戳記);(d)量測 FPS，比較「電腦端」與「ESP32-CAM 端上」的效能差異;(e)討論題：承第 2 章三架構——這個門禁若要加上「辨識是誰」的功能，偵測與辨識應各自放在端上、近端還是雲端?請畫出你的系統架構圖並說明理由。

本章重點整理

- 二值影像每像素僅前景/背景兩值，多為第 9 章分割的產物;型態學以集合論為基礎，用結構元素探測前景形狀，結構元素的形狀與大小是處理關鍵。
- 侵蝕使前景縮水、消滅小細節(對灰階即最小值濾波);膨脹使前景長大、填補小孔(對灰階即最大值濾波);兩者為對偶運算。
- 斷開(先侵蝕後膨脹)去除背景雜點且維持主體大小;閉合(先膨脹後侵蝕)填補前景孔洞且維持主體大小;是分割後清理的常用組合。
- 型態學梯度求輪廓;頂帽萃取亮細節並校正不均勻背景(與同態濾波同功);黑帽萃取暗細節;Hit-or-Miss 偵測特定形狀，為細線化與骨架化的基礎。
- 連通元件標記回答「有幾個物件」並提供面積、外接矩形、質心等統計;距離轉換為每個前景像素計算到最近背景的距離，用於求骨架與分離相黏物件(分水嶺標記)。
- Viola-Jones 以 Haar 特徵、積分影像、AdaBoost 級聯三支柱達成即時人臉偵測，輕量而適合 ESP32-CAM 等端上部署;擅長正面人臉，側臉與遮擋則需第 16 章的深度學習方法。

習題

1. 二值影像通常從何而來?為什麼把影像化約成非黑即白反而有價值?分割產生的二值影像常有哪些缺陷?

提示與參考解答：多來自第 9 章的分割(門檻、Otsu、邊緣偵測)。價值在於：當只關心物件的位置、形狀、數量時，二值化讓形狀分析極為高效。常見缺陷：前景內

有小孔洞(反光/雜訊)、背景有孤立小白點、該分開的物件相黏、該相連的物件斷裂——這些正是型態學要清理的。

2. 什麼是結構元素?它與空間濾波的核有何異同?為什麼選擇不同形狀的結構元素會影響結果?

提示與參考解答: 結構元素是型態學用來探測前景的小形狀, 相當於濾波的核, 但作用是集合運算而非數值加權。形狀(方/十字/圓/線)會影響結果: 例如水平線形結構元素會強化水平結構、削弱垂直結構, 圓形則各方向一視同仁。選對形狀與大小是型態學的關鍵。

3. 說明侵蝕與膨脹的規則與效果, 並指出它們與第 7 章哪兩種濾波等價。

提示與參考解答: 侵蝕: 結構元素完全落在前景內時原點才保留, 效果是前景縮水、消滅小細節, 等價於灰階的最小值濾波(去鹽雜訊)。膨脹: 結構元素與前景有交集即設為前景, 效果是前景長大、填補小孔, 等價於最大值濾波(去胡椒雜訊)。

4. 斷開與閉合各是什麼順序的組合?為什麼它們能在去除缺陷的同時大致維持物件原本大小?各自的口訣是什麼?

提示與參考解答: 斷開=先侵蝕後膨脹, 去除背景雜點(「開」掉背景雜訊);閉合=先膨脹後侵蝕, 填補前景孔洞(「閉」合前景破洞)。維持大小的原因: 一縮一脹(或一脹一縮)相互抵消, 主體回到接近原始尺寸, 只有小於結構元素的缺陷被永久去除或填補。

5. 頂帽轉換如何校正不均勻的背景照明?它與第 6 章的哪個技術解決同一問題?兩者作用的域有何不同?

提示與參考解答: 頂帽=原圖減去斷開結果。用大結構元素做斷開會得到緩慢變化的背景照明, 原圖減去它即抽出均勻照明下的前景亮細節。它與第 6.3.3 節的同態濾波解決同一問題(不均勻照明): 頂帽在空間域用形狀運算, 同態濾波在頻率域用對數加高低頻分離。

6. 細線化與骨架化的目的是什麼?各舉一個應用。骨架化與距離轉換有何關聯?

提示與參考解答: 目的是把物件化約成單像素寬的線或中軸, 以分析其拓撲結構。細線化應用: 指紋脊線細化、文字筆畫分析。骨架化應用: 電路走線分析、道路網路萃取。關聯: 距離轉換的山脊(局部最大值連線)正是物件中軸, 提供求骨架的另一途徑。

7. 連通元件標記回答什麼問題?它建立在第 3 章的什麼概念上?
connectedComponentsWithStats 除了標記還提供哪些資訊?

提示與參考解答: 回答「畫面中有幾個相連的前景區域(物件)」。建立在第 3 章的連通性(4-連通或 8-連通, 選擇不同計數可能不同)。額外資訊: 每個元件的面積、外接矩形(位置與大小)、質心座標, 可直接用於計數與量測, 並以面積門檻濾除雜點。

8. 距離轉換如何幫助分開相黏的物件?請說明它在第 9 章分水嶺流程中扮演的角色。

提示與參考解答: 對相黏的圓形物件做距離轉換, 每個物件的中心離背景最遠、形成一個局部極大亮點。取這些亮點作為分水嶺的「確定前景」標記, 由此注水, 即可在兩物件的接縫處築起分界, 正確分開它們。這就是第 9 章程式 9-4 中距離轉換那一步的原理。

9. Viola-Jones 的三大技術支柱是什麼?各自解決了什麼問題?

提示與參考解答：一、Haar-like 特徵：以相鄰矩形亮度差捕捉人臉明暗規律(如眼比頰暗)。二、積分影像：預先累加，使任意矩形區域總和查表三次加減即得、與大小無關，解決海量特徵的計算速度。三、AdaBoost 級聯：從十八萬個特徵篩選少數有效者，並串成級聯前段快速淘汰非人臉、後段精細確認，達成即時偵測。

10. 積分影像的「預先計算換取常數時間查詢」思想，與本章哪個技術相通？這屬於哪一類演算法設計思維？

提示與參考解答：與連通元件標記的「單次掃描完成統計」相通，也與部分實作中的累加思想一致。這屬於「空間換時間」的演算法設計思維：預先付出計算與儲存成本，換取後續查詢時的極高速度。

11. detectMultiScale 的 scaleFactor 與 minNeighbors 各控制什麼？調大或調小如何影響「漏檢」與「誤報」？這與第 9、16 章的哪些取捨同類？

提示與參考解答：scaleFactor 是每次尺度縮放比例：愈接近 1 搜尋愈細密、愈不漏檢但愈慢。minNeighbors 是確認一個偵測所需的鄰近偵測數：愈大愈能抑制誤報但可能漏掉真臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 雙門檻、第 16 章物件偵測的信心門檻同類。

12. (綜合題)為什麼在 ESP32-CAM 上偵測人臉，2001 年的 Viola-Jones 可能比 2020 年代的 YOLO 更合適？請以第 2 章三架構與第 9 章「方法沒有絕對優劣」的觀點論證。

提示與參考解答：ESP32-CAM 算力與記憶體極為有限，YOLO 等深度模型難以即時執行；Viola-Jones 輕量、不需 GPU，適合端上運算(低延遲、保護隱私、不需連網)。以第 2 章三架構看，輕量偵測放端上最務實；若需辨識身分再上傳雲端。呼應第 9 章：方法沒有絕對優劣，只有適不適合任務與硬體——在低功耗節點偵測正面人臉的場景，舊演算法反而勝出。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 9(型態學影像處理之標準參考)。
- Viola, P., & Jones, M., Rapid Object Detection using a Boosted Cascade of Simple Features, CVPR, 2001.(本章叫獸開講之史料來源)
- Viola, P., & Jones, M., Robust Real-Time Face Detection, IJCV, 57(2), 2004.
- Serra, J., Image Analysis and Mathematical Morphology, Academic Press, 1982.(數學型態學經典)
- Soille, P., Morphological Image Analysis: Principles and Applications, 2nd ed., Springer, 2003.
- OpenCV 官方教學：Morphological Transformations 與 Cascade Classifier。
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 11 章 小波與正交轉換

本章學習目標

- 說明正交轉換的共同思想，理解「換一組基底表示影像」為何有利於壓縮與分析。
- 闡述傅立葉轉換「有頻率、無位置」的根本弱點，說明短時傅立葉與小波如何回應。
- 推導 Haar 小波與多解析度分析，實作二維離散小波轉換的多層分解。
- 說明離散餘弦轉換(DCT)的原理，完整拆解 JPEG 壓縮的五個步驟與方塊效應成因。
- 比較 JPEG 與 JPEG 2000(小波)的差異，解釋小波為何避免方塊效應。
- 運用小波於影像去雜訊、融合與浮水印，並認識其在深度學習時代的定位。

叫獸開講|FBI 的指紋危機：三千五百萬張卡片與小波的救援

1990 年代初，美國聯邦調查局(FBI)面臨一個幾乎要把系統壓垮的難題。它的檔案庫裡堆放著超過三千五百萬張紙本指紋卡，還有大量待分類的積壓，而每天湧入的指紋比對請求又不斷增加。唯一的出路是全面數位化——但這裡藏著一個駭人的數字：以當年的技術把整個指紋庫掃描存檔，將產生大約一千一百四十兆位元組(超過一千 TB)的影像資料。光是儲存就是天文數字，更別提用當年的數據機傳輸一張指紋卡要花上好幾分鐘。不壓縮，數位化根本不可能。

最直覺的選擇是用現成的 JPEG 壓縮。但 FBI 與國家標準暨技術研究院(NIST)的早期測試發現一個致命問題：JPEG 在高壓縮率下會產生「方塊效應」——影像被切成 8×8 的小方塊各自處理，壓得愈狠，方塊之間的接縫就愈明顯，像一面貼歪的磁磚牆。對一般照片這或許可以忍受，但對指紋是災難：指紋辨識靠的是脊線的細微特徵點(minutiae)——分岔、端點、間距，而 JPEG 的方塊效應與高頻細節流失，恰恰會破壞或偽造這些用於法律鑑識的關鍵特徵。一個被壓縮演算法「抹掉」或「無中生有」的特徵點，在法庭上可能就是天壤之別。

FBI 找上了洛杉磯阿拉莫斯國家實驗室。由 FBI 的 Tom Hopper 與洛杉磯阿拉莫斯的 Jonathan Bradley、Chris Brislawn 領軍的團隊，最終採用了一套基於小波的方案，稱為「小波純量量化」(Wavelet Scalar Quantization, WSQ)，並在 1993 年定為國家標準。與 JPEG 把影像硬切成方塊不同，小波轉換是對整張影像做多解析度分解，不存在方塊邊界，因此從根本上避免了方塊效應。WSQ 能在 20:1 的壓縮率下，達到人眼幾乎無法察覺的品質損失，同時保留鑑識級的脊線細節。三千五百萬張指紋卡的數位化危機，就這樣被一個當時還很年輕的數學工具——小波——化解了。

這個故事點出了本章的核心。第 6 章我們讚嘆傅立葉轉換的威力，但也埋下一個伏筆：它是「全域」的，只告訴你影像含有哪些頻率，卻不告訴你這些頻率出現在哪裡。JPEG 用「切成小方塊」來勉強取得局部性，代價就是方塊效應。而小波提供了一個更優雅的答案：它能同時告訴你「有哪些頻率」與「頻率在哪裡」——這就是多解析度分析。本章要講的，正是一場「如何同時掌握頻率與位置」的數學奮鬥史，以及它如何撐起你每天使用的影像壓縮。

思考題：JPEG 為了取得「局部性」而把影像切成 8×8 方塊，結果付出了方塊效應的代價。小波用不切塊的方式達到局部性。這讓你想到什麼工程上的通則？（提示：當一個系統為了某個好處而做出粗暴的妥協時，往往存在更精巧的替代方案）

11.1 正交轉換的共同思想

11.1.1 換一組基底看世界

本章與第 6 章共享一個深刻的思想：把影像從一組基底轉換到另一組基底表示。第 6 章的傅立葉轉換，是把影像表示成一堆「不同頻率的弦波(斜條紋圖)」的加權和；本章的離散餘弦轉換與小波轉換，只是換用不同的基底函數，但精神完全一致——都是在問「這張影像，用這組積木來拼，各需要多少？」

為什麼要換基底？因為在一個好的基底下，影像的資訊會變得「稀疏」——絕大部分的能量集中在少數幾個係數上，其餘係數接近於零。而稀疏正是壓縮的黃金鑰匙：只要記住那少數重要的係數、把大量接近零的係數丟棄或粗略記錄，就能用極少的資料重建出接近原貌的影像。這是所有轉換式壓縮(JPEG、JPEG 2000)的共同原理。自然影像在空間域(像素域)並不稀疏(每個像素都攜帶資訊)，但轉換到頻率域或小波域後就變得非常稀疏——這就是轉換的價值。

11.1.2 正交的意義

這些轉換之所以稱為「正交」(Orthogonal)，是因為它們的基底函數彼此正交——用向量的語言說，任兩個基底函數的內積為零，彼此獨立、互不重疊。正交性帶來三個好處：轉換可逆(能無失真地變回原影像)、能量守恆(轉換前後總能量不變，即 Parseval 定理，這讓我們能在轉換域中判斷丟棄某係數會損失多少能量)、以及係數之間互不干擾(改動一個係數不影響其他係數的意義)。第 6 章的傅立葉、本章的 DCT 與正交小波，都具備這些性質。

11.2 傅立葉的弱點與小波的動機

11.2.1 有頻率，無位置

第 6.6.5 節已經預告過傅立葉轉換的根本弱點，這裡完整展開。傅立葉轉換用「無限延伸的弦波」作為基底——每一個弦波都從負無窮延伸到正無窮，在整張影像上均勻存在。這帶來一個後果：當你在頻譜上看到某個頻率成分很強，你知道「這張影像含有這個頻率」，但你完全不知道「這個頻率來自影像的哪個位置」。

用一個聽覺的比喻：傅立葉分析一首樂曲，能告訴你整首曲子用到了哪些音符，卻無法告訴你這些音符出現的先後順序——它把樂譜壓扁成了一張「音符清單」。對於平穩(statistics 不隨位置改變)的訊號，這不成問題;但影像恰恰是高度非平穩的：左半邊可能是平滑的天空(只有低頻)，右半邊可能是茂密的樹葉(充滿高頻)。傅立葉把這兩者的頻率混在一起報告，喪失了寶貴的位置資訊。

11.2.2 短時傅立葉與測不準

第一個補救是短時傅立葉轉換(STFT)：把影像切成小塊，對每一塊各自做傅立葉轉換，如此就能知道「某頻率出現在某塊」。JPEG 的 8×8 分塊 DCT 正是這個思路的體現。但 STFT 有一個先天的困境：窗口大小固定。窗口切得大，頻率分辨得準、但位置模糊;窗口切得小，位置定得準、但頻率模糊。你無法讓兩者同時精準——這就是訊號處理版的測不準原理(第 6.3.2 節提過「函數不可能在兩個域同時很窄」)。JPEG 的方塊效應，本質上就是固定窗口大小付出的代價。

11.2.3 小波的洞見：讓窗口大小自己調整

小波的天才之處，在於它放棄了「固定窗口」。它的觀察是：低頻成分變化緩慢需要長窗口才能看清(犧牲位置換頻率精度);高頻成分變化劇烈且短暫，需要短窗口才能定位(犧牲頻率換位置精度)。小波用一個「有限長度的小波形」(mother wavelet)作為基底，透過縮放(scaling，拉長或壓縮波形以對應不同頻率)與平移(translation，移動波形以對應不同位置)，自動實現「低頻用長窗、高頻用短窗」的自適應分析。

這正是多解析度分析(Multi-Resolution Analysis)的精髓：在粗的尺度上看大輪廓(低頻、低空間解析度)，在細的尺度上看小細節(高頻、高空間解析度)，不同尺度各司其職。它同時提供了頻率與位置資訊，徹底回應了傅立葉的弱點——這也是小波能避免方塊效應的根本原因：它從一開始就不需要把影像切成方塊。

11.3 Haar 小波與多解析度分析

11.3.1 最簡單的小波：Haar

Haar 小波是最早、也最簡單的小波，早在 1909 年就由 Alfréd Haar 提出(比小波理論的正式成熟早了數十年)。它的一維運算簡單到可以手算：對相鄰的一對數值，計算它們的平均(得到「概略」資訊，對應低頻)與差值的一半(得到「細節」資訊，對應高頻)。例如數列 $[8, 4]$ ：平均是 6、差的一半是 2，於是 $[8, 4]$ 被表示成 $[6, 2]$ ——其中 6 是概略、2 是細節。要還原也容易： $8 = 6 + 2$ 、 $4 = 6 - 2$ 。這個「平均與差分」的配對，就是最基本的小波分解。

對整個數列反覆施行：先兩兩配對得到一半長度的概略序列與一半長度的細節序列，再對概略序列繼續兩兩配對……如此層層遞進，就是多解析度分解。每往上一層，概略的解析度減半(看得更粗略)，而每一層的細節被分別保留。這解釋了小波「多解析度」之名的由來，也說明了它與傅立葉的根本不同：傅立葉一次把訊號攤平成頻譜，小波則是一層一層地剝出不同尺度的細節。

11.3.2 二維小波轉換與四個子帶

把一維小波推廣到二維影像的標準做法是：先對每一列做一維小波轉換(水平方向)，再對每一行做一維小波轉換(垂直方向)。一層轉換後，影像被分解成四個子帶(subband)，各佔原影像四分之一大小：

1. LL(低-低)子帶：水平垂直都取概略，是原影像的縮小版(縮略圖)，集中了絕大部分能量。
2. LH(低-高)子帶：水平概略、垂直細節，凸顯水平方向的邊緣。
3. HL(高-低)子帶：水平細節、垂直概略，凸顯垂直方向的邊緣。
4. HH(高-高)子帶：兩方向都取細節，凸顯對角線方向的細節與雜訊。

多層分解時，只對 LL 子帶(縮略圖)繼續往下分解，形成一個由粗到細的金字塔結構。這個結構的美妙之處在於：大部分能量集中在最頂層的小小 LL 子帶，而其他子帶的係數大多接近於零(自然影像的細節是稀疏的)——這正是 11.1.1 節說的「稀疏性」，壓縮的機會就藏在這裡。

程式 11-1 安裝 PyWavelets

```
# 安裝小波處理套件
pip install PyWavelets
```

程式 11-2 二維小波分解與重建

```
import cv2, numpy as np, pywt

gray = cv2.imread("fingerprint.jpg",
cv2.IMREAD_GRAYSCALE).astype(float)

# --- 單層二維 Haar 小波轉換 ---
coeffs = pywt.dwt2(gray, "haar")
LL, (LH, HL, HH) = coeffs
print("四個子帶尺寸:", LL.shape)      # 各為原圖的四分之一

# 把四個子帶拼成一張圖觀察
def norm(x):
    return cv2.normalize(x, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
top = np.hstack([norm(LL), norm(LH)])
bot = np.hstack([norm(HL), norm(HH)])
cv2.imwrite("wavelet_subbands.png", np.vstack([top, bot]))

# --- 多層分解(三層) ---
coeffs_multi = pywt.wavedec2(gray, "haar", level=3)
arr, slices = pywt.coeffs_to_array(coeffs_multi)
cv2.imwrite("wavelet_pyramid.png", norm(arr))

# --- 反轉換:確認可完整還原 ---
recon = pywt.idwt2(coeffs, "haar")
print("還原最大誤差:", np.abs(gray - recon).max())    # 接近 0
```

除了 Haar，實務上常用的小波還有 Daubechies(db 系列，由 Ingrid Daubechies 於 1988 年提出，是小波理論成熟的里程碑)、Biorthogonal(雙正交，JPEG 2000 採用)等。不同的小波在「平滑度」與「支撐長度」上各有取捨，選擇適合訊號特性的小波，是小波應用的一門學問——FBI 的 WSQ 正是精心挑選了適合指紋脊線特性的小波。

11.4 離散餘弦轉換與 JPEG 壓縮

在小波稱霸之前(以及在絕大多數日常照片中至今)，影像壓縮的主力是離散餘弦轉換(DCT)與建立其上的 JPEG。理解 JPEG，是理解影像壓縮不可跳過的一課，它也把第 3、6、8 章的許多概念串在一起。

11.4.1 離散餘弦轉換

DCT 是傅立葉家族的近親，差別在於它只使用餘弦函數作為基底。這帶來一個實用的好處：對實數影像做 DCT 得到的係數也是實數(不像傅立葉會產生複數)，處理起來更簡便。更重要的是，DCT 有優異的能量集中特性：對自然影像的區塊做 DCT，能量會高度集中在少數幾個低頻係數(左上角)上，而高頻係數(右下角)大多接近零——這種稀疏性正是壓縮的基礎(呼應 11.1.1 節)。

11.4.2 JPEG 壓縮的五個步驟

JPEG 是一條精心設計的管線，每個步驟都呼應本書前面的概念：

5. 色彩空間轉換與降取樣：先把 RGB 轉為 YCbCr(第 8 章)，分離亮度 Y 與色度 Cb、Cr；因為人眼對色度不敏感，對 Cb、Cr 做 4:2:0 降取樣(解析度砍四分之一)，這一步就先省下一半資料且幾乎無感——這是第 8 章「人眼對色度遲鈍」的直接應用。
6. 分塊與 DCT：把每個通道切成 8×8 的小區塊，對每一塊各自做 DCT。切塊正是 11.2.2 節短時傅立葉的思路——用分塊取得局部性，但也埋下方塊效應的種子。
7. 量化(Quantization)：這是 JPEG 唯一「失真」的步驟，也是壓縮的核心。用一張「量化表」把每個 DCT 係數除以對應的量化步長再取整數——高頻係數用較大的步長(除以大數字)，許多因此變成零。品質參數(quality)控制的正是量化表的縮放：品質愈低、量化步長愈大、被歸零的高頻愈多、檔案愈小、失真愈大。這一步利用了人眼對高頻細節較不敏感的特性，大膽丟棄「看不太出來」的資訊。
8. Zigzag 掃描與遊程編碼：量化後， 8×8 區塊的右下角(高頻)佈滿了零。以 zigzag(之字形)順序掃描，把二維區塊排成一維序列，能讓這些零連續排在一起，再用遊程編碼(把「連續 20 個零」記成一個符號)大幅壓縮。
9. 熵編碼(Entropy Coding)：最後用 Huffman 編碼(或算術編碼)對結果做無失真壓縮，把出現頻繁的符號用較短的碼表示。這一步不丟資訊，純粹榨乾統計冗餘。

解壓縮則是完全反向的過程。整條管線中，只有第三步「量化」是不可逆的失真來源——這也是為什麼反覆存檔 JPEG 會愈存愈糊：每存一次就量化一次。

程式 11-3 DCT 量化與 JPEG 品質實驗

```

import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)

# --- 手動示範 JPEG 的核心:8x8 區塊 DCT 與量化 ---
# JPEG 標準亮度量化表(quality~50)
Q = np.array([
    [16,11,10,16,24,40,51,61], [12,12,14,19,26,58,60,55],
    [14,13,16,24,40,57,69,56], [14,17,22,29,51,87,80,62],
    [18,22,37,56,68,109,103,77], [24,35,55,64,81,104,113,92],
    [49,64,78,87,103,121,120,101],[72,92,95,98,112,100,103,99]],
    dtype=np.float32)

block = gray[:8, :8] - 128          # 置中
dct_block = cv2.dct(block)          # 8x8 DCT
quantized = np.round(dct_block / Q) # 量化(失真在此發生)
print("非零係數數量:", np.count_nonzero(quantized), "/ 64")

# 反量化 + 反 DCT 還原
recovered = cv2.idct(quantized * Q) + 128

# --- 實用:直接用不同品質存 JPEG,觀察方塊效應 ---
img = cv2.imread("photo.jpg")
for q in [90, 50, 10]:
    cv2.imwrite(f"jpeg_q{q}.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, q])
    back = cv2.imread(f"jpeg_q{q}.jpg")
    print(f"品質 {q}: PSNR = {cv2.PSNR(img, back):.2f} dB")
    # 放大 q10 的結果,可清楚看到 8x8 方塊邊界

```

11.5 小波壓縮與 JPEG 2000

理解了 JPEG 的方塊之痛，小波壓縮的優勢就不言自明。JPEG 2000 是小波壓縮的標準化成果，它用二維離散小波轉換取代了 JPEG 的分塊 DCT。關鍵差異在於：小波轉換作用於整張影像(或大的圖塊)，不把影像硬切成獨立的 8×8 方塊，因此從根本上不存在方塊邊界——高壓縮率下，JPEG 2000 的影像是整體變得柔和模糊，而非碎裂成磁磚，視覺上遠優於同壓縮率的 JPEG。

小波壓縮的一般流程與 WSQ、JPEG 2000 一致：對影像做多層小波分解 → 對各子帶的係數做量化(利用能量集中於 LL 子帶、其餘子帶稀疏的特性) → 熵編碼。除了避免方塊效應，小波壓縮還有一個附加優點：天然支援漸進式傳輸與多解析度存取——先傳最頂層的 LL(縮略圖)就能顯示模糊的全貌，隨著更多子帶傳入逐步變清晰，非常適合網路傳輸與大圖瀏覽。

比較面向	JPEG(DCT)	JPEG 2000 / WSQ(小波)
轉換單位	8×8 獨立方塊	整張影像多解析度分解
高壓縮率瑕疵	方塊效應、蚊狀雜訊	整體柔化模糊，無方塊
漸進式傳輸	有限(progressive 模式)	天然支援，多解析度

運算複雜度	較低，普及極廣	較高
典型應用	一般照片、網頁、相機	指紋(WSQ)、醫療、電影數位母帶

表 11-1 JPEG 與小波壓縮的比較

那為什麼今天絕大多數照片仍是 JPEG? 因為 JPEG「夠好、夠快、夠普及」——它的運算簡單、硬體支援完善、相容性無敵。JPEG 2000 雖然技術更優，卻因複雜度較高與相容性問題，主要用在對品質要求極高的專業領域(醫療影像、電影數位母帶、指紋鑑識)。這是工程世界的常態：最好的技術不一定勝出，「足夠好加上生態系」往往才是贏家。這個觀察值得每一位工程師記在心裡。

11.6 小波的其他應用

11.6.1 小波去雜訊

小波提供了一種優雅的去雜訊思路，與第 7 章的方法互補。核心觀察是：自然影像的重要結構(邊緣、輪廓)在小波分解後，會集中成少數幾個「大係數」；而雜訊則均勻散布成大量「小係數」。因此去雜訊的策略非常直觀——小波門檻化(Wavelet Thresholding)：把小波係數中絕對值小於某門檻的通通歸零(視為雜訊)，保留大係數(視為訊號)，再反轉換。

這個方法之所以強大，是因為它在「同時掌握頻率與位置」的小波域中操作：它能區分「邊緣附近的高頻(該保留)」與「平坦區的高頻(是雜訊，該去除)」——這是第 7 章純空間域或第 6 章純頻率域方法難以做到的。它與第 7 章的非局部平均、維納濾波各擅勝場，共同構成去雜訊的工具箱。

11.6.2 影像融合

影像融合(Image Fusion)把多張影像的資訊合併成一張。典型應用：把一張對焦近物、一張對焦遠物的照片融合成「全景深」清晰影像；把紅外線影像與可見光影像融合以兼得夜視與細節；醫學上把 CT(骨骼)與 MRI(軟組織)融合。小波是影像融合的常用工具：先把各影像做小波分解，再依規則合併各子帶的係數(例如細節子帶取絕對值較大者，代表該處較清晰)，最後反轉換得到融合影像。

11.6.3 數位浮水印

數位浮水印(Watermarking)把識別資訊(版權標記、序號)隱藏進影像。把浮水印嵌入小波域的中頻係數，是常見的強健做法：它既不像嵌入低頻那樣容易被肉眼察覺，也不像嵌入高頻那樣容易被壓縮或濾波破壞，能在影像被 JPEG 壓縮、縮放後依然存活。這在版權保護與影像鑑識中很重要。

11.7 小波在深度學習時代的定位

進入深度學習時代，壓縮與特徵擷取的許多工作逐漸被神經網路接手，小波是否過時了? 答案與第 9、10 章一致：它轉變了角色，但並未退場。

三個觀察值得學生留意。其一，小波仍是專業壓縮的中流砥柱：指紋(WSQ)、醫療影像、電影數位母帶仍大量使用小波，因為這些領域對「無方塊、可鑑識」的要求無可妥協。其二，小波與深度學習正在融合：近年出現不少「小波卷積網路」的研究，把小波的多解析度分解作為神經網路的一層，結合兩者的長處——用小波提供的多尺度結構，幫助網路更有效率地學習。其三，多解析度的思想無所不在：第 13 章 SIFT 的尺度空間、卷積網路中的多層降取樣(每一層看到的都是更粗尺度的特徵)，骨子裡都與小波「由粗到細、多尺度分析」的精神相通。

換言之，小波作為一個「具體演算法」或許在日常應用中不如深度學習搶眼，但它所承載的「多解析度分析」思想，已經深深融入現代電腦視覺的血液。理解小波，就是理解這個貫穿全書(第 6 章頻率、第 9 章尺度、第 13 章 SIFT、第 15 章 CNN)的多尺度思維。

11.8 應用案例

11.8.1 案例一：指紋壓縮與 WSQ

呼應叫獸開講，這是小波壓縮的旗艦案例。情境：執法機構需要儲存與傳輸海量的 500 ppi 指紋影像，且不能損失用於鑑識的脊線細節。WSQ 的流程：對整張指紋影像做多層離散小波分解(精選適合指紋脊線頻率特性的小波)→ 對各子帶做純量量化(依子帶的重要性分配不同的量化精度，脊線所在的頻帶量化得細)→ Huffman 熵編碼。在 20:1 壓縮率下，人眼幾乎無法察覺品質損失，關鍵的特徵點得以保留。這個案例完美展示了小波「無方塊、保細節」相對 JPEG 的決定性優勢——在鑑識這種細節攸關人命的場合，方塊效應是不可接受的。

11.8.2 案例二：多焦點影像融合

情境：用顯微鏡或微距鏡頭拍攝時，景深很淺，一張照片只能拍清楚某個深度的物體。要得到「全部清晰」的影像，可拍攝一系列對焦在不同深度的照片，再融合。小波做法：對每張照片做小波分解 → 在每個位置，細節子帶取「絕對值最大」的係數(絕對值大代表該處有清晰的邊緣細節，即該張在此處對焦準確)→ 反轉換得到處處清晰的融合影像。這在顯微攝影、微距攝影、工業檢測(檢查有高低起伏的工件)中極為實用。

11.8.3 案例三：小波去雜訊對照第 7 章

情境：同一張含高斯雜訊的影像，分別用第 7 章的方法(高斯、中值、非局部平均、維納)與本章的小波門檻化去雜訊，比較成效。小波法的特色在於它對「邊緣的保留」通常較佳——因為它能在小波域中區分「邊緣的高頻(大係數，保留)」與「雜訊的高頻(小係數，歸零)」，而空間域的高斯模糊會無差別地抹平兩者。以 PSNR 與 SSIM 量化，並附上邊緣區域的局部放大，學生能清楚看到不同方法的取捨。這個案例把第 7 章與第 11 章的去雜訊工具箱完整地擺在一起比較。

11.8.4 案例四：壓縮率與品質的權衡實驗

情境：對同一張影像，分別以不同壓縮率存成 JPEG 與 JPEG 2000(或用 PyWavelets 模擬小波壓縮)，繪製「壓縮率對 PSNR/SSIM」的率失真曲線(rate-distortion curve)，並在高壓縮率處並排比較兩者的視覺瑕疵。學生會看到：低壓縮率時兩者差異不大，但當壓縮率推到極高(如 50:1),JPEG 碎裂成明顯的方塊，而小波僅是整體柔化——這正是叫獸開講中 FBI 捨 JPEG 就小波的量化證據。這個案例也是理解「率失真」這個壓縮領域核心概念的最佳實作。

11.9 本章與全書的觀念連結總表

相關章節	共用的觀念	本章的延伸或差異
第 3 章 數位影像基礎	位元深度、PSNR/SSIM、檔案格式	壓縮的品質以 PSNR/SSIM 量化;RAW 對照壓縮格式
第 5 章 空間域強化	平滑去雜訊	小波門檻化是另一種去雜訊途徑
第 6 章 頻率域	換基底、頻率、測不準	小波補上傳立葉缺的位置資訊(多解析度)
第 7 章 影像還原	去雜訊工具箱	小波門檻化與維納、NLM 互補，較能保邊緣
第 8 章 色彩處理	YCbCr、色度降取樣	JPEG 第一步即 YCbCr 轉換與 4:2:0 降取樣
第 9 章 影像分割	尺度空間	多解析度分析與 LoG/DoG 的尺度思想相通
第 13 章 特徵擷取	SIFT 尺度空間	SIFT 的多尺度金字塔與小波多解析度同源
第 15 章 深度學習	多層降取樣、多尺度特徵	CNN 的層級降取樣與小波多解析度精神相通

表11-2 第11章與全書其他章節的觀念連結

程式實作 Lab 11

Lab 11-1 手算 Haar 小波

目標：用最簡單的方式理解小波。(a)不用任何函式庫，對一維數列 [8, 4, 6, 10, 12, 8, 4, 2] 手算(或用 Python 純運算)一層 Haar 分解，得到 4 個概略值與 4 個細節值，並驗證能無失真還原;(b)繼續對概略序列做第二、三層分解，觀察多解析度結構;(c)把最小的細節係數歸零後還原，計算與原數列的誤差——體會「丟棄小係數」的壓縮思想;(d)用 PyWavelets 的 pywt.wavedec 驗證你的手算結果。

Lab 11-2 影像的小波金字塔

目標：視覺化二維小波分解。(a)以程式 11-2 對一張影像(建議用指紋圖或紋理豐富的圖)做單層分解，把 LL、LH、HL、HH 四個子帶拼成一張圖顯示，說明每個子帶凸顯了什麼方向的結構;(b)做三層分解，顯示金字塔結構，觀察能量如何集中在頂層 LL;(c)統計各子帶中「絕對值小於某門檻」的係數比例，驗證細節子帶的稀疏性;(d)只保留最大的 5%、10%、25% 係數，其餘歸零後重建，計算各自的 PSNR——這就是小波壓縮的核心實驗。

Lab 11-3 JPEG 方塊效應解剖

目標：親眼見證叫獸開講的方塊效應。(a)以程式 11-3 對一張影像用品質 90、50、30、10 分別存成 JPEG，計算各自的 PSNR 與檔案大小，繪製率失真曲線;(b)把品質 10 的結果放大到像素級，清楚標出 8×8 方塊邊界與蚊狀雜訊;(c)手動實作單一 8×8 區塊的 DCT 與量化，統計量化後非零係數的數量，理解壓縮如何發生;(d)找一張含銳利邊緣(如文字)的影像，觀察高壓縮率下邊緣附近的蚊狀雜訊——說明它與第 6 章振鈴效應的關聯。

Lab 11-4 小波去雜訊擂台

目標：把第 7 章與第 11 章的去雜訊方法同場競技。(a)對一張乾淨影像加入已知強度的高斯雜訊;(b)分別用高斯模糊、中值、非局部平均(第 7 章)與小波門檻化(本章)去雜訊;(c)以 PSNR 與 SSIM 製作比較表，並附上「邊緣區域局部放大」的對照圖;(d)特別觀察：小波法在「邊緣銳利度」與「平坦區乾淨度」的平衡上表現如何?為什麼它能同時做到(提示：在小波域區分邊緣高頻與雜訊高頻)?(e)討論：什麼情況下你會選小波去雜訊，什麼情況下選非局部平均?

本章重點整理

- 正交轉換的共同思想是「換一組基底表示影像」，使能量集中於少數係數(稀疏)，而稀疏是壓縮的黃金鑰匙;正交性帶來可逆、能量守恆、係數互不干擾三大好處。
- 傅立葉的根本弱點是「有頻率、無位置」(全域基底);短時傅立葉以固定窗口取得局部性但受測不準限制;小波以縮放與平移實現「低頻用長窗、高頻用短窗」的自適應多解析度分析，同時掌握頻率與位置。
- Haar 小波以「相鄰值的平均與差分」做分解，可手算;二維小波一層產生 LL(縮略圖)、LH、HL、HH 四個子帶，多層時只續分 LL，形成能量集中於頂層的金字塔。
- DCT 只用餘弦基底、係數為實數、能量集中;JPEG 五步驟為 YCbCr 降取樣、 8×8 分塊 DCT、量化(唯一失真來源)、zigzag 與遊程編碼、熵編碼;反覆存檔會累積量化失真。
- 小波壓縮(JPEG 2000、WSQ)作用於整張影像不切塊，從根本避免方塊效應，並天然支援漸進式與多解析度存取;但 JPEG 因夠好、夠快、夠普及而仍是日常主流。

- 小波應用：門檻化去雜訊(能區分邊緣高頻與雜訊高頻，較保邊緣)、多焦點/多模態影像融合、中頻域數位浮水印;其「多解析度」思想延續到 SIFT 尺度空間與 CNN 的層級降取樣。

習題

1. 正交轉換的共同思想是什麼?為什麼「換基底使能量稀疏」有利於壓縮?自然影像在空間域是否稀疏?

提示與參考解答：共同思想是把影像從一組基底轉換到另一組基底表示。換到好的基底(頻率域、小波域)後能量集中於少數係數(稀疏)，只需記住少數重要係數、丟棄大量近零係數即可重建，故利於壓縮。自然影像在空間域不稀疏(每個像素都攜帶資訊)，但轉換後變稀疏——這正是轉換的價值。

2. 「正交」是什麼意思?它為轉換帶來哪三個好處?

提示與參考解答：正交指基底函數彼此內積為零、互相獨立不重疊。三個好處：轉換可逆(能無失真變回原影像)、能量守恆(Parseval 定理，轉換前後總能量不變，可判斷丟棄係數的能量損失)、係數互不干擾(改一個不影響其他)。

3. 傅立葉轉換的根本弱點是什麼?用一個生活化的比喻說明。短時傅立葉如何補救，又受什麼限制?

提示與參考解答：弱點是「有頻率、無位置」：基底是無限延伸的弦波，只知道有哪些頻率、不知道頻率來自哪個位置。比喻：分析樂曲只能列出用到哪些音符，卻不知先後順序。短時傅立葉把影像切塊各自轉換以取得位置，但受測不準限制：窗口大則頻率準位置模糊，窗口小則位置準頻率模糊，無法兼得。

4. 小波如何同時掌握頻率與位置?請說明「低頻用長窗、高頻用短窗」背後的觀察。

提示與參考解答：小波用有限長度的小波形為基底，透過縮放(對應頻率)與平移(對應位置)分析。背後觀察：低頻成分變化緩慢，需長窗才看得清(換取頻率精度);高頻成分變化劇烈短暫，需短窗才能定位(換取位置精度)。小波自動實現這種自適應，即多解析度分析。

5. 對數對 [8, 4] 做一層 Haar 小波分解，寫出概略值與細節值，並驗證還原。

提示與參考解答：概略值(平均) = $(8+4)/2 = 6$; 細節值(差的一半) = $(8-4)/2 = 2$ 。表示為 [6, 2]。還原: $8 = 6+2$ 、 $4 = 6-2$ ，無失真。

6. 二維小波一層分解產生哪四個子帶?各凸顯什麼?多層分解時只對哪個子帶繼續分解，為什麼?

提示與參考解答：LL(縮略圖，集中能量)、LH(水平邊緣)、HL(垂直邊緣)、HH(對角細節與雜訊)。多層時只續分 LL，因為能量集中在 LL，其餘子帶已稀疏;續分 LL 可得到由粗到細的金字塔，大部分能量集中在最頂層小 LL，其餘近零——這就是壓縮的機會。

7. 請完整列出 JPEG 壓縮的五個步驟，並指出哪一步是唯一的失真來源、為什麼反覆存檔 JPEG 會愈存愈糊。

提示與參考解答：一、YCbCr 轉換與色度 4:2:0 降取樣;二、 8×8 分塊做 DCT;三、量化(唯一失真來源);四、zigzag 掃描與遊程編碼;五、Huffman 熵編碼。量化把係數除以量化步長取整，不可逆。每次存檔都量化一次，失真累積，故愈存愈糊。

8. JPEG 的方塊效應從何而來?小波壓縮(JPEG 2000/WSQ)為什麼能避免它?

提示與參考解答：方塊效應源於JPEG 把影像切成獨立的 8×8 方塊各自DCT 與量化，高壓縮率下相鄰方塊的量化差異在邊界顯現。小波轉換作用於整張影像(不切塊)，不存在方塊邊界，高壓縮率下只是整體柔化模糊而非碎裂，故避免方塊效應。

9. 承叫獸開講：FBI 為什麼不用 JPEG 而選擇小波(WSQ)來壓縮指紋？這對「技術選型」有何啟示？

提示與參考解答：指紋辨識靠脊線的細微特徵點(分岔、端點)，JPEG 的方塊效應與高頻流失會破壞或偽造這些鑑識關鍵特徵，法律上不可接受。小波無方塊、能保脊線細節，在20:1 壓縮下無感知損失。啟示：技術選型須看應用的核心需求——一般照片能容忍方塊，鑑識則不能；沒有最好的壓縮，只有最適合任務的壓縮。

10. 小波門檻化如何去雜訊？為什麼它在保留邊緣方面常優於空間域的高斯模糊？

提示與參考解答：自然影像的重要結構在小波域集中成少數大係數，雜訊散布成大量小係數；把小於門檻的小係數歸零(視為雜訊)、保留大係數(視為訊號)，再反轉換。它在「同時掌握頻率與位置」的小波域操作，能區分「邊緣附近的高頻(保留)」與「平坦區的高頻(雜訊，去除)」；高斯模糊則無差別抹平所有高頻，故傷邊緣。

11. 為什麼技術更優的 JPEG 2000 至今仍未取代 JPEG 成為日常主流？這反映了什麼工程通則？

提示與參考解答：JPEG 運算簡單、硬體支援完善、相容性無敵、「夠好夠快夠普及」；JPEG 2000 複雜度較高、相容性較差，主要用於醫療、電影母帶、指紋等高要求專業領域。通則：最好的技術不一定勝出，「足夠好加上成熟生態系」往往才是贏家(與第9、10 章「適合勝於最優」呼應)。

12. (綜合題)小波的「多解析度分析」思想，如何呼應第9 章的尺度空間、第13 章的 SIFT 與第15 章的 CNN？

提示與參考解答：多解析度=由粗到細、多尺度分析。第9 章LoG/DoG 在不同尺度找邊緣；第13 章SIFT 建高斯尺度金字塔在多尺度找特徵點；第15 章CNN 透過層級降取樣，每層看到更粗尺度的特徵。它們與小波「不同尺度各司其職」的精神同源——多尺度思維貫穿全書，是現代電腦視覺的核心觀念之一。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6-7、8(小波、多解析度與影像壓縮)。
- Bradley, J. N., Brislawn, C. M., & Hopper, T., The FBI Wavelet/Scalar Quantization Standard for Gray-Scale Fingerprint Image Compression, 1993.(本章叫獸開講之史料來源)
- Daubechies, I., Ten Lectures on Wavelets, SIAM, 1992.(小波理論經典)
- Mallat, S., A Theory for Multiresolution Signal Decomposition: The Wavelet Representation, IEEE TPAMI, 1989.
- Wallace, G. K., The JPEG Still Picture Compression Standard, Comm. ACM, 1991.
- Taubman, D., & Marcellin, M., JPEG2000: Image Compression Fundamentals, Standards and Practice, Springer, 2002.
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 12 章 視訊與動態影像處理

本章學習目標

- 說明視訊的本質(連續影格)與時間維度為影像處理帶來的新機會與新挑戰。
- 以 OpenCV 讀取、處理、寫出視訊與即時攝影機串流，理解影格率與編解碼器。
- 運用影格差分與背景相減完成移動偵測，並以型態學清理前景遮罩。
- 闡述光流的概念，區分稀疏光流(Lucas-Kanade)與稠密光流(Farneback)。
- 實作物件追蹤，區分「偵測」與「追蹤」的差異，並串接第 8、9、10、16 章。
- 認識視訊穩定、動作辨識與時序深度學習，理解時間維度在機器人視覺中的角色。

叫獸開講|一場關於奔馬的爭論，如何意外催生了電影

故事要從一個看似無聊的問題說起：一匹馬全速奔跑時，會不會有那麼一瞬間，四隻蹄子同時離地？在 1870 年代，這個問題無法回答——馬腿動得太快，人眼根本看不清。傳統繪畫總把奔馬畫成「飛躍式」：四腿向前後極力伸展、如同騰空飛翔。但這真的是馬奔跑的樣子嗎？沒有人能證明。

加州的鐵路大亨、前州長 Leland Stanford(後來史丹佛大學的創辦人)對這個問題著了迷。1872 年，他請來英國攝影師 Eadweard Muybridge 來解決它。流傳最廣的版本說，Stanford 為此下了兩萬五千美元的賭注——不過要誠實說，不少歷史學家認為這個「賭局」缺乏一手史料佐證，很可能是後人加油添醋的傳說。無論賭注是真是假，可以確定的是：這是一項需要當時最頂尖攝影技術的艱鉅挑戰，而 Stanford 願意持續投入重金。

經過六年的技術摸索(期間 Muybridge 還一度因槍殺妻子的情夫而中斷工作、受審)，1878 年 6 月，他終於在史丹佛的帕羅奧圖農場架設起一整排相機——沿著跑道平行排列，相機的快門連接絆線，當名為「Sallie Gardner」的母馬奔馳而過、依序碰斷絆線，便觸發一台台相機，以大約二十五分之一秒的間隔連續拍下一整組動作。照片沖洗出來，答案揭曉：確實有一個瞬間，馬的四蹄全部離地——但不是傳統繪畫想像的伸展飛躍姿，而是四腿收攏於腹下的那一刻。從此，「飛躍式奔馬」的錯誤畫法在藝術界銷聲匿跡。

但這個故事真正的遺產遠不止於此。當 Muybridge 把這一系列靜態照片依序快速播放，觀眾看到的竟是一匹「活生生奔跑的馬」——史上第一次，連續的靜態影像創造出了「動態」的錯覺。這組名為《奔跑的馬》(Sallie Gardner at a Gallop)的照片，被公認為電影的源頭。一場關於馬蹄的爭論，意外開啟了整個電影工業。而它揭示的原理，正是本章的基石：視訊，本質上就是一疊「快速翻頁的照片」；而所謂的「動態」，是我們的視覺系統把一連串靜態影格腦補連接起來的結果。

這也帶出了視訊處理的核心洞見。前十一章，我們把每一張影像當成孤立的個體來處理；但視訊多了一個維度——時間。相鄰的影格之間有著極強的關聯（這一影格和上一影格通常只差一點點），這個「時間上的連續性」既是新的資訊來源（可以偵測移動、追蹤物件、估計速度），也是新的挑戰（要即時處理、要在影格與影格之間維持一致）。學會利用時間這個維度，是本章的目標，也是機器人「看懂動態世界」的關鍵。

思考題：Muybridge 用「一排相機依序觸發」來把時間切成一格一格。今天的手機用單一感光元件、以固定影格率連續讀取來達成同樣的事。想一想：如果影格率不夠高（取樣時間的間隔太大），拍攝高速旋轉的電風扇或車輪，會看到什麼奇怪的現象？這與第 3、6 章的哪個概念有關？

12.1 視訊的本質：多了一個時間維度

12.1.1 從影像到視訊

一張影像是二維的（寬、高），而視訊多了第三個維度——時間。視訊是由一連串影格（frame）以固定的時間間隔排列而成，每一個影格都是一張完整的影像。播放的速度稱為影格率（frame rate），單位是每秒影格數（fps）：電影通常是 24 fps，電視是 25 或 30 fps，而流暢的遊戲與高速攝影可達 60 fps 以上。當影格切換得夠快（一般認為超過每秒十餘格），人類的視覺暫留就會把離散的影格「腦補」成連續的動態——這正是叫獸開講中 Muybridge 的發現，也是所有動態影像的生理基礎。

這個「時間軸」的加入，徹底改變了影像處理的格局。它同時帶來了機會與挑戰，理解這個雙面性，是掌握本章的關鍵。

12.1.2 時間維度帶來的機會

相鄰影格之間有極強的時間關聯性——這一影格和上一影格，通常絕大部分內容都相同，只有移動的物件或變化的區域不同。這個「幾乎不變」的特性，是一座資訊金礦：

- 偵測移動：把兩影格相減，不變的背景相消為零，只剩下移動的物件浮現（12.3 節）。單張影像永遠做不到這件事——「移動」本身就是一個跨越時間的概念。
- 追蹤物件：知道一個物件在這一影格的位置，由於下一影格它不會跳太遠，可以有效率地在附近找到它、串起它的軌跡（12.5 節）。
- 估計運動與速度：量測物件在連續影格間的位移，除以時間，就得到速度——這是機器人閃避障礙、預測行人動向的基礎（12.4 節光流）。
- 提升品質：多影格之間的冗餘可用於去雜訊（第 5 章夜景模式的多影格平均）、超解析度、視訊穩定。

12.1.3 時間維度帶來的挑戰

但時間維度也帶來三個新挑戰：其一是即時性——視訊以每秒數十影格的速度湧入，處理演算法必須跟得上，否則就會掉影格或延遲，這對第 17 章的邊緣部署是嚴

苛的考驗(一個在單張影像上跑得很好、但每一影格要一秒的演算法，對 30 fps 的視訊毫無用處)。其二是時間一致性——處理結果在影格與影格之間必須穩定，若每一影格獨立處理，偵測框可能會抖動、閃爍，追蹤編號可能會跳號。其三是資料量——視訊的資料量是影像的數十倍，儲存與傳輸都仰賴第 11 章的壓縮技術(視訊編碼更進一步利用了「影格間冗餘」，只記錄影格與影格之間的差異)。

12.2 視訊的讀取、處理與寫出

OpenCV 以 VideoCapture 讀取視訊(檔案或即時攝影機)，以 VideoWriter 寫出。核心觀念很簡單：視訊處理就是一個迴圈——逐影格讀取、把每一影格當成一張影像套用前面十一章的任何技術、再逐影格輸出或顯示。

程式 12-1 視訊讀取、處理與寫出的基本框架

```
import cv2

# 開啟視訊來源:檔名 → 讀檔;數字 0 → 預設攝影機;網址 → 串流
cap = cv2.VideoCapture("traffic.mp4") # 或 0,或 ESP32-CAM 串流網址

# 查詢視訊屬性
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"影格率={fps}, 尺寸={width}x{height}")

# 準備寫出(fourcc 指定編解碼器)
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("output.mp4", fourcc, fps, (width, height))

while True:
    ok, frame = cap.read() # 逐影格讀取
    if not ok: # 讀完或讀取失敗
        break

    # === 把每一影格當成一張影像:套用前 11 章的任何技術 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 80, 160) # 第 9 章
    result = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

    out.write(result) # 寫出這一影格
    cv2.imshow("video", result)
    if cv2.waitKey(1) == 27: # ESC 離開;waitKey 值影響播放速度
        break

cap.release() # 務必釋放資源
out.release()
cv2.destroyAllWindows()
```

幾個實務要點：第一，fourcc 是編解碼器的四字元代碼(如 mp4v、XVID、H264)，它決定輸出檔的壓縮方式，選錯會導致無法寫出或檔案過大——這直接關聯第 11 章的壓縮。第二，cap.read() 回傳的成功旗標一定要檢查，它

是判斷「視訊是否讀完」的依據。第三，`cv2.waitKey(n)` 的 `n` 值(毫秒)控制每一影格之間的等待，影響顯示的播放速度；處理即時攝影機時通常設 1。第四，務必呼叫 `release()` 釋放資源，否則攝影機或檔案可能被鎖住。

12.3 移動偵測：影格差分與背景相減

移動偵測是視訊處理最基礎也最實用的任務，它完美體現了「時間維度是資訊金礦」的觀念。核心思想只有一句：不變的是背景，變化的是前景。

12.3.1 影格差分

最簡單的方法是影格差分(Frame Differencing)：把相鄰兩影格相減，取絕對值。靜止的背景在兩影格中相同，相減後為零(黑)；移動的物件在兩影格中位置不同，相減後留下非零的差異(亮)。再對差值做門檻化(第 9 章)，就得到移動區域的遮罩。它的優點是極簡、極快、能自動適應緩慢的光線變化；缺點是：只能偵測到「正在移動」的部分，物件若暫停就消失；且移動物件的內部(顏色均勻處)前後影格相同，會被誤判為靜止，只留下物件的「邊緣輪廓」——這稱為「鬼影」或空心問題。

12.3.2 背景相減

更強大的方法是背景相減(Background Subtraction)：先建立一個「純背景」的模型(沒有任何移動物件時的畫面)，再把每一影格與這個背景模型相減，差異顯著處就是前景。相較影格差分，它能偵測到完整的物件(而非只有邊緣)，且暫停的物件也不會消失。

關鍵難點在於「背景模型」如何建立與維護——真實場景的背景並非一成不變：光線會漸變(日照移動)、樹葉會搖動、水面會波動、攝影機會輕微震動。因此優秀的背景相減器必須是自適應的：它持續、緩慢地更新背景模型，把「長期穩定存在」的東西學習為背景，把「突然出現、快速移動」的東西判定為前景。OpenCV 提供的 MOG2(高斯混合模型)與 KNN 背景相減器就是這類自適應方法，它們為每個像素維護一個統計模型，能應付上述干擾，還能偵測並標記陰影。

程式 12-2 影格差分與背景相減移動偵測

```
import cv2, numpy as np

cap = cv2.VideoCapture("surveillance.mp4")

# --- 方法一：影格差分 ---
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 方法二：自適應背景相減器(實務首選) ---
backsub = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=16, detectShadows=True)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

while True:
    ok, frame = cap.read()
    if not ok:
```



```

        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 影格差分
    diff = cv2.absdiff(prev_gray, gray)
    _, motion = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
    prev_gray = gray

    # 背景相減 → 前景遮罩
    fgmask = backsub.apply(frame)

    # 型態學清理(第 10 章):去雜點、補破洞
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel)
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_CLOSE, kernel)

    # 連通元件框出移動物件(第 10 章)
    cnts, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)

    for c in cnts:
        if cv2.contourArea(c) > 500:          # 面積門檻濾雜訊
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imshow("motion", frame)
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()

```

這段程式是本書多章的綜合：背景相減得到前景 → 第 10 章的型態學清理雜點與破洞 → 第 10 章的連通元件與第 9 章的輪廓框出物件。它是智慧監控、人流計數、交通流量分析、入侵偵測的骨幹，也正是第 9.7.4 節「視訊中的移動物件分割」的完整實作。

12.4 光流：估計像素的運動

移動偵測只回答「哪裡有東西在動」，光流(Optical Flow)則更進一步，回答「每個點往哪個方向、以多快的速度移動」——它為影像中的點估計出運動向量。這是機器人理解動態世界的核心工具：知道了場景中每個點的運動，就能推算自身的移動、偵測逼近的障礙、預測物件的軌跡。

12.4.1 基本假設與孔徑問題

光流建立在一個核心假設上：亮度恆定——同一個物體點在相鄰兩影格之間，亮度不變，只是位置移動了。由這個假設可推導出光流方程式。但這裡有一個著名的困境：孔徑問題(Aperture Problem)。透過一個小孔觀察一條移動的直線邊緣，你只能看出它「垂直於邊緣方向」的運動分量，而沿著邊緣方向的運動則無從察覺(就像只透過吸管看一根滑動的長棍，無法判斷它沿長軸滑了多少)。因此，單看一個像素的局部資訊，無法唯一決定它的運動——必須引入額外的假設(例如鄰近像素運動相似)才能求解。

12.4.2 稀疏光流：Lucas-Kanade

Lucas-Kanade 法計算稀疏光流：它不追蹤每一個像素，而是只追蹤少數「適合追蹤的特徵點」（通常用第 13 章的 Shi-Tomasi 角點——因為角點在兩個方向都有明顯變化，能繞過孔徑問題）。它假設一個小鄰域內的所有像素運動相同，藉此求解每個特徵點的運動向量。它高效、精準，適合追蹤特定的關鍵點，是視訊穩定、運動估計、視覺里程計(第 17 章機器人定位)的常用工具。

12.4.3 稠密光流：Farneback

稠密光流(如 Farneback 法)則為影像中的每一個像素都計算運動向量，得到一張完整的「運動場」。它資訊更豐富(能看出整個場景的運動結構)，但運算量遠大於稀疏光流。常用第 8 章的虛擬色彩把運動場視覺化：用色相(H)表示運動方向、飽和度或明度表示運動速度，於是整個畫面的運動一目了然。稠密光流用於動作辨識、視訊分割、慢動作生成(影格內插)等需要全域運動資訊的任務。

程式 12-3 稀疏與稠密光流

```
import cv2, numpy as np

cap = cv2.VideoCapture("walking.mp4")
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 稀疏光流:先找適合追蹤的角點(第 13 章 Shi-Tomasi) ---
pts = cv2.goodFeaturesToTrack(prev_gray, maxCorners=100,
                              qualityLevel=0.3, minDistance=7)

while True:
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Lucas-Kanade 稀疏光流:追蹤這些點到新位置
    new_pts, status, err = cv2.calcOpticalFlowPyrLK(
        prev_gray, gray, pts, None)
    for (nx, ny), (ox, oy) in zip(new_pts[status==1],
                                  pts[status==1]):
        cv2.line(frame, (int(nx),int(ny)), (int(ox),int(oy)),
                  (0,255,0), 2)
        cv2.circle(frame, (int(nx),int(ny)), 3, (0,0,255), -1)

    # --- 稠密光流:每個像素的運動,用色彩視覺化 ---
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
                                         0.5, 3, 15, 3, 5, 1.2, 0)
    mag, ang = cv2.cartToPolar(flow[...,0], flow[...,1])
    hsv = np.zeros_like(frame)
    hsv[...,0] = ang * 180 / np.pi / 2      # 方向 → 色相(第 8 章)
    hsv[...,1] = 255
    hsv[...,2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    flow_vis = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
```

```

cv2.imshow("sparse", frame)
cv2.imshow("dense", flow_vis)
prev_gray = gray
pts = new_pts[status==1].reshape(-1,1,2)
if cv2.waitKey(30) == 27:
    break

cap.release(); cv2.destroyAllWindows()

```

12.5 物件追蹤

12.5.1 偵測與追蹤的差別

這是視訊分析中一組容易混淆卻至關重要的概念。偵測(Detection)是在單一影格中「找出物件在哪裡」，每一影格獨立進行，彼此無關;追蹤(Tracking)則是在連續影格中「維持同一個物件的身分」，把它在時間軸上的位置串成一條軌跡，並給它一個穩定的編號(ID)。

兩者為什麼要分工?因為各有優劣。偵測(尤其是第 16 章的 YOLO)準確但通常較耗算力，且它不知道「這一影格的人」和「上一影格的人」是不是同一個。追蹤則利用時間連續性，運算快(只需在上一影格位置附近搜尋)，還能維持身分——但它可能會「跟丟」(物件被遮擋、快速移動時)或「跟錯」(兩個物件交錯時身分互換)。

實務上最強健的方案是兩者結合，稱為「偵測式追蹤」(Tracking-by-Detection)：每隔幾影格用偵測器(YOLO)重新定位所有物件(校正累積誤差、發現新物件)，中間的影格則用輕量的追蹤演算法快速更新位置並維持 ID。著名的 SORT、DeepSORT 演算法就是這個框架，廣泛用於多物件追蹤(MOT)——這也是第 16 章 YOLO 追蹤功能的底層邏輯。

12.5.2 追蹤演算法

傳統追蹤器種類繁多：MeanShift 與 CamShift 基於顏色直方圖(第 8 章)，追蹤顏色特徵明顯的物件;KCF、CSRT 等相關濾波器類追蹤器在速度與準確度間各有取捨(CSRT 較準、KCF 較快);卡爾曼濾波器(Kalman Filter)則從運動模型預測物件下一刻的位置，能在短暫遮擋時「猜測」物件的去向，是追蹤系統的經典組件。OpenCV 的 tracking 模組提供了這些追蹤器的現成實作。

程式 12-4 單物件追蹤

```

import cv2

cap = cv2.VideoCapture("target.mp4")
ok, frame = cap.read()

# 第一影格:框選要追蹤的物件(可手動或用第 16 章 YOLO 偵測)
bbox = cv2.selectROI("select", frame, False)

# 建立追蹤器(CSRT 較準,KCF 較快)
tracker = cv2.TrackerCSRT_create()

```

```

tracker.init(frame, bbox)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    success, bbox = tracker.update(frame)    # 在新影格中更新位置
    if success:
        x, y, w, h = [int(v) for v in bbox]
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    else:
        cv2.putText(frame, "Lost!", (20, 40),
                     cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    cv2.imshow("tracking", frame)
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()

```

12.6 進階主題：穩定、動作辨識與時序深度學習

12.6.1 視訊穩定

手持或車載攝影機的畫面總是晃動的。視訊穩定(Video Stabilization)的原理是：估計相鄰影格之間因抖動造成的整體運動(用第 6.7.2 節的相位相關法，或第 12.4 節的光流，或第 13 章的特徵匹配)，再反向補償這個運動(用第 4 章的幾何轉換把每一影格「拉回」穩定位置)，最後裁掉補償後露出的黑邊。這串起了第 4、6、12、13 章——是本書多項技術的綜合應用。

12.6.2 動作辨識

動作辨識(Action Recognition)要回答「影片中的人在做什麼」(走路、揮手、跌倒)。它與單張影像分類的根本差別在於：動作是時間性的——單看一張「手舉起來」的照片，無法分辨是在打招呼還是要打人，必須看時間序列。傳統方法結合外觀特徵與光流(運動資訊)；深度學習方法則用能處理時間維度的網路(如 3D 卷積、雙流網路、或後來的時序 Transformer)。應用包括：照護場景的跌倒偵測、運動分析、人機協作的手勢指令、異常行為監控。

12.6.3 時序深度學習與時間一致性

進入深度學習，處理時間維度成為一個核心課題。早期用循環神經網路(RNN/LSTM)處理影格序列；3D CNN 把卷積從二維空間擴展到「空間加時間」的三維；近年的視訊 Transformer 則以注意力機制捕捉長距離的時間關聯。這些方法讓機器不只看懂單一影格，更能理解「事件的發展」。

但無論用什麼方法，視訊深度學習都要面對 12.1.3 節提過的時間一致性挑戰：若對每一影格獨立套用模型，結果會閃爍抖動(這一影格偵測到、下一影格又消失)。解

決之道包括：在時間軸上做平滑、用追蹤維持身分、或設計本身就考慮時間關聯的模型。這也是機器人視覺與靜態影像辨識最大的不同——機器人活在連續的時間裡，它需要的不是一張張孤立的判斷，而是對「正在發生什麼」的連貫理解。

12.7 應用案例

12.7.1 案例一：智慧監控與人流計數

情境：在店門口或走廊上方架設攝影機，自動統計進出的人數。流程：背景相減得到移動前景(12.3 節)→ 型態學清理、連通元件框出每個人(第 10 章)→ 用追蹤器維持每個人的 ID(12.5 節)→ 在畫面中畫一條虛擬計數線，當某人的軌跡越過此線，依方向計數(進或出)。關鍵在於追蹤——若只靠偵測，同一個人可能被重複計數；有了穩定的 ID，才能確保「一個人只算一次」。這是零售分析、場館人流管理、防疫人數管制的實用系統。

12.7.2 案例二：車流量與車速偵測

情境：道路監控攝影機自動統計車流量與估計車速。流程：背景相減或第 16 章 YOLO 偵測車輛 → 追蹤每輛車的軌跡 → 由軌跡在連續影格的位移，結合第 4 章相機校正得到的「像素對應實際距離」的比例，以及已知的影格率(位移除以時間即速度)，換算出每輛車的實際速度。這串起了第 4 章(相機校正把像素換算成公尺)、第 12 章(追蹤取得位移)、以及影格率(提供時間基準)——是一個把「像素運動」轉換成「物理速度」的漂亮範例，也是智慧交通、區間測速的基礎。

12.7.3 案例三：照護場景的跌倒偵測

情境：在長者居家或病房中，自動偵測跌倒並發出警示。這是動作辨識(12.6.2 節)的重要應用，也凸顯了「時間維度不可或缺」——單看一張「人躺在地上」的照片，無法分辨是跌倒還是正常躺下休息；必須看時間序列：一個「快速的、由站立到倒地的垂直運動」才是跌倒的特徵。傳統做法可結合人體姿態(第 16 章 YOLO Pose)的重心高度變化與運動速度來判斷；深度學習則用時序模型直接學習跌倒的動作模式。此案例同時觸及第 16 章的姿態估計，展示了視訊、姿態、時序三者的整合。

這裡也值得提醒一個貫穿全書的價值：照護應用直接關係人的安危與尊嚴，系統的誤報(把正常動作誤判為跌倒，頻繁打擾)與漏報(真跌倒卻沒偵測到)都有嚴重後果，且涉及被監控者的隱私。這類系統的設計，技術之外更需要對使用者處境的體貼——這是工程倫理的一環。

12.7.4 案例四：機器人的視覺里程計

情境：一台自主移動機器人(或無人機)需要知道自己移動了多少、轉了多少，即使沒有 GPS。視覺里程計(Visual Odometry)的核心正是本章的光流與追蹤：機器人前進時，攝影機看到的靜態場景會「反向流動」；透過追蹤場景中的特徵點(第 13 章)、分析它們在連續影格間的運動(12.4 節光流)，就能反推出攝影機自身的運動——這是機器人「自我定位」的視覺基礎，也是第 17 章機器人視覺整合的重要前置技術。這個案例揭示了一個深刻的觀點：同樣是「場景中的運動」，換個角度看，就成了「觀

察者自身的運動」——運動是相對的，而機器人正是利用這個相對性來認識自己在世界中的位置。

12.8 本章與全書的觀念連結總表

相關章節	共用的觀念	視訊處理中的角色
第 3 章 數位影像基礎	取樣定理	影格率不足造成時間混疊(車輪倒轉錯覺)
第 4 章 幾何轉換	仿射/透視轉換、相機校正	視訊穩定的運動補償;像素換算實際速度
第 5 章 空間域強化	多影格平均	夜景模式即時間維度的去雜訊
第 6 章 頻率域	相位相關法	視訊穩定的影格間運動估計
第 8 章 色彩處理	HSV、顏色追蹤、虛擬色彩	稠密光流視覺化;CamShift 顏色追蹤
第 9、10 章 分割/二值	門檻、型態學、連通元件	前景遮罩的清理與物件框選
第 11 章 小波	壓縮	視訊編碼進一步利用影格間冗餘
第 13 章 特徵擷取	角點、特徵匹配	Shi-Tomasi 角點供稀疏光流追蹤
第 16 章 物件偵測	YOLO、姿態估計	偵測式追蹤;跌倒偵測結合姿態與時序
第 17 章 邊緣部署	即時性、視覺里程計	視訊處理的算力預算;機器人自我定位

表 12-1 第 12 章與全書其他章節的觀念連結

程式實作 Lab 12

Lab 12-1 視訊處理管線

目標：熟悉視訊讀寫框架。(a)以程式 12-1 為基礎，讀取一段視訊或即時攝影機，對每一影格套用前面各章的至少三種技術(如 CLAHE、Canny、HSV 顏色分割)，並排顯示原始與處理結果;(b)在畫面上即時顯示影格率(用 `cv2.getTickCount` 計時)，觀察不同處理的耗時;(c)把結果寫成輸出檔，嘗試不同的 `fourcc` 編解碼器，比較輸出檔的大小與品質;(d)討論題：哪一種處理最耗時?若要在 30 fps 即時執行，每一影格的處理時間預算是多少毫秒?

Lab 12-2 移動偵測監控系統

目標：實作 12.7.1 節的監控系統。(a)以程式 12-2 對一段監控影片做背景相減，比較「影格差分」與「MOG2 背景相減」在偵測完整物件、應對暫停物件、抵抗光線

變化上的差異;(b)加入型態學清理與面積門檻，穩定地框出移動物件;(c)畫一條虛擬計數線，實作越線計數(需簡單追蹤以避免重複計數);(d)測試干擾：讓畫面中有搖動的樹葉或漸變的光線，觀察 MOG2 如何自適應;(e)挑戰題：加入陰影偵測(detectShadows)，避免把物體的影子也框進去。

Lab 12-3 光流視覺化

目標：直觀理解光流。(a)以程式 12-3 對一段有明顯運動的影片(如行走的人、車流)，同時顯示稀疏光流(特徵點的運動軌跡)與稠密光流(色彩運動場);(b)解讀稠密光流的色彩圖：不同顏色代表什麼方向?為什麼靜止的背景是黑色?(c)拍攝或找一段「攝影機自己在移動」的影片(如手持前進)，觀察整個畫面的光流呈現什麼模式——這與 12.7.4 節的視覺里程計有何關聯?(d)驗證孔徑問題：對著一條緩慢移動的直線邊緣，觀察光流估計是否只捕捉到垂直於邊緣的分量。

Lab 12-4 物件追蹤與偵測式追蹤

目標：體會偵測與追蹤的分工。(a)以程式 12-4 用 CSRT 追蹤器追蹤一個手選的物件，測試它在物件被短暫遮擋、快速移動、與其他物件交錯時是否跟丟或跟錯;(b)比較 KCF 與 CSRT 在速度與準確度上的差異(量測 FPS);(c)概念實作偵測式追蹤：每隔 N 影格「重新框選」(模擬偵測器校正)，中間影格用追蹤器更新，觀察這樣是否比純追蹤更穩健;(d)討論題：承第 16 章，若把手動框選換成 YOLO 自動偵測，並對多個物件各自維持 ID，你會如何設計?(這正是 DeepSORT 的思路)

本章重點整理

- 視訊是以固定影格率排列的連續影格，多了「時間」維度;視訊處理的基本框架是逐影格讀取、當成影像處理、逐影格輸出的迴圈。時間維度帶來偵測移動、追蹤、估計速度、多影格提質的機會，也帶來即時性、時間一致性、資料量三大挑戰。
- 移動偵測基於「背景不變、前景變化」：影格差分極簡但只偵測到移動邊緣、暫停物件會消失;背景相減能偵測完整物件，MOG2/KNN 等自適應方法可應對光線漸變與搖動干擾。
- 光流估計每個點的運動向量，基於亮度恆定假設，受孔徑問題限制;Lucas-Kanade 為稀疏光流(追蹤角點),Farnebäck 為稠密光流(每像素運動場，常以虛擬色彩視覺化)。
- 偵測是單一影格找物件、追蹤是跨影格維持身分;偵測式追蹤(SORT/DeepSORT)結合兩者——定期偵測校正、中間追蹤維持 ID;傳統追蹤器有 MeanShift/CamShift(顏色)、KCF/CSRT(相關濾波)、卡爾曼濾波(運動預測)。
- 視訊穩定綜合光流/特徵匹配估計抖動、以幾何轉換反向補償;動作辨識需時間序列(單張影像無法分辨動作);時序深度學習(3D CNN、視訊 Transformer)須處理時間一致性以免結果閃爍。
- 視訊處理串起全書：第 3 章取樣(時間混疊)、第 4 章幾何(穩定與速度換算)、第 8 章色彩(光流視覺化)、第 9/10 章(前景清理)、第 13 章(角點追蹤)、第 16 章(偵測式追蹤、跌倒偵測)、第 17 章(視覺里程計)。

習題

1. 視訊相較於單張影像，多了什麼維度？這個維度帶來哪些機會與哪些挑戰？

提示與參考解答：多了「時間」維度。機會：相鄰影格高度關聯，可偵測移動(影格相減)、追蹤物件、估計速度、多影格提質去雜訊。挑戰：即時性(須跟上每秒數十影格)、時間一致性(結果不可影格間抖動閃爍)、資料量(數十倍於影像，仰賴壓縮與影格間編碼)。

2. 承叫獸開講的思考題：用影格率不足的相機拍攝高速旋轉的車輪，會看到車輪倒轉或靜止的錯覺，為什麼？這與第 3、6 章的什麼概念相關？

提示與參考解答：這是時間維度的混疊(時間 aliasing)。車輪旋轉是一個高頻的週期運動，影格率(取樣頻率)不足以滿足 Nyquist 定理時，取樣點恰好錯過真實運動，大腦把離散樣本腦補成較低頻的運動(倒轉或靜止)。這與第 3 章取樣定理、第 6 章混疊(高頻摺疊成低頻)同源，只是發生在時間軸上。

3. 視訊處理的基本框架是什麼？請說明 VideoCapture、逐影格迴圈、VideoWriter 三者的角色，以及 fourcc 與 release 的重要性。

提示與參考解答：框架是一個迴圈：VideoCapture 逐影格讀取來源(檔案/攝影機/串流)→每一影格當成影像套用任何影像處理技術→VideoWriter 逐影格寫出。fourcc 指定輸出的編解碼器(決定壓縮方式，選錯無法寫出)，release 釋放攝影機/檔案資源以免被鎖住。cap.read 的成功旗標用於判斷視訊是否讀完。

4. 影格差分與背景相減有何差異？各自的優缺點是什麼？為什麼影格差分會產生「空心」的移動物件？

提示與參考解答：影格差分：相鄰兩影格相減，極簡極快、自動適應光線，但只偵測到移動的邊緣，暫停物件會消失，且物件內部顏色均勻處前後影格相同、相減為零，故只留輪廓(空心/鬼影)。背景相減：與一個學習得到的背景模型相減，能偵測完整物件、暫停物件不消失，但需自適應維護背景模型以應對光線與搖動干擾(MOG2/KNN)。

5. 為什麼優秀的背景相減器必須是「自適應」的？它要應對哪些真實場景的干擾？

提示與參考解答：因為真實背景並非一成不變：日照移動使光線漸變、樹葉搖動、水面波動、攝影機輕微震動。自適應背景相減器持續緩慢更新背景模型，把「長期穩定存在」的學為背景、「突然出現快速移動」的判為前景，並可偵測陰影，才能在這些干擾下正確運作。

6. 光流的核心假設是什麼？什麼是孔徑問題？它為什麼使單一像素的運動無法唯一決定？

提示與參考解答：核心假設是亮度恆定：同一物體點在相鄰影格間亮度不變、只是位置移動。孔徑問題：透過小孔(局部)觀察移動的直線邊緣，只能看出垂直於邊緣方向的運動分量，沿邊緣方向的運動無從察覺。故單看一個像素的局部資訊無法唯一決定運動，須引入額外假設(如鄰域運動相似)才能求解。

7. 稀疏光流與稠密光流有何差別？Lucas-Kanade 為什麼要選角點來追蹤？

提示與參考解答：稀疏光流(Lucas-Kanade)只追蹤少數適合追蹤的特徵點，高效精準；稠密光流(Farneback)為每個像素計算運動向量，資訊豐富但運算量大，常以虛擬色彩視覺化。Lucas-Kanade 選角點(Shi-Tomasi)是因為角點在兩個方向都有明顯灰階變化，能繞過孔徑問題、運動可唯一決定。

8. 「偵測」與「追蹤」有何根本差別?為什麼「偵測式追蹤」結合兩者會更強健?

提示與參考解答：偵測是在單一影格找出物件位置、影格間獨立無關;追蹤是跨影格維持同一物件的身分(ID)並串成軌跡。偵測準但耗算力、不知影格間是否同一物件;追蹤快且維持身分但可能跟丟或跟錯。偵測式追蹤(SORT/DeepSORT)定期用偵測器校正並發現新物件，中間影格用輕量追蹤維持ID，兼得準確、效率與身分一致性。

9. 為什麼動作辨識(如跌倒偵測)必須用時間序列，而不能只看單張影像?

提示與參考解答：動作是時間性的：單看一張「人躺在地上」的照片無法分辨是跌倒還是正常躺下休息;單看「手舉起」無法分辨是打招呼還是攻擊。必須看時間序列——例如「快速的、由站立到倒地的垂直運動」才是跌倒特徵。故需結合光流/姿態的時間變化，或用3D CNN、視訊Transformer等時序模型。

10. 什麼是視訊深度學習的「時間一致性」問題?若對每一影格獨立套用模型會發生什麼?如何緩解?

提示與參考解答：時間一致性指處理結果在影格與影格之間應穩定連貫。若每一影格獨立套用模型，結果會閃爍抖動(偵測框跳動、物件時有時無、ID跳號)。緩解：在時間軸上平滑結果、用追蹤維持身分、或設計本身考慮時間關聯的模型(3D CNN、時序Transformer)。這是機器人視覺與靜態辨識最大的不同。

11. 視訊穩定的原理是什麼?它綜合了本書哪幾章的技術?

提示與參考解答：原理：估計相鄰影格因抖動造成的整體運動，再反向補償、拉回穩定位置，最後裁掉黑邊。綜合：第6章相位相關法或第12章光流或第13章特徵匹配(估計運動)、第4章幾何轉換(反向補償)、第12章視訊框架。是多章技術的整合應用。

12. (綜合題)請設計一個「道路車速偵測」系統，說明如何從影片算出每輛車的實際速度(km/h)，並指出用到本書哪幾章的技術。

提示與參考解答：流程：一、偵測車輛(背景相減或第16章YOLO);二、追蹤每輛車的軌跡並維持ID(第12章);三、量測車在連續影格間的像素位移;四、用第4章相機校正得到「每像素對應的實際距離(公尺)」把像素位移換算成實際位移;五、除以時間(位移影格數除以影格率得秒數)得速度，再換算km/h。用到第4章(校正/像素換算)、第12章(偵測、追蹤、影格率提供時間基準)、第16章(YOLO偵測)。

參考資料與延伸閱讀

1. Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018.(相關章節)
2. Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 9(視訊處理與運動)。
3. Muybridge, E., The Horse in Motion (Sallie Gardner at a Gallop), 1878, Library of Congress.(本章叫獸開講之史料來源)
4. Lucas, B. D., & Kanade, T., An Iterative Image Registration Technique with an Application to Stereo Vision, IJCAI, 1981.
5. Farnebäck, G., Two-Frame Motion Estimation Based on Polynomial Expansion, SCIA, 2003.

6. Bewley, A., et al., Simple Online and Realtime Tracking (SORT), ICIP, 2016;Wojke, N., et al., DeepSORT, ICIP, 2017.
7. 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 13 章 特徵擷取

本章學習目標

- 說明「特徵」在電腦視覺中的意義，理解好的特徵應具備重複性、獨特性與不變性。
- 推導 Harris 與 Shi-Tomasi 角點偵測，說明角點為何是良好的追蹤與匹配基準。
- 闡述 SIFT 的尺度不變原理與 ORB 的工程取捨，說明兩者的授權與效能差異。
- 運用 BF 與 FLANN 進行特徵匹配，並以比值測試與 RANSAC 去除錯誤匹配。
- 完成影像拼接、模板比對與物件定位，並實作車牌偵測的特徵前處理。
- 理解手工特徵與深度學習特徵的關係，說明本章在傳統與現代視覺之間的橋梁地位。

叫獸開講|SIFT 的二十年專利長跑：一個演算法的解禁之日

2020 年 3 月，全球電腦視覺社群悄悄迎來一個值得慶祝的日子。開源函式庫 OpenCV 的開發者們，終於可以名正言順地把一個名叫 SIFT 的演算法，從那個帶著警意味的「非自由」(non-free)模組，搬進人人可自由使用的主庫。這一天，許多工程師等了將近二十年。

故事要從 1999 年說起。加拿大英屬哥倫比亞大學的 David Lowe 教授，發表了尺度不變特徵轉換(Scale-Invariant Feature Transform，簡稱 SIFT)。這是一個劃時代的成就：在 SIFT 之前，電腦視覺一直被一個根本難題困擾——當同一個物體以不同的大小、不同的角度、不同的光線出現時，電腦很難認出「這是同一個東西」。SIFT 優雅地解決了這個問題，它能在影像中找出一些獨特的「關鍵點」，而這些關鍵點對縮放、旋轉都保持不變，對光線變化與雜訊也相當強健——就像替影像中的每個特徵蓋上一枚無論如何變形都認得出來的「指紋」。SIFT 迅速成為物件辨識、影像拼接、3D 重建的基石，是深度學習興起前最重要的視覺演算法之一。

但 SIFT 有一道緊箍咒：它被英屬哥倫比亞大學申請了專利(美國專利 6,711,293,1999 年 3 月提出臨時申請，2004 年正式授予)。這意味著在商業產品中使用 SIFT 需要授權。因此，儘管 OpenCV 收錄了 SIFT 的實作，卻只能把它放進一個特別的「非自由」模組，並附上明確警示：使用前請自行確認專利與授權問題。全球工程師只能一邊讚嘆它的威力，一邊小心翼翼地繞道而行。這道緊箍咒也催生了替代品：2011 年，OpenCV 團隊自己提出了 ORB(Oriented FAST and Rotated BRIEF)，一個完全免費、速度更快、專為「沒有專利負擔」而設計的方案——工程界對專利限制的回應，往往就是造一個自由的輪子。

專利的保護期是二十年。2020 年 3 月，SIFT 的專利終於到期。消息傳開，社群一片歡騰，OpenCV 隨即在 4.4.0(以及 3.4.11)版本中，把 SIFT 正式移入自

由的主庫。一個沉睡了二十年的經典，終於解除封印，回到每一位工程師都能自由取用的工具箱中。

這個故事給了我們兩個超越技術的啟示。其一，技術選型不只看效能，還要看授權：在 SIFT 被專利綁住的年代，免費的 ORB 儘管精度稍遜，卻因為「可以自由商用」而被廣泛採用——工程決策從來不是在真空中做的。其二，好的基礎研究會持續發光：即使深度學習已經興起，SIFT 所奠定的「尺度不變、局部特徵」思想，依然深深影響著今天的視覺演算法。這一章，我們就來認識這些為影像蓋「指紋」的經典方法。

思考題：在 SIFT 被專利保護的二十年間，ORB 這個「免費替代品」蓬勃發展你能想到其他領域中，因為某個技術被專利或授權限制，而催生出自由替代方案的例子嗎？(提示：作業系統、資料庫、影音編碼)

13.1 什麼是好的特徵

13.1.1 從像素到特徵：再一次的抽象躍升

回顧全書的旅程：第 1 到 8 章我們處理「像素」，第 9、10 章我們把像素聚成「區域與物件」，而本章要再往上一階——從影像中萃取出精簡而有代表性的「特徵」(Feature)。所謂特徵，是影像中一小塊「值得注意、容易辨認、可重複找到」的局部結構。與其比對成千上萬個像素，不如只比對數十個關鍵特徵——這讓影像的比對、辨識、對齊變得高效而強健。

這個抽象躍升的價值，可以用一個生活比喻理解：要向朋友描述一個人，你不會逐一報告他臉上每個毛孔的位置與顏色(像素層級)，而是說「他有濃眉、鷹勾鼻、左臉有顆痣」(特徵層級)——用少數幾個獨特的標記，就足以精準辨認。電腦視覺的特徵擷取，做的正是同一件事：為影像找出它最具辨識度的「標記」。

13.1.2 好特徵的三個條件

並非影像中任何一點都適合當特徵。一個理想的特徵應具備三個性質：

1. 重複性(Repeatability)：同一個場景在不同條件下(不同視角、光線、雜訊)拍攝，同一個特徵都應該能被再次偵測到。若一個特徵這次找得到、下次就消失，它就無法用於比對。
2. 獨特性(Distinctiveness)：特徵應該與眾不同、容易與其他特徵區分。一片平坦的牆面、一段筆直的邊緣都不夠獨特(牆上每一點都長得一樣，邊緣上的點沿邊緣滑動也分不出來)，而角點、紋理豐富的斑塊則獨一無二。
3. 不變性(Invariance)：特徵的描述應該對各種變化保持穩定——尺度不變(物體放大縮小仍認得)、旋轉不變(物體轉了角度仍認得)、光照不變(明暗改變仍認得)。不變性愈強，特徵愈能在惡劣條件下發揮作用。

這三個條件也解釋了為什麼「角點」是特徵偵測的寵兒。承接第 12 章的孔徑問題：平坦區域在任何方向移動都看不出變化(無法定位)；邊緣只能定出垂直於邊緣的方向(一維模糊)；唯有角點在兩個方向都有明顯變化，能被唯一地定位——它同時滿足重複性與獨特性。這就是下一節 Harris 角點偵測的出發點。

13.2 角點偵測：Harris 與 Shi-Tomasi

13.2.1 Harris 角點的直覺

Harris 角點偵測(1988 年由 Chris Harris 與 Mike Stephens 提出)的核心直覺非常優雅：拿一個小視窗在影像上滑動，觀察視窗內的灰階如何變化。有三種情況——若視窗在平坦區域移動，無論往哪個方向，視窗內容都幾乎不變；若視窗沿著邊緣移動(平行於邊緣)，內容不變，但垂直於邊緣移動則劇烈變化；若視窗位於角點，則往任何方向移動，內容都會顯著改變。「往任何方向移動都變化很大」正是角點的數學特徵。

Harris 把這個直覺量化成一個「結構張量」(從視窗內的梯度統計而來)，其兩個特徵值反映了兩個主要方向上的變化強度：兩者都小是平坦區、一大一小是邊緣、兩者都大則是角點。Harris 巧妙地用一個不需實際計算特徵值的響應函數 R 來判斷，計算高效。這裡也再次呼應第 9 章：梯度(Sobel)是這一切的基礎——角點偵測建立在灰階變化(梯度)的分析之上。

13.2.2 Shi-Tomasi 的改良

1994 年，Jianbo Shi 與 Carlo Tomasi 在論文《Good Features to Track》中提出改良：與其用 Harris 的響應函數，不如直接取「兩個特徵值中較小者」作為角點強度——只要較小的特徵值夠大，就保證兩個方向都有足夠變化。這個判準在實務上(尤其是用於第 12 章的光流追蹤)表現更穩定，因此 OpenCV 的 `goodFeaturesToTrack` 函式預設採用 Shi-Tomasi。名稱「適合追蹤的好特徵」也點明了它與第 12 章光流的血緣。

程式 13-1 Harris 與 Shi-Tomasi 角點偵測

```
import cv2, numpy as np

img = cv2.imread("building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# --- Harris 角點 ---
harris = cv2.cornerHarris(np.float32(gray), blockSize=2,
                           ksize=3, k=0.04)
img_h = img.copy()
img_h[harris > 0.01 * harris.max()] = [0, 0, 255] # 標紅

# --- Shi-Tomasi(適合追蹤,第 12 章光流的輸入) ---
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100,
                                   qualityLevel=0.01, minDistance=10)
img_s = img.copy()
for c in np.int0(corners):
    x, y = c.ravel()
    cv2.circle(img_s, (x, y), 4, (0, 255, 0), -1)

cv2.imwrite("harris.png", img_h)
cv2.imwrite("shitomasi.png", img_s)
```

角點偵測的一個侷限是：它不具備尺度不變性。一個在原尺寸下是「尖銳角點」的結構，把影像放大數倍後，同一個角點在局部看來變成了「平緩的曲線」，角點響應消失。要處理「同一物體以不同大小出現」的問題，就需要下一節登場的主角——SIFT。

13.3 尺度不變特徵：SIFT 與 ORB

13.3.1 SIFT 的四個步驟

承叫獸開講，SIFT 的偉大之處在於它同時解決了尺度與旋轉不變性。它的完整流程可以拆成四步，每一步都巧妙地呼應了本書前面的概念：

4. 尺度空間極值偵測：把影像用一系列愈來愈強的高斯模糊處理，建構出一個「尺度金字塔」(這正是第 11 章多解析度分析的思想!)。相鄰兩層相減得到「高斯差」(DoG)，它是第 9 章 LoG 的高效近似，也是一個帶通濾波器(第 6 章)。在這個「空間加尺度」的三維空間中尋找極值點——一個點若在自己的尺度與相鄰尺度中都是局部極值，它就是候選關鍵點。關鍵在於：因為在多個尺度中搜尋，無論物體以什麼大小出現，都能在對應的尺度層被找到，這就實現了尺度不變性。
5. 關鍵點精確定位：剔除對比度太低(易受雜訊干擾)與位於邊緣上(定位不穩)的候選點，留下真正穩定的關鍵點。
6. 方向指派：分析關鍵點周圍的梯度方向分布，把最主要的方向指派給該關鍵點作為「基準方向」。之後所有的描述都相對於這個基準方向來計算——這就實現了旋轉不變性(無論物體轉了多少度，基準方向會跟著轉，描述保持一致)。
7. 描述子生成：在關鍵點周圍取一個區塊，依基準方向對齊後，統計區塊內各子區域的梯度方向直方圖，組成一個 128 維的向量——這就是該關鍵點的「指紋」(描述子)。這個向量經過正規化以抵抗光照變化，最終得到一個對尺度、旋轉、光照都穩健的特徵描述。

值得回味的是，SIFT 幾乎是本書前半的集大成：高斯模糊(第 5 章)、多尺度(第 11 章)、DoG/帶通(第 6、9 章)、梯度(第 9 章)、直方圖(第 5 章)、正規化(第 5 章)——這些看似分散的工具，在 SIFT 中被組織成一個優雅的整體。這也說明了為什麼要循序學完前面各章：它們是理解進階演算法的積木。

13.3.2 ORB：免費而快速的替代方案

SIFT 雖強，卻有兩個現實問題：受專利保護(叫獸開講的核心)、且運算較慢(128 維浮點描述子，計算與比對都吃資源)，不適合即時或資源受限的場合。2011 年 OpenCV 團隊提出的 ORB，正是為了解決這兩點。

ORB 是兩個既有技術的聰明融合：用 FAST 角點偵測器(極快)找關鍵點，並加上方向資訊使其具旋轉不變性；用 BRIEF 二進位描述子(以一連串「這個點比那個點亮嗎」的是非比較，產生一串 0 與 1)來描述特徵，並加以旋轉校正。二進位描述子的好處是：比對時用漢明距離(算兩串位元有幾位不同)，可以用 CPU 指令極速完成，遠快於 SIFT 的浮點運算。ORB 完全免費、速度可比 SIFT 快數十倍，雖然在尺度變化極大

或精度要求極高的場合略遜 SIFT，但在多數即時應用(尤其是機器人、手機、ESP32/Jetson 等第 17 章的邊緣平台)中是務實首選。

比較面向	SIFT	ORB
授權	原有專利(2020 年到期)	完全免費開源
描述子	128 維浮點向量	二進位字串(漢明距離比對)
速度	較慢	快數十倍，可即時
尺度不變性	優異	中等(尺度變化大時較弱)
適用場合	高精度離線比對、拼接	即時、行動與邊緣裝置

表13-1 SIFT 與ORB 的比較

程式 13-2 SIFT 與 ORB 特徵擷取

```
import cv2

img = cv2.imread("object.jpg", cv2.IMREAD_GRAYSCALE)

# --- SIFT(專利已於 2020 到期,可自由使用) ---
sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(img, None)
print("SIFT 關鍵點數:", len(kp_sift),
      "描述子維度:", des_sift.shape)      # (N, 128)

# --- ORB(免費、快速) ---
orb = cv2.ORB_create(nfeatures=500)
kp_orb, des_orb = orb.detectAndCompute(img, None)
print("ORB 關鍵點數:", len(kp_orb),
      "描述子維度:", des_orb.shape)      # (N, 32) 二進位

# 畫出關鍵點(含尺度與方向)
vis = cv2.drawKeypoints(img, kp_sift, None,
                        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("keypoints.png", vis)
```

13.4 特徵匹配與外點去除

偵測出兩張影像各自的特徵與描述子後，下一步是匹配(Matching)：找出「這張影像的哪個特徵，對應到那張影像的哪個特徵」。這是影像拼接、物件辨識、3D 重建的關鍵一步。

13.4.1 匹配器：BF 與 FLANN

最直接的方法是暴力匹配(Brute-Force,BF)：把第一張影像的每個描述子，與第二張影像的所有描述子逐一比對距離，取最近的作為匹配。距離的計算方式取決於描述子類型——SIFT 的浮點描述子用歐氏距離，ORB 的二進位描述子用漢明距離。BF 準確但當特徵量大時很慢。

FLANN(快速近似最近鄰函式庫)則以近似的方式加速搜尋：它建立特殊的資料結構(如 KD 樹)，不保證找到絕對最近的匹配，但速度快得多，適合大量特徵的場合。BF 與 FLANN 的取捨，又是一次「準確 vs. 速度」的工程權衡——與第 9 章門檻、第 10 章偵測參數、第 12 章追蹤器選擇一脈相承。

13.4.2 比值測試：過濾模糊匹配

原始匹配充滿錯誤——因為並非每個特徵都在另一張影像中有真正的對應(可能被遮擋、或根本不存在)。David Lowe 在 SIFT 論文中提出了一個極為有效的過濾法：比值測試(Ratio Test)。對每個特徵，找出它在另一張影像中最近與次近的兩個匹配；若「最近距離」明顯小於「次近距離」(例如小於 0.75 倍)，代表這個匹配獨一無二、可信；若兩者接近，代表這個特徵在對方影像中有多個相似的候選、難以區分，應該捨棄。這個簡單的比值，能濾掉大量不可靠的匹配。

13.4.3 RANSAC：在錯誤中找出真相

即使經過比值測試，仍會有一些「看起來可信、實則錯誤」的匹配(稱為外點，outliers)。RANSAC(隨機抽樣一致法)是對抗外點的利器，其思想極為漂亮：與其相信所有資料，不如反覆隨機抽取少數幾組匹配，假設它們是正確的、據此計算出一個幾何模型(例如第 4 章的透視轉換矩陣)，再看有多少其他匹配符合這個模型；重複多次，「獲得最多支持者」的模型就是正確答案，而不符合它的匹配就是外點，予以剔除。

RANSAC 的精神與第 9 章霍夫轉換異曲同工——都是「投票」：霍夫讓每個邊緣點投票給可能的直線，RANSAC 讓每個匹配投票給可能的幾何模型，得票最高者勝出。這種「用多數證據對抗少數雜訊」的思想，是穩健估計的核心，在充滿雜訊與錯誤的真實資料中至關重要。

程式 13-3 特徵匹配、比值測試與 RANSAC 物件定位

```
import cv2, numpy as np

img1 = cv2.imread("box.jpg", cv2.IMREAD_GRAYSCALE)      # 要找的物件
img2 = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)    # 場景

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# --- BF 匹配 + 比值測試 ---
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)                  # 取最近兩個
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:                  # Lowe 比值測試
        good.append(m)
print("比值測試後的匹配數:", len(good))

# --- RANSAC 求單應性矩陣, 剔除外點 ---
if len(good) > 10:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
```

```

dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-
1,1,2)
H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
print("RANSAC 判定的內點數:", int(mask.sum()))

# 在場景中畫出物件的位置(第 4 章透視轉換)
h, w = img1.shape
corners = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-
1,1,2)
projected = cv2.perspectiveTransform(corners, H)
scene = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)
cv2.polylines(scene, [np.int32(projected)], True, (0,255,0), 3)
cv2.imwrite("object_located.png", scene)

```

13.5 應用：影像拼接、模板比對與物件定位

13.5.1 影像拼接(全景圖)

全景照片是特徵匹配最迷人的應用。原理：對兩張有重疊區域的照片各自擷取特徵 → 匹配並用 RANSAC 求出它們之間的透視轉換(第 4 章) → 把一張影像依此轉換「拉」到與另一張對齊的位置 → 融合重疊區域(消除接縫)。手機相機的「全景模式」、Google 街景的環景圖，骨幹都是這條流程。它綜合了本章(特徵匹配)、第 4 章(透視轉換)、第 6 章(相位相關可做初始對齊)、第 11 章(多頻帶融合可消接縫)。

13.5.2 模板比對與其侷限

模板比對(Template Matching)是最樸素的「找物件」方法：拿一個小範本(template)在大影像上逐位置滑動，計算每個位置的相似度，相似度最高處就是物件所在。OpenCV 的 matchTemplate 提供此功能。它簡單直接，但侷限明顯：對尺度與旋轉極度敏感——範本一旦與目標的大小或角度不符就會失敗。這恰恰反襯出特徵匹配(SIFT/ORB)的價值：後者的尺度與旋轉不變性，正是為了克服模板比對的死穴而生。何時用哪個？目標大小角度固定(如生產線上位置固定的工件)用模板比對即可；目標會變大變小變角度，就需要特徵匹配。

13.5.3 物件定位

程式 13-3 已示範了完整的物件定位：給定一張「目標物件」的參考圖與一張「場景」圖，透過特徵匹配加 RANSAC，不僅能判斷場景中「有沒有」該物件，還能精確框出它「在哪裡、以什麼角度出現」——即使物件被部分遮擋(SIFT 論文指出，少至三個正確匹配的特徵就足以定位物件)。這是深度學習物件偵測(第 16 章)普及之前，工業界物件辨識與定位的主力方法，至今在樣本稀少、不便訓練深度模型的場合仍有一席之地。

13.6 案例深探：車牌偵測

車牌偵測是一個絕佳的綜合案例，它把本書多章的技術串成一條完整的實用管線也是您在課程中反覆演練的經典題目。完整的車牌辨識(License Plate

Recognition, LPR) 分為「偵測定位」與「字元辨識」兩大階段，本節聚焦前者(偵測)，字元辨識則銜接第 10 章(字元分割)與第 15 章(字元分類)。

13.6.1 傳統車牌偵測管線

傳統方法充分利用車牌的視覺特性——矩形、高對比、字元密集(高頻紋理豐富)。典型流程：

8. 前處理：轉灰階、以第 5 章的方法增強對比、去雜訊。
9. 邊緣與紋理分析：車牌區域因字元密集而有大量垂直邊緣。用第 9 章的 Sobel 求垂直梯度，車牌區域會呈現密集的邊緣響應，凸顯出候選區域。
10. 型態學處理：用第 10 章的閉合運算把密集的邊緣連成一整塊矩形區域，再用斷開去除雜點。
11. 候選區域篩選：用第 10 章的連通元件標記找出候選區塊，依車牌的長寬比(約 3:1 到 4:1)、面積、填充率等幾何條件過濾掉不像車牌的區域。
12. 透視校正：若車牌是斜拍的，用第 4 章的透視轉換把它「拉正」成正面矩形，方便後續字元分割。

這條管線是第 4、5、9、10 章的完美綜合，展示了傳統影像處理如何以「一連串簡單步驟的巧妙串接」解決真實問題——這正是叫獸開講中 Canny、Viola-Jones 都體現過的工程智慧。

13.6.2 特徵匹配在車牌辨識中的角色

特徵匹配(本章主題)在車牌辨識中也有用武之地：例如以特徵匹配比對車牌上的地區標誌或特殊符號、或在多影格影片中追蹤同一張車牌(結合第 12 章)。而在現代系統中，整個偵測階段愈來愈多改用第 16 章的 YOLO——直接訓練一個「車牌偵測器」一步到位框出車牌，對傾斜、模糊、光線變化的強健度遠勝傳統管線。這再次呼應全書的主軸：傳統方法(可解釋、不需訓練、樣本少也能用)與深度學習(強健、需資料與算力)各擅勝場，成熟的工程師懂得依場景選擇，甚至混用。

程式 13-4 傳統車牌偵測管線

```
import cv2, numpy as np

img = cv2.imread("car.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. 前處理(第 5 章)
gray = cv2.bilateralFilter(gray, 11, 17, 17) # 保邊去雜訊

# 2. 垂直邊緣:車牌字元密集,垂直邊緣豐富(第 9 章)
edges = cv2.Sobel(gray, cv2.CV_8U, 1, 0, ksize=3)
_, edges = cv2.threshold(edges, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3. 型態學:把密集邊緣連成矩形塊(第 10 章)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (17, 5))
closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)
```

```
# 4. 連通元件 + 幾何條件篩選候選車牌(第 10 章)
cnts, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)
for c in cnts:
    x, y, w, h = cv2.boundingRect(c)
    ratio = w / float(h)
    if 2.5 < ratio < 5.0 and w > 80:          # 車牌長寬比約 3~4:1
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
        plate = gray[y:y+h, x:x+w]          # 裁出車牌供字元分割(第 10 章)

cv2.imwrite("plate_detected.png", img)
```

13.7 從手工特徵到學習特徵

本章介紹的 Harris、SIFT、ORB，都屬於手工設計特徵(hand-crafted features)——它們是人類專家憑藉對影像的深刻理解，精心設計出來的偵測與描述規則。在深度學習興起前的二十年，手工特徵是電腦視覺的絕對主流，SIFT 更是其中的皇冠。

2012 年後，深度學習帶來了典範轉移(這正是第 15 章的主題)：卷積神經網路不再依賴人類設計特徵，而是從海量資料中自動學習特徵。但這裡有一個深刻且美麗的連結——第 6.6.7 節已經預告過：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣偵測器與方向性帶通濾波器，與人工設計的 Gabor、SIFT 的梯度方向分析驚人地相似！換言之，深度學習並沒有推翻手工特徵的智慧，而是「重新發明」並「自動化」了它——網路透過梯度下降，自己學會了人類專家花了數十年才設計出的東西，並且能組合出更複雜、更抽象的高層特徵。

因此，學習本章絕非學習「過時的技術」。理由有三：其一，理解手工特徵，才能真正理解 CNN 學到了什麼、為什麼有效——手工特徵是深度特徵的「白箱版本」。其二，在樣本稀少、算力受限、需要可解釋性的場合(許多工業與機器人場景)，手工特徵至今仍是最務實的選擇。其三，現代許多系統是混合式的：用深度學習做高層理解、用手工特徵(如 ORB)做即時的幾何對齊與定位(例如許多 SLAM 系統的核心仍是 ORB 特徵)。手工特徵與學習特徵不是取代關係，而是各據所長、相互補充——這也是本章作為「傳統視覺」與「現代視覺」橋梁的意義所在。

13.8 本章與全書的觀念連結總表

相關章節	共用的觀念	特徵擷取中的角色
第 4 章 幾何轉換	透視轉換、單應性	RANSAC 求轉換做物件定位與影像拼接
第 5 章 空間域強化	高斯模糊、直方圖、正規化	SIFT 尺度空間、方向直方圖、描述子正規化
第 6 章 頻率域	帶通、Gabor、DoG	SIFT 的 DoG 即帶通;Gabor 為紋理特徵
第 9 章 影像分割	梯度、Sobel、LoG	角點與 SIFT 建立於梯度;DoG 近似 LoG

第 10 章 二值影像	型態學、連通元件	車牌偵測的候選區域生成與篩選
第 11 章 小波	多解析度分析	SIFT 尺度金字塔與小波多尺度同源
第 12 章 視訊處理	光流、追蹤、視訊穩定	Shi-Tomasi 供光流追蹤;特徵匹配做穩定
第 15、16 章 深度學習	CNN 卷積核、物件偵測	CNN 首層重現手工特徵;YOLO 取代傳統偵測
第 17 章 邊緣部署	即時性、SLAM	ORB 輕量，為機器人 SLAM 的核心特徵

表13-2 第13 章與全書其他章節的觀念連結

程式實作 Lab 13

Lab 13-1 角點偵測與特徵比較

目標：建立對特徵的直覺。(a)對同一張影像分別套用 Harris、Shi-Tomasi、SIFT、ORB，比較各自偵測到的關鍵點分布——它們都偏好落在什麼樣的區域?(b)把影像放大兩倍、旋轉 45 度、調暗，重新偵測，量化比較哪種方法的「重複性」最好(同一特徵在變換後仍被偵測到的比例);(c)驗證角點偵測不具尺度不變性：放大影像後，Harris 角點是否消失?SIFT 是否仍找得到?(d)討論：為什麼平坦區與邊緣上偵測不到角點?這與第 12 章孔徑問題有何關聯?

Lab 13-2 物件定位器

目標：實作程式 13-3 的完整物件定位。(a)拍一張小物件(如書本封面、商標)作為「目標」，再拍幾張把它放進雜亂場景(不同角度、部分遮擋)的照片作為「場景」;(b)用 SIFT/ORB 加 BF 匹配加比值測試加 RANSAC，在場景中框出目標的位置與方向(c)測試極限：目標旋轉多少、遮擋多少、縮放多少時開始失敗?(d)比較 SIFT 與 ORB 的定位成功率與速度(量測 FPS);(e)討論：比值測試的門檻(0.7、0.75、0.8)如何影響匹配的數量與品質?RANSAC 剔除了多少外點?

Lab 13-3 全景圖拼接

目標：做出自己的全景照片。(a)手持相機水平平移，拍攝三到五張有重疊(建議重疊三成以上)的照片;(b)相鄰兩張擷取特徵、匹配、RANSAC 求透視轉換，把後一張拉到與前一張對齊，逐步拼接;(c)處理接縫：比較「直接覆蓋」與「重疊區漸變融合」的視覺差異(進階可用第 11 章的多頻帶融合);(d)直接用 OpenCV 的 cv2.Stitcher 對照你的手工結果;(e)討論：什麼情況下拼接會失敗(重疊太少、場景無紋理、有移動物件)?

Lab 13-4 車牌偵測管線

目標：實作 13.6 節的傳統車牌偵測。(a)以程式 13-4 為基礎，對車輛照片完成「前處理→垂直邊緣→型態學→連通元件→幾何篩選」的完整管線，框出車牌;(b)調

校型態學結構元素的尺寸與長寬比門檻，提高偵測率、降低誤報;(c)對斜拍的車牌，用第 4 章透視轉換拉正;(d)串接第 10 章：對拉正後的車牌做二值化與連通元件，分割出各個字元;(e)討論：傳統管線在什麼情況下失敗(強光、髒污、傾斜過大)?對照第 16 章，YOLO 車牌偵測會如何改善這些問題?各自的成本與適用場景為何?

本章重點整理

- 特徵是影像中值得注意、易辨認、可重複找到的局部結構;好特徵須具重複性、獨特性、不變性(尺度/旋轉/光照)。角點在兩方向都有變化，能唯一定位，是理想特徵(呼應第 12 章孔徑問題)。
- Harris 角點以視窗滑動的灰階變化與結構張量特徵值判斷：兩者都大即角點;Shi-Tomasi 取較小特徵值作強度，更適合追蹤，為 goodFeaturesToTrack 的預設;兩者皆不具尺度不變性。
- SIFT 四步驟(尺度空間 DoG 極值、關鍵點定位、方向指派、128 維描述子)達成尺度與旋轉不變，是第 5、6、9、11 章工具的集大成;ORB(FAST+BRIEF)免費、快數十倍、二進位描述子用漢明距離比對，適合即時與邊緣裝置。
- 匹配：BF 準確、FLANN 快速近似;Lowe 比值測試(最近/次近距離比)過濾模糊匹配;RANSAC 以「投票」思想反覆抽樣建模、剔除外點，精神與霍夫轉換一致。
- 應用：影像拼接(特徵匹配+透視轉換+融合)、模板比對(簡單但不具尺度旋轉不變性)、物件定位(匹配+RANSAC，可抗遮擋);車牌偵測綜合第 4、5、9、10 章，是傳統管線的典範。
- Harris/SIFT/ORB 為手工特徵;CNN 首層自動學出的濾波器與之高度相似，深度學習是「重新發明並自動化」了手工特徵。兩者互補：手工特徵可解釋、樣本少可用、即時(SLAM 用 ORB)，學習特徵強健、能學高層語意。

習題

1. 什麼是影像的「特徵」?為什麼用少數特徵比對比逐像素比對更好?請用一個生活比喻說明。

提示與參考解答：特徵是影像中值得注意、易辨認、可重複找到的局部結構(如角點、獨特斑塊)。用少數特徵比對高效且強健，避免比對成千上萬像素。生活比喻：描述一個人不會逐一報告每個毛孔，而是說「濃眉、鷹勾鼻、左臉有痣」——用少數獨特標記就能精準辨認。

2. 好的特徵應具備哪三個條件?各舉一個「不符合」該條件的反例。

提示與參考解答：重複性(不同條件下都能再次偵測到;反例：只在特定光線下出現的反光點)、獨特性(與眾不同易區分;反例：平坦牆面上的點，每點都一樣)、不變性(對尺度/旋轉/光照穩定;反例：Harris 角點放大後消失，不具尺度不變性)。

3. 為什麼角點是理想的特徵，而平坦區與邊緣不是?這與第 12 章的孔徑問題有何關聯?

提示與參考解答：角點在兩個方向都有明顯灰階變化，能被唯一定位(重複+獨特)。平坦區任何方向移動都無變化，無法定位;邊緣沿邊緣方向移動無變化，只能定出垂

直方向(一維模糊)。這正是第12章孔徑問題的體現——局部只能看出垂直於邊緣的運動/位置，唯有角點無此歧義。

- 說明 Harris 角點的核心直覺，以及結構張量兩個特徵值的三種組合分別代表什麼。

提示與參考解答：直覺：小視窗在影像上滑動，觀察灰階變化——平坦區任何方向都不變、邊緣僅垂直方向變、角點任何方向都劇烈變。結構張量兩特徵值：都小=平坦區，一大一小=邊緣，都大=角點。Harris 用不需實際算特徵值的響應函數 R 高效判斷。

- Shi-Tomasi 相對 Harris 做了什麼改良？為什麼它更適合用於第12章的光流追蹤？

提示與參考解答：Shi-Tomasi 直接取兩個特徵值中較小者作為角點強度，只要較小者夠大就保證兩方向都有足夠變化，判準更穩定。因此在追蹤場合表現更好，OpenCV 的 `goodFeaturesToTrack` 預設採用它，論文名〈Good Features to Track〉即點明與光流的血緣——這些角點是 Lucas-Kanade 稀疏光流理想的追蹤點。

- SIFT 如何達成尺度不變性與旋轉不變性？請分別說明對應的步驟。

提示與參考解答：尺度不變性：建構高斯尺度金字塔(多解析度)，在「空間+尺度」三維空間找 DoG 極值——物體無論多大都能在對應尺度層被找到。旋轉不變性：分析關鍵點周圍梯度方向分布，指派主方向為基準，之後所有描述相對此基準計算——物體轉多少度，基準跟著轉，描述保持一致。

- SIFT 的四個步驟分別用到本書前面的哪些概念？這說明了什麼學習上的道理？

提示與參考解答：高斯模糊(第5章)、多尺度金字塔(第11章)、DoG 帶通/近似 LoG(第6、9章)、梯度(第9章)、方向直方圖(第5章)、描述子正規化抗光照(第5章)。說明 SIFT 是前面各章工具的集大成，循序學完基礎章節，是理解進階演算法的積木——基礎不牢，進階難懂。

- ORB 為什麼比 SIFT 快？它由哪兩個技術融合而成？二進位描述子如何加速比對？

提示與參考解答：ORB 融合 FAST 角點偵測器(極快)加方向資訊，與 BRIEF 二進位描述子(一連串「這點比那點亮嗎」的是非比較，產生 0/1 字串)加旋轉校正。二進位描述子比對時用漢明距離(算兩串位元有幾位不同)，可用 CPU 指令極速完成，遠快於 SIFT 的 128 維浮點歐氏距離運算。

- 承叫獸開講：為什麼在 2020 年之前，免費的 ORB 儘管精度稍遜 SIFT 卻被廣泛採用？這對「技術選型」有何啟示？

提示與參考解答：因為 SIFT 受專利保護(US 6,711,293, 2020 年到期)，商業使用需授權；ORB 完全免費可自由商用。啟示：技術選型不只看效能，還要看授權、成本、生態系——工程決策不在真空中做，一個「夠好且無授權負擔」的方案常勝過「最強但受限」的方案(呼應第11章 JPEG vs. JPEG 2000)。

- 說明 Lowe 比值測試的原理。為什麼「最近距離明顯小於次近距離」代表匹配可信？

提示與參考解答：對每個特徵找出另一影像中最近與次近兩個匹配，若最近距離明顯小於次近(如小於 0.75 倍)則匹配可信，否則捨棄。原理：若一個特徵在對方影像中有唯一明確的對應，最近的會遠比次近的近；若最近與次近相當，代表有多個相似候選、難以區分，很可能匹配錯誤，應捨棄。

11. RANSAC 如何從充滿錯誤的匹配中找出正確的幾何模型?它與第 9 章的哪個演算法精神相同?

提示與參考解答：RANSAC 反覆隨機抽取少數匹配、假設正確、據此算出幾何模型(如透視轉換)，再統計有多少其他匹配符合此模型；重複多次，得票最高(支持者最多)的模型勝出，不符者為外點剔除。與第 9 章霍夫轉換精神相同——都是「投票」：用多數證據對抗少數雜訊，是穩健估計的核心。

12. 模板比對的最大侷限是什麼?特徵匹配如何克服?各自適合什麼場合?

提示與參考解答：模板比對對尺度與旋轉極度敏感，範本一旦與目標大小/角度不符就失敗。特徵匹配(SIFT/ORB)具尺度與旋轉不變性，正是為克服此死穴而生。場合：目標大小角度固定(如產線上位置固定的工件)用模板比對即可；目標會變大小變角度，需用特徵匹配。

13. (綜合題)請設計一條傳統車牌偵測管線，列出每一步用到本書哪一章的技術，並說明相對於第 16 章 YOLO 方案的優缺點。

提示與參考解答：管線：前處理(第 5 章：灰階、去雜訊、增強)→ 垂直邊緣分析(第 9 章 Sobel，車牌字元密集邊緣多)→ 型態學閉合連成矩形塊、斷開去雜(第 10 章)→ 連通元件標記加長寬比/面積/填充率篩選(第 10 章)→ 透視校正拉正(第 4 章)→ 字元分割(第 10 章)。相對 YOLO：傳統管線可解釋、不需訓練資料、樣本少也能用、算力低；但對強光、髒污、大傾斜較不強健。YOLO 強健但需大量標註與 GPU。依場景與資源選擇或混用。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 11(特徵擷取)。
- Lowe, D. G., Distinctive Image Features from Scale-Invariant Keypoints, IJCV, 60(2), 2004.(SIFT;US Patent 6,711,293, 本章叫獸開講之史料來源)
- Harris, C., & Stephens, M., A Combined Corner and Edge Detector, Alvey Vision Conference, 1988.
- Shi, J., & Tomasi, C., Good Features to Track, CVPR, 1994.
- Rublee, E., et al., ORB: An Efficient Alternative to SIFT or SURF, ICCV, 2011.
- Fischler, M. A., & Bolles, R. C., Random Sample Consensus (RANSAC), Comm. ACM, 1981.
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 14 章 機器學習基礎

本章學習目標

- 說明機器學習的定義與典範轉移，區分監督式、非監督式與強化學習。
- 理解影像分類的完整流程：資料、特徵、模型、訓練、評估。
- 認識重要的影像資料集(MNIST、CIFAR、COCO、ImageNet/ILSVRC)及其歷史角色。
- 操作 kNN 與 SVM 兩種經典分類器，理解其原理與適用場合。
- 正確切分訓練/驗證/測試集，以準確率、混淆矩陣、精確率與召回率評估模型。
- 診斷過度擬合與擬合不足，並理解偏差-變異權衡在機器學習中的核心地位。

叫獸開講|MNIST：讀懂支票的神經網路與七萬個手寫數字

1980 年代末，一位名叫 Yann LeCun 的年輕研究者來到美國紐澤西州的貝爾實驗室——那個誕生過電晶體、資訊理論、C 語言、Unix 的傳奇之地。他手上的任務很實際：讓機器學會辨認信封上手寫的郵遞區號，好讓美國郵政的信件自動分揀。手寫數字千變萬化，每個人的「7」都不一樣，傳統那套「人工設計規則」的方法在這裡撞牆了——你永遠寫不完「什麼樣算是 7」的規則。

LeCun 走了一條當時還很非主流的路：與其教電腦規則，不如給它大量範例，讓它自己「學」。1989 年，他和同事打造出第一個以反向傳播訓練的卷積神經網路 LeNet，成功辨識美國郵政的手寫郵遞區號。這個系統不斷改良，到 1990 年代初已經進化到能讀取銀行支票上的金額。這不是實驗室的玩具：搭載這套技術的支票辨識系統於 1990 年代初商業化部署，進入美國與歐洲的自動櫃員機；到了 1990 年代末，全美國約有百分之十以上的支票是由 LeCun 團隊的系統自動判讀的。一個學術演算法，悄悄地每天處理著數以百萬計的真實金錢交易。

但要訓練與比較這些「會學習」的系統，需要一套公平的考卷。LeCun 與同事 Corinna Cortes、Chris Burges 於是整理出了 MNIST 資料集：七萬張手寫數字影像(六萬張用於訓練、一萬張用於測試)，每張都是 28×28 像素的灰階圖，標好了 0 到 9 的正確答案。它的來源有個有趣的細節：MNIST 混合自美國國家標準局的兩個資料庫——一個由人口普查局員工書寫(字跡工整、較容易)，一個由高中生書寫(字跡潦草、較困難)，刻意混合是為了讓訓練與測試的難度一致、結論才可靠。這個看似瑣碎的設計考量，其實蘊含了本章要教的重要觀念：資料的品質與切分方式，往往比演算法本身更決定成敗。

MNIST 從此成為機器學習界的「Hello World」——幾乎每一個學習視覺辨識的人，第一個訓練的模型都是 MNIST 手寫數字分類器，您在課程中的跨框架 MNIST 實作(從 TensorFlow 到 Keras 到 PyTorch)正是這個傳統的延續。它也是無數新演算法的第一塊試金石。一個為了解決郵政與銀行實際問題而生的資料集，

意外地推動了整個領域三十多年——這印證了一句在機器學習界流傳的名言：「資料，才是驅動這一切的燃料。」

思考題：LeCun 沒有去寫「什麼樣算是數字 7」的規則，而是給機器看幾萬個 7 的範例讓它自己學。這個「從範例學習」而非「被明確告知規則」的思路，與你過去學程式的方式(工程師寫死每一條 if-else)有什麼根本不同？這個轉變為什麼被稱為「典範轉移」？

14.1 什麼是機器學習

14.1.1 典範轉移：從寫規則到給範例

回顧本書前十三章，我們解決問題的方式有一個共同點：人類專家分析問題、設計演算法。想去雜訊，我們設計濾波器；想找邊緣，我們設計 Canny；想描述特徵，我們設計 SIFT。這是「傳統程式設計」的典範——工程師明確地把「怎麼做」一步一步寫給電腦。

機器學習代表了一次根本的典範轉移(正是叫獸開講思考題的核心)。它不再由人類寫死規則，而是：給電腦大量的「輸入-輸出」範例，讓它自己從中歸納出規律。用一句話對比：傳統程式是「人給規則，電腦套用到資料上得出答案」；機器學習是「人給資料和答案，電腦自己歸納出規則」。這個轉變之所以重要，是因為有太多問題——例如辨認手寫數字、辨識人臉、理解語音——其規則複雜到人類根本寫不出來，但範例卻俯拾皆是。既然寫不出規則，那就讓機器從範例中自己學。

學術上常引用的定義來自 Tom Mitchell：「若一個電腦程式在某任務 T 上、以某效能指標 P 衡量的表現，能隨著經驗 E 而提升，則稱它從經驗 E 中學習。」以 MNIST 為例：任務 T 是辨認數字、效能 P 是辨識準確率、經驗 E 是那六萬張標好答案的訓練影像。看得愈多、學得愈好——這就是「學習」。

14.1.2 三種學習範式

1. 監督式學習(Supervised Learning)：訓練資料同時提供「輸入」與「正確答案(標籤)」，模型學習從輸入預測答案。本章與 MNIST 都屬此類——每張數字影像都附有正確的數字標籤。它又分為分類(輸出是類別，如「這是 3」)與迴歸(輸出是連續值，如「房價是多少」)。影像辨識絕大多數是分類問題。
2. 非監督式學習(Unsupervised Learning)：訓練資料只有輸入、沒有標籤，模型自己發現資料中的結構。例如分群(把相似的影像自動歸為一類，而沒人告訴它類別是什麼)、降維(把高維資料壓縮到低維以便視覺化或去除冗餘)。
3. 強化學習(Reinforcement Learning)：模型(稱為智能體 agent)透過與環境互動、依據行動的獎懲來學習策略，沒有現成的正確答案，只有「做得好不好」的回饋。它是機器人控制、遊戲 AI(如下圍棋的 AlphaGo)、自駕決策的核心，與機器人學程的關聯尤深——不過本章聚焦影像分類，強化學習留待進階課程。

本章與第 15、16 章的深度學習，主軸都是監督式學習中的影像分類與偵測。理解監督式學習的完整流程，是踏入深度學習的必要基礎——事實上，深度學習只是機器學習的一個分支(用「深層神經網路」作為模型)，它遵循的訓練、評估、過度擬合

診斷等原則，與本章介紹的完全一致。先打好本章的地基，第 15 章的 CNN 才不會是空中樓閣。

14.2 影像分類的完整流程

一個監督式影像分類系統，從資料到可用模型，要走過五個環節。這條流程貫穿本章與後續深度學習各章，務必牢記：

4. 資料(Data)：收集大量影像並標註正確答案。這是整個流程的地基——正如叫獸開講所示，資料的品質與數量往往比演算法更決定成敗。14.3 節專門討論資料集。
5. 特徵(Features)：把原始像素轉換成更有代表性的數值向量。傳統機器學習高度依賴人工設計特徵(第 13 章的 SIFT、HOG、顏色直方圖等)——這一步的好壞直接決定成敗。而深度學習(第 15 章)最大的突破，正是讓模型自動學出特徵，免去了人工設計的負擔。
6. 模型(Model)：選擇一個能從特徵映射到類別的數學模型，如本章的 kNN、SVM，或第 15 章的神經網路。
7. 訓練(Training)：用訓練資料調整模型的參數，使它在訓練資料上的預測盡量正確。
8. 評估(Evaluation)：用「模型沒看過的」測試資料檢驗它的真實表現——這是判斷模型好壞的關鍵，14.5 節詳述。

請特別注意「特徵」這一環，它是理解傳統機器學習與深度學習分野的鑰匙。在傳統流程中，特徵是人工設計的，模型只負責「從特徵到類別」這後半段；而深度學習把「從像素到特徵」與「從特徵到類別」整合進同一個網路，端到端一起學習——這就是第 13.7 節說的「從手工特徵到學習特徵」的典範轉移。本章聚焦傳統流程，正是為了讓你清楚看見：深度學習到底改變了哪一環、又保留了哪一環。

14.3 影像資料集：機器學習的燃料

如果說演算法是引擎，那麼資料就是燃料——沒有燃料，再好的引擎也跑不動。機器學習的發展史，某種程度上就是一部「資料集推動演算法」的歷史。以下四個資料集，不僅是實作練習的素材，更是理解整個領域演進的關鍵座標。

14.3.1 MNIST：入門的經典

承叫獸開講，MNIST 是七萬張 28×28 灰階手寫數字(六萬訓練、一萬測試)，分 0 到 9 共十類。它體積小、乾淨、易上手，是所有人學習機器學習的「Hello World」，也是您課程中跨框架實作的主角。它的優點(簡單)也是它的侷限：現代模型在 MNIST 上輕鬆達到 99% 以上準確率，已無法區分演算法的優劣，因此僅適合入門教學，不再用於嚴肅的演算法評比。

14.3.2 CIFAR-10 與 CIFAR-100：邁向真實

CIFAR 資料集由多倫多大學整理，是六萬張 32×32 的彩色自然影像。CIFAR-10 分十類(飛機、汽車、鳥、貓、狗、船等)，CIFAR-100 則細分為一百類。相較 MNIST

的黑白數字，CIFAR 是彩色的真實世界物體，有複雜的背景與多變的視角，難度大幅提升，是驗證中小型模型的常用基準。從 MNIST 到 CIFAR，反映了資料集「由簡入繁、逼近真實」的演進軌跡。

14.3.3 ImageNet 與 ILSVRC：改變歷史的資料集

ImageNet 是一個劃時代的資料集，由史丹佛大學的李飛飛(Fei-Fei Li)團隊自 2009 年起建構，包含超過一千四百萬張影像、涵蓋兩萬多個類別，以人工標註。基於它舉辦的年度競賽 ILSVRC(ImageNet 大規模視覺辨識挑戰賽)，使用約一千個類別的子集，成為衡量視覺演算法的最高殿堂。

ImageNet 的歷史地位無可取代：2012 年，一個名為 AlexNet 的深度卷積神經網路在 ILSVRC 上以壓倒性差距奪冠，錯誤率大幅領先第二名，一舉引爆了深度學習革命(這正是第 15 章叫獸開講的主角)。可以說，沒有 ImageNet 這個規模空前的資料集，就沒有深度學習的爆發——它完美印證了「資料是燃料」的道理：是李飛飛「與其改進演算法、不如先建立夠大的資料集」的洞見，為 AlexNet 的成功鋪好了跑道。這也呼應本章的核心訊息：資料，常常比演算法更重要。

14.3.4 COCO：走向複雜場景

COCO(Common Objects in Context)由微軟推出，它的特點是「情境中的物件」：一張影像中包含多個物件、彼此有空間關係，且提供了比「這張圖是什麼」更豐富的標註——每個物件的精確位置(邊界框)、像素級輪廓(分割遮罩)、甚至人體關鍵點。因此 COCO 不只用於分類，更是物件偵測、實例分割、姿態估計的標準基準，直接銜接第 16 章的 YOLO。從 ImageNet(單物件分類)到 COCO(多物件偵測與分割)，又是一次「由簡入繁」的演進。

資料集	規模與尺寸	主要任務	歷史角色
MNIST	7 萬張, 28×28 灰階	手寫數字分類	機器學習的 Hello World
CIFAR-10/100	6 萬張, 32×32 彩色	自然物體分類	中小型模型基準
ImageNet	逾 1400 萬張	大規模分類(ILSVRC)	2012 引爆深度學習革命
COCO	逾 30 萬張	偵測、分割、姿態	複雜場景基準，接第 16 章

表 14-1 重要影像資料集比較

14.4 兩個經典分類器：kNN 與 SVM

14.4.1 k 最近鄰(kNN)

kNN 是最直觀的分類器，它的哲學是「物以類聚」：要判斷一個新樣本屬於哪一類，就看它在特徵空間中最靠近的 k 個鄰居，由這些鄰居「投票」決定——多數鄰居屬於哪一類，新樣本就歸為哪一類。例如 k=5，新樣本最近的五個鄰居中有三個是貓、兩個是狗，就判為貓。

kNN 有一個奇特的性質：它「不訓練」——沒有需要學習調整的參數，只是把所有訓練樣本記住，預測時才逐一計算距離找鄰居。這使它實作極簡，但也帶來缺點：預測慢(每次都要與所有訓練樣本比距離)、記憶體耗費大(要存全部訓練資料)、且對特徵的尺度與維度敏感(高維空間中「距離」會失去意義，稱為維度災難)。k 值的選擇是關鍵：k 太小容易受雜訊影響，k 太大則會把界線模糊掉——這又是一個貫穿全書的取舍。kNN 常作為理解分類概念的第一站，以及新問題的簡單基準線。

14.4.2 支持向量機(SVM)

SVM 是深度學習興起前最強大、最受推崇的分類器之一。它的核心思想是找出一條「最好的分界線」(在高維空間中稱為超平面)來分開兩類資料。什麼叫「最好」？SVM 的答案很優雅：不只要能分開，還要讓分界線離兩邊最近的樣本盡可能遠——這個「最大間隔」(maximum margin)原則，讓分類器對新樣本更有容錯能力、泛化更好。那些恰好落在間隔邊界上、決定了分界線位置的關鍵樣本，就稱為「支持向量」，SVM 之名由此而來。

SVM 還有一個威力強大的祕密武器——核技巧(Kernel Trick)：當資料在原始空間無法用直線分開時(例如一類包在另一類中間)，核技巧能把資料「隱含地」映射到更高維的空間，在那裡它們就變得線性可分了，而且計算上不必真的算出高維座標。這讓 SVM 能處理複雜的非線性分類。在深度學習普及前，「人工設計特徵(如 SIFT/HOG)+ SVM 分類」是影像辨識的黃金組合，至今在樣本較少的場合仍是可靠選擇。

程式 14-1 kNN 與 SVM 手寫數字分類

```
import cv2, numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# 載入手寫數字(類 MNIST 的小型資料集)
digits = load_digits()
X = digits.data          # 每張影像攤平成特徵向量
y = digits.target        # 標籤 0~9

# 切分訓練/測試(14.5 節的核心觀念)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# --- kNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)      # kNN 的 fit 只是記住資料
pred_knn = knn.predict(X_te)
print("kNN 準確率:", accuracy_score(y_te, pred_knn))

# --- SVM(RBF 核) ---
svm = SVC(kernel="rbf", C=10, gamma=0.001)
svm.fit(X_tr, y_tr)
pred_svm = svm.predict(X_te)
```

```
print("SVM 準確率:", accuracy_score(y_te, pred_svm))
print("SVM 混淆矩陣:\n", confusion_matrix(y_te, pred_svm))
```

14.5 資料切分與模型評估

14.5.1 訓練、驗證、測試：為什麼要三分

這是機器學習最重要、也最容易被初學者忽略的觀念：評估模型，絕不能用它訓練時看過的資料。道理很簡單——一個學生若考的是他背過答案的題目，高分不代表他真懂。模型也一樣：它可能只是「記住」了訓練資料，而非「學會」了規律。因此我們把資料切成三份：

9. 訓練集(Training Set)：用來調整模型參數，佔最大比例(如 60~80%)。
10. 驗證集(Validation Set)：在訓練過程中用來調整「超參數」(如 kNN 的 k、SVM 的 C)、選擇模型、決定何時停止訓練。它不直接參與參數調整，但會影響我們的決策。
11. 測試集(Test Set)：模型與所有決策都定案後，最後才用它評估——它模擬「模型上線後遇到全新資料」的真實表現。測試集必須嚴格保密，絕不能在開發過程中反覆使用，否則就等於間接讓模型「看過」了它，評估結果會失真、過度樂觀。

這也解釋了叫獸開講中 MNIST 的設計巧思：LeCun 團隊刻意混合「人口普查局員工(易)」與「高中生(難)」兩種來源，並確保訓練集與測試集的難度分布一致——若訓練集全是工整字跡、測試集全是潦草字跡，模型的測試表現會低估它的真實能力，反之則高估。資料切分的公平性，是可靠評估的前提。當資料量較少時，還可用「交叉驗證」(把資料輪流當訓練與驗證，取平均)來更充分地利用資料。

14.5.2 評估指標：準確率的陷阱

最直觀的指標是準確率(Accuracy)：預測正確的比例。但準確率有一個著名的陷阱——類別不平衡時會嚴重誤導。舉例：某罕見疾病的盛行率是 1%，一個「無論如何都判斷沒病」的模型，準確率高達 99%，卻毫無用處(它抓不出任何一個病人)。因此，只看準確率是危險的，必須輔以更細緻的指標。

混淆矩陣(Confusion Matrix)是評估的基礎工具：它是一張表格，列出「實際類別」與「預測類別」的所有組合，清楚顯示模型把哪些類別混淆了(例如把 4 誤判成 9)。從混淆矩陣可導出兩個關鍵指標：精確率(Precision，模型說是陽性的裡面，有多少真的是陽性——衡量「不誤報」)與召回率(Recall，所有真正的陽性中，模型抓出了多少——衡量「不漏抓」)。這兩者常有取捨：提高召回(寧可錯抓也不放過)往往降低精確(誤報變多)，反之亦然；F1 分數則是兩者的調和平均，綜合衡量。

選擇哪個指標，取決於應用的代價結構——這是工程判斷，不是數學問題。癌症篩檢重召回(漏抓一個病人的代價，遠大於多做幾次複檢)；垃圾郵件過濾重精確(把重要郵件誤判為垃圾的代價，大於偶爾漏掉一封垃圾)；這與第 10 章 Viola-Jones 的「漏檢 vs. 誤報」、第 9 章 Canny 雙門檻，是同一類「依代價權衡」的思維，貫穿整本書。

14.6 過度擬合、擬合不足與偏差-變異權衡

這是機器學習最核心的理論觀念，也是所有實務調校的指南針。它回答一個根本問題：為什麼模型在訓練資料上表現很好，到了真實世界卻不行？

14.6.1 擬合不足與過度擬合

擬合不足(Underfitting)是模型太簡單，連訓練資料的規律都抓不住——訓練表現與測試表現都差。好比用一條直線去擬合明顯彎曲的資料，怎麼畫都不貼。

過度擬合(Overfitting)則相反，是模型太複雜，把訓練資料「背」得太死，連其中的雜訊與偶然細節都當成規律學了進去——訓練表現極好，但一遇到沒看過的測試資料就崩潰。好比一個學生死背了考古題的每個答案，換個題目就不會了。過度擬合是實務中最常見、最需要警惕的問題。

診斷方法很簡單：比較模型在訓練集與驗證集上的表現。兩者都差是擬合不足；訓練好、驗證差且落差很大，就是過度擬合。理想的模型落在中間——複雜度剛好抓住真實規律，又不至於背下雜訊。

14.6.2 偏差-變異權衡

這兩種失敗的背後，是一組深刻的張力：偏差-變異權衡(Bias-Variance Tradeoff)。偏差(Bias)是模型因為太簡單而產生的系統性誤差(對應擬合不足)；變異(Variance)是模型因為太複雜、對訓練資料太敏感而產生的不穩定(對應過度擬合)。兩者往往此消彼長：提高模型複雜度會降低偏差、但增加變異；降低複雜度則反之。機器學習調校的藝術，很大程度就是在這兩者間尋找最佳平衡點——這也呼應了全書反覆出現的「取舍」主題。

14.6.3 對抗過度擬合的武器

對抗過度擬合有幾件常備武器，它們在第 15 章訓練神經網路時會反覆出現：其一，更多資料——資料愈多，模型愈難靠死背過關，這再次呼應「資料是燃料」；其二，資料增強(Data Augmentation)——對訓練影像做旋轉、平移、翻轉、調亮度(全是第 4、5 章的技術!)人工擴充資料的多樣性，這是影像領域對抗過度擬合最有效的手段之一；其三，正則化(Regularization)——限制模型的複雜度，懲罰過大的參數，這與第 7 章影像還原的正則化是同一個數學思想(以約束對抗病態)；其四，及早停止——在驗證表現開始變差時就停止訓練，不讓模型繼續往「背誦」的方向走。

請留意這裡的美妙連結：對抗過度擬合的資料增強，用的是第 4、5 章的幾何轉換與影像強化；正則化的思想來自第 7 章。機器學習不是憑空而來的新學問，它站在前面十三章的肩膀上——這正是本教科書把機器學習放在第 14 章、而非一開頭的用意。

14.7 應用案例

14.7.1 案例一：手寫數字辨識(傳統流程)

這是呼應叫獸開講、也是您課程 MNIST 實作的傳統版本。情境：辨識 0 到 9 的手寫數字，但不使用深度學習，以凸顯「特徵」這一環的重要。流程：對每張數字影像萃取特徵——可以是簡單的「像素攤平」，也可以是更聰明的 HOG(方向梯度直方圖，第 13 章特徵思想的延伸)→ 用 SVM 或 kNN 訓練分類器 → 以測試集評估、看混淆矩陣哪些數字最容易搞混(通常是 4 與 9、3 與 8、7 與 1)。這個案例讓學生親身體會：同樣的分類器，換用更好的特徵(HOG 優於原始像素)，準確率會顯著提升——這正是第 15 章深度學習要「自動學特徵」的動機所在。

14.7.2 案例二：水果或零件的品質分類

情境：農產分級(將水果依外觀分為特級、良級、次級)或工業零件的良品/不良品分類——這是機器學習在雲林在地農業與製造業最實際的應用。流程：拍攝大量已分級的樣本並標註 → 萃取特徵(第 8 章的顏色直方圖判斷成熟度、第 13 章的形狀特徵判斷外形、紋理特徵判斷表面瑕疵)→ 訓練 SVM 分類器 → 部署到分級生產線。這個案例的價值在於：它示範了前面各章的「特徵工程」如何餵給機器學習模型，是傳統影像處理與機器學習銜接的典型範例，也說明了為什麼在樣本有限的中小企業場景，「特徵工程 + SVM」常比動輒需要上萬標註樣本的深度學習更務實。

14.7.3 案例三：資料切分陷阱的真實教訓

這個案例不寫程式，而是講一個發人深省的故事，凸顯 14.5 節資料切分的重要。曾有研究團隊訓練模型從胸部 X 光辨識肺炎，在測試集上準確率極高，上線後卻表現崩壞。追查發現：訓練資料中，重症病人的 X 光多來自某幾家配備較舊機器的醫院，模型竟然「偷懶」學會了辨識影像角落的醫院標記與機器雜訊特徵，而非真正的病灶——它在資料中找到了一條與答案高度相關、卻毫無醫學意義的「捷徑」。因為測試集也來自同樣的醫院分布，這個捷徑在測試時依然有效，直到面對新醫院的資料才現形。

這個真實教訓有兩層啟示：其一，資料的收集與切分若有偏差(不同類別來自不同來源)，模型會學到「捷徑」而非真正的規律，測試表現會給出虛假的信心。其二，這也呼應了第 7 章「還原 vs. 腦補」與第 9 章醫學影像的倫理提醒——在醫療等高風險領域，一個「準確率很高」的模型未必可信，必須深究它到底學到了什麼。機器學習工程師的責任，不只是把準確率調高，更是理解模型為何做出判斷、資料是否公平。這是技術素養，也是工程倫理。

14.8 本章與全書的觀念連結總表

相關章節	共用的觀念	機器學習中的角色
第 4 章 幾何轉換	旋轉、平移、翻轉	資料增強對抗過度擬合的主要手段

第 5 章 空間域強化	亮度、對比調整	資料增強的另一組操作
第 7 章 影像還原	正則化	對抗過度擬合的正則化與病態問題正則化同源
第 8 章 色彩處理	顏色直方圖	品質分類的顏色特徵
第 13 章 特徵擷取	SIFT、HOG、形狀特徵	傳統機器學習的「特徵」環節
第 15 章 深度學習	神經網路、自動特徵	深度學習是機器學習的分支，遵循同樣的訓練評估原則
第 16 章 物件偵測	COCO、mAP	COCO 資料集;評估指標延伸自本章
第 9、10 章 分割/二值	準確、召回、誤報漏檢	評估指標的取舍思維一以貫之

表14-2 第14章與全書其他章節的觀念連結

程式實作 Lab 14

Lab 14-1 分類器擂台：kNN vs. SVM

目標：比較兩種經典分類器。(a)以程式 14-1 在手寫數字資料集上訓練 kNN 與 SVM，比較準確率與預測速度;(b)調整 kNN 的 k 值(1、5、15、50)，觀察準確率如何變化，找出最佳 k，並解釋 k 太小與太大各自的問題;(c)調整 SVM 的 C 與 gamma，觀察對結果的影響;(d)畫出兩者的混淆矩陣，找出最容易被搞混的數字對(如 4 與 9)，並思考為什麼;(e)討論：kNN 為什麼「不訓練」?這帶來什麼優點與缺點?

Lab 14-2 特徵的威力

目標：親身體會「特徵」這一環的決定性。(a)對同一個手寫數字資料集，分別用兩種特徵訓練 SVM：一是「原始像素攤平」，二是「HOG 特徵」;(b)比較兩者的準確率——通常 HOG 會明顯勝出;(c)思考：為什麼更好的特徵能提升同一個分類器的表現?這說明了傳統機器學習中「特徵工程」的重要性;(d)延伸討論：第 15 章的深度學習如何「自動」完成這件事，免去人工設計特徵的負擔?

Lab 14-3 過度擬合的現形與對抗

目標：親眼見證過度擬合。(a)刻意用一個很複雜的模型(如 SVM 的 gamma 設得很大)在少量訓練資料上訓練，記錄訓練準確率(會很高)與測試準確率(會很低)，這就是過度擬合;(b)畫出「訓練集大小 vs. 訓練/測試準確率」的學習曲線，觀察兩條線的落差如何隨資料增加而縮小——驗證「更多資料對抗過度擬合」;(c)加入資料增強(用第 4 章旋轉、第 5 章調亮度擴充訓練集)，觀察測試準確率是否提升;(d)討論：如何從學習曲線判斷一個模型是擬合不足還是過度擬合?各該怎麼補救?

Lab 14-4 品質分類器(整合前面各章)

目標：打造一個結合影像處理與機器學習的實用系統。(a)拍攝或收集兩三類物件的影像(如不同成熟度的水果、良品與不良品的零件)，各數十張並標註;(b)萃取特徵：用第 8 章顏色直方圖 + 第 13 章形狀/紋理特徵，組成特徵向量;(c)正確切分訓練/測試集，訓練 SVM 分類器;(d)以混淆矩陣、精確率、召回率評估，並依應用場景(如「不良品絕不能放過」)判斷該重召回還是重精確;(e)反思：你的資料收集有沒有落入 14.7.3 節的「捷徑陷阱」?(例如所有良品都在同一背景下拍攝，模型會不會學到背景而非物件?)

本章重點整理

- 機器學習是典範轉移：從「人寫規則」到「給範例讓機器歸納規則」;分監督式(有標籤，含分類與迴歸)、非監督式(無標籤，分群/降維)、強化學習(獎懲互動)三類，本章與深度學習聚焦監督式影像分類。
- 影像分類五環節：資料、特徵、模型、訓練、評估;傳統機器學習依賴人工設計特徵(第 13 章)，深度學習則自動學特徵——這是兩者的分野。
- 重要資料集：MNIST(入門 Hello World)、CIFAR(彩色自然物體)、ImageNet/ILSVRC(2012 引爆深度學習)、COCO(複雜場景偵測分割);資料是機器學習的燃料，常比演算法更決定成敗。
- kNN 以最近 k 個鄰居投票、不訓練但預測慢;SVM 以最大間隔超平面分類，核技巧處理非線性，是深度學習前的黃金分類器。
- 資料須切分訓練/驗證/測試，測試集嚴格保密以模擬真實表現;準確率在類別不平衡時會誤導，須輔以混淆矩陣、精確率(不誤報)、召回率(不漏抓)，依應用代價權衡。
- 擬合不足(太簡單、訓練測試都差)、過度擬合(太複雜、背下雜訊、訓練好測試差)源於偏差-變異權衡;對抗過度擬合靠更多資料、資料增強(第 4、5 章)、正則化(第 7 章)、及早停止。

習題

- 機器學習相對於傳統程式設計，發生了什麼「典範轉移」?為什麼有些問題非用機器學習不可?

提示與參考解答：傳統程式：人寫死規則，電腦套用到資料得答案。機器學習：人給資料和答案，電腦自己歸納出規則。轉移在於「從寫規則到給範例」。有些問題(辨認手寫數字、人臉、語音)的規則複雜到人類根本寫不出來，但範例俯拾皆是，只能讓機器從範例自己學。

- 請說明監督式、非監督式、強化學習的差別，各舉一個影像或機器人相關的例子。

提示與參考解答：監督式：資料有輸入與正確標籤，學習預測答案(如 MNIST 數字分類)。非監督式：資料無標籤，自己發現結構(如把相似影像自動分群)。強化學習：智能體與環境互動、依獎懲學策略(如機器人學會抓取物件、AlphaGo 下圍棋)。

3. 影像分類的五個環節是什麼?傳統機器學習與深度學習在哪一個環節產生了根本差異?

提示與參考解答：五環節：資料、特徵、模型、訓練、評估。根本差異在「特徵」：傳統機器學習依賴人工設計特徵(SIFT、HOG)，模型只做「特徵到類別」；深度學習把「像素到特徵」與「特徵到類別」整合進同一網路，端到端自動學習特徵。

4. 為什麼說「資料是機器學習的燃料」?請以 ImageNet 對深度學習的影響說明。

提示與參考解答：演算法是引擎、資料是燃料，沒燃料引擎跑不動。ImageNet(李飛飛團隊，逾1400萬張影像)提供了規模空前的訓練資料，2012年AlexNet正是靠它在ILSVRC奪冠、引爆深度學習革命。沒有ImageNet就沒有深度學習爆發——是「先建大資料集」的洞見為演算法鋪好跑道，印證資料常比演算法更重要。

5. 承叫獸開講：MNIST 為什麼要混合「人口普查局員工」與「高中生」兩種來源的手寫數字?這與資料切分有什麼關聯?

提示與參考解答：普查局員工字跡工整(較易)、高中生字跡潦草(較難)，混合是為了讓訓練集與測試集的難度分布一致，結論才可靠。若訓練全易、測試全難會低估模型能力，反之則高估。這正是資料切分公平性的體現——訓練與測試的分布必須一致，評估才可信。

6. KNN 的分類原理是什麼?為什麼說它「不訓練」?它有哪些缺點?

提示與參考解答：原理：看新樣本在特徵空間最近的k個鄰居，由它們投票決定類別(物以類聚)。「不訓練」是因為它沒有需學習調整的參數，只把訓練樣本記住，預測時才算距離找鄰居。缺點：預測慢(每次都要比全部樣本)、記憶體耗費大、對特徵尺度與高維敏感(維度災難)。

7. SVM 的「最大間隔」原則是什麼?為什麼它有助於泛化?什麼是核技巧?

提示與參考解答：最大間隔：不只要能分開兩類，還要讓分界線(超平面)離兩邊最近樣本盡可能遠；決定分界線的關鍵樣本即支持向量。它有助泛化，因為留有餘裕、對新樣本更有容錯能力。核技巧：當資料在原空間線性不可分時，隱含地映射到更高維空間使其線性可分，且不必真的計算高維座標，讓SVM能處理非線性分類。

8. 為什麼評估模型不能用訓練時看過的資料?訓練、驗證、測試三個集合各扮演什麼角色?

提示與參考解答：用看過的資料評估，無法分辨模型是「學會規律」還是只「背下答案」(如考背過的題)。訓練集調整參數、驗證集調超參數、選模型、決定何時停止；測試集在一切定案後最後評估，模擬上線遇到全新資料的表現，須嚴格保密不可反覆使用，否則評估會過度樂觀。

9. 準確率在什麼情況下會誤導?請以罕見疾病為例，並說明精確率與召回率如何補救。

提示與參考解答：類別不平衡時誤導：某病盛行率1%，「一律判沒病」的模型準確率99%卻抓不出任何病人。精確率(判為陽性中真陽性的比例，衡量不誤報)與召回率(真陽性中被抓出的比例，衡量不漏抓)能揭示這個問題；癌症篩檢重召回(漏抓代價大)，垃圾郵件重精確(誤判重要郵件代價大)。

10. 過度擬合與擬合不足各是什麼?如何從訓練與驗證表現診斷?它們與偏差-變異權衡有何關係?

提示與參考解答：擬合不足：模型太簡單、抓不住規律，訓練與測試都差(高偏差)。過度擬合：模型太複雜、背下雜訊，訓練好但測試崩壞(高變異)。診斷：兩者都差

是擬合不足;訓練好、驗證差且落差大是過度擬合。偏差-變異權衡:提高複雜度降偏差但增變異,反之亦然,調校即在兩者間找平衡。

11. 對抗過度擬合有哪四種常備武器?其中資料增強與正則化分別與本書哪幾章相關?

提示與參考解答:四武器:更多資料、資料增強、正則化、及早停止。資料增強用第4章幾何轉換(旋轉、平移、翻轉)與第5章影像強化(調亮度、對比)擴充訓練資料多樣性;正則化(限制模型複雜度、懲罰過大參數)與第7章影像還原的正則化同源——都是以約束對抗病態/過度擬合。

12. (綜合思辨題)承 14.7.3 節肺炎 X 光的教訓:一個測試準確率很高的模型,為什麼上線後可能崩壞?這給機器學習工程師什麼責任上的啟示?

提示與參考解答:模型可能學到與答案高度相關、卻無實質意義的「捷徑」(如醫院標記、機器雜訊),而非真正的病灶。若測試集與訓練集來自同樣的偏差分布,捷徑在測試時仍有效,直到面對新分布才現形。啟示:資料收集與切分若有偏差,模型會學捷徑、測試給出虛假信心;工程師的責任不只是調高準確率,更要理解模型為何做出判斷、資料是否公平——這是技術素養,也是工程倫理(呼應第7、9章)。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 12(物件辨識與分類基礎)。
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P., Gradient-Based Learning Applied to Document Recognition, Proc. IEEE, 1998.(LeNet 與 MNIST, 本章叫獸開講之史料來源)
- Deng, J., et al. (Li, F.-F.), ImageNet: A Large-Scale Hierarchical Image Database, CVPR, 2009.
- Cortes, C., & Vapnik, V., Support-Vector Networks, Machine Learning, 1995.(SVM 原始論文)
- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.
- Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.(機器學習經典教科書)
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 15 章 深度學習

本章學習目標

- 說明從手工特徵到學習特徵的典範轉移，理解深度學習「端到端」的核心精神。
- 推導神經網路的基本構件：神經元、激勵函數、損失函數與反向傳播。
- 闡述卷積神經網路(CNN)的三大構件：卷積層、池化層、全連接層，理解其為何適合影像。
- 認識經典 CNN 架構譜系 (LeNet→AlexNet→VGG→GoogLeNet→ResNet→MobileNet)的演進脈絡。
- 以三大框架(TensorFlow、Keras、PyTorch)實作 MNIST 手寫數字分類。
- 運用遷移學習與資料增強，在小資料上訓練出高效能的影像模型。

叫獸開講|黃仁勳與 AlexNet：兩顆遊戲顯卡掀起的革命

2012 年 9 月，電腦視覺界發生了一場地震。在權威的 ImageNet 影像辨識競賽(ILSVRC)上，多年來各隊都靠著人工設計特徵配上傳統分類器，辛苦地把錯誤率一個百分點、一個百分點地往下磨，當時的最佳成績卡在約 26% 的 top-5 錯誤率。然後，來自多倫多大學的一支隊伍——Alex Krizhevsky、Ilya Sutskever，以及他們的指導教授、後來被稱為「深度學習教父」的 Geoffrey Hinton——投下了一顆震撼彈：他們的模型把錯誤率一舉壓到 15.3%，足足領先第二名近 11 個百分點。在一個習慣以零點幾個百分點論輸贏的領域，這個差距簡直是不同次元的存在。這個模型，就是後來以第一作者命名的 AlexNet。

AlexNet 憑什麼？它其實是一個八層的卷積神經網路(五層卷積、三層全連接)，概念上並非全新——它的祖先正是第 14 章 LeCun 讀支票的 LeNet。真正讓它脫胎換骨的是三個關鍵：第一，它用了 ReLU 激勵函數，讓深層網路的訓練快了好幾倍；第二，它用了 Dropout 這個新招來對抗過度擬合(第 14 章的老朋友)；第三——也是最出人意料的——它是在兩張 NVIDIA GTX 580 上訓練的。沒錯，是遊戲玩家用來跑 3A 大作的那種顯示卡。

這第三點，正是這個故事最迷人的轉折。原本為了「讓遊戲畫面更逼真」而設計的圖形處理器(GPU)，因為天生擅長大量平行的數值運算，竟意外地與神經網路的訓練需求完美契合——訓練神經網路，本質上就是海量的矩陣乘法，而這正是 GPU 的看家本領。AlexNet 團隊用兩張遊戲顯卡，把原本需要數週的訓練壓縮到五、六天，讓「訓練一個夠深的網路」從不切實際變成了可行。這個發現讓一家原本以電玩顯卡聞名的公司——NVIDIA，以及它的創辦人黃仁勳——站上了 AI 時代的浪頭。黃仁勳多次談到，AlexNet 是 NVIDIA 全力轉向 AI 運算的關鍵時刻：公司從此不再只是賣遊戲顯卡，而是打造整個 AI 運算的基礎設施。今天訓練大型

語言模型的資料中心裡，滿滿都是 NVIDIA 的晶片——這條路的起點，就是 2012 年那兩張跑 AlexNet 的 GTX 580。

AlexNet 之後，深度學習如野火燎原：Google、Facebook、微軟、百度紛紛重金投入，ImageNet 的冠軍模型愈疊愈深，到 2015 年 ResNet 的錯誤率已低於人類水準。這場革命的三大燃料——大數據(第 14 章的 ImageNet)、強大算力(GPU)、以及深層網路架構——缺一不可。而它給我們的最大啟示，呼應了第 13、14 章一路鋪陳的主軸：深度學習最革命性的地方，不是它「更準」，而是它讓機器自己學會了特徵。AlexNet 較低的網路層自動長出了邊緣與顏色的偵測器，較高的層則認得出輪子、狗臉——這些過去要靠人類專家嘔心瀝血設計(SIFT、HOG)的東西，網路從資料中自己學會了。這一章，我們就來理解這場革命的技术核心。

思考題：GPU 原本是為了電玩遊戲的畫面而設計，卻意外成為 AI 革命的引擎。這種「一項技術在原本用途之外找到革命性新應用」的現象，你還能想到哪些例子？這對「基礎技術的投資」有什麼啟示？

15.1 從手工特徵到學習特徵：典範轉移

第 13.7 節與第 14 章已經多次預告這場典範轉移，現在正式展開。回顧傳統的影像辨識流程(第 14.2 節的五環節)：人類專家設計特徵(第 13 章的 SIFT、HOG)，再把特徵餵給分類器(第 14 章的 SVM)。這裡的瓶頸顯而易見——特徵是人設計的，而人的設計能力有限：我們能想出邊緣、角點、梯度方向，但面對「什麼特徵能區分狗和狼」這種問題，人類的直覺很快就見底了。

深度學習的核心突破，是把「特徵設計」這件事，從人類手中交給了機器。一個深度神經網路，把「從像素到特徵」與「從特徵到類別」整合進同一個模型，端到端(end-to-end)一起訓練：你只需給它影像與答案，它會自己學出「什麼特徵最有助於分類」。這就是叫獸開講中最關鍵的啟示——AlexNet 的網路層自動長出了邊緣偵測器與物件辨識器，無需人類插手。

而這正呼應了第 6.6.7 節與第 13.7 節埋下的伏筆：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣與方向性帶通濾波器，與人工設計的 Gabor、SIFT 驚人地相似。深度學習沒有推翻前面十四章的智慧，而是「自動重新發明」了它，並能組合出人類難以手工設計的、更抽象的高層特徵。這也是為什麼要先學完前面各章：唯有理解手工特徵，才能真正看懂 CNN 到底學到了什麼、為什麼有效。

15.2 神經網路的基本構件

15.2.1 神經元與激勵函數

神經網路的靈感來自生物神經元。一個人工神經元做的事很簡單：把多個輸入各自乘上一個權重(weight)、加總、再加上一個偏置(bias)，然後通過一個激勵函數(activation function)輸出。權重與偏置就是神經元「學習」的對象——訓練的過程，就是不斷調整這些數值，讓網路的輸出愈來愈接近正確答案。

激勵函數為網路注入了「非線性」，這一點至關重要：若沒有它，無論疊多少層整個網路都只是一連串線性運算的組合，等效於單一層，無法學習複雜的規律。常見的激勵函數有：Sigmoid(把輸出壓到 0 與 1 之間，早期常用，但在深層網路會導致梯度消失)、Tanh(類似 Sigmoid 但範圍是 -1 到 1)、以及 ReLU(修正線性單元，負值歸零、正值保留)。ReLU 正是叫獸開講中 AlexNet 的關鍵創新之一——它計算極簡、且大幅緩解了梯度消失問題，讓深層網路的訓練從「幾乎不可能」變得「又快又穩」，至今仍是最主流的選擇。

15.2.2 多層感知器

把許多神經元排成一層、再把多層堆疊起來，就構成了多層感知器(MLP)：輸入層接收資料、若干隱藏層逐層萃取愈來愈抽象的表示、輸出層給出最終預測(例如十個輸出對應 MNIST 的十個數字)。「深度學習」的「深」，指的正是隱藏層的層數多——層數愈多，網路能表達的函數愈複雜，但也愈難訓練(這正是 AlexNet 之前深層網路難以成功的原因)。

15.2.3 損失函數與梯度下降

網路怎麼知道自己預測得好不好?靠損失函數(loss function)：它量化「預測」與「正確答案」之間的差距——差距愈大、損失愈高。分類問題常用交叉熵損失(cross-entropy)，迴歸問題常用均方誤差(MSE，正是第 3 章的老朋友)。訓練的目標，就是調整所有權重與偏置，讓損失盡可能小。

怎麼調?靠梯度下降(gradient descent)。想像損失是一片高低起伏的地形，而我們走到最低點(最小損失)。梯度指出了「當前位置最陡上坡的方向」，那麼往反方向(下坡)走一小步，損失就會下降一點;反覆走，終能逼近谷底。每一步的大小由「學習率」(learning rate)控制——太大會在谷底附近來回震盪甚至衝過頭，太小則收斂慢得令人絕望，這又是一個貫穿全書的取舍。實務上用「隨機梯度下降」(SGD)及其改良版(如 Adam)，每次只用一小批(mini-batch)資料估計梯度，兼顧效率與穩定。

15.2.4 反向傳播

最後一塊拼圖是反向傳播(backpropagation)——它回答了「如何算出每一個權重對損失的影響(梯度)」這個關鍵問題。它的核心是微積分的連鎖律：先前向計算得到預測與損失，再從輸出層往輸入層「逆向」逐層傳遞誤差，依連鎖律算出每個權重的梯度。這個演算法讓「調整深層網路裡數百萬個參數」變得可行，是神經網路能夠訓練的數學引擎。LeCun 在 1989 年正是用反向傳播訓練了 LeNet(第 14 章)，而 AlexNet 也建立在同一個基礎上。所幸，今天的深度學習框架(15.5 節)會自動完成反向傳播的所有計算，使用者無需手動推導——但理解它的原理，能幫助你診斷訓練為何失敗(如梯度消失或爆炸)。

15.3 卷積神經網路(CNN)

多層感知器有一個致命問題：它把影像「攤平」成一長串像素再處理，徹底破壞了影像的空間結構(相鄰像素的關聯性)，而且參數量爆炸——一張小小的 224×224

彩色圖攤平就是十五萬個輸入，若第一層隱藏層有一千個神經元，光第一層就要一億五千萬個權重。卷積神經網路(CNN)正是為影像量身打造，優雅地解決了這兩個問題，也是叫獸開講中 AlexNet 的骨幹。

15.3.1 卷積層：本書最熟悉的老朋友

CNN 的核心是卷積層，而「卷積」正是我們從第 5、6 章一路認識到現在的老朋友！卷積層用一組小小的卷積核(filter)在影像上滑動，計算局部區域的加權和——這與第 5 章的空間濾波在數學上是同一件事。差別只有一個、卻是革命性的：第 5 章的濾波核是人設計的(高斯核、Sobel 核)，而 CNN 的卷積核是學出來的——網路透過訓練自己決定核裡的每個數值，以萃取出最有助於任務的特徵。

這個設計帶來三個關鍵優勢。其一，局部連接：每個卷積核只看局部區域，符合「影像的意義來自局部結構(邊緣、紋理)」的直覺。其二，權重共享：同一個卷積核滑遍整張影像，用同一組權重——這既大幅減少參數(一個 3×3 核只有 9 個權重，不管影像多大)，又帶來平移不變性(物件出現在哪裡都能被同一個核偵測到)。其三，層級特徵：淺層的卷積核學到邊緣、顏色等低階特徵，深層的則組合出紋理、部件乃至完整物件等高階特徵——這正是叫獸開講中 AlexNet 「淺層學邊緣、深層認物件」的由來，也再次呼應第 6、11 章的多解析度、由粗到細的思想。

15.3.2 池化層

池化層(pooling)負責「降取樣」——縮小特徵圖的尺寸。最常見的最大池化(max pooling)把每個小區域(如 2×2)只保留最大值，尺寸減半。它有三個作用：減少後續運算量與參數、擴大後續卷積核的「視野」(讓深層能看到更大範圍)、並提供一定的位置容錯(物件稍微移動，池化結果變化不大)。有趣的是，最大池化與最小池化，本質上就是第 10 章的型態學膨脹與侵蝕——又一次一體多面的印證。

15.3.3 全連接層與整體結構

一個典型的 CNN，是「卷積層 + 池化層」的多次堆疊(負責萃取由低階到高階的特徵)，最後接上幾層全連接層(第 15.2 節的 MLP，負責把萃取到的特徵映射到最終的類別)。這個「卷積池化萃取特徵、全連接做決策」的結構，從 1998 年的 LeNet 到 2012 年的 AlexNet 一脈相承，也是下一節架構譜系的共同骨架。

15.4 經典 CNN 架構譜系

自 AlexNet 引爆革命後，CNN 架構快速演進，每一代都針對前一代的瓶頸提出突破。理解這條譜系，就是理解深度學習視覺這十餘年的發展主線。

1. LeNet(1998, LeCun): CNN 的開山始祖，約七層，用於 MNIST 手寫數字辨識(第 14 章)。它奠定了「卷積-池化-全連接」的基本範式，但受限於當年的資料與算力，未能大放異彩。
2. AlexNet(2012, Krizhevsky 等): 叫獸開講的主角，八層。以 ReLU、Dropout、GPU 訓練與資料增強，在 ImageNet 上一舉成名，引爆深度學習革命。它證明了「夠深的網路 + 大數據 + GPU」的威力。

3. VGGNet(2014, 牛津): 把網路加深到 16、19 層, 並確立了一個優雅的原則——全部只用小小的 3×3 卷積核堆疊。它證明了「深度」本身的價值, 結構規整、易於理解, 至今仍是教學與遷移學習的常用骨幹。
4. GoogLeNet / Inception(2014, Google): 同年冠軍, 提出「Inception 模組」——在同一層並行使用多種尺寸的卷積核(1×1 、 3×3 、 5×5), 讓網路自己選擇適合的尺度(呼應多尺度思想); 並用 1×1 卷積大幅壓縮參數, 以 22 層的深度卻比 AlexNet 更省參數。
5. ResNet(2015, 微軟何愷明等): 解決了一個致命瓶頸——網路太深時, 梯度難以傳遞、反而更難訓練(退化問題)。它的神來之筆是「殘差連接」(skip connection): 讓資訊可以「跳過」若干層直接往後傳, 使得訓練上百層的網路成為可能。ResNet 在 ImageNet 上的錯誤率首度低於人類水準(約 5%), 是深度學習史上又一里程碑, 殘差連接的思想至今無所不在(包括 Transformer)。
6. MobileNet(2017, Google): 前面的架構愈做愈深、愈準, 但也愈龐大、愈耗算力, 無法在手機或嵌入式裝置上執行。MobileNet 反其道而行, 以「深度可分離卷積」大幅壓縮運算量與模型大小, 在可接受的精度損失下, 讓 CNN 能在手機、ESP32、Jetson 等資源受限的邊緣裝置上即時執行——這正是第 17 章邊緣部署的關鍵, 也呼應第 9、10、13 章反覆強調的「適合勝於最強」。

這條譜系揭示了一個清晰的張力：一邊是「愈深愈準」的極致追求(VGG、ResNet), 一邊是「夠好且能部署」的務實考量(MobileNet)。成熟的工程師懂得依場景選擇——雲端伺服器追求極致精度, 邊緣裝置追求效率, 沒有一個架構適用於所有場合。這也是為什麼您的教材要完整涵蓋整條譜系：它們不是「新的取代舊的」, 而是各自佔據不同的應用生態位。

架構	年份	關鍵創新	歷史意義
LeNet	1998	卷積-池化-全連接範式	CNN 開山始祖(讀支票)
AlexNet	2012	ReLU、Dropout、GPU 訓練	引爆深度學習革命
VGGNet	2014	全用 3×3 小核堆疊加深	證明深度的價值, 結構規整
GoogLeNet	2014	Inception 多尺度、 1×1 壓縮	更深卻更省參數
ResNet	2015	殘差連接(跳接)	可訓練上百層, 錯誤率低於人類
MobileNet	2017	深度可分離卷積	輕量化, 可在邊緣裝置執行

表15-1 經典CNN 架構譜系

15.5 三大框架與 MNIST 實作

手動實作神經網路(尤其是反向傳播)極為繁瑣, 所幸有深度學習框架代勞——它們自動處理梯度計算、GPU 加速、以及大量現成的網路元件。三大主流框架各有特色:

7. TensorFlow(Google,2015): 工業界部署的重鎮, 生態完整, 尤其擅長大規模生產環境與行動端(TensorFlow Lite 可部署到手機與 ESP32, 呼應第 17 章)。
8. Keras: 原為獨立的高階 API、現已整合進 TensorFlow, 以極簡潔的語法著稱, 幾行程式就能搭出一個網路, 是初學者與快速原型的首選——也是您課程 MNIST 教學的理想起點。
9. PyTorch(Meta,2016): 學術研究的主流, 以直觀的「動態計算圖」與貼近 Python 的風格深受研究者喜愛, 除錯容易, 近年在產業界也快速普及。

您的教材採用「TensorFlow → Keras → PyTorch」的跨框架 MNIST 實作, 用意深遠: 同一個手寫數字分類任務, 用三種框架各做一遍, 學生能清楚看見「網路架構的思想是共通的, 只是語法不同」——這培養的是不被單一工具綁架的核心理解力。以下以 Keras 示範最簡潔的版本。

程式 15-1 Keras 實作 MNIST CNN 分類器

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. 載入 MNIST(第 14 章的經典資料集)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0  # 正規化到 0~1
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

# 2. 建構 CNN(卷積-池化-全連接, 15.3 節的結構)
model = keras.Sequential([
    layers.Conv2D(32, 3, activation="relu", input_shape=(28,28,1)),
    layers.MaxPooling2D(),  # 池化降取樣
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),  # 攤平接全連接
    layers.Dropout(0.5),  # 對抗過度擬合(第 14 章)
    layers.Dense(10, activation="softmax")  # 10 類輸出
])

# 3. 編譯: 指定損失函數與最佳化器(15.2 節)
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 4. 訓練
model.fit(x_train, y_train, epochs=5, batch_size=128,
         validation_split=0.1)  # 切出驗證集(第 14 章)

# 5. 評估(用沒看過的測試集, 第 14 章)
loss, acc = model.evaluate(x_test, y_test)
print(f"測試準確率: {acc:.4f}")  # 通常可達 99% 以上
```

請留意這段程式如何綜合了前兩章的觀念：正規化(第 3 章)、卷積與池化(本章)、Dropout 對抗過度擬合(第 14 章)、切驗證集與用測試集評估(第 14 章)、交叉熵損失與 Adam 最佳化器(本章)。短短二十行，是全書多章觀念的結晶。

15.6 遷移學習與資料增強

15.6.1 遷移學習：站在巨人的肩膀上

訓練一個像 ResNet 這樣的大型網路，需要 ImageNet 那樣的百萬級資料與大量 GPU 時間——這對絕大多數應用(尤其是資料稀少的中小企業與學生專題)是不切實際的。遷移學習(Transfer Learning)是這個困境的救星，其思想極為漂亮：一個在 ImageNet 上訓練好的網路，它較低的層已經學會了通用的視覺特徵(邊緣、紋理、形狀)——這些特徵對任何影像任務都有用，不必從頭再學。

因此，我們可以「借用」一個預訓練模型：保留它已經學好的特徵萃取層(凍結權重)，只替換並重新訓練最後負責分類的幾層，使其適應我們自己的任務。這樣一來，即使只有幾百張自己的影像，也能訓練出高效能的模型——因為困難的「學特徵」部分已經由巨人(ImageNet 預訓練)完成了，我們只需教會它「把這些特徵對應到我的類別」。這呼應第 14 章「資料是燃料」：遷移學習讓你借用別人燒過的燃料所產生的成果。對雲林在地的農業分級、工業檢測等樣本有限的應用，遷移學習往往是比從頭訓練更務實、更高效的選擇。

15.6.2 資料增強：第 4、5 章的華麗回歸

第 14 章介紹過，資料增強是對抗過度擬合最有效的手段之一，而它用的正是本書前面的技術！對訓練影像做隨機的旋轉、平移、翻轉(第 4 章幾何轉換)、亮度與對比調整(第 5 章空間域強化)、乃至加入雜訊(第 7 章)——每一種變換都產生一張「新」的訓練樣本，人工擴充了資料的多樣性，逼迫模型學習真正穩健的特徵(而非死背某張圖的細節)。

叫獸開講中的 AlexNet 正是資料增強的早期實踐者——它用隨機裁切、水平翻轉、色彩擾動來擴充 ImageNet，顯著降低了過度擬合。這個美妙的連結值得再次強調：深度學習訓練中不可或缺的資料增強，骨子裡就是第 4、5、7 章的影像處理技術。這再次證明，深度學習不是憑空而來的新學問，而是穩穩地站在傳統影像處理的地基之上——這正是本教科書把深度學習放在最後、而非一開頭的深意。

程式 15-2 遷移學習與資料增強

```
from tensorflow import keras
from tensorflow.keras import layers

# --- 資料增強:全是第 4、5、7 章的影像處理技術 ---
augment = keras.Sequential([
    layers.RandomFlip("horizontal"),      # 第 4 章:翻轉
    layers.RandomRotation(0.1),           # 第 4 章:旋轉
    layers.RandomZoom(0.1),               # 第 4 章:縮放
    layers.RandomContrast(0.2),           # 第 5 章:對比
])
```

```
# --- 遷移學習:借用 ImageNet 預訓練的 MobileNetV2 ---
base = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,          # 去掉原本的分類層
    weights="imagenet")        # 載入預訓練權重
base.trainable = False         # 凍結:保留學好的通用特徵

model = keras.Sequential([
    augment,                    # 先做資料增強
    base,                      # 借用的特徵萃取器
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(5, activation="softmax") # 換成自己的 5 類
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
# 只需少量自己的資料即可訓練出高效能模型
```

15.7 深度學習的侷限與展望

深度學習威力驚人，但絕非萬靈丹。理解它的侷限，是負責任地使用它的前提，也呼應了第 7、14 章一路強調的工程倫理。

第一，資料飢渴：深度學習需要大量標註資料，取得成本高昂；遷移學習與資料增強能緩解，但無法完全消除。第二，算力昂貴：訓練大型模型耗費可觀的電力與硬體，有其環境與成本代價。第三，黑箱難解：深度網路的決策過程難以解釋——它為什麼判斷這是貓？我們往往說不清。這在醫療、司法等高風險領域是嚴重問題（呼應第 14 章的肺炎 X 光捷徑陷阱：模型可能學到與答案相關卻無意義的捷徑，而我們難以察覺）。第四，對抗脆弱性：對影像加入人眼難以察覺的微小擾動，可能讓模型完全誤判——這在安全攸關的應用（如自駕、人臉辨識門禁）中是真實的風險。第五，分布偏移：模型在訓練分布外的資料上表現會急劇下滑（第 7.7.3 節已詳述）。

展望未來，幾個方向正在改變格局：Transformer 架構（源於自然語言處理）已進入視覺領域（Vision Transformer），在大資料上表現超越 CNN；自監督學習試圖擺脫對人工標註的依賴，讓模型從無標籤資料中自學；而生成式 AI（擴散模型、多模態大模型）則模糊了「辨識」與「創造」的界線。但無論架構如何演進，本章的基礎——神經元、卷積、損失、梯度下降、反向傳播、以及對抗過度擬合的智慧——都是理解這一切的根基。技術的表層日新月異，底層的原理歷久彌新。

15.8 本章與全書的觀念連結總表

相關章節	共用的觀念	深度學習中的角色
第 3 章 數位影像基礎	正規化、MSE	輸入正規化；MSE 為迴歸損失函數
第 4、5 章 幾	旋轉、翻轉、亮度對比	資料增強的具體操作

何/強化		
第 5、6 章 濾波/頻率	卷積、濾波核	卷積層即可學習的濾波核;首層自發學成邊緣/帶通
第 7 章 影像還原	正則化、分布偏移	對抗過度擬合;深度學習的分布偏移侷限
第 10 章 二值影像	膨脹、侵蝕	最大/最小池化即型態學膨脹/侵蝕
第 11 章 小波	多解析度、由粗到細	CNN 層級特徵與多解析度分析同源
第 13 章 特徵擷取	手工特徵、Gabor	CNN 自動學出特徵，首層重現手工特徵
第 14 章 機器學習	訓練評估、過度擬合、資料集	深度學習是機器學習的分支，共用全部原則
第 16、17 章	YOLO、邊緣部署	CNN 為 YOLO 骨幹;MobileNet 供邊緣部署

表15-2 第15 章與全書其他章節的觀念連結

程式實作 Lab 15

Lab 15-1 跨框架 MNIST 三部曲

目標：體會「架構思想共通、語法各異」。(a)以程式 15-1 用 Keras 訓練 MNIST CNN，達到 99% 以上準確率;(b)用 PyTorch 實作同樣架構的網路，比較兩者的程式風格(Keras 的宣告式 vs. PyTorch 的命令式);(c)用純 TensorFlow(低階 API)實作，體會框架自動化了多少細節;(d)三者訓練同一任務，比較準確率是否一致(應該相近)、程式碼行數與可讀性;(e)討論：為什麼說「不被單一工具綁架的核心理解力」比熟練某個框架更重要？

Lab 15-2 卷積核視覺化

目標：親眼見證叫獸開講「網路自動學出特徵」的核心。(a)訓練一個 CNN 後，把第一層卷積核視覺化成小圖;(b)觀察它們是否自發演化成邊緣偵測器、方向性條紋(對照第 6 章 Gabor、第 13 章 SIFT);(c)把一張影像餵入網路，視覺化各層的「特徵圖」(feature map)，觀察淺層與深層分別對什麼有反應;(d)討論：這與第 13.7 節「CNN 重新發明手工特徵」的說法是否吻合?為什麼理解手工特徵有助於理解 CNN?

Lab 15-3 遷移學習實戰

目標：用少量資料訓練高效能模型。(a)收集自己的小資料集(如三到五類水果或零件，每類數十張);(b)以程式 15-2 借用 ImageNet 預訓練的 MobileNetV2，凍結特徵層、只訓練分類層;(c)對照組：用同樣的資料從頭訓練一個 CNN，比較兩者的準確率與訓練時間——遷移學習應在小資料上明顯勝出;(d)加入資料增強，觀察測試準

確率的提升;(e)討論：對雲林在地的農業或製造業應用，為什麼遷移學習加資料增強常比從頭訓練更務實？

Lab 15-4 過度擬合的深度學習版

目標：在深度學習情境重現第 14 章的過度擬合診斷。(a)訓練一個 CNN，畫出訓練與驗證的準確率/損失隨 epoch 變化的曲線;(b)觀察過度擬合的典型徵兆：訓練準確率持續上升、但驗證準確率停滯甚至下降;(c)分別加入 Dropout、資料增強、及早停止，觀察各自如何改善過度擬合(呼應第 14 章四大武器);(d)討論：深度學習的過度擬合對抗手段，與第 14 章的傳統機器學習有何異同？為什麼深度網路特別容易過度擬合？

本章重點整理

- 深度學習的核心典範轉移：把特徵設計從人類手中交給機器，端到端一起學習；CNN 首層自發學出的濾波器與第 6、13 章的手工特徵(Gabor、SIFT)高度相似——深度學習是「自動重新發明」了手工特徵。
- 神經網路構件：神經元(加權求和加激活)、激勵函數(注入非線性，ReLU 為主流且緩解梯度消失)、損失函數(交叉熵/MSE 量化誤差)、梯度下降(往損失下坡走，學習率控制步長)、反向傳播(以連鎖律逆向算梯度，框架自動完成)。
- CNN 三大構件：卷積層(可學習的濾波核，局部連接、權重共享、層級特徵)、池化層(降取樣，即型態學膨脹/侵蝕)、全連接層(做最終決策)；專為影像設計，解決 MLP 破壞空間結構與參數爆炸的問題。
- 架構譜系：LeNet(範式開創)→AlexNet(ReLU/Dropout/GPU，引爆革命)→VGG(3×3 加深)→GoogLeNet(Inception 多尺度)→ResNet(殘差連接，低於人類錯誤率)→MobileNet(深度可分離卷積，可邊緣部署)；「愈深愈準」與「夠好可部署」兩條路線並存。
- 三大框架 TensorFlow(部署)、Keras(簡潔入門)、PyTorch(研究)；跨框架 MNIST 實作培養不被工具綁架的核心理解；程式綜合了正規化、卷積池化、Dropout、驗證測試切分等全書觀念。
- 遷移學習借用預訓練模型的通用特徵、只重訓分類層，少量資料即可高效；資料增強(第 4、5、7 章的影像處理技術)對抗過度擬合。深度學習侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱、分布偏移——負責任使用需正視這些。

習題

1. 深度學習相對於傳統影像辨識(第 13、14 章)，最革命性的改變是什麼？請以「特徵」說明。

提示與參考解答：最革命性的是把「特徵設計」從人類手中交給機器：傳統流程由人設計特徵(SIFT、HOG)再餵給分類器，受限於人的設計能力；深度學習把「像素到特徵」與「特徵到類別」整合進同一網路端到端學習，自己學出最有助於任務的特徵。如叫獸開講，AlexNet 網路層自動長出邊緣偵測器與物件辨識器。

2. 為什麼激勵函數是必要的？若把所有激勵函數拿掉，深層網路會變成什麼？ReLU 相對 Sigmoid 有何優勢？

提示與參考解答：激勵函數注入非線性。若拿掉，無論疊多少層都只是一連串線性運算的組合，等效於單一層，無法學複雜規律。ReLU(負值歸零、正值保留)相對 Sigmoid：計算極簡、且大幅緩解梯度消失問題，讓深層網路訓練又快又穩，是 AlexNet 的關鍵創新之一。

3. 用「地形與下坡」的比喻說明梯度下降。學習率太大或太小分別會怎樣？

提示與參考解答：損失是一片高低起伏的地形，目標是走到最低點(最小損失)。梯度指出最陡上坡方向，往反方向(下坡)走一小步損失就下降，反覆走逼近谷底。學習率控制步長：太大會在谷底附近震盪甚至衝過頭，太小則收斂慢得令人絕望——又一個取舍。

4. 反向傳播解決了什麼問題？它的數學核心是什麼？為什麼理解它仍然重要(即使框架會自動完成)？

提示與參考解答：它解決「如何算出每個權重對損失的影響(梯度)」的問題，使調整深層網路數百萬參數變得可行，是神經網路能訓練的數學引擎。核心是微積分的連鎖律：前向算損失，再從輸出往輸入逆向逐層傳遞誤差算梯度。框架雖自動完成，但理解它能幫助診斷訓練失敗的原因(如梯度消失或爆炸)。

5. 多層感知器直接處理影像有哪兩個致命問題？CNN 如何分別解決？

提示與參考解答：問題一：把影像攤平破壞了空間結構(相鄰像素關聯)。問題二：參數量爆炸(小圖攤平就數萬輸入，全連接層權重上億)。CNN 解決：卷積的局部連接保留空間結構，權重共享(同一核滑遍全圖)大幅減少參數並帶來平移不變性。

6. CNN 的卷積層與第 5 章的空間濾波有何相同、有何關鍵不同？這說明了什麼？

提示與參考解答：相同：都是用小核在影像上滑動計算局部加權和，數學上是同一件事(卷積)。關鍵不同：第 5 章的核是人設計的(高斯、Sobel)，CNN 的核是訓練學出來的——網路自己決定核裡的數值以萃取最有用的特徵。說明深度學習「自動學特徵」正是建立在傳統卷積之上，是它的自動化與升級。

7. 卷積層的「權重共享」帶來哪兩個好處？池化層有什麼作用？它與第 10 章的什麼運算相同？

提示與參考解答：權重共享(同一核滑遍全圖用同組權重)：其一大幅減少參數(一個 3×3 核只 9 個權重，不管影像多大)，其二帶來平移不變性(物件在哪都能被同一核偵測)。池化(如最大池化)降取樣：減少運算、擴大深層視野、提供位置容錯。最大/最小池化即第 10 章型態學的膨脹/侵蝕。

8. 請依序說明 CNN 架構譜系從 LeNet 到 MobileNet 各自的關鍵創新，並指出它們回應了什麼瓶頸。

提示與參考解答：LeNet(1998)開創卷積-池化-全連接範式。AlexNet(2012)以 ReLU/Dropout/GPU 引爆革命，回應「深層難訓練」。VGG(2014)全用 3×3 加深，證明深度價值。GoogLeNet(2014)Inception 多尺度加 1×1 壓縮，回應「更深卻要省參數」。ResNet(2015)殘差連接，回應「太深反而難訓練的退化問題」，錯誤率低於人類。MobileNet(2017)深度可分離卷積，回應「大模型無法在邊緣裝置執行」。

9. ResNet 的「殘差連接」解決了什麼問題？為什麼它讓訓練上百層的網路成為可能？

提示與參考解答：解決退化問題：網路太深時梯度難以傳遞、反而更難訓練、精度不升反降。殘差連接(skip connection)讓資訊可以「跳過」若干層直接往後傳，梯

度也能沿捷徑順暢回傳，避免梯度消失，使訓練上百層網路成為可能。此思想至今無所不在(含Transformer)。

10. 遷移學習的核心思想是什麼?為什麼它對「資料稀少」的應用特別有價值?

提示與參考解答：核心思想：預訓練模型較低的層已學會通用視覺特徵(邊緣、紋理、形狀)，對任何影像任務都有用、不必重學，因此保留(凍結)特徵萃取層，只重新訓練最後的分類層以適應自己的任務。對資料稀少的應用特別有價值，因為困難的「學特徵」已由ImageNet 預訓練的巨人完成，只需少量自己的資料教它「特徵對應到我的類別」。

11. 資料增強用到了本書前面哪幾章的技術?為什麼說「深度學習站在傳統影像處理的地基上」?

提示與參考解答：資料增強用第4章幾何轉換(旋轉、平移、翻轉、縮放)、第5章空間域強化(亮度、對比)、第7章(加雜訊)。AlexNet 即以隨機裁切、翻轉、色彩擾動擴充資料。這說明深度學習不是憑空而來：資料增強是傳統影像處理、卷積源於空間濾波、正則化源於第7章、訓練評估原則源於第14章——深度學習穩穩站在前十四章的地基上，這正是本書把它放在最後的深意。

12. (綜合思辨題)深度學習有哪些主要侷限?為什麼在醫療、司法等高風險領域使用時要格外謹慎?請結合第7、14章的觀點。

提示與參考解答：侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱(微小擾動致誤判)、分布偏移(訓練分布外表現急降)。高風險領域謹慎的原因：黑箱使我們難以解釋模型為何判斷，可能學到與答案相關卻無意義的捷徑(第14章肺炎X光)、或在新分布上崩壞(第7.7.3節);清晰不等於正確(第7章還原vs. 腦補)。工程師的責任不只調高準確率，更要理解模型學到什麼、資料是否公平——這是技術素養與工程倫理。

參考資料與延伸閱讀

- Goodfellow, I., Bengio, Y., & Courville, A., Deep Learning, MIT Press, 2016.(深度學習權威教科書)
- Krizhevsky, A., Sutskever, I., & Hinton, G. E., ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.(AlexNet, 本章叫獸開講之史料來源)
- LeCun, Y., Bengio, Y., & Hinton, G., Deep Learning, Nature, 2015.
- Simonyan, K., & Zisserman, A., Very Deep Convolutional Networks (VGG), ICLR, 2015; He, K., et al., Deep Residual Learning (ResNet), CVPR, 2016.
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.
- OpenCV、TensorFlow、Keras、PyTorch 官方文件與教學。
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 16 章 物件偵測 YOLO 實戰

本章學習目標

- 區分影像分類、物件偵測、語意分割與實例分割四種視覺任務。
- 說明兩階段與單階段偵測器的差異，理解 YOLO「只看一次」的核心思想。
- 運用 IoU、NMS、mAP 等關鍵概念評估與後處理物件偵測結果。
- 以 Ultralytics YOLO11 完成推論、訓練自訂資料集與模型驗證。
- 操作 YOLO 的分割、姿態估計與追蹤等延伸功能，並串接第 12 章視訊處理。
- 思辨電腦視覺技術的倫理責任，理解工程師在技術應用中的角色。

叫獸開講|YOLO 之父的告別：當創造者選擇放下

2016 年，華盛頓大學的研究生 Joseph Redmon 在頂級會議 CVPR 上發表了一篇論文，提出一個名叫 YOLO(You Only Look Once, 「你只需看一次」)的物件偵測演算法。在那之前，主流的偵測方法要分兩階段：先產生大量候選框、再逐一分類，又慢又笨重。Redmon 的想法極為大膽：何不讓神經網路「看一次」整張影像，就直接同時輸出所有物件的位置與類別？這個一氣呵成的設計讓偵測速度快到能即時執行，論文一舉拿下 OpenCV 群眾票選獎，隔年的改良版 YOLO9000 更獲得 CVPR 最佳論文榮譽提名。YOLO 成了物件偵測的代名詞，從自駕車、工廠檢測到手機相機，無所不在。

Redmon 本人也是個充滿個人風格的研究者——他的個人網站與履歷以幽默不羈聞名，論文裡甚至夾雜著玩笑與自嘲。但在 2018 年的 YOLOv3 論文結尾，他寫下了一段不太像技術論文的文字，流露出對自己研究可能被誤用的不安。

2020 年 2 月，這份不安化為行動。當時學術界正在討論 NeurIPS 會議的新規定——要求論文作者加寫一段「廣泛影響」，說明研究可能帶來的正面與負面社會後果。有人質疑：研究者真的有必要因為「廣泛影響」而決定不投稿嗎？就在這場討論中，Redmon 現身分享了自己的親身經歷：他說，他已經停止電腦視覺研究了，因為他看見自己的研究成果帶來的影響——儘管熱愛這份工作，但軍事應用與隱私疑慮已經到了無法忽視的地步。這位親手創造出當代最流行偵測演算法的人，選擇了放下。

這件事在社群中引發了持久的辯論，而雙方都有道理。反對者說：技術本身沒有道德，刀子可以殺人也可以救人，問題出在使用者而非發明者；而且你不做，別人也會做，退出只是讓自己失去影響力。支持者則說：創造者對自己創造物的流向並非全然無責，當一項技術的下行風險遠大於上行效益時，選擇不推進它，本身就是一種負責任的表態。

這一章我們要學的，正是 Redmon 創造的 YOLO——它強大、快速、易用，是您在課程中要親手部署的主力工具。但在按下訓練指令之前，叫獸希望你先記

住這個故事。作為工程師，我們手上的技術是中性的，但我們選擇把它用在哪裡、為誰服務、防範什麼樣的濫用，從來就不是中性的。這門課從第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤，一路鋪陳到這裡，談的都是同一件事：技術能力愈強大，對使用者處境的體貼與對後果的想像，就愈是專業素養不可分割的一部分。

思考題：「技術本身沒有道德，問題在使用者」與「創造者對技術的流向也有責任」——你比較認同哪一方？如果你設計的視覺系統可能被用於你不認同的用途，你會怎麼做？(想想：授權條款、使用限制、公開透明、或如 Redmon 一般選擇退出)

16.1 四種視覺任務

進入 YOLO 之前，必須先分清楚四種常被混淆的視覺任務。它們的差別在於「輸出的精細程度」，難度也依序遞增：

1. 影像分類(Classification)：輸出整張影像屬於哪一類——「這是一張貓的照片」。這是第 14、15 章的主題，只回答「是什麼」，不回答「在哪裡」。
2. 物件偵測(Object Detection)：輸出影像中每個物件的類別與位置(邊界框)——「這裡有一隻貓、那裡有兩個人」。它同時回答「是什麼」與「在哪裡」，是本章的主軸。
3. 語意分割(Semantic Segmentation)：為每一個像素標上類別——「這些像素是路面、那些是天空」。它給出精確的輪廓，但不區分同類的不同個體(三個人會被標成同一片「人」的區域)。這是第 9.6 節介紹過的 U-Net 所擅長的。
4. 實例分割(Instance Segmentation)：結合偵測與分割——不僅標出每個像素的類別，還區分同類的不同個體(三個人各自有獨立的輪廓與編號)。這是最精細、也最困難的任務。

另有一個常見的延伸任務：姿態估計(Pose Estimation)，它偵測的不是物件的框，而是人體(或物體)的關鍵點——頭、肩、肘、腕、髖、膝、踝等骨架節點。它在動作分析、人機協作、以及第 12 章的跌倒偵測中極為重要。現代的 YOLO 已經把偵測、分割、姿態估計、追蹤整合進同一個工具中，這正是它成為業界主力的原因。

16.2 物件偵測的演進與 YOLO 的核心思想

16.2.1 從傳統方法到深度學習

回顧本書，我們其實已經走過了物件偵測的整部演進史。第 10 章的 Viola-Jones(2001)以 Haar 特徵加級聯分類器，是第一代能即時執行的偵測器，但只擅長正面人臉；第 13 章的特徵匹配加 RANSAC，能定位特定物件但難以泛化到「所有的貓」；而傳統做法的共同侷限是：依賴人工設計特徵(第 15 章已詳述這個瓶頸)。

深度學習時代的偵測器分兩大流派。兩階段偵測器(如 R-CNN 系列)先產生候選區域、再對每個區域分類與微調框，準確度高但速度慢——因為同一張影像要被網路處理很多次。單階段偵測器(如 YOLO、SSD)則一次前向傳播就同時輸出所有物件的位置與類別，速度快得多，這正是 YOLO 名稱「You Only Look Once」的由來。

16.2.2 YOLO 的核心思想

YOLO 的巧思在於把「偵測」重新定義成一個「迴歸問題」：它把影像切成網格，讓網路對每個網格直接預測「這裡有沒有物件、框在哪裡、是什麼類別、信心多高」。整張影像只需通過網路一次，所有預測一氣呵成——這與兩階段方法「先提候選再逐一檢查」的思路截然不同，是一次漂亮的觀點翻轉(呼應第 9 章霍夫轉換的叫獸開講)。

這個設計還有一個額外好處：因為 YOLO 每次都看整張影像，它能利用全域的脈絡資訊(例如「這個區域周圍都是道路，所以這個模糊的物體比較可能是車而非船」)，背景誤判反而較少。而它早期的弱點是對小物件與密集群體較不擅長——這在後續版本中已大幅改善。

16.2.3 YOLO 的世代演進

YOLO 從 2016 年至今經歷了快速的世代更迭：v1 到 v3 由 Redmon 本人主導(叫獸開講的主角)，v3 引入多尺度預測(呼應第 11 章多解析度思想)大幅改善小物件偵測；此後由社群與不同團隊接棒，陸續推出 v4、v5、v6、v7、v8，直到近年的 YOLO11 等版本。演進的主軸始終不變：更快、更準、更容易使用，並逐步把分割、姿態估計、追蹤等功能整合進同一套框架。

本章實作採用 Ultralytics 提供的 YOLO11。它把整套流程封裝得極為簡潔——載入模型、推論、訓練自訂資料集都只需幾行程式，同時提供 n(nano)、s、m、l、x 等不同大小的模型，讓使用者依算力需求選擇：nano 版可在第 17 章的邊緣裝置上即時執行，x 版則追求最高精度。這個「同一架構、多種尺寸」的設計，正呼應了第 15 章 MobileNet 所代表的「夠好且能部署」的務實路線。

16.3 物件偵測的關鍵概念

16.3.1 邊界框與 IoU

物件偵測的輸出是邊界框(bounding box)，通常以「左上角座標加寬高」或「中心座標加寬高」表示，並附帶類別與信心分數。要評估一個預測框準不準，需要一個量化的標準——這就是 IoU(Intersection over Union，交集聯集比)：把預測框與真實框的「重疊面積」除以「聯集面積」。IoU 為 1 代表完美重合，為 0 代表完全不重疊。實務上常以 IoU 大於某門檻(如 0.5)判定為「偵測正確」。

16.3.2 非極大值抑制(NMS)

偵測器對同一個物件，往往會輸出好幾個高度重疊的框。非極大值抑制(NMS)負責去蕪存菁：先取信心分數最高的框，再把與它 IoU 超過門檻的其他框都刪掉(視為重複)，接著對剩下的框重複這個程序，直到沒有框可處理。這個「保留局部最強、抑制鄰近重複」的思想，你應該覺得眼熟——它正是第 9 章 Canny 邊緣偵測第三步「非極大值抑制」的同名兄弟，兩者的精神完全一致。這種概念的跨章重現，是本書一再強調的：影像處理的核心思想並不多，但它們會以不同面貌反覆出現。

16.3.3 評估指標：mAP

物件偵測的標準評估指標是 mAP(mean Average Precision, 平均精度均值)。它建立在第 14 章的精確率與召回率之上：對每一個類別，調整信心門檻可得到一條「精確率-召回率曲線」，曲線下的面積即為該類別的 AP；把所有類別的 AP 取平均，就是 mAP。常見的寫法如 mAP@0.5(以 IoU 門檻 0.5 判定正確)或 mAP@0.5:0.95(在多個 IoU 門檻下取平均，標準更嚴格，是 COCO 資料集的主要指標)。

請留意這裡與第 14 章的連貫：同樣是「精確率 vs. 召回率」的權衡，同樣是「依應用代價選擇門檻」的工程判斷。一個工安監控系統可能寧可誤報也不能漏報(重召回)，一個自動分揀系統則可能相反(重精確)。指標的選擇從來不只是數學問題。

概念	定義	本書中的呼應
IoU	預測框與真實框的交集除以聯集	量化重疊程度，判定偵測是否正確
NMS	保留信心最高框、刪除高度重疊者	與第 9 章 Canny 非極大值抑制同名同理
信心分數	模型對該預測的把握程度	門檻取舍即第 14 章精確率與召回率權衡
mAP	各類別精確率-召回率曲線下面積的平均	建立於第 14 章的精確率與召回率

表 16-1 物件偵測的關鍵概念

16.4 YOLO 實戰：推論、訓練與驗證

16.4.1 環境安裝與快速推論

Ultralytics 把 YOLO 的使用門檻降到極低。安裝一個套件、載入預訓練模型，幾行程式就能偵測 COCO 資料集(第 14 章)的八十個類別——人、車、貓、狗、瓶子、椅子等日常物件。

程式 16-1 安裝 Ultralytics YOLO

```
# 安裝(建議在虛擬環境中)
pip install ultralytics
```

程式 16-2 YOLO 影像推論

```
from ultralytics import YOLO
import cv2

# 載入預訓練模型(n/s/m/l/x 依算力需求選擇)
model = YOLO("yolo11n.pt")          # nano:最輕量,適合邊緣裝置

# --- 單張影像推論 ---
results = model("street.jpg", conf=0.25, iou=0.45)
#   conf:信心門檻(調高減少誤報、調低減少漏檢)
#   iou :NMS 的 IoU 門檻
```

```

for r in results:
    for box in r.bboxes:
        cls = int(box.cls[0])           # 類別編號
        name = model.names[cls]         # 類別名稱
        conf = float(box.conf[0])       # 信心分數
        x1, y1, x2, y2 = box.xyxy[0].tolist() # 邊界框座標
        print(f"{name}: {conf:.2f} @ ({x1:.0f},{y1:.0f})-({x2:.0f},{y2:.0f})")

    annotated = r.plot()                # 畫出所有偵測框
    cv2.imwrite("detected.jpg", annotated)

```

16.4.2 訓練自訂資料集

真正的價值在於訓練自己的偵測器——偵測您關心的物件(工件瑕疵、農產品、特定零件)。流程分三步：

5. 準備資料：蒐集影像並標註邊界框(常用工具如 Labellmg、Roboflow、CVAT)。標註格式為 YOLO 格式：每張影像對應一個 txt 檔，每行記錄「類別編號 中心 x 中心 y 寬 高」(座標皆正規化到 0 至 1)。務必依第 14 章原則切分訓練/驗證/測試集。
6. 撰寫設定檔：一個 YAML 檔，指定資料集路徑與類別名稱。
7. 訓練與驗證：呼叫 train() 開始訓練(內建資料增強、及早停止等第 14、15 章的機制)，再用 val() 在驗證集上計算 mAP。

程式 16-3 資料集設定檔

```

# data.yaml - 資料集設定檔
path: /home/user/mydata
train: images/train
val: images/val
names:
  0: good_part
  1: scratch
  2: crack

```

程式 16-4 訓練自訂 YOLO 模型

```

from ultralytics import YOLO

# 從預訓練權重開始訓練 = 遷移學習(第 15 章)
model = YOLO("yolo11n.pt")

results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20,           # 及早停止(第 14 章對抗過度擬合)
    degrees=10,           # 資料增強:旋轉(第 4 章)
    hsv_v=0.4,            # 資料增強:亮度(第 5、8 章)
    flipplr=0.5,          # 資料增強:水平翻轉(第 4 章)

```

```

project="runs", name="defect_v1")

# 驗證:計算 mAP
metrics = model.val()
print("mAP@0.5      :", metrics.box.map50)
print("mAP@0.5:0.95 :", metrics.box.map)

# 匯出為部署格式(第 17 章邊緣部署)
model.export(format="onnx")      # 或 engine(TensorRT)、tflite

```

請注意程式 16-4 中的訓練參數：degrees、hsv_v、fliplr 全是第 4、5、8 章的影像處理技術，在此作為資料增強使用；patience 是第 14 章的及早停止；而「從預訓練權重開始」正是第 15 章的遷移學習。這再次印證全書的主軸——最先進的工具，骨子裡仍是前面各章觀念的組合。

16.5 延伸功能：分割、姿態估計與追蹤

現代 YOLO 已不只是偵測器，而是一套整合的視覺工具箱，只需更換模型權重即可切換任務。

16.5.1 實例分割

使用 seg 模型，YOLO 除了邊界框還會輸出每個物件的像素級遮罩(mask)，達成 16.1 節所述的實例分割。這比單純的框更精確——量測物件的實際面積、計算不規則形狀的尺寸、或做背景去除時，遮罩遠優於矩形框。它也是第 9 章傳統分割方法的深度學習後繼者。

16.5.2 姿態估計

使用 pose 模型，YOLO 會輸出人體的關鍵點(通常十七個：鼻、雙眼、雙耳、雙肩、雙肘、雙腕、雙髖、雙膝、雙踝)，形成骨架。這開啟了豐富的應用：動作分析與運動姿勢矯正、人機協作中判斷作業員的手部位置以確保安全、手勢指令控制、以及第 12 章討論過的跌倒偵測——透過追蹤重心高度的急遽變化與骨架的姿態，能比單純的框更可靠地判斷跌倒。

16.5.3 物件追蹤

YOLO 內建追蹤功能，直接實現了第 12.5 節介紹的「偵測式追蹤」(tracking-by-detection)：每一影格用 YOLO 偵測，再以 ByteTrack、BoT-SORT 等演算法把跨影格的偵測結果關聯起來、賦予穩定的 ID。這正是第 12 章所說 SORT/DeepSORT 框架的現代實作。有了穩定的 ID，才能做人流計數、車流分析、軌跡記錄——這些在第 12.7 節的案例中都已鋪陳過，現在終於能用最先進的工具實現。

程式 16-5 分割、姿態估計與追蹤

```

from ultralytics import YOLO
import cv2

# --- 實例分割:輸出像素級遮罩 ---
seg_model = YOLO("yolo11n-seg.pt")

```

```

r = seg_model("parts.jpg")[0]
if r.masks is not None:
    print("偵測到物件數:", len(r.masks))
    mask = r.masks.data[0].cpu().numpy()      # 第一個物件的遮罩
    area_px = int(mask.sum())                  # 像素面積(可量測)
    print("面積(像素):", area_px)

# --- 姿態估計:輸出人體 17 個關鍵點 ---
pose_model = YOLO("yolo11n-pose.pt")
r = pose_model("person.jpg")[0]
if r.keypoints is not None:
    kpts = r.keypoints.xy[0]                  # 第一個人的關鍵點座標
    print("鼻子座標:", kpts[0].tolist())
    print("左肩座標:", kpts[5].tolist())

# --- 追蹤:第 12 章的偵測式追蹤 ---
cap = cv2.VideoCapture("traffic.mp4")
while True:
    ok, frame = cap.read()
    if not ok:
        break
    # persist=True 讓 ID 跨影格維持
    res = seg_model.track(frame, persist=True,
tracker="bytetrack.yaml")[0]
    if res.bboxes.id is not None:
        for box, tid in zip(res.bboxes.xyxy, res.bboxes.id):
            x1, y1, x2, y2 = box.tolist()
            cv2.putText(frame, f"ID {int(tid)}", (int(x1), int(y1)-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
    cv2.imshow("track", res.plot())
    if cv2.waitKey(1) == 27:
        break
cap.release(); cv2.destroyAllWindows()

```

16.6 應用案例

16.6.1 案例一：AOI 瑕疵偵測

情境：產線上自動偵測工件表面的刮痕、裂紋、缺件。相較第 6 章的頻率域紋理檢測與第 9、10 章的傳統管線，YOLO 的優勢是能處理形態多變、難以用規則描述的瑕疵，且能同時分類多種瑕疵類型。流程：蒐集並標註各類瑕疵影像(通常需數百至數千張)→ 以遷移學習訓練 YOLO → 用 mAP 驗證 → 匯出為 ONNX 或 TensorRT 部署到產線電腦或 Jetson(第 17 章)。

關鍵挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少。對策包括：資料增強(第 4、5 章)、合成瑕疵樣本、或改用只學習「正常」樣貌的異常偵測方法。這也解釋了為什麼第 6 章的傳統方法至今仍有價值——它不需要瑕疵樣本，對「沒見過的新瑕疵」天生有效，常與 YOLO 互補使用。

16.6.2 案例二：交通場景分析

情境：路口監控自動統計車流量、分類車種、偵測違規。這是第 12 章與本章的完整整合：YOLO 偵測並分類車輛與行人 → 內建追蹤器維持每個目標的 ID → 依軌跡越線計數(第 12.7.1 節)、結合第 4 章相機校正換算實際速度(第 12.7.2 節)、或判斷是否闖紅燈、逆向。這條管線串起了第 4、12、16 章，是智慧交通的標準架構。

16.6.3 案例三：農業與畜牧應用

情境：這是雲林在地產業最有潛力的應用之一。以無人機或固定攝影機搭配 YOLO，可自動計數果樹上的果實以估算產量、偵測病蟲害的葉片斑點、辨識雜草以做精準除草、或在畜牧場中計數與追蹤牲畜、透過姿態估計監控其行為異常(如久臥不起可能是生病徵兆)。

這類應用的工程重點在於：田間環境的光線變化劇烈(第 5、8 章的白平衡與對比強化成為必要的前處理)、物件常相互遮擋(密集果實的偵測是難點)、且需在無網路的田間即時執行(第 17 章的邊緣部署)。這也再次體現本書一貫的訊息：先進的 AI 模型不是孤立運作的，它需要前面各章的影像處理技術一起支撐，才能在真實世界中可靠工作。

16.6.4 案例四：人機協作的安全監控

情境：在協作型機器人(cobot)的工作區，以 YOLO 的姿態估計即時追蹤作業員的手部與身體位置，當人員進入機器人的危險作業半徑時，自動減速或停機。這是工業安全的重要應用，也直接連結智慧機器人學程的核心關懷。

但這個案例也必須談責任：安全攸關的系統，不能單靠一個可能誤判的 AI 模型。工程上的正確做法是「多層防護」——視覺系統作為第一道預警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制。呼應叫獸開講與第 15 章談過的深度學習侷限(對抗脆弱、分布偏移)，在人命關天的場合，對 AI 保持審慎、設計冗餘，是工程師的基本責任。

16.7 技術倫理：工程師的責任

本節不談程式，而是承接叫獸開講，把全書零散提及的倫理線索收攏起來——因為 YOLO 這樣強大而易用的工具，正是最需要談責任的地方。

回顧本書的倫理伏筆：第 7 章的「還原 vs. 腦補」提醒我們區分「找回的資訊」與「編造的資訊」，並在司法、醫療場合誠實標註；第 9 章的醫學影像分割強調自動結果僅供輔助、須由醫師確認；第 12 章的跌倒偵測提醒照護應用涉及被監控者的隱私與尊嚴；第 14 章的肺炎 X 光案例揭露資料偏誤如何讓模型學到無意義的捷徑；第 15 章列舉了深度學習的黑箱與對抗脆弱。這些線索指向同一個核心：技術能力愈強，對後果的想像力就愈重要。

物件偵測尤其敏感，因為它的用途橫跨光譜的兩端：同一套人流分析技術，可以用於商場最佳化動線、也可以用於監控特定族群；同一套目標追蹤技術，可以用於搜救失蹤者、也可以用於軍事鎖定。Redmon 的選擇是退出，但那不是唯一的答案。工程

師還有許多實際的作為：在系統設計中內建隱私保護(如只統計數量不儲存人臉、在裝置端處理而非上傳雲端)、對資料來源與模型限制保持透明、在授權條款中明訂禁止的用途、在部署前評估誤判的後果與受影響者、以及——最基本卻最重要的——在被要求做一件你認為不對的事情時，有能力說出自己的疑慮。

叫獸想說的是：這門課教你把技術做出來，但沒有任何一行程式碼能替你決定「該不該做」。那是你作為一個專業工作者、也作為一個人的判斷。而判斷力，和寫程式一樣，是需要練習的——從現在開始，每當你部署一個系統，多問自己一句：「誰會受益?誰可能受害?如果它出錯了，誰要承擔?」

16.8 本章與全書的觀念連結總表

相關章節	共用的觀念	物件偵測中的角色
第 4、5、8 章	旋轉、亮度對比、HSV	訓練時的資料增強參數;田間光線的前處理
第 9 章 影像分割	非極大值抑制、語意分割	NMS 與 Canny 同名同理;YOLO-seg 為傳統分割的後繼
第 10 章 二值影像	Viola-Jones 級聯	第一代即時偵測器，YOLO 的前輩
第 12 章 視訊處理	偵測式追蹤、SORT	YOLO track 即 tracking-by-detection 的實作
第 13 章 特徵擷取	特徵匹配定位、車牌偵測	傳統定位方法的深度學習後繼
第 14 章 機器學習	COCO、精確率召回率、資料切分	mAP 建立於精確率召回率;COCO 為預訓練資料集
第 15 章 深度學習	CNN、遷移學習、MobileNet	YOLO 骨幹為 CNN;從預訓練權重訓練即遷移學習
第 17 章 邊緣部署	ONNX、TensorRT、Jetson	模型匯出與邊緣裝置上的即時推論

表16-2 第16章與全書其他章節的觀念連結

程式實作 Lab 16

Lab 16-1 YOLO 快速上手

目標：熟悉推論與參數。(a)安裝 Ultralytics，以程式 16-2 對數張照片(街景、室內、多物件場景)推論，觀察 COCO 八十類的偵測結果;(b)調整 conf 門檻(0.1、0.25、0.5、0.8)，記錄漏檢與誤報如何變化，對照第 14 章的精確率-召回率權衡;(c)調整 iou 門檻，觀察 NMS 如何影響重疊框的保留;(d)比較 yolo11n 與 yolo11s/m 的偵測結果與推論速度(FPS)，體會精度與速度的取舍;(e)討論：在什麼應用中你會把 conf 調低?什麼應用中會調高?

Lab 16-2 訓練自訂偵測器

目標：完成一個屬於自己的偵測器。(a)選定二到三類物件(如實驗室器材、特定零件、水果)，各拍攝並標註五十張以上影像(建議用 Labellmg 或 Roboflow);(b)依第 14 章原則切分訓練/驗證集，撰寫 data.yaml;(c)以程式 16-4 從預訓練權重開始訓練(遷移學習)，觀察訓練過程的損失曲線與 mAP 變化;(d)在驗證集上計算 mAP@0.5 與 mAP@0.5:0.95，並找出模型最常誤判的情況;(e)刻意減少訓練資料到二十張重訓，比較 mAP 差異——驗證第 14 章「資料是燃料」;(f)加入或移除資料增強參數，觀察對過度擬合的影響。

Lab 16-3 姿態估計與跌倒偵測

目標：實作第 12 章預告的跌倒偵測。(a)以程式 16-5 的 pose 模型對人物影片做姿態估計，視覺化十七個關鍵點與骨架;(b)計算人體重心(可用髖部關鍵點近似)的高度與其變化速率;(c)設計一個簡單的跌倒判斷規則：重心高度在短時間內急遽下降、且之後維持在低位;(d)測試：正常坐下、蹲下、躺下與真正的跌倒，你的規則能否區分?(e)討論：承第 12.7.3 節，這類照護系統的誤報與漏報各有什麼代價?如何在系統設計中兼顧隱私(例如只傳送骨架資料而非影像)?

Lab 16-4 交通流量分析系統

目標：整合第 12 章與本章，完成完整的視訊分析系統。(a)以 YOLO 的 track 模式對交通影片偵測並追蹤車輛，確認 ID 跨影格穩定;(b)畫一條虛擬計數線，依軌跡越線方向統計雙向車流量;(c)分類統計不同車種(汽車、機車、公車、卡車)的數量;(d)進階：結合第 4 章相機校正，把像素位移換算成實際速度(第 12.7.2 節);(e)量測系統的 FPS，評估在你的硬體上能否即時處理，並思考第 17 章的部署選項(用更小的模型?降低解析度?用 GPU?)。

本章重點整理

- 四種視覺任務依精細度遞增：分類(是什麼)、偵測(是什麼加在哪裡)、語意分割(每像素類別、不分個體)、實例分割(每像素類別且分個體);姿態估計則輸出關鍵點骨架。
- 兩階段偵測器先提候選再分類(準但慢)，單階段偵測器一次前向傳播直接輸出所有結果(快);YOLO 把偵測重新定義為迴歸問題，只看一次全圖，並能利用全域脈絡減少背景誤判。
- 關鍵概念：IoU(交集聯集比，判定框準不準)、NMS(保留信心最高框、抑制重疊者，與第 9 章 Canny 同名同理)、mAP(建立於第 14 章精確率與召回率，常用 mAP@0.5 與 mAP@0.5:0.95)。
- Ultralytics YOLO11 提供 n/s/m/l/x 多種尺寸供依算力選擇;訓練自訂資料集三步驟：標註資料(YOLO 格式)、撰寫 data.yaml、train 與 val;訓練參數中的旋轉、亮度、翻轉即第 4、5 章的資料增強，從預訓練權重開始即第 15 章的遷移學習。

- 延伸功能：seg 模型輸出像素級遮罩(可精確量測面積)、pose 模型輸出十七個人體關鍵點(動作分析、跌倒偵測、人機協作安全)、track 模式實現第 12 章的偵測式追蹤並維持穩定 ID。
- 技術倫理：同一套技術可用於截然不同的目的;工程師的實際作為包括內建隱私保護、對限制保持透明、授權中明訂禁止用途、評估誤判後果、以及在必要時表達疑慮。安全攸關系統須設計多層防護，不可單靠可能誤判的 AI 模型。

習題

1. 請區分影像分類、物件偵測、語意分割與實例分割，說明各自的輸出形式與難度差異。

提示與參考解答：分類：輸出整張影像的類別(是什麼)，不含位置。偵測：輸出每個物件的類別與邊界框(是什麼加在哪裡)。語意分割：每個像素標上類別，有精確輪廓但不分同類個體(三個人是同一片「人」)。實例分割：每像素類別且區分不同個體(三個人各有獨立輪廓與編號)，最精細也最困難。

2. 兩階段與單階段偵測器有何差異?YOLO 的「只看一次」具體是什麼意思?

提示與參考解答：兩階段(如 R-CNN 系列)先產生候選區域再逐一分類與微調框，準確度高但慢(同一影像被網路處理多次)。單階段(YOLO、SSD)一次前向傳播就同時輸出所有物件的位置與類別。YOLO 把影像切成網格，讓網路對每個網格直接預測有無物件、框的位置、類別與信心，整張影像只需通過網路一次，故名 You Only Look Once。

3. YOLO 每次都看整張影像，這帶來什麼額外好處?它早期的弱點是什麼?

提示與參考解答：好處：能利用全域脈絡資訊(如「周圍都是道路，這個模糊物體較可能是車而非船」)，背景誤判較少。早期弱點：對小物件與密集群體較不擅長;YOLOv3 引入多尺度預測(呼應第 11 章多解析度)後大幅改善。

4. 什麼是 IoU?它如何用於判定偵測是否正確?

提示與參考解答：IoU(Intersection over Union)是預測框與真實框的交集面積除以聯集面積。IoU 為 1 代表完美重合、0 代表完全不重疊。實務上以 IoU 超過某門檻(常用 0.5)判定該偵測為正確;也用於 NMS 判斷兩框是否重複。

5. 說明 NMS 的運作步驟。它與第 9 章的哪個步驟同名同理?

提示與參考解答：步驟：取信心分數最高的框，刪除與它 IoU 超過門檻的其他框(視為重複)，對剩下的框重複此程序直到處理完畢。它與第 9 章 Canny 邊緣偵測第三步的「非極大值抑制」同名同理——都是「保留局部最強、抑制鄰近重複」，體現本書「核心思想以不同面貌反覆出現」的主軸。

6. mAP 是如何計算的?它建立在第 14 章的哪兩個指標之上?mAP@0.5 與 mAP@0.5:0.95 有何差別?

提示與參考解答：對每個類別，調整信心門檻得到精確率-召回率曲線，曲線下面積為該類的 AP;所有類別的 AP 取平均即 mAP。它建立於第 14 章的精確率與召回率。mAP@0.5 以 IoU 門檻 0.5 判定正確;mAP@0.5:0.95 在多個 IoU 門檻(0.5 到 0.95)下取平均，標準更嚴格，是 COCO 的主要指標。

7. 訓練自訂 YOLO 模型的三個步驟是什麼?YOLO 標註格式的每一行記錄什麼?

提示與參考解答：三步驟：一、準備並標註資料(用LabelImg/Roboflow 等，依第 14 章切分訓練/驗證/測試集)；二、撰寫data.yaml 設定檔指定路徑與類別名稱；三、呼叫train() 訓練、val() 計算mAP。標註格式每行為「類別編號 中心x 中心y 寬 高」，座標皆正規化到0 至1。

8. 程式 16-4 的訓練參數 degrees、hsv_v、fliplr、patience 分別對應本書哪幾章的觀念？「從預訓練權重開始訓練」又是什麼？

提示與參考解答：degrees(旋轉)、fliplr(水平翻轉)是第4 章幾何轉換；hsv_v(亮度)是第5、8 章；三者皆作為資料增強對抗過度擬合(第14 章)。patience 是第14 章的及早停止。從預訓練權重開始訓練即第15 章的遷移學習——借用已學好的通用特徵，少量資料也能訓練出高效能模型。

9. YOLO 的 seg、pose、track 三種模式各輸出什麼？各舉一個應用。

提示與參考解答：seg：輸出像素級遮罩(實例分割)，應用如精確量測不規則物件面積、背景去除。pose：輸出人體十七個關鍵點骨架，應用如動作分析、跌倒偵測、人機協作安全監控。track：每一影格偵測並跨影格關聯維持穩定ID(第12 章偵測式追蹤)，應用如人流計數、車流分析、軌跡記錄。

10. AOI 瑕疵偵測用 YOLO 會遇到什麼資料上的挑戰？為什麼第 6 章的傳統方法至今仍有價值？

提示與參考解答：挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少，難以訓練。對策：資料增強、合成瑕疵樣本、或改用只學正常樣貌的異常偵測。第6 章的頻率域紋理檢測不需要瑕疵樣本(策略是「濾掉正常」)，對「沒見過的新瑕疵」天生有效，故與YOLO 互補使用。

11. 為什麼人機協作的安全監控不能單靠 YOLO 姿態估計？應如何設計？

提示與參考解答：因為深度學習模型可能誤判(第15 章：對抗脆弱、分布偏移、黑箱難解)，而人機協作安全攸關人命。正確做法是多層防護：視覺系統作為第一道預警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制，形成冗餘。對 AI 保持審慎、設計冗餘，是工程師的基本責任。

12. (思辨題)承叫獸開講：Redmon 因軍事應用與隱私疑慮退出電腦視覺研究。請說明支持與反對的論點，並提出「除了退出之外」工程師還能有哪些負責任的作為。

提示與參考解答：本題無標準答案，評分重點在論證品質。反對方：技術本身中性，道德在於使用者；你不做別人也會做，退出反而失去影響力。支持方：創造者對創造物的流向並非全然無責；當下行風險遠大於上行效益時，不推進本身即是負責任的表態。其他作為：系統設計內建隱私保護(只統計數量不存人臉、端上處理不上傳)、對資料來源與模型限制保持透明、授權條款明訂禁止用途、部署前評估誤判後果與受影響者、以及在被要求做不對的事時有能力表達疑慮。

參考資料與延伸閱讀

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A., You Only Look Once: Unified, Real-Time Object Detection, CVPR, 2016.(YOLO 原始論文)
- Redmon, J., & Farhadi, A., YOLO9000: Better, Faster, Stronger, CVPR, 2017;YOLOv3: An Incremental Improvement, arXiv, 2018.

- Synced Review, YOLO Creator Joseph Redmon Stopped CV Research Due to Ethical Concerns, Feb. 2020.(本章叫獸開講之史料來源)
- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.(第 14 章亦引用)
- Ultralytics YOLO 官方文件: <https://docs.ultralytics.com>
- Zhang, Y., et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box, ECCV, 2022.
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 17 章 邊緣 AI 與機器人視覺部署

本章學習目標

- 說明邊緣運算的動機，比較端上、近端邊緣、雲端三種 AI 辨識架構的取舍。
- 認識 ESP32-CAM 與 NVIDIA Jetson 兩大實作平台的定位與能力差異。
- 運用量化、剪枝、ONNX、TensorRT 等技術最佳化模型以在邊緣裝置上執行。
- 量測與最佳化推論效能，理解延遲、吞吐量與功耗的權衡。
- 將視覺模組整合進 ROS 2 機器人系統，理解節點與話題的通訊架構。
- 完成一個端到端的機器人視覺部署，並評估系統的可靠度與失效模式。

叫獸開講|五美元的革命：ESP32 如何讓每個人都能做物聯網

2014 年 8 月，一群歐美的創客(maker)在網路論壇上議論紛紛。一款來自中國的小模組出現在市面上，名叫 ESP-01，由深圳的安信可(Ai-Thinker)製造，核心是上海樂鑫科技(Espressif Systems)的 ESP8266 晶片。它引起注意的原因很單純：價格。當時要讓一塊 Arduino 連上 Wi-Fi，得外掛一個動輒十到二十美元、甚至更貴的無線模組；而這個小東西，只要五美元左右，晶片本身在量產時甚至低到一兩美元。

但真正的震撼還在後頭。當時這款晶片幾乎沒有英文技術文件，創客們只好自己動手翻譯中文資料、逆向摸索——結果他們發現了一件被忽略的事：ESP8266 根本不只是一個「Wi-Fi 模組」，它裡面藏著一顆完整的三十二位元微控制器！換言之，你不需要外接 Arduino，這個五美元的小板子本身就能獨立跑程式、連網、控制感測器與馬達。一個原本被定位成「配件」的東西，搖身一變成了「主角」。

消息傳開後，全球創客社群沸騰了。人們自發地撰寫開發工具、移植 Arduino 開發環境、翻譯文件、分享程式庫；樂鑫也順勢投入資源提供官方 SDK 與多語文件。這家 2008 年由新加坡工程師 Teo Swee Ann 在上海創立、早年經營得相當辛苦的公司，靠著 ESP8266 一舉成名。2016 年 9 月，更強大的 ESP32 問世——雙核心處理器、Wi-Fi 加藍牙、更多記憶體與周邊，價格依然親民。而搭載攝影機的 ESP32-CAM 模組，更把「能拍照、能連網、能運算」的完整視覺節點價格，壓到了一頓便當的水準。

這場「五美元革命」的意義，遠不只是省錢。它把物聯網與邊緣運算的門檻，從企業實驗室拉低到了每一位學生、每一位創客的桌上。過去要做一個聯網的視覺感測節點，需要的預算與知識門檻讓多數人望而卻步；現在，一個高中生用零用錢就能買齊材料，做出會看、會想、會傳訊息的裝置。您課程中的 ESP32-CAM 實作，以及本書一路貫穿的「雙平台」設計(ESP32-CAM 入門、Jetson 進階)，背後的教育理念正是如此：讓每個人都有動手的機會，是創新最肥沃的土壤。

這也帶出本章的主題。前面十六章，我們的程式大多在筆電或桌機上執行——那裡有充裕的算力、記憶體與電力。但機器人不是桌機：它要在戶外奔跑、靠電池供電、即時反應、可能還沒有網路。把演算法從實驗室搬到真實的機器人身上，是一門獨立的學問，也是本課程最後的技術關卡。

思考題：ESP8266 原本被設計成「Wi-Fi 模組」，卻因為創客們發現它其實是一顆微控制器而爆紅。這個「使用者發現了設計者沒想到的價值」的故事，對你思考「技術的價值由誰定義」有什麼啟發？

17.1 為什麼需要邊緣運算

17.1.1 從雲端回到現場

在深度學習興起的頭幾年，主流想法是「把運算放到雲端」：裝置只負責拍照，把影像傳到雲端伺服器，由強大的 GPU 完成辨識，再把結果傳回來。這個架構有明顯的優點——裝置可以很便宜、模型可以很大、更新也方便。但用在機器人與工業現場，它很快就撞牆了。

邊緣運算(Edge Computing)的核心主張是：把運算搬回資料產生的現場。理由有五，每一個都攸關機器人視覺的成敗：

1. 延遲(Latency)：影像上傳、雲端運算、結果回傳，一來一回動輒數百毫秒。對一台以每秒一公尺行進的機器人，五百毫秒的延遲代表它「看到」障礙物時已經又前進了半公尺——這在避障、人機協作安全上是不可接受的。
2. 頻寬與成本：一路 1080p 30fps 的影像串流，持續上傳的頻寬與雲端運算費用相當可觀；若是一整條產線數十支攝影機，成本更是驚人。
3. 可靠性：工廠、田間、隧道、災難現場往往網路不穩或根本沒有網路。一個「斷網就瞎掉」的機器人，是不可靠的機器人。
4. 隱私：影像是高度敏感的資料。承第 16 章的倫理討論，把人臉影像上傳雲端，與在裝置上處理完只回報「有幾個人」，對隱私的意涵天差地別。端上處理是最有力的隱私保護設計之一。
5. 功耗與自主性：電池供電的機器人與無人機，無法負擔持續高功率的無線傳輸；而自主系統的本質，就是不依賴外部連線也能運作。

17.1.2 三種 AI 辨識架構

這正是第 2 章就介紹過、並在第 10、16 章反覆呼應的三種架構，現在可以完整地整合起來：

比較面向	端上運算(On-device)	近端邊緣(Edge)	雲端運算(Cloud)
典型硬體	ESP32-CAM、微控制器	Jetson、工業電腦	資料中心 GPU
算力	極低	中至高	極高
延遲	最低(毫秒級)	低	高(受網路影響)

隱私	最佳(資料不出裝置)	佳(資料不出廠區)	需審慎處理
斷網可用	可	可	不可
適用任務	簡單偵測、觸發、預篩	即時偵測、SLAM、控制	大模型訓練、複雜分析

表 17-1 三種 AI 辨識架構的比較

關鍵觀念是：這三者不是互斥的選擇，而是可以分層協作的。第 10.7.3 節的 ESP32-CAM 門禁案例就是典型的混合架構——端上用輕量演算法偵測「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把影像交給雲端做「是誰」的精細辨識(低頻、高算力)。這種「端上預篩、雲端精算」的分層設計，兼顧了即時性、成本與隱私，是實務上最常見的架構模式。

17.2 兩大實作平台：ESP32-CAM 與 Jetson

本書一路採用雙平台教學，現在正式比較它們的定位——這也是您在課程中希望學生建立的核心判斷力：選對工具，比用最強的工具更重要。

17.2.1 ESP32-CAM：入門與感測節點

承叫獸開講，ESP32-CAM 是 ESP32 加上攝影機模組的組合，價格極低、功耗極小、體積如郵票。它的定位是「會看的感測節點」：能拍照、能串流影像到網路、能執行極輕量的視覺運算(如第 10 章的 Haar 級聯人臉偵測、第 8 章的顏色追蹤、簡單的移動偵測)。

它的限制也很明確：處理器與記憶體都很有限，無法執行 YOLO 這類深度模型(即使是 nano 版也太重)。因此典型的用法有兩種：一是端上做極簡的偵測與觸發(偵測到移動就拍照、偵測到人臉就通知);二是作為「網路攝影機」，把影像串流給更強的裝置處理——這正是本書前面多章 Lab 中「以 ESP32-CAM 串流網址作為 OpenCV 輸入」的做法。它的價值不在算力，而在於用極低的成本與功耗，把「視覺」佈署到任何角落。

17.2.2 NVIDIA Jetson：機器人的視覺大腦

Jetson 系列是 NVIDIA 專為邊緣 AI 設計的模組化電腦，內建 GPU，能在數瓦到數十瓦的功耗下執行完整的深度學習模型。它跑的是 Linux，支援 CUDA、TensorRT、PyTorch、ROS 2 等完整生態系——本質上是一台「能塞進機器人裡的小型 AI 電腦」。

對機器人視覺而言，Jetson 的意義在於：它讓第 15、16 章的深度學習模型能真正「上車」。YOLO 即時偵測、語意分割、姿態估計、視覺 SLAM——這些過去只能在桌機上跑的任務，現在能在機器人身上以合理的功耗即時執行。呼應第 15 章叫獸開講：當年 AlexNet 用兩張桌上型顯卡開啟了深度學習革命，如今同一家公司的晶片，把這份算力縮小到了掌心大小，裝進了機器人的胸腔裡。

選擇建議：感測、觸發、串流、教學入門用 ESP32-CAM;需要即時深度學習推論、SLAM、多感測器融合的機器人主控用 Jetson。兩者也常搭配使用——多個 ESP32-CAM 分佈在環境中作為眼睛，Jetson 作為集中處理的大腦。

17.3 模型最佳化：讓大模型跑在小裝置上

直接把訓練好的模型丟到邊緣裝置，往往跑不動或跑太慢。模型最佳化是邊緣部署的核心技術，目標是在可接受的精度損失下，大幅降低運算量、記憶體與功耗。

17.3.1 量化

量化(Quantization)是最有效的手段：訓練時模型的權重與運算通常用 32 位元浮點數(FP32)，但推論時未必需要這麼高的精度。把它降到 16 位元浮點(FP16)或 8 位元整數(INT8)，模型大小可縮為四分之一、運算速度大幅提升、功耗顯著下降，而精度往往只損失一點點。這個「用精度換效率」的思想，你應該覺得眼熟——它正是第 3 章量化(把連續的光強度離散化)與第 11 章 JPEG 量化(把 DCT 係數粗略化)的同一個核心概念，在模型層次的再現。本書反覆出現的觀念，再一次以新的面貌登場。

17.3.2 剪枝與知識蒸餾

剪枝(Pruning)把網路中「貢獻很小」的權重或整個通道移除，讓模型變稀疏、變小。知識蒸餾(Knowledge Distillation)則訓練一個小模型(學生)去模仿大模型(老師)的輸出，讓小模型繼承大模型的能力。兩者都是「把模型變小但盡量保住能力」的策略，常與量化搭配使用。

17.3.3 推論引擎：ONNX 與 TensorRT

訓練用的框架(PyTorch、TensorFlow)為了靈活性，推論效率並非最佳。部署時通常會轉換成專用的推論格式：ONNX 是一個開放的中介格式，讓模型能在不同框架與硬體間流通(第 16 章程式 16-4 的 export 就是在做這件事);TensorRT 則是 NVIDIA 針對自家 GPU 的推論最佳化引擎，它會分析模型結構、融合運算層、選擇最佳核心、並套用量化，在 Jetson 上通常能帶來數倍的加速。對 ESP32 這類微控制器，則有 TensorFlow Lite for Microcontrollers 等專用的輕量執行環境。

架構層面的選擇同樣重要：與其把大模型硬壓小，不如一開始就選用為邊緣設計的架構——第 15 章的 MobileNet、第 16 章 YOLO 的 nano 版本，都是為此而生。這呼應了第 9、10、13 章反覆強調的判準：適合勝於最強。

程式 17-1 模型匯出與效能量測

```
from ultralytics import YOLO
import time, cv2

model = YOLO("yolo11n.pt")          # nano:為邊緣裝置設計

# --- 匯出為部署格式 ---
model.export(format="onnx")          # 通用中介格式
# model.export(format="engine", half=True)    # TensorRT + FP16(Jetson)
# model.export(format="tflite", int8=True)    # TFLite + INT8(行動/微控)
```



```
# --- 量測推論效能(部署前必做) ---
img = cv2.imread("test.jpg")
model(img)                                # 暖機一次(第一次含初始化)

N = 50
t0 = time.time()
for _ in range(N):
    model(img, verbose=False)
dt = (time.time() - t0) / N
print(f"平均延遲:{dt*1000:.1f} ms,吞吐量:{1/dt:.1f} FPS")

# 效能不足時的調整順序:
# 1. 換更小的模型(n < s < m < l < x)
# 2. 降低輸入解析度(imgsz=640 → 416 → 320)
# 3. 量化(FP16 / INT8)
# 4. 用 TensorRT 等推論引擎
# 5. 降低處理影格率(不必每一影格都偵測)
```

17.4 效能量測與系統設計

17.4.1 三個關鍵指標

部署前必須量測三個指標，它們常被混淆卻意義迥異：延遲(Latency)是「單張影像從輸入到輸出所需的時間」，決定系統的反應速度，對即時控制最關鍵；吞吐量(Throughput，常以 FPS 表示)是「每秒能處理幾張影像」，決定系統的處理能力；功耗(Power)則決定電池續航與散熱設計。

延遲與吞吐量並不等價：透過批次處理(一次處理多張)可以提高吞吐量，但單張的延遲反而變長——因為要等湊滿一批。對機器人避障這種需要即刻反應的任務，低延遲遠比高吞吐量重要；對離線的影片分析，則反之。這又是一個「依應用選指標」的工程判斷，與第 14、16 章的精確率召回率取捨同出一轍。

17.4.2 端到端的延遲預算

一個常見的錯誤是：只量測模型推論的時間，就以為那是系統的反應速度。實際上，一個機器人視覺系統的端到端延遲包含：影像擷取(相機曝光與傳輸)、前處理(縮放、正規化，第 4、5 章)、模型推論、後處理(NMS、座標轉換，第 16 章)、決策與通訊、以及致動器的反應時間。模型推論往往只佔其中一部分。

因此系統設計要做「延遲預算」：先確定任務允許的總延遲(例如機器人以 0.5 公尺/秒行進、要在碰撞前 1 公尺停下，總延遲須小於某個上限)，再回頭分配給各個環節。這種「從需求反推規格」的思維，是工程設計的基本功，也是本章從「會寫演算法」邁向「會設計系統」的關鍵一步。

17.5 整合進機器人系統：ROS 2

17.5.1 為什麼需要機器人作業系統

一個機器人不只有視覺：它有馬達、輪子、雷射雷達、IMU、機械臂、電池管理，以及導航、定位、路徑規劃、任務決策等軟體模組。如果把所有東西寫成一支巨大的程式，將難以開發、除錯與維護。ROS 2(Robot Operating System 2)提供的正是一套「讓各個模組獨立開發、彼此通訊」的框架。

它的核心概念很簡單：每個功能模組是一個節點(Node)，節點之間透過話題(Topic)以「發布-訂閱」的方式交換訊息。例如相機節點持續發布影像到 /camera/image 話題，視覺辨識節點訂閱它、處理後發布偵測結果到 /detections 話題，導航節點再訂閱偵測結果來避障。各節點可以用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作。這種鬆耦合的設計，讓團隊能分工開發、也讓模組能重複使用。

17.5.2 視覺節點的典型結構

把本課程學到的視覺能力包裝成 ROS 2 節點，是機器人視覺工程師的核心工作。典型結構是：訂閱相機影像話題 → 把 ROS 影像訊息轉成 OpenCV 格式(用 cv_bridge)→ 套用本書的處理流程(前處理、YOLO 推論、後處理)→ 把結果包裝成 ROS 訊息發布出去，供其他節點使用。

程式 17-2 ROS 2 視覺節點骨架

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray
from cv_bridge import CvBridge
from ultralytics import YOLO

class VisionNode(Node):
    def __init__(self):
        super().__init__("vision_node")
        self.bridge = CvBridge()
        self.model = YOLO("yololln.engine")    # TensorRT 最佳化版

        # 訂閱相機影像
        self.sub = self.create_subscription(
            Image, "/camera/image_raw", self.on_image, 10)
        # 發布偵測結果
        self.pub = self.create_publisher(
            Detection2DArray, "/detections", 10)

    def on_image(self, msg):
        # ROS 影像訊息 → OpenCV 格式
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")

        # 套用本書所學的處理流程
        results = self.model(frame, verbose=False)[0]
```

```

# 包裝成 ROS 訊息發布給導航等其他節點
out = Detection2DArray()
out.header = msg.header          # 保留時間戳記,供時間同步
# ... 填入 results.bboxes 的內容 ...
self.pub.publish(out)

def main():
    rclpy.init()
    rclpy.spin(VisionNode())
    rclpy.shutdown()

```

請留意 header 那一行：保留時間戳記至關重要，因為機器人要把「影像看到什麼」與「當時機器人在哪裡」對應起來——這需要精確的時間同步，也需要第 4 章的座標轉換(ROS 2 的 tf2 系統正是齊次座標與座標框架的實作，第 4.1.2 節已預告過這個連結)。

17.6 應用案例

17.6.1 案例一：自主移動機器人的視覺避障

情境：一台自主移動機器人在走廊或廠區中行進，需即時偵測前方障礙物(人、箱子、其他車輛)並避開。系統架構：Jetson 作為主控 → 相機節點發布影像 → 視覺節點以 YOLO 偵測障礙物與其位置 → 結合深度資訊(深度相機或雙目視覺)估計距離 → 發布給導航節點調整路徑。

本案例的工程重點在延遲預算(17.4.2 節)：機器人速度、煞車距離、總延遲三者必須匹配。若量測發現延遲過高，依程式 17-1 的順序最佳化——換更小的模型、降解析度、量化、用 TensorRT。同時要設計失效模式：若視覺節點當機或延遲暴增，機器人應自動減速或停止，而非盲目前進。這呼應第 16 章人機協作案例的「多層防護」原則。

17.6.2 案例二：分散式 ESP32-CAM 環境監測網

情境：在溫室、倉庫或校園中佈署數十個低成本視覺節點，監測作物生長、人員流動或設備狀態。架構：多個 ESP32-CAM 分散佈署(端上做簡單的移動偵測與觸發，平時不傳資料以省電省頻寬)→ 觸發時把影像傳給近端的 Jetson 或工業電腦 → 集中做 YOLO 辨識與分析 → 結果上傳雲端儀表板。

這是三層架構協作的典範：端上負責「有沒有事情發生」的高頻低算力判斷，近端負責「發生了什麼」的精細辨識，雲端負責長期趨勢分析與遠端存取。成本、頻寬、隱私、即時性四者在此取得平衡——這正是叫獸開講中「五美元革命」讓大規模佈署成為可能的實際體現。

17.6.3 案例三：農業無人機的田間巡檢

情境：無人機飛越農田，即時辨識作物病害、雜草分布或缺株位置。這個案例把邊緣運算的每個理由都體現得淋漓盡致：田間沒有穩定網路(可靠性)、無人機電池有

限(功耗)、需即時調整航線(延遲)、影像資料量龐大(頻寬)。因此運算必須在機上完成——用輕量模型加量化，在 Jetson 或類似的機載電腦上即時推論，只把「哪裡有病害」的座標與少量關鍵影像回傳。

技術上還需整合第 4 章的相機校正(把像素座標換算成實際的田間位置)、第 8 章的色彩處理(戶外光線變化劇烈，白平衡與對比校正是必要前處理)、第 12 章的視訊處理(連續影格的追蹤與拼接)。這個案例最能說明：邊緣部署不是把模型搬過去就好，而是整個系統的重新設計。

17.6.4 案例四：產線 AOI 的邊緣部署

情境：產線上每秒通過數個工件，需即時檢測瑕疵並剔除不良品。這是延遲要求最嚴苛的場景之一——工件通過檢測站到剔除機構的時間可能只有數百毫秒，整條處理鏈必須在此之內完成。

系統設計要點：相機用全域快門避免運動模糊(第 7.6.3 節已詳述)、打光穩定以避免第 8 章的「洋裝效應」、模型用量化與 TensorRT 最佳化、並保留餘裕以應對突發的處理延遲。此外，產線 AOI 常混用傳統與深度學習方法(第 6 章頻率域紋理檢測、第 9、10 章的傳統管線、第 16 章的 YOLO)，因為傳統方法不需瑕疵樣本、運算極輕可解釋性高——在邊緣裝置上，這些優勢格外珍貴。

17.7 部署的實務課題

把系統從「實驗室能跑」帶到「現場能用」，還有幾個常被低估的課題：

- 環境變異：實驗室的光線、背景、角度都是理想的，現場則充滿變數。這正是第 15 章談過的分布偏移——模型在訓練分布外表現會急劇下滑。對策：在真實環境蒐集訓練資料、大量使用資料增強、並持續監控上線後的表現。
- 溫度與散熱：邊緣裝置長時間高負載執行會發熱，過熱時會自動降頻，導致效能突然下降。散熱設計與功耗管理是硬體工程的一環。
- 失效模式設計：系統一定會出錯，問題是「出錯時會怎樣」。安全的設計應該是「失效時進入安全狀態」(如停止、警示)，而非繼續盲目運作。這與第 16 章人機協作的多層防護一脈相承。
- 模型更新與維運：部署不是終點。現場條件會改變、新的物件類型會出現，模型需要定期用新資料重訓與更新。如何遠端安全地更新邊緣裝置上的模型，是系統設計的一部分。
- 資料回饋迴圈：上線後蒐集模型出錯的案例、加以標註並納入下一輪訓練，是提升系統的最有效途徑。設計時就應規劃這個迴圈——這也呼應第 14 章「資料是燃料」。

17.8 本章與全書的觀念連結總表

相關章節	共用的觀念	邊緣部署中的角色
第 2 章 開發環境	三種辨識架構、ESP32-CAM	本章完整整合三架構的取舍分析

第 3、11 章	量化、壓縮	模型量化與影像量化、JPEG 量化同一思想
第 4 章 幾何轉換	相機校正、齊次座標	像素換算實際位置;ROS 2 的 tf2 座標框架
第 5、6 章	可分離濾波、頻率域成本	演算法的運算成本評估
第 7 章 影像還原	運動模糊、全域快門	產線 AOI 的取像設計
第 10 章 二值影像	Haar 級聯輕量偵測	ESP32-CAM 端上運算的可行方案
第 12 章 視訊處理	即時性、視覺里程計	影格率預算;機器人自我定位
第 13 章 特徵擷取	ORB 輕量特徵	SLAM 的核心特徵，適合邊緣運算
第 15、16 章	MobileNet、YOLO nano、匯出	為邊緣設計的架構;ONNX/TensorRT 部署

表17-2 第17章與全書其他章節的觀念連結

程式實作 Lab 17

Lab 17-1 效能量測與最佳化

目標：建立效能工程的實作能力。(a)以程式 17-1 量測 YOLO 在你的裝置上的延遲與 FPS，注意排除第一次推論的初始化時間;(b)依序測試五種最佳化：換小模型(n/s/m)、降解析度(640/416/320)、FP16 量化、ONNX 匯出、跳影格處理，記錄每種對延遲與 mAP 的影響，製成取舍表;(c)畫出「精度 vs. 速度」的散點圖，標出各種組合的位置，找出符合你需求的最佳點;(d)討論：若目標是 30 FPS 即時處理，你會選哪個組合?若目標是最高精度而不在意速度呢?

Lab 17-2 ESP32-CAM 視覺節點

目標：實作端上運算與串流兩種模式。(a)燒錄 ESP32-CAM 韌體，設定 Wi-Fi 並取得串流網址;(b)模式一(端上)：在 ESP32-CAM 上執行簡單的移動偵測(影格差分，第 12 章)，偵測到變化才拍照儲存或通知;(c)模式二(串流)：以 OpenCV 讀取串流，在電腦端執行 YOLO 偵測;(d)比較兩種模式的延遲、頻寬使用與功耗;(e)討論：什麼應用適合模式一?什麼適合模式二?如何設計成「端上預篩、近端精算」的混合架構(第 17.1.2 節)?

Lab 17-3 端到端延遲預算

目標：從系統角度思考。(a)為一個「移動機器人視覺避障」系統做延遲預算：設定機器人速度與煞車距離，推算允許的總延遲上限;(b)實測系統各環節的耗時：影像擷取、前處理、推論、後處理、通訊，製成延遲分解表;(c)找出瓶頸環節並最佳化;(d)

設計失效模式：若視覺節點延遲超過門檻或當機，系統該如何反應？寫出你的安全邏輯；(e)討論：為什麼「只量測模型推論時間」是危險的？

Lab 17-4 ROS 2 視覺節點整合(期末專題前導)

目標：把本課程的視覺能力包裝成機器人可用的模組。(a)以程式 17-2 為骨架，建立一個 ROS 2 節點，訂閱相機影像、執行 YOLO 偵測、發布偵測結果；(b)用 `ros2 topic echo` 驗證訊息正確發布，用 `rqt_graph` 查看節點與話題的關係圖；(c)加入 `header` 時間戳記，確認時間同步正確；(d)進階：再寫一個簡單的訂閱節點，依偵測結果印出「前方有障礙物，建議停止」等決策訊息，模擬視覺與導航的協作；(e)這個節點將是第 18 章期末專題的基礎模組，請妥善保存並撰寫使用說明文件。

本章重點整理

- 邊緣運算的五個理由：低延遲(即時控制)、省頻寬與成本、斷網仍可靠、保護隱私(資料不出裝置)、低功耗與自主性；三種架構(端上、近端邊緣、雲端)非互斥，常以「端上預篩、近端精算、雲端分析」分層協作。
- ESP32-CAM 定位為極低成本、低功耗的視覺感測節點(端上做簡單偵測或串流影像)；Jetson 為內建 GPU 的邊緣 AI 電腦，能即時執行深度模型與 ROS 2，是機器人的視覺大腦。
- 模型最佳化手段：量化(FP32→FP16/INT8，與第 3、11 章的量化同一思想)、剪枝、知識蒸餾；部署格式 ONNX(通用)、TensorRT(NVIDIA GPU)、TFLite(行動與微控制器)；更根本的是選用為邊緣設計的架構(MobileNet、YOLO nano)。
- 三個效能指標：延遲(反應速度，即時控制的關鍵)、吞吐量(FPS，處理能力)、功耗；兩者不等價，批次處理提高吞吐量卻增加延遲。系統設計應做端到端的延遲預算，而非只量測模型推論時間。
- ROS 2 以節點與話題的發布訂閱架構整合機器人各模組；視覺節點的典型結構為訂閱影像、`cv_bridge` 轉換、處理推論、發布結果，並須保留時間戳記以供時間同步與座標轉換(`tf2`，呼應第 4 章齊次座標)。
- 部署實務課題：環境變異造成的分布偏移、溫度與散熱降頻、失效模式應設計為「失效即安全」、模型更新與遠端維運、以及蒐集錯誤案例重訓的資料回饋迴圈。

習題

1. 請列出邊緣運算相較純雲端架構的五個理由，並各舉一個機器人或工業場景說明。

提示與參考解答：一、低延遲：機器人避障需毫秒級反應，雲端一來一回數百毫秒不可接受。二、頻寬與成本：一整條產線數十支攝影機持續上傳成本驚人。三、可靠性：田間、隧道、災難現場網路不穩或無網路。四、隱私：人臉影像在端上處理完只回報數量，遠優於上傳雲端。五、功耗與自主性：電池供電的無人機無法負擔持續高功率傳輸，自主系統本質上不應依賴外部連線。

2. 比較端上、近端邊緣、雲端三種架構的算力、延遲、隱私與斷網可用性。為什麼說它們不是互斥的？

提示與參考解答：端上(ESP32-CAM)：算力極低、延遲最低、隱私最佳、斷網可用。近端(Jetson)：算力中高、延遲低、隱私佳、斷網可用。雲端：算力極高、延遲高且受網路影響、隱私需審慎、斷網不可用。三者可分層協作：端上做高頻低算力的預篩(有沒有事發生)、近端做精細辨識(發生了什麼)、雲端做長期分析與遠端存取，兼顧即時性、成本與隱私。

3. ESP32-CAM 的能力邊界在哪裡？它的兩種典型用法是什麼？

提示與參考解答：邊界：處理器與記憶體極有限，無法執行YOLO 等深度模型(即使 nano 版也太重)。兩種用法：一是端上做極簡偵測與觸發(移動偵測、Haar 級聯人臉偵測、顏色追蹤，偵測到才拍照或通知)；二是作為網路攝影機，串流影像給更強的裝置(電腦或Jetson)處理。價值不在算力，而在極低成本與功耗讓視覺能佈署到任何角落。

4. 模型量化是什麼？它與第 3 章的影像量化、第 11 章的 JPEG 量化在思想上有何共通？

提示與參考解答：模型量化把權重與運算從 32 位元浮點降到 16 位元浮點或 8 位元整數，模型縮小、加速、省電，精度僅損失少許。共通思想是「用精度換效率」：第 3 章把連續光強度離散成有限灰階、第 11 章 JPEG 把 DCT 係數除以量化步長取整(丟棄人眼不敏感的高頻)、本章把模型參數的精度降低——都是在可接受的品質損失下換取儲存與運算的大幅節省。

5. ONNX 與 TensorRT 各扮演什麼角色？為什麼訓練框架不適合直接用於部署？

提示與參考解答：訓練框架(PyTorch、TensorFlow)為靈活性而設計，推論效率非最佳。ONNX 是開放的中介格式，讓模型能在不同框架與硬體間流通。TensorRT 是 NVIDIA 針對自家 GPU 的推論最佳化引擎，會分析模型結構、融合運算層、選最佳核心並套用量化，在 Jetson 上常帶來數倍加速。微控制器則用 TensorFlow Lite for Microcontrollers 等輕量執行環境。

6. 延遲與吞吐量有何差異？為什麼提高吞吐量有時反而增加延遲？各適合什麼應用？

提示與參考解答：延遲是單張影像從輸入到輸出的時間(反應速度)；吞吐量是每秒能處理幾張(處理能力)。批次處理一次處理多張可提高吞吐量，但要等湊滿一批，單張延遲反而變長。機器人避障等即時控制重延遲，離線影片分析重吞吐量。這與第 14、16 章「依應用選指標」的工程判斷同出一轍。

7. 什麼是端到端延遲預算？為什麼「只量測模型推論時間」是危險的？

提示與參考解答：端到端延遲包含影像擷取、前處理、模型推論、後處理、決策通訊、致動器反應等全部環節，模型推論往往只佔一部分。延遲預算是先確定任務允許的總延遲(如依機器人速度與煞車距離推算)，再回頭分配給各環節。只量測推論時間會嚴重低估實際反應時間，導致系統在真實場景中反應不及。

8. 若量測發現 YOLO 在你的邊緣裝置上跑不到目標 FPS，你會依什麼順序嘗試最佳化？

提示與參考解答：依程式 17-1 的順序：一、換更小的模型($n < s < m < l < x$)；二、降低輸入解析度(640→416→320)；三、量化(FP16 或 INT8)；四、改用 TensorRT 等推論引擎；五、降低處理影格率(不必每一影格都偵測，可搭配第 12 章的追蹤填補中間影格)。每一步都要重新量測延遲與 mAP，評估精度損失是否可接受。

9. ROS 2 的節點與話題是什麼?這種架構帶來什麼好處?

提示與參考解答：節點是獨立的功能模組(相機、視覺辨識、導航等)，話題是節點間以「發布-訂閱」方式交換訊息的通道。好處：鬆耦合設計讓各模組能獨立開發、除錯與維護，可用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作，模組也能重複使用，團隊可分工。

10. ROS 2 視覺節點為什麼要保留影像訊息的時間戳記(header)?這與第 4 章有什麼關聯?

提示與參考解答：因為機器人要把「影像看到什麼」與「當時機器人在哪裡」對應起來，需要精確的時間同步(影像有處理延遲，不能用當下的位置)。與第 4 章的關聯：ROS 2 的 tf2 座標轉換系統正是齊次座標與座標框架的實作，把相機座標系的偵測結果轉換到機器人或世界座標系，才能用於導航決策。

11. 部署到真實環境時，有哪些實驗室不會遇到的課題?請至少說明三項及其對策。

提示與參考解答：一、環境變異造成分布偏移(第 15 章)：現場光線背景角度多變，模型表現下滑；對策是在真實環境蒐集資料、大量資料增強、持續監控上線表現。二、溫度與散熱：長時間高負載發熱會自動降頻導致效能驟降；需散熱設計與功耗管理。三、失效模式：系統必然會出錯，應設計成「失效即安全」(停止、警示)而非盲目運作。另有模型更新與遠端維運、資料回饋迴圈等。

12. (綜合設計題)請為「校園自主巡邏機器人」設計視覺系統架構：列出硬體選擇、模型與最佳化策略、ROS 2 節點劃分、延遲預算、以及失效安全設計，並說明每項決策的理由。

提示與參考解答：本題為開放設計題，評分重點在決策理由的完整性。參考方向——硬體：Jetson 作為主控(需即時深度學習與 ROS 2)，搭配相機與可能的深度感測器；若需多點監測可加 ESP32-CAM 節點。模型：YOLO nano 或小型模型加 FP16/TensorRT 最佳化，兼顧速度與精度。節點劃分：相機節點、視覺辨識節點、定位節點、導航節點、決策節點，以話題通訊並保留時間戳記。延遲預算：依巡邏速度與煞車距離反推總延遲上限，分配給擷取、前處理、推論、後處理、通訊、致動各環節。失效安全：視覺節點逾時或當機則減速停止並警示；多層防護不單靠視覺(可加超音波或碰撞感測器，呼應第 16 章)。另應考慮隱私(校園有大量人員，建議端上處理、不儲存人臉)。

參考資料與延伸閱讀

- Espressif Systems 官方文件與 ESP32 技術規格：
<https://www.espressif.com>(本章叫獸開講之史料來源)
- NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南：
<https://developer.nvidia.com/embedded-computing>
- ROS 2 官方教學文件：<https://docs.ros.org>
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.(第 15 章亦引用)
- Jacob, B., et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR, 2018.
- Ultralytics 部署與匯出文件：<https://docs.ultralytics.com>
- 補充教材投影片下載：<https://page.line.me/prof>(機器人叫獸)

影像處理 Image Processing

國立雲林科技大學 智慧科技學院 智慧機器人學程 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 18 章 期末專題：4P 實踐

本章學習目標

- 闡述 4P 創意學習(Projects、Passion、Peers、Play)的理念與其在工程教育中的意義。
- 說明創意學習螺旋的五個階段，並以之規劃專題的迭代開發流程。
- 選定一個兼具熱情與可行性的專題題目，並拆解為可執行的里程碑。
- 運用本書十七章的技術，整合出一個端到端的視覺應用系統。
- 進行有效的團隊協作、版本管理與技術文件撰寫。
- 完成專題成果的展示、反思與評估，並規劃後續的學習路徑。

叫獸開講|終身幼兒園：千年來最重要的發明是什麼？

世紀之交，一場研討會上有人拋出了一個問題：過去一千年來，最重要的發明是什麼？與會者陸續給出答案——有人說是印刷術，讓知識得以大量流通；有人說是蒸汽機，啟動了工業革命；有人說是電燈泡，把人類的活動延伸到夜晚。輪到麻省理工學院媒體實驗室的 Mitchel Resnick 教授時，他給了一個讓全場愣住的答案：幼兒園。

這聽起來像個玩笑，但 Resnick 是認真的。他的理由是：一百多年前誕生的幼兒園，是人類教育史上一次根本的創新——它不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解。孩子們用積木蓋城堡、用蠟筆畫故事、在遊戲中學會協商與分享。這個過程，正是創造力的搖籃。

Resnick 觀察到一個令人遺憾的現象：離開幼兒園之後，學校教育就變了樣——孩子開始長時間坐在課桌前，填寫習題、聽講、應付考試。更糟的是，近年連幼兒園本身都在「小學化」，積木與手指畫被數學練習卷與識字卡取代。他的主張因此格外鮮明：不該讓幼兒園變得像其他學校，而該讓其他學校——甚至整個人生——變得更像幼兒園。

這不是要大人去玩積木，而是要保留幼兒園那套學習方式的精髓。Resnick 帶領的研究群就叫「終身幼兒園」(Lifelong Kindergarten)，他們把這套精髓提煉成四個以 P 開頭的原則：專題(Projects)、熱情(Passion)、同儕(Peers)、遊玩(Play)——讓人們基於自己的熱情、與同儕協作、以遊玩的精神，投入專題的創作。這個研究群的成果你可能早就用過：他們開發了全球最大的兒童程式學習平台 Scratch，也與樂高合作催生了 LEGO Mindstorms 機器人套件。Resnick 的學術血脈同樣耀眼——他的博士指導教授是建構主義學習理論的先驅 Seymour Papert。

而這，正是這門課從第一章走到現在的隱藏主線。回頭看看：每一章開頭的「叫獸開講」，講的都是一個真實的人、在真實的處境中，因為某種熱情或不甘

而做出的創造——傅立葉在行政庶務之餘寫下被退稿的論文、張正友用一張紙取代昂貴的標定物、Muybridge 為了一場關於馬蹄的爭論架起一整排相機、Redmon 在成就的頂點選擇放下、樂鑫的一群工程師讓五美元的晶片改變了創客世界。這些故事不是花絮，它們是想告訴你：技術從來不是冷冰冰的公式，而是活生生的人，帶著熱情與判斷，一步步做出來的。

現在，輪到你了。期末專題不是一份要交出去的作業，而是這門課給你的一次機會：去做一個你真心想做的東西，和你的夥伴一起，帶著玩的心情，把這十七章學到的技術變成一個真實存在的作品。做出來之後，它會是你的——不是老師的、不是課本的，是你的。

思考題：回想你的求學歷程，哪一次學習經驗讓你最投入、學得最深刻？它符合 4P 中的哪幾項？為什麼那次經驗與「為了考試而讀書」感覺如此不同？

18.1 4P 創意學習

4P 是本課程的教學設計骨架，也是期末專題的評量精神。理解它，你才會明白為什麼這門課要這樣上、專題要這樣做。

18.1.1 Projects：專題

人們在動手創造具體事物的過程中學得最深。聽講與寫習題能傳遞資訊，但真正的理解來自於「試著做出來卻卡住，然後想辦法解決」的那個當下。當你為了讓機器人認出障礙物而反覆調整參數、當你發現訓練出來的模型在教室裡很準、拿到走廊就失靈——那些挫折時刻學到的東西，遠比課本上的段落深刻。這也是為什麼本書每一章都有 Lab，而整門課以專題收尾：知識要透過創造才能真正內化。

18.1.2 Passion：熱情

這是 4P 中最容易被忽略、卻最關鍵的一項。當人們投入自己真正在乎的題目時，願意付出更多時間、承受更多挫折、學得更多東西。所以選題時，叫獸的建議永遠是：選一個你自己會想用、會想給家人朋友看的東西。是替阿嬤的果園做病蟲害偵測？是幫社團管理器材借還？是做一個能陪你打電動的機器人？題目不必偉大，但必須是你的。

請特別注意：熱情不是「一開始就有」，更多時候是「做著做著長出來的」。若你暫時想不到有熱情的題目，就先動手做一個看起來有趣的，過程中的小成就感會慢慢點燃它。

18.1.3 Peers：同儕

創造很少是孤獨的事。與同伴協作、互相給予回饋、分享彼此的發現，能讓每個人走得更遠。專題採分組進行，不只是為了分攤工作量，更是因為：當你必須向隊友解釋你的做法時，你會發現自己理解上的漏洞；當隊友提出你沒想過的角度時，你的方案會變得更好。

同儕也不限於同組隊友。向其他組請教、參考開源社群的做法、在網路上提問——這些都是同儕學習。承第 13 章 ORB 的故事、第 17 章 ESP32 的故事：整個開源與創客文化，本身就是「同儕協作」在全球尺度上的展現。

18.1.4 Play: 遊玩

遊玩的精神，指的是願意嘗試、不怕失敗、樂於實驗的心態。它與「不認真」正好相反——遊玩需要高度的投入，只是這份投入來自好奇而非壓力。在專題中，這意味著：大膽試試看那個不確定會不會成功的想法、把出錯的結果當成有趣的線索而非災難、在做出基本功能後想想「如果加上這個會怎樣」。

工程上，遊玩的精神有一個實際的名字叫「快速原型」：先做一個粗糙但能動的版本，從中學習，再改進。這比在紙上完美規劃三個月然後一次做對，更符合真實的創造歷程。

18.2 創意學習螺旋

Resnick 觀察幼兒園裡孩子玩積木的過程，提煉出一個循環，稱為創意學習螺旋——這也是專題開發最貼近真實的流程模型：

1. 想像(Imagine)：產生一個想法——「我想做一個能自動辨識回收物的垃圾桶」。
2. 創造(Create)：動手把它做出來，哪怕只是最粗糙的版本。
3. 遊玩(Play)：實際玩玩看、測試它，在各種情況下試探它的行為。
4. 分享(Share)：給別人看、聽取回饋——同學、老師、家人的反應往往能點出你沒發現的問題。
5. 反思(Reflect)：想想哪裡好、哪裡不好、為什麼——然後回到「想像」，展開下一輪。

之所以稱為「螺旋」而非「循環」，是因為每繞一圈，你都回到起點但站在更高的位置：你的想法更成熟、技術更純熟、對問題的理解更深。這正是專題應有的節奏——不要期待一次做對，而要規劃多次迭代。

這個螺旋與工程界的敏捷開發、與您在本書各章 Lab 中反覆經歷的「試參數、看結果、再調整」，骨子裡是同一件事。而它也解釋了為什麼 18.3 節的專題時程要切成多個里程碑：每個里程碑就是螺旋的一圈。

18.3 專題的規劃與執行

18.3.1 選題：熱情與可行性的交集

好的題目落在兩個圓的交集：一邊是「你真心想做」(Passion)，另一邊是「在時間與資源內做得出來」(可行性)。只有熱情而不可行，會做到一半崩潰；只有可行而無熱情，會做得索然無味、成果平庸。

可行性的評估要誠實面對四件事：時間(一學期實際能投入多少小時?)、資料(需要多少標註影像?蒐集得到嗎?)、硬體(有沒有相機、Jetson、機構?)、技術落差(有多少是你沒學過、需要現學的?)。一個實用的技巧是設定「最小可行版本」：先定義一個「一定做得出來」的核心功能，確保專題有底線；行有餘力再往上加。這比訂一個宏大目標卻做不完要好得多。

選題方向的建議：從你熟悉的生活場域出發——雲林在地的農業(果實計數、病害辨識、雜草偵測)、製造業(工件檢測、儀表判讀、安全監控)、校園生活(教室人數統計、器材管理、無障礙輔助)、或個人興趣(運動姿勢分析、寵物監控、遊戲互動)。在地與生活化的題目，往往最能激發熱情，也最容易取得資料。

18.3.2 里程碑規劃

把一學期的專題切成四到五個里程碑，每個里程碑對應創意學習螺旋的一圈——都要有可展示的成果，而非只是「進度報告」：

里程碑	階段	主要工作	可展示的成果
M1	提案與可行性	選題、調查現有做法、評估資源	提案簡報、系統架構草圖
M2	資料與原型	蒐集標註資料、跑通最小可行版本	資料集、能動的粗糙原型
M3	核心功能	完成主要演算法、量化評估	準確率/mAP 等指標、實測影片
M4	整合與部署	系統整合、效能最佳化、實地測試	可執行的完整系統
M5	展示與反思	文件、簡報、成果發表	報告、demo、反思紀錄

表18-1 專題里程碑規劃

請特別重視 M2 的「粗糙原型」：許多專題失敗的原因，是把太多時間花在規劃與準備，直到後期才發現核心技術根本行不通。先做一個能動的版本(哪怕只有百分之三十的準確率)，你就能及早發現真正的困難點，並據此調整方向——這正是遊玩精神的工程版本。

18.3.3 團隊協作與工程實務

三個實務建議，它們是專業工程師的基本功：

- 版本管理：用 Git 管理程式碼與文件。它不只是備份，更讓你能安心嘗試(改壞了隨時可以回復)、讓多人能平行開發、也讓你回頭檢視「當時為什麼這樣改」。
- 分工與整合：依模組分工(如影像處理、模型訓練、硬體整合、介面)，但務必約定好模組之間的介面(輸入什麼、輸出什麼格式)，並儘早進行整合測試。太晚整合是團隊專題最常見的災難。
- 工程筆記：記錄每次實驗的參數、結果與觀察。三個月後你絕對不會記得「那次準確率 92% 是用什麼設定跑出來的」。這份筆記也是期末報告最寶貴的素材。

18.4 專題方向與技術地圖

本節把全書十七章的技術整理成一張「技術地圖」，幫助你在選定題目後，知道該回去複習哪幾章。

專題類型	範例題目	主要用到的章節
農業與生態	果實計數估產、病害辨識、雜草偵測、生物監測	第 8(色彩)、14-16(模型)、17(部署)
工業檢測	表面瑕疵偵測、零件計數量測、儀表判讀	第 5-7、9-11(傳統)、16(YOLO)
機器人視覺	循跡避障、物件抓取定位、視覺導航	第 4(校正)、12-13(追蹤特徵)、17(ROS 2)
人機互動	手勢控制、運動姿勢分析、跌倒偵測	第 12(視訊)、16(Pose)、17(即時)
文件與辨識	車牌辨識、表單數位化、手寫辨識	第 4、9-10(分割)、13-15(辨識)
影像品質與修復	低光增強、老照片修復、去雜訊比較	第 5-7(強化還原)、11(小波)
智慧生活	人流計數、垃圾分類、無障礙輔助	第 12(追蹤)、16(偵測)、17(邊緣)

表 18-2 專題方向與技術地圖

選題時的一個重要提醒：不要為了用最新的技術而選題，要為了解決問題而選技術。承第 9、10、13、16 章反覆強調的判準——一個用 Otsu 加連通元件就能穩定運作的零件計數系統，價值不亞於一個用上最新模型卻不穩定的系統。工程的價值在於解決問題，不在於炫技。

18.5 成果展示與評估

18.5.1 如何做一場好的展示

技術做得好，還要說得清楚——這是工程師常被低估的能力。一場好的專題展示建議這樣安排：

- 從問題出發：先說「為什麼要做這個」，用一個具體的情境或痛點抓住聽眾。不要一開場就講你用了什麼模型。
- 展示成果：盡早讓聽眾看到「它會動」——現場 demo 或實測影片，比任何投影片都有說服力。
- 說明方法：再回頭解釋你怎麼做到的，重點放在關鍵決策與取捨(為什麼選這個方法而非那個)，而非流水帳。

9. 誠實呈現限制：說明系統在什麼情況下會失敗、目前的準確率、還有什麼沒做到。誠實不會扣分，反而顯示你真正理解自己的系統——這也呼應第 7、16 章的工程倫理。
10. 反思與展望：分享你學到什麼、若重來會怎麼做、下一步可以往哪走。

18.5.2 評量向度

本專題的評量對應 4P 精神，不只看最終成果，更看過程與成長：

向度	評量重點	對應的 4P
技術實作	技術選擇是否適當、實作是否正確、有無量化評估	Projects
問題意識	題目是否有明確的問題與價值、動機是否真誠	Passion
團隊協作	分工是否合理、整合是否順暢、是否互相學習	Peers
迭代與探索	是否經歷多次迭代、是否勇於嘗試與修正	Play
溝通與文件	報告與展示是否清楚、文件是否完整可重現	Peers / Projects
反思與倫理	是否誠實面對限制、是否考量使用者與社會影響	全部

表 18-3 專題評量向度

請注意最後一列「反思與倫理」。承第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤、第 16 章的 Redmon 故事、第 17 章的隱私設計——在你的專題報告中，請務必回答三個問題：誰會從這個系統受益？誰可能受到傷害？如果它出錯了，後果由誰承擔？這不是加分題，而是專業工作者的基本功。

18.6 專題案例參考

以下四個案例示範不同類型的專題如何整合全書技術。它們不是標準答案，而是幫助你想像可能性。

18.6.1 案例一：果園產量估計系統

問題：小農難以在採收前準確估計產量，影響銷售規劃與人力調度。做法：用手機或 ESP32-CAM 拍攝果樹 → 以 YOLO 偵測並計數果實(第 16 章) → 用第 8 章的色彩分析判斷成熟度 → 用第 4 章的相機校正估計果實大小 → 統計輸出估產報告。

技術挑戰與對策：果實相互遮擋(可結合多角度拍攝，或訓練模型學習遮擋樣態)、光線變化劇烈(第 5、8 章的前處理)、標註資料不足(第 15 章的遷移學習與資料增強)。最小可行版本：先做單一品種、固定距離拍攝的計數，再逐步擴充。

18.6.2 案例二：工件瑕疵檢測站

問題：人工目視檢測易疲勞、標準不一致。做法：固定打光與相機(第 17 章的取像設計)→前處理拉平背景(第 5 章 CLAHE 或第 10 章頂帽)→雙軌檢測：傳統管線(第 6 章頻率域紋理異常、第 9、10 章分割與型態學)與深度學習(第 16 章 YOLO)並行→比較兩者的準確率、速度與對「未見過瑕疵」的表現。

這個題目的教育價值在於：它讓你親身體驗全書反覆強調的判準——傳統方法不需瑕疵樣本、可解釋、運算輕；深度學習強健、能處理複雜形態。透過實測數據來論證「什麼時候該用什麼」，遠比背誦結論深刻。

18.6.3 案例三：互動式手勢控制機器人

問題：讓人以自然的手勢指揮機器人，而非透過遙控器。做法：以 YOLO Pose 或手部關鍵點偵測取得手勢(第 16 章)→定義手勢語彙(如張開手掌代表停止、指向代表前進方向)→加入時序判斷避免誤觸發(第 12 章)→透過 ROS 2 發布控制指令給機器人(第 17 章)→在 Jetson 上即時執行。

這個案例最能體現 Play 的精神——手勢的設計本身就是一種創作，你可以玩出各種有趣的互動方式。工程重點在即時性(第 17 章的延遲預算)與可靠性(誤判會讓機器人做出非預期動作，須設計安全機制)。

18.6.4 案例四：低光影像增強比較研究

問題：夜間或室內昏暗環境下拍攝的影像品質不佳，哪種方法最有效？做法：自建一組含「暗處拍攝」與「充足光線參考」配對的影像資料集→分別實作與比較：Gamma 校正與 CLAHE(第 5 章)、多影格平均(第 5 章夜景模式)、維納濾波與非局部平均(第 7 章)、小波門檻化(第 11 章)、深度學習增強模型(第 15 章)→以 PSNR、SSIM(第 3 章)與主觀評分量化比較。

這是一個「研究型」而非「系統型」的專題，適合對演算法本身有興趣的同學。它的價值在於系統性的比較與誠實的數據呈現——這正是工程研究的核心素養，也直接對應 M3 里程碑要求的量化評估。

18.7 全書技術整合地圖

走到這裡，你已經走完了從像素到系統的完整旅程。這張表回顧全書的四篇架構也是你規劃專題時的索引。

篇	章節	核心能力
第一篇 基礎	第 1-3 章	影像的本質、開發環境、取樣量化與品質評估
第二篇 核心	第 4-12 章	幾何轉換、強化、頻率域、還原、色彩、分割、型態學、小波、視訊
第三篇 學習	第 13-16 章	特徵擷取、機器學習、深度學習、物件偵測
第四篇 整合	第 17-18 章	邊緣部署與機器人整合、專題實踐

表18-4 全書四篇架構與核心能力

也請記得那些貫穿全書、比任何單一技術更重要的思想：換一個域看問題(第 6 章)、多解析度與由粗到細(第 11、13、15 章)、投票對抗雜訊(第 9、13 章)、正則化對抗病態(第 7、14 章)、精確率與召回率的取捨(第 9、10、14、16 章)、以及「適合勝於最強」(第 9、10、11、13、16、17 章)。技術會更迭，這些思考方式不會。

程式實作 Lab 18：期末專題

Lab 18-1(M1)提案與可行性評估

目標：找到一個你真心想做的題目。(a)腦力激盪：每人先獨立列出五個想做的題目，不要自我審查；(b)組內分享並票選，依「熱情」與「可行性」兩軸把候選題目畫在座標圖上；(c)對選定的題目做調查：是否已有類似作品？他們怎麼做？你的切入點是什麼？(d)畫出系統架構草圖：輸入是什麼、經過哪些處理、輸出什麼、用什麼硬體；(e)定義最小可行版本與延伸目標；(f)產出提案簡報，包含問題描述、目標、方法、時程與分工。

Lab 18-2(M2)資料蒐集與原型

目標：及早驗證核心技術可行。(a)蒐集資料：注意第 14 章的教訓——不同類別要在相同條件下拍攝，避免模型學到「捷徑」；記錄拍攝條件；(b)標註並依第 14 章原則切分訓練/驗證/測試集；(c)做出最小可行原型，哪怕準確率很低、介面很醜——重點是「能從頭跑到尾」；(d)記錄第一次的量化結果作為基準線；(e)反思：實際做過之後，原本的規劃有哪些需要調整？

Lab 18-3(M3-M4)核心開發與整合部署

目標：把原型變成可用的系統。(a)迭代改進核心演算法，每次改動都記錄參數與結果(工程筆記)；(b)量化評估：依任務選擇適當指標(準確率、mAP、PSNR 等)，並誠實記錄失敗案例；(c)系統整合：把各模組串接，進行端到端測試；(d)效能最佳化與部署(第 17 章)：量測延遲與 FPS，依需要最佳化；(e)實地測試：把系統帶到真實環境(果園實驗室、走廊)測試，記錄與實驗室結果的差異——這往往是最有價值的學習；(f)依測試結果再迭代一輪。

Lab 18-4(M5)展示、文件與反思

目標：完整交付並沉澱學習。(a)撰寫技術文件：系統架構、環境安裝、使用說明、程式碼結構——標準是「另一個人照著做能重現你的成果」；(b)準備展示：依 18.5.1 節的五個步驟編排，務必現場 demo 或實測影片；(c)誠實呈現限制與失敗案例；(d)回答三個倫理問題：誰受益、誰可能受害、出錯了誰承擔；(e)個人反思：這學期你學到最重要的一件事是什麼？若重來一次會怎麼做？接下來想繼續學什麼？(f)把成果整理成作品集——這是你日後求職、升學最有力的證明。

本章重點整理

- 4P 創意學習：Projects(在創造中學習)、Passion(投入真心在乎的題目)、Peers(協作與回饋)、Play(勇於嘗試、不怕失敗);源自 MIT Media Lab 的終身幼兒園研究群。
- 創意學習螺旋：想像、創造、遊玩、分享、反思，循環迭代且每繞一圈都站得更高;對應專題的多次迭代與里程碑規劃。
- 選題落在「熱情」與「可行性」的交集;可行性須誠實評估時間、資料、硬體與技術落差，並設定最小可行版本以確保底線。
- 里程碑 M1 提案、M2 資料與原型、M3 核心功能、M4 整合部署、M5 展示反思;每個里程碑都要有可展示的成果，並儘早做出粗糙原型以及早發現困難。
- 團隊實務：用 Git 版本管理、依模組分工但約定好介面並儘早整合、撰寫工程筆記記錄實驗參數與結果。
- 展示應從問題出發、盡早展示成果、說明關鍵取舍、誠實呈現限制、並反思展望;評量對應 4P，涵蓋技術、問題意識、協作、迭代、溝通與倫理反思。

習題

1. 請說明 4P 各自的意義。為什麼說「熱情」是其中最容易被忽略卻最關鍵的一項？

提示與參考解答：Projects：在動手創造具體事物的過程中學得最深。Passion：投入真心在乎的題目。Peers：與同伴協作、互相回饋。Play：願意嘗試、不怕失敗的實驗心態。熱情最關鍵是因為它決定了你願意投入多少時間、承受多少挫折——而學習的深度往往正比於投入；它也最易被忽略，因為傳統作業多是老師指定題目，學生沒有選擇的餘地。

2. 承叫獸開講：Resnick 為什麼認為「幼兒園」是千年來最重要的發明？他對學校教育的主張是什麼？

提示與參考解答：因為幼兒園是教育史上的根本創新——不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解，這是創造力的搖籃。他的主張：不該讓幼兒園變得像其他學校(近年幼兒園小學化，積木被練習卷取代)，而該讓其他學校、甚至整個人生變得更像幼兒園——保留那套學習方式的精髓(即 4P)。

3. 請說明創意學習螺旋的五個階段。為什麼稱為「螺旋」而非「循環」？

提示與參考解答：五階段：想像(產生想法)、創造(動手做出來)、遊玩(測試試探)、分享(給人看、聽回饋)、反思(想想哪裡好壞與為什麼)，然後回到想像。稱為螺旋是因為每繞一圈都回到起點但站在更高位置——想法更成熟、技術更純熟、理解更深。它提醒我們專題不應期待一次做對，而要規劃多次迭代。

4. 好的專題題目應落在哪兩個條件的交集？若只有其中之一會發生什麼？

提示與參考解答：落在「熱情(你真心想做)」與「可行性(時間與資源內做得出來)」的交集。只有熱情而不可行：會做到一半崩潰、無法交付。只有可行而無熱情：會做得索然無味、成果平庸、學習有限。可行性須誠實評估時間、資料、硬體、技術落差四項。

5. 什麼是「最小可行版本」?為什麼 M2 階段就要做出粗糙原型,而不是先完整規劃?

提示與參考解答:最小可行版本是一個「一定做得出來」的核心功能,確保專題有底線,行有餘力再往上加。及早做粗糙原型的理由:許多專題失敗是因為把太多時間花在規劃準備,後期才發現核心技術行不通。先做能從頭跑到尾的版本(哪怕準確率很低),就能及早發現真正的困難點並調整方向——這是遊玩精神的工程版本(快速原型)。

6. 團隊專題有哪三項工程實務建議?為什麼「太晚整合」是常見的災難?

提示與參考解答:一、用Git 版本管理(不只備份,更讓你安心嘗試、多人平行開發、回顧修改理由)。二、依模組分工但務必約定介面(輸入輸出格式)並儘早整合測試。三、撰寫工程筆記記錄每次實驗的參數、結果與觀察。太晚整合的災難:各模組單獨都能跑,合起來卻因介面不符、時序問題、環境衝突而全面卡關,而此時已無時間修正。

7. 一場好的專題展示建議依什麼順序安排?為什麼「誠實呈現限制」不會扣分?

提示與參考解答:順序:一、從問題出發(為什麼要做,用具體情境抓住聽眾);二、盡早展示成果(demo 或實測影片最有說服力);三、說明方法(重點在關鍵決策與取舍,而非流水帳);四、誠實呈現限制(何時會失敗、目前準確率、還有什麼沒做到);五、反思與展望。誠實不扣分,反而顯示你真正理解自己的系統——這也是第7、16 章工程倫理的體現;隱藏限制的系統反而危險。

8. 在選擇專題技術時,為什麼「不要為了用最新技術而選題」?請以本書的判準說明。

提示與參考解答:因為工程的價值在於解決問題,不在炫技。承第9、10、11、13、16、17 章反覆強調的「適合勝於最強」:一個用Otsu 加連通元件就能穩定運作的零件計數系統,價值不亞於用上最新模型卻不穩定的系統;受控環境用傳統方法可能又快又省又可解釋,複雜開放場景才需要深度學習。應為了解決問題而選技術。

9. 你的專題報告中應回答哪三個倫理問題?請以本書各章的倫理線索說明為什麼這是「基本功」而非加分題。

提示與參考解答:三問題:誰會受益?誰可能受到傷害?如果它出錯了,後果由誰承擔?這是基本功的理由:第7 章「還原vs. 腦補」提醒區分找回與編造的資訊、第9 章醫學影像須醫師確認、第12 章照護監控涉及隱私尊嚴、第14 章資料偏誤讓模型學到捷徑、第16 章Redmon 因軍事與隱私疑慮退出、第17 章端上處理保護隱私——技術能力愈強,對後果的想像力就愈是專業素養不可分割的一部分。

10. 請從表 18-2 選一個專題方向,列出你會用到本書哪些章節的技術,並說明各自負責什麼環節。

提示與參考解答:本題為開放題,評分重點在技術鏈的完整與合理。範例(果園產量估計):第4 章相機校正(像素換算實際尺寸以估果實大小)、第5 章CLAHE 與白平衡(戶外光線變化前處理)、第8 章色彩分析(成熟度判斷)、第14 章資料切分與評估、第15 章遷移學習與資料增強(樣本不足)、第16 章YOLO(果實偵測計數)、第17 章邊緣部署(田間無網路須機上運算)。

11. 請說出三個貫穿本書、比任何單一技術更重要的思想,並各舉兩章說明它如何反覆出現。

提示與參考解答：可任選三個：一、換一個域看問題(第6章頻率域解週期雜訊、第11章小波補位置資訊)。二、多解析度由粗到細(第11章小波金字塔、第13章SIFT尺度空間、第15章CNN層級特徵)。三、投票對抗雜訊(第9章霍夫轉換、第13章RANSAC)。四、正則化對抗病態(第7章維納濾波、第14章對抗過度擬合)。五、精確率與召回率的取捨(第9章Canny雙門檻、第10章Viola-Jones參數、第14、16章評估指標)。六、適合勝於最強(第10章ESP32上的Haar、第11章JPEG勝過JPEG 2000、第17章邊緣模型選擇)。

12. (反思題)請回顧本學期：你學到最重要的一件事是什麼？哪一章的內容最出乎你意料？接下來你想繼續學什麼？請具體說明並規劃你的下一步。

提示與參考解答：本題無標準答案，重點在反思的具體與真誠。好的回答應：指出具體的學習事件(而非泛泛而談「學到很多」)、說明它為什麼改變了你的理解、並提出可執行的下一步(如深入某個主題、參加競賽、做開源貢獻、修習進階課程如3D視覺/SLAM/生成式模型、或把專題延伸為更完整的作品)。

結語：從像素到世界

十八章的旅程到這裡告一段落。回頭看，我們從一張影像最基本的像素開始，學會了如何強化它、還原它、分析它的頻率、分割出物件、萃取特徵，再一路走到讓機器自己學習、自己辨識，最後把這一切裝進機器人裡，讓它能在真實世界中看見與行動。

但叫獸真正希望你帶走的，不是那些函式的用法——那些查得到、也會更新。我希望你帶走的是三樣東西。第一是判斷力：面對一個問題，知道有哪些工具可選、各自的代價是什麼、什麼情況下該用哪一個。這門課花了大量篇幅在比較與取捨，就是為了培養這件事。第二是連結的眼光：當你發現卷積、量化、多尺度、投票、正則化這些概念在不同章節中以不同面貌反覆出現時，你就開始像一個真正的工程師那樣思考了——看見表象之下的共通結構。第三是責任感：每一章的故事裡都有人在做選擇，而技術愈強大，選擇就愈重要。

最後，請記得叫獸開講裡那些人。他們不是教科書上的名字，而是一個個在自己的處境裡，帶著熱情、好奇與不甘，把不可能變成可能的人。傅立葉的論文被退了稿，他繼續改；哈伯望遠鏡近乎失明，工程師用數學把影像救回來；一群創客翻譯著沒人看得懂的中文文件，意外打開了物聯網的大門。他們做的事，和你在期末專題裡要做的事，本質上是同一件事——只是規模不同而已。

所以，去做一個你真心想做的東西吧。和你的夥伴一起，帶著玩的心情，允許自己失敗、允許自己重來。這門課結束了，但那個螺旋不會停——想像、創造、遊玩、分享、反思，然後再一次想像。

祝你玩得開心。我們在你的作品裡見。

——機器人叫獸 李世淵

參考資料與延伸閱讀

- Resnick, M., Lifelong Kindergarten: Cultivating Creativity through Projects, Passion, Peers, and Play, MIT Press, 2017.(本章叫獸開講之史料來源;2018 年 PROSE 教育實務獎)
- MIT Media Lab, Lifelong Kindergarten Group:<https://www.media.mit.edu/groups/lifelong-kindergarten>
- Papert, S., Mindstorms: Children, Computers, and Powerful Ideas, Basic Books, 1980.(建構主義學習經典, Resnick 的師承)
- Scratch 程式學習平台: <https://scratch.mit.edu>
- 本書第 1 至 17 章之各章參考資料。
- 補充教材投影片下載: <https://page.line.me/prof>(機器人叫獸)

索引

本索引標註各詞條主要出現的章次。中文詞條依注音符號排序,英文與縮寫詞條依字母排序。

一、中文詞條

白平衡 1、3、5、8、16-18
背景相減 9、12
比值測試 13
閉合 10、13
邊界框 14、16
邊界處理 5
邊緣偵測 1、4-7、9-10、13、15-16
邊緣運算 1-2、10、17
病態問題 7、14
膨脹 7、9-10、15
頻譜 2-3、5-7、11
模型量化 3、17
摩爾紋 3、6
門檻法 9-10
描述子 6-7、13
模板比對 13
非極大值抑制 7、9、16
非監督式學習 14
非局部平均 6-7、11、18
反卷積 1、5-7
反向傳播 14-15
分水嶺法 9
仿射轉換 4
方塊效應 3、6、11
負片 2、5
傅立葉轉換 6、11
帶通濾波 6、13、15
單應性 4、13
低通濾波 3-4、6-7
點擴散函數 7
頂帽轉換 10
多解析度分析 6、11、13、15
多影格合成 5
對比度受限的自適應直方圖等化 5
對數轉換 3、5-6
斷開 10、13
端上運算 1-2、10、17
動態範圍 1、3、5-6
動作辨識 12
特徵匹配 4-5、7、12-13、16
特徵工程 14

特徵擷取 1、6-7、9-18
透視轉換 4、12-13
梯度 5-7、9-10、13-15
梯度下降 5-6、13、15
推論引擎 17
吞吐量 17
桶狀畸變 4
同態濾波 6、10
同色異譜 8
內部參數 4
內插 3-6、8、12
擬合不足 14
逆濾波 7
鳥瞰圖 4
拉普拉斯運算子 5-7、9
離散餘弦轉換 6、11
劣化模型 5-7
連通性 3、9-10
連通元件標記 3、6、9-10、13
量化 1-9、11、13、15-18
濾波器核 5
率失真 3、11
高通濾波 6-7、9
高斯濾波 5-6、9
高斯雜訊 3、5-7、11
骨架化 3、10
過度分割 9
過度擬合 1、4、14-16、18
光流 5、8、12-13
可分離濾波 5-6、17
快速傅立葉轉換 6
孔徑問題 12-13
空間頻率 6
霍夫轉換 1、4、9、13、16、18
灰階轉換 2-6
灰階世界 5、8-9
混疊 3-4、6、12
混淆矩陣 14
幾何轉換 1、4-6、12-18
級聯分類器 10、16
計算攝影 1、5、7
積分影像 10
激勵函數 15
機器學習 1、6、10、14-16、18
解析度 1、3、6-8、11-13、15-18
結構元素 3、10、13
角點偵測 4、7、13
交叉驗證 7、14
椒鹽雜訊 5-7、10

剪枝 17
監督式學習 14
鏡頭畸變 4
精確率 1、14、16-18
距離轉換 3、8-10
卷積 1、5-7、9、11-15、18
卷積定理 5-7
卷積神經網路 5-7、13-15
卷積層 1、5、15
均值濾波 5-7
齊次座標 4、17
遷移學習 1、15-16、18
侵蝕 7、9-10、15
強化學習 14
取樣 1、3-6、8、11-12、15、18
全連接層 15
區域成長 9
稀疏光流 12-13
小波轉換 6、11
陷波濾波 3、6-7
像素 2-18
相位譜 6
相位相關法 5-6、12
相關 2-17
相機校正 2、4、7、10、12、16-18
型態學梯度 10
虛擬色彩 3、8、12
旋轉不變性 13
訓練集 7、14
直方圖 2、5、7-9、12-14
直方圖等化 5、7、9
知識蒸餾 17
遮罩銳化 5
召回率 1、14、16-18
週期性雜訊 6-7
振幅譜 6
振鈴效應 5-7、11
偵測式追蹤 12、16
正則化 1、7、14-15、18
準確率 1、7-8、14-15、18
重投影誤差 4
中值濾波 5-7
尺度不變性 13
池化層 15
遲滯連接 9
車牌辨識 1、4、10、13、18
稠密光流 8、12
創意學習螺旋 1、18
實例分割 9、14、16

視覺里程計 1、12、17
視訊 1-5、8-9、12-13、16-18
視訊穩定 4、12-13
手工特徵 13-15
神經網路 3-8、11-16
深度學習 1-2、5-18
數學型態學 10
數位浮水印 11
雙邊濾波 5-7
銳化 1、3、5-7
自適應中值濾波 7
姿態估計 1、12、14、16-17
資料集 1、14-16、18
資料增強 1、4-5、7-8、14-18
測試集 1、14-16、18
殘差連接 15
參數空間 9
色彩空間 2、5-6、8、11
色彩恆常性 8-9
散粒雜訊 3、5、7
損失函數 15
二值影像 3、7、9-10、13、15-17
延遲 2-3、10、12、17-18
驗證集 14-16
影格率 3、12、17
影格差分 12、17
影像拼接 4、6、13
影像分割 6-7、9-11、13、16
影像還原 1、3、5-7、9-11、14-15、17
影像融合 11
影像壓縮 3、6、11
物件偵測 1、7、9-10、12-14、16、18
物件追蹤 2、8、12、16
外部參數 4
維納濾波 1、6-7、11、18
位元深度 1-3、5、7、11
語意分割 9、16-17
約束最小平方 7

二、英文與縮寫詞條

AlexNet 1、14-15、17
AOI 1、5-10、16-17
Bayer 3
BGR 2-3、5、8-10、12-13、17
Canny 1-2、4-7、9-10、12-14、16、18
CLAHE 5、8-10、12、18
CMYK 8

CNN 1、5-7、9、11-18
COCO 1、14、16
CT 1-18
DCT 3、6、11、17
DFT 5-6
Dropout 1、15
ESP32-CAM 1-2、4、8、10、12、17-18
FAST 7、13、16
FFT 6-7
FLANN 13
Gabor 6、13、15
Gamma 3、5、7、14、18
GoogLeNet 15
Haar 1-2、10-11、16-18
Harris 7、13
HDR 3、5
HSL 8
HSV 2、8-9、12、16
ImageNet 1、14-15
IoU 16
Jetson 1-2、5-7、13、15-18
JPEG 1-3、5-8、11、13、17-18
JPEG 2000 1、11、13、18
Keras 14-15
kNN 9、12-14
LeNet 14-17
LoG 5-13
Lucas-Kanade 12-13
mAP 3-4、8、14-18
MNIST 1、14-15
MobileNet 15-17
MRI 1、3、6-7、9、11
NMS 16-17
NumPy 2-14、16
ONNX 16-17
OpenCV 1-10、12-13、15-17
ORB 4、7、13、17-18
Otsu 5-6、9-10、13、18
PSNR 1、3-7、11、18
PyTorch 2、14-15、17
RANSAC 1、4-5、13、16、18
RAW 2-3、5、7-8、11、13、17
ReLU 15
ResNet 15
RGB 2-3、8、11
ROI 2、6-7、9-10、12
ROS 2 1-2、4、17-18
SIFT 1-2、4-9、11、13-15、18
SLAM 1-2、13、17-18

Sobel 5-6、9-10、13、15
SORT 2、5、12、16
SSIM 1、3、5-7、11、18
SVM 14-15
TensorFlow 14-15、17
TensorRT 16-17
U-Net 9、16
VGGNet 15
WSQ 11
YCbCr 3、5-6、8、11
YOLO 1-2、4、7-10、12-18

— 全書完 —